# BÁO CÁO ĐỒ ÁN CUỐI KỲ LẬP TRÌNH WEB
# HỆ THỐNG QUẢN LÝ HỌC TẬP EDULEARN LMS

## Trang bìa

ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN
BÁO CÁO ĐỒ ÁN CUỐI KỲ MÔN LẬP TRÌNH WEB
Đề tài: Hệ thống quản lý học tập EduLearn LMS
Nhóm: Nhóm 1 — Lớp/mã học phần: Lập trình Web (HK1 2026-2027)
Giảng viên hướng dẫn: Giảng viên phụ trách môn Lập trình Web
Sinh viên thực hiện: Nguyễn Minh Trí — MSSV: 24110359
TP. HCM, năm 2026

## Thông tin nhóm và giảng viên hướng dẫn

Hệ thống được phát triển và báo cáo bởi sinh viên Nguyễn Minh Trí (MSSV: 24110359), lớp Lập trình Web. Thông tin phân công và danh mục chức năng được trình bày chi tiết tại Phụ lục A.

## Bảng phân công và đóng góp của từng thành viên

Chi tiết phân công nhiệm vụ, kiến trúc các module và tỷ lệ đóng góp được đối chiếu với lịch sử commit thực tế của repository tại Phụ lục A.

## Lời cảm ơn

Nhóm xin cảm ơn giảng viên phụ trách môn Lập trình Web đã cung cấp yêu cầu và tiêu chí để định hướng thực hiện đồ án. Qua đề tài quản lý học tập, nhóm có cơ hội hệ thống hóa các nội dung về tổ chức ứng dụng, dữ liệu, xác thực, phân quyền và đánh giá chức năng. Báo cáo được đối chiếu với source ở phiên bản đã ghi. Các phần còn thiếu hình, dữ liệu nhóm và kết quả kiểm thử được đánh dấu để bổ sung trước nộp; nhóm mong nhận được góp ý để hoàn thiện sản phẩm và cách trình bày.

# Mục lục

Mục lục cấu trúc phân cấp các chương và phụ lục của báo cáo Đồ án cuối kỳ:

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI
CHƯƠNG 2 KHẢO SÁT HIỆN TRẠNG
CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG
CHƯƠNG 4 KIẾN TRÚC VÀ CÔNG NGHỆ
CHƯƠNG 5 XÂY DỰNG VÀ MINH HỌA CÁC CHỨC NĂNG
CHƯƠNG 6 KIỂM THỬ VÀ ĐÁNH GIÁ
CHƯƠNG 7 CÀI ĐẶT TRIỂN KHAI VÀ HƯỚNG DẪN SỬ DỤNG
CHƯƠNG 8 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN
TÀI LIỆU THAM KHẢO
PHỤ LỤC
A Phân công và GitHub
B Danh mục API
C Sử dụng AI trong quá trình thực hiện
D Minh chứng kiểm thử và giao diện
E Hồ sơ và kịch bản báo cáo cuối kỳ
F Kết quả rà soát source và phạm vi báo cáo

[Sau khi chốt hình và dàn trang, tạo/cập nhật mục lục có số trang trong bản nộp.]

# Danh mục hình

Các hình minh chứng đã chèn:

Hình 3.73 Trang chủ khi chưa đăng nhập — khu vực giới thiệu.
Hình 3.74 Trang chủ — nhóm chức năng, lời mời đăng ký và chân trang.
Hình 5.1 Form đăng ký tài khoản với vai trò Học viên.
Hình 5.2 Giao diện thông báo đăng ký tài khoản thành công.
Hình 5.3 Danh sách khóa học với ô tìm kiếm, bộ lọc danh mục và độ khó.
Hình 5.4 Chi tiết khóa học và khu vực đăng ký tham gia.
Hình 5.5 Hồ sơ cá nhân của tài khoản có nhãn INSTRUCTOR.

Các hình khảo sát, UML và giao diện khác còn chờ bổ sung. Số trang sẽ cập nhật sau khi hoàn tất dàn trang.

# Danh mục bảng

Danh mục dưới đây khớp tên và số thứ tự của 100 bảng hiện có. Chưa ghi số trang vì tài liệu còn chờ hình và kết quả thực tế.

Bảng Thuật ngữ và từ viết tắt
Bảng 2.1 So sánh các nền tảng quản lý học tập
Bảng 3.1 Ma trận vai trò và quyền truy cập
Bảng 3.2 Đặc tả UC001 Đăng ký tài khoản
Bảng 3.3 Đặc tả UC002 Gửi và xác minh OTP email
Bảng 3.4 Đặc tả UC003 Đăng nhập bằng tài khoản
Bảng 3.5 Đặc tả UC004 Đăng nhập bằng Google OAuth
Bảng 3.6 Đặc tả UC005 Đăng xuất

Bảng 3.7 Đặc tả UC006 Quên và đặt lại mật khẩu
Bảng 3.8 Đặc tả UC007 Làm mới phiên đăng nhập
Bảng 3.9 Đặc tả UC008 Cập nhật hồ sơ và ảnh đại diện
Bảng 3.10 Đặc tả UC009 Thay đổi mật khẩu
Bảng 3.11 Đặc tả UC010 Xem trang chủ và giới thiệu
Bảng 3.12 Đặc tả UC011 Tìm kiếm lọc và phân trang khóa học
Bảng 3.13 Đặc tả UC012 Xem chi tiết khóa học
Bảng 3.14 Đặc tả UC013 Đăng ký tham gia khóa học
Bảng 3.15 Đặc tả UC014 Xem khóa học đã tham gia
Bảng 3.16 Đặc tả UC015 Xem cấu trúc và nội dung bài học
Bảng 3.17 Đặc tả UC016 Cập nhật và theo dõi tiến độ
Bảng 3.18 Đặc tả UC017 Xem lịch học và thông báo
Bảng 3.19 Đặc tả UC018 Tra cứu bài tập và quiz
Bảng 3.20 Đặc tả UC019 Làm và nộp bài quiz
Bảng 3.21 Đặc tả UC020 Xem điểm và lịch sử làm quiz
Bảng 3.22 Đặc tả UC021 Soạn chạy thử và nộp bài lập trình
Bảng 3.23 Đặc tả UC022 Xem kết quả chấm và lịch sử submission
Bảng 3.24 Đặc tả UC023 Xem bảng xếp hạng
Bảng 3.25 Đặc tả UC024 Tạo sửa xóa và quản lý khóa học
Bảng 3.26 Đặc tả UC025 Quản lý chương mục và cấu trúc khóa học
Bảng 3.27 Đặc tả UC026 Quản lý bài học và tài nguyên
Bảng 3.28 Đặc tả UC027 Mời và quản lý học viên trong khóa học
Bảng 3.29 Đặc tả UC028 Khóa và mở khóa khóa học bởi Admin
Bảng 3.30 Đặc tả UC029 Quản lý lịch học và liên kết Google Meet
Bảng 3.31 Đặc tả UC030 Quản lý thông báo khóa học
Bảng 3.32 Đặc tả UC031 Quản lý ngân hàng câu hỏi và đáp án
Bảng 3.33 Đặc tả UC032 Tạo sửa và quản lý quiz
Bảng 3.34 Đặc tả UC033 Nhập quiz từ template
Bảng 3.35 Đặc tả UC034 Tạo sửa và quản lý bài lập trình
Bảng 3.36 Đặc tả UC035 Quản lý test case
Bảng 3.37 Đặc tả UC036 Nhập bài lập trình từ template
Bảng 3.38 Đặc tả UC037 Xem bảng điểm và thống kê học viên
Bảng 3.39 Đặc tả UC038 Tìm kiếm và xem bài viết
Bảng 3.40 Đặc tả UC039 Tạo sửa xóa bài viết cá nhân
Bảng 3.41 Đặc tả UC040 Quản lý bài viết cá nhân
Bảng 3.42 Đặc tả UC041 Bình luận và trả lời bình luận
Bảng 3.43 Đặc tả UC042 Bình chọn bài viết và bình luận
Bảng 3.44 Đặc tả UC043 Quản lý tag và tệp đính kèm
Bảng 3.45 Đặc tả UC044 Hỏi đáp với chatbot học tập
Bảng 3.46 Đặc tả UC045 Xem lịch sử và lưu trữ phiên chat
Bảng 3.47 Đặc tả UC046 Sinh quiz bằng AI
Bảng 3.48 Đặc tả UC047 Sinh test case bằng AI
Bảng 3.49 Đặc tả UC048 Quản lý người dùng và vai trò
Bảng 3.50 Đặc tả UC049 Quản lý khóa học toàn hệ thống
Bảng 3.51 Đặc tả UC050 Quản lý danh mục khóa học
Bảng 3.52 Đặc tả UC051 Quản lý nội dung blog
Bảng 3.53 Đặc tả UC052 Theo dõi hoạt động chấm bài lập trình
Bảng 3.54 Đặc tả UC053 Quản lý nội dung website

Bảng 3.55 Đặc tả UC054 Theo dõi dashboard và sức khỏe dịch vụ
Bảng 3.56 Từ điển dữ liệu users
Bảng 3.57 Từ điển dữ liệu refresh_tokens
Bảng 3.58 Từ điển dữ liệu email_verifications
Bảng 3.59 Từ điển dữ liệu categories
Bảng 3.60 Từ điển dữ liệu courses
Bảng 3.61 Từ điển dữ liệu enrollments
Bảng 3.62 Từ điển dữ liệu lesson_progress
Bảng 3.63 Từ điển dữ liệu schedules
Bảng 3.64 Từ điển dữ liệu announcements
Bảng 3.65 Từ điển dữ liệu course_categories
Bảng 3.66 Từ điển dữ liệu course_learning_points
Bảng 3.67 Từ điển dữ liệu course_requirements
Bảng 3.68 Từ điển dữ liệu questions
Bảng 3.69 Từ điển dữ liệu answers
Bảng 3.70 Từ điển dữ liệu quizzes
Bảng 3.71 Từ điển dữ liệu quiz_questions
Bảng 3.72 Từ điển dữ liệu quiz_attempts
Bảng 3.73 Từ điển dữ liệu quiz_answer_records
Bảng 3.74 Từ điển dữ liệu problems
Bảng 3.75 Từ điển dữ liệu test_cases
Bảng 3.76 Từ điển dữ liệu grades
Bảng 3.77 Từ điển dữ liệu posts
Bảng 3.78 Từ điển dữ liệu tags
Bảng 3.79 Từ điển dữ liệu comments
Bảng 3.80 Từ điển dữ liệu votes
Bảng 3.81 Từ điển dữ liệu post_tags liên kết bài viết và tag
Bảng 3.82 Từ điển dữ liệu course_structures
Bảng 3.83 Từ điển dữ liệu lessons
Bảng 3.84 Từ điển dữ liệu submissions
Bảng 3.85 Từ điển dữ liệu chat_sessions
Bảng 3.86 Từ điển dữ liệu chat_messages
Bảng 4.1 Thành phần kiến trúc và trách nhiệm
Bảng 4.2 Các lớp kiểm tra quyền và giới hạn
Bảng 6.1 Kiểm kê test trong source và trạng thái chạy
Bảng 6.2 Trạng thái thực hiện kiểm thử và build
Bảng 7.1 So sánh các phương án chạy ứng dụng
Bảng A.1 Thông tin thành viên cần bổ sung
Bảng B.1 Danh mục mapping Identity
Bảng B.2 Danh mục mapping Course
Bảng B.3 Danh mục mapping Assignment
Bảng B.4 Danh mục mapping Blog
Bảng B.5 Danh mục mapping Chatbot
Bảng E.1 Các nội dung còn cần bổ sung trước khi nộp

# Danh mục từ viết tắt

Danh sách thuật ngữ được dùng thống nhất trong phần dưới.
STUDENT/INSTRUCTOR/ADMIN là tên role trong source; Guest là trạng thái chưa đăng nhập. Manager không phải role riêng đã có, SSE không gọi là WebSocket và ngữ cảnh lấy qua client không tự được gọi RAG embedding.

**Bảng Thuật ngữ và từ viết tắt**

| Viết tắt/thuật ngữ | Nghĩa sử dụng trong báo cáo |
|---|---|
| LMS | Learning Management System, hệ thống quản lý học tập |
| API / REST | Giao diện lập trình ứng dụng; các HTTP endpoint trong source |
| DTO | Đối tượng dữ liệu request/response |
| JPA / ORM | Ánh xạ entity và dữ liệu quan hệ |
| JWT / OTP | Token xác thực / mã dùng một lần theo luồng xác minh |
| OAuth | Ủy quyền; được ứng dụng dùng trong luồng đăng nhập Google |
| RBAC | Kiểm soát quyền theo vai trò; cần thêm kiểm tra owner |
| PK / FK / UNIQUE | Khóa chính / khóa ngoại / ràng buộc duy nhất |
| ERD / UML | Mô hình quan hệ thực thể / ký pháp mô hình hóa |
| SSE | Server-Sent Events, phản hồi stream chatbot |
| CI / Build | Tích hợp liên tục / biên dịch, đóng gói; không tự đồng nghĩa test đạt |
| CRUD | Tạo, đọc, cập nhật, xóa |
| Cache / TTL | Dữ liệu đệm / thời gian hiệu lực |
| E2E / Mock | Kiểm thử xuyên luồng / đối tượng giả lập phụ thuộc |
| AC / WA / CE | Accepted / Wrong Answer / Compilation Error theo judge |
| TLE / MLE / RE / IE | Lỗi giới hạn thời gian/bộ nhớ/chạy/chấm hệ thống theo engine |
| PENDING / JUDGING | Trạng thái chờ/đang chấm submission, chưa là điểm cuối |

# cHƯƠNG 1 TỔNG QUAN ĐỀ TÀI

## 1.1 Giới thiệu đề tài

### 1.1.1 Bối cảnh quản lý học tập trực tuyến

Trong hoạt động học tập trực tuyến, tài liệu, bài tập, kết quả đánh giá và thông báo cần được tổ chức theo cùng một khóa học. Nếu mỗi loại thông tin nằm ở một công cụ riêng, học viên phải chuyển giữa nhiều nơi để học bài, nộp bài và xem kết quả; giảng viên cũng khó theo dõi toàn bộ quá trình trong một luồng thống nhất.
Đề tài xây dựng một hệ thống quản lý học tập trên nền web, tập trung vào học nội dung khóa học và thực hành lập trình. Bối cảnh này là bài toán thiết kế của nhóm, không phải kết quả một cuộc điều tra thống kê tại trường. Hệ thống được đặt tên EduLearn LMS theo README của phiên bản source dùng cho báo cáo.

### 1.1.2 Vấn đề cần giải quyết

Bài toán thứ nhất là tổ chức học liệu: khóa học cần có cấu trúc chương mục, bài học và tài nguyên để người học biết mình đang học phần nào. Bài toán thứ hai là đánh giá: quiz và bài lập trình có cách nộp, chấm và lưu kết quả khác nhau nhưng phải liên kết với hoạt động học tập.
Bài toán thứ ba là quản lý quyền. Học viên chỉ được thực hiện hoạt động học tập phù hợp; giảng viên quản lý nội dung của mình; Admin quản lý dữ liệu toàn hệ thống theo các API được cấp quyền. Ngoài ra, ứng dụng cần xác thực tài khoản, xử lý đầu vào sai, lưu tiến độ và hỗ trợ tra cứu. Các vấn đề này được triển khai qua Identity, Course, Assignment, Blog và Chatbot Service.

### 1.1.3 Lý do lựa chọn đề tài

Nhóm lựa chọn LMS vì đề tài có nghiệp vụ rõ ràng, có nhiều vai trò và tạo điều kiện vận dụng kiến thức lập trình web vào một quy trình hoàn chỉnh. Một thao tác như nộp bài không chỉ cần giao diện mà còn cần kiểm tra phiên đăng nhập, quyền tham gia, tính hợp lệ của dữ liệu, xử lý nghiệp vụ và lưu kết quả.
Điểm tập trung của phiên bản này là kết hợp học bài, quiz và thực hành code trong cùng ứng dụng; đồng thời có blog và chatbot hỗ trợ học tập. Nhóm không đặt mục tiêu thay thế toàn bộ chức năng của một LMS thương mại. Chất lượng đồ án được đánh giá bằng mức độ đúng và hoàn chỉnh của các luồng đã chọn, cùng khả năng giải thích source và minh chứng vận hành.

## 1.2 Mục tiêu của hệ thống

### 1.2.1 Mục tiêu đối với học viên

Đối với học viên, mục tiêu là cung cấp luồng sử dụng từ tìm khóa học đến xem kết quả học tập: tìm kiếm và xem chi tiết, đăng ký tham gia, truy cập bài học, cập nhật tiến độ, làm quiz và thực hành lập trình. Lịch sử làm bài giúp người học xem lại kết quả thay vì chỉ nhận thông báo điểm tại thời điểm nộp.

Giao diện còn có dashboard, lịch học, thông báo, blog và chatbot. Đây là các nhóm chức năng xác định từ source, chưa phải cam kết rằng mọi trường hợp đều vận hành thành công. Các tiêu chí nghiệm thu cụ thể, như quyền truy cập bài học và cách ghi nhận điểm, được trình bày và kiểm tra ở các chương sau.

### 1.2.2 Mục tiêu đối với giảng viên

Đối với giảng viên, mục tiêu là quản lý nội dung và đánh giá trong phạm vi khóa học được phép phụ trách. Giảng viên có thể tổ chức cấu trúc khóa học, bài học, tài nguyên, danh sách học viên, lịch học và thông báo; xây dựng ngân hàng câu hỏi, quiz, ibài lập trình và test case.
Hệ thống cung cấp bảng điểm và thống kê để hỗ trợ theo dõi kết quả. Tính năng AI hỗ trợ đề xuất quiz hoặc test case được xem là công cụ tạo bản nháp; nội dung cần được người phụ trách kiểm tra trước khi sử dụng. Quyền INSTRUCTOR không đồng nghĩa được sửa mọi khóa học: nhiều service còn kiểm tra instructorId của khóa học.

### 1.2.3 Mục tiêu đối với quản trị viên

Đối với Admin, mục tiêu là quản lý tài khoản, khóa học và các nội dung dùng chung, đồng thời theo dõi hoạt động hệ thống qua dashboard và trạng thái service. Khóa hoặc mở khóa khóa học thuộc quyền Admin trong CourseController, giúp phân biệt nghiệp vụ quản trị với thao tác biên soạn nội dung của giảng viên.
Báo cáo phân biệt các chức năng có API lưu trữ với thành phần cấu hình mới nằm ở frontend. Chẳng hạn, danh mục khóa học có API, nhưng ảnh truyền cảm hứng lưu bằng localStorage và phần thông tin người sáng lập chưa có thao tác lưu backend. Các giới hạn này được ghi nhận khi đánh giá mức hoàn thiện.

## 1.3 Đối tượng và phạm vi

### 1.3.1 Khách chưa đăng nhập Guest

Guest là người chưa đăng nhập, không phải giá trị role trong hệ thống. Spring MVC Controller & Route Mapping cho phép truy cập các trang công khai như giới thiệu, danh sách khóa học, chi tiết khóa học và blog; các màn hình đăng ký, đăng nhập và quên mật khẩu nằm trong nhóm xác thực.
Khách có thể tìm hiểu nội dung trước khi tạo tài khoản. Việc truy cập trang chi tiết không tự cấp quyền xem đầy đủ bài học, nộp bài hoặc sửa dữ liệu. Quyền API còn được xác định bởi cấu hình bảo mật và kiểm tra nghiệp vụ tại backend.

### 1.3.2 Học viên STUDENT

STUDENT là vai trò học viên. Nhánh giao diện được bảo vệ bằng Bộ lọc bảo vệ Spring Security và Bộ lọc kiểm tra vai trò Spring Security, bao gồm dashboard, khóa học đã tham gia, khu vực bài tập, quiz, bài lập trình, kết quả, lịch sử và bảng xếp hạng. Học viên còn có màn hình viết và quản lý bài blog cá nhân.
Phạm vi dữ liệu không được xác định chỉ bằng role. Ví dụ, xem nội dung một khóa học còn phụ thuộc việc tham gia khóa học; sửa bài viết phải gắn với tác giả. Báo cáo sẽ sử dụng cả route và điều kiện service để mô tả quyền.

### 1.3.3 Giảng viên INSTRUCTOR

INSTRUCTOR là vai trò giảng viên trong source. Các màn hình chính gồm quản lý khóa học, trung tâm nội dung khóa học, quản lý quiz và bài lập trình, kết quả học viên và blog. Chức năng biên soạn nội dung gắn với trách nhiệm của giảng viên đối với khóa học.
Backend cho phép cả Admin và Instructor ở một số API, nhưng có kiểm tra quyền sở hữu ở tầng service. Vai trò này được mô tả theo đúng tên source; không đổi tên thành MANAGER rồi khẳng định ứng dụng có một role Manager riêng.

### 1.3.4 Quản trị viên ADMIN

ADMIN là vai trò quản trị hệ thống. Nhánh frontend có quản lý người dùng, tài khoản quản trị, khóa học, blog, nội dung website và hoạt động chấm bài. Các API quản trị tài khoản hỗ trợ những thao tác như cập nhật thông tin, thay đổi trạng thái hoạt động và quản lý tài khoản Admin theo chức năng cụ thể.
Một số API quản lý bài tập hoặc nội dung cũng chấp nhận Admin. Tuy nhiên, quyền backend và màn hình frontend là hai căn cứ khác nhau; báo cáo không khẳng định mọi quyền có sẵn đều đã có một giao diện quản trị hoàn chỉnh.

### 1.3.5 Phạm vi chức năng và giới hạn đề tài

Phạm vi đồ án gồm xác thực và tài khoản; quản lý khóa học và học liệu; đăng ký học và tiến độ; quiz; bài lập trình và chấm tự động; blog; chatbot; các chức năng quản trị và cấu hình triển khai. Giao diện dùng React, backend dùng Spring Boot microservices, với MySQL, MongoDB, Redis và MinIO phục vụ các loại dữ liệu khác nhau.
Đề tài không triển khai luồng bán hàng, giỏ hàng, vận chuyển hay thanh toán khóa học trong phạm vi đã xác định. Chưa có role MANAGER độc lập; custom judge chưa hoàn thiện; chưa xác nhận đáp ứng Bootstrap. Kết quả kiểm thử, ảnh demo và đánh giá hiệu năng chỉ được bổ sung khi có minh chứng thực tế.

## 1.4 Phương pháp và kế hoạch thực hiện

### 1.4.1 Phương pháp khảo sát và phân tích

Phương pháp khảo sát hiện tại là nghiên cứu tài liệu chính thức của Moodle, Google Classroom và Canvas để xác định cách tổ chức khóa học, giao bài và phản hồi kết quả. Những nhận xét về mức phù hợp với đề tài được trình bày như đánh giá của nhóm, không giả định đã phỏng vấn người dùng hoặc đo thời gian sử dụng.
Phân tích source tiến hành theo tuyến giao diện → API → service → repository → dữ liệu. Mỗi chức năng cần xác định tác nhân, đầu vào, đầu ra, điều kiện trước và ngoại lệ; từ đó xây dựng Use Case, sơ đồ hoạt động, sơ đồ tuần tự và mô hình dữ liệu.

### 1.4.2 Phương pháp phát triển và kiểm thử

Quy trình kỹ thuật đề xuất cho mỗi chức năng gồm xác định yêu cầu, thiết kế API và dữ liệu, xây dựng backend, nối frontend, kiểm tra validation và phân quyền, sau đó chạy kịch bản chức năng. Các bước này là phương pháp dùng để hoàn thiện và kiểm chứng đồ án, không được dùng thay cho lịch sử thực hiện thực tế của nhóm.

Source đã có test backend với JUnit và test frontend với Vitest cùng Testing Library. Đợt rà soát ban đầu chưa chạy các bộ test này. Chương kiểm thử ghi rõ môi trường, lệnh chạy và vị trí bổ sung kết quả và trường hợp chưa xác minh, thay vì suy số test đạt từ số lượng file.

### 1.4.3 Kế hoạch và các mốc thực hiện

Kế hoạch hoàn thiện báo cáo được chia theo thứ tự phụ thuộc: xác định phạm vi source; viết tổng quan và khảo sát; phân tích yêu cầu; thiết kế sơ đồ và dữ liệu; mô tả kiến trúc; giải thích triển khai từng module; kiểm thử và hướng dẫn cài đặt; cuối cùng tổng hợp kết luận và hồ sơ nộp.
Các mốc phát triển phần mềm, ngày bắt đầu, thời gian hoàn thành và tiến độ của từng thành viên cần lấy từ kế hoạch nhóm và lịch sử GitHub. Không dùng lịch viết báo cáo để thay thế lịch phát triển ứng dụng.

### 1.4.4 Phân công theo vai trò và nhóm chức năng

Phân công cần gắn với một tập chức năng có thể kiểm chứng, bao gồm backend, frontend và kiểm thử liên quan. Mỗi phần phải chỉ ra người chịu trách nhiệm chính, người phối hợp, trạng thái hoàn thành và minh chứng commit hoặc pull request; đồng thời phù hợp yêu cầu báo cáo theo vai trò.
Các tên có sẵn trong giao diện giới thiệu của source không được tự động dùng làm thành viên nhóm nộp bài. Phụ lục A là nơi lưu bảng phân công chi tiết.

### 1.4.5 Quy trình cộng tác và commit GitHub

GitHub được dùng làm nguồn quản lý phiên bản của đồ án. Khi đánh giá đóng góp, cần liên hệ commit với file và chức năng thay đổi; số lượng commit riêng lẻ không đủ chứng minh khối lượng hoặc chất lượng công việc. Một thay đổi cần được người phụ trách hiểu và giải thích trước khi tích hợp.
Báo cáo tham chiếu repository LMSmau, nhánh main, phiên bản đã rà soát có commit rút gọn 4288b95. Nhóm cần bổ sung lịch sử phát triển, các nhánh hoặc pull request có thật và kết quả kiểm tra khi tích hợp; chưa khẳng định đã áp dụng một quy trình review cụ thể nếu thiếu minh chứng.

### 1.4.6 Quy trình sử dụng và kiểm chứng code do AI hỗ trợ

AI có thể hỗ trợ phân tích yêu cầu, gợi ý code, tạo kịch bản kiểm thử và diễn giải lỗi. Người thực hiện cần đọc kết quả, đối chiếu với cấu trúc project, kiểm tra API và thư viện đang dùng, sửa những phần không phù hợp rồi kiểm thử. Trách nhiệm đối với code vẫn thuộc thành viên phụ trách.
Phụ lục C sẽ ghi công cụ, prompt thực tế, phần code được hỗ trợ, chỉnh sửa của nhóm và cách kiểm chứng. Không tạo lại lịch sử prompt giả để làm minh chứng. AI hỗ trợ phát triển khác với chatbot tích hợp trong ứng dụng; hai nội dung được đánh giá và mô tả riêng.

## 1.5 Đối chiếu yêu cầu môn học

### 1.5.1 Ánh xạ vai trò LMS với yêu cầu đề tài không bán hàng

Theo yêu cầu đề tài không bán hàng, vai trò và chức năng cần được thống nhất với giảng viên. Trong source, Guest tương ứng người chưa đăng nhập; STUDENT phục vụ người học; INSTRUCTOR phụ trách khóa học; ADMIN quản trị toàn hệ thống.
Source chưa có MANAGER độc lập. Việc dùng INSTRUCTOR làm vai trò quản lý nghiệp vụ giảng dạy là một phương án ánh xạ, chưa phải bằng chứng giảng viên đã chấp thuận.

### 1.5.2 Đối chiếu chức năng chung và bảo mật

Source có các luồng đăng ký, đăng nhập, đăng xuất, gửi và xác minh OTP email, quên và đặt lại mật khẩu. Identity Service cấu hình Spring Security và BCryptPasswordEncoder với strength 10; ứng dụng còn có JWT, refresh token và Google OAuth.
Sự có mặt của cấu hình không đủ kết luận mọi luồng an toàn. Các đợt tiếp theo sẽ kiểm tra điều kiện OTP, trạng thái tài khoản, quyền sở hữu và vòng đời token; việc gửi email thành công phải được xác minh với SMTP thực tế. Chức năng tìm kiếm và lọc được đối chiếu với khóa học thay vì sản phẩm bán hàng.

### 1.5.3 Đối chiếu mô hình ba tầng trong từng microservice

Kiến trúc tổng thể là microservices; bên trong các service nghiệp vụ có controller tiếp nhận HTTP, service xử lý nghiệp vụ và repository truy cập dữ liệu. DTO, mapper, entity, document và validator hỗ trợ ranh giới giữa các tầng.
Do đó, yêu cầu ba tầng cần được giải thích bằng luồng xử lý cụ thể trong từng service, không chỉ bằng sơ đồ thư mục. Tầng giao diện Thymeleaf (web-portal / web-app) là ứng dụng riêng; Gateway và Eureka là thành phần hạ tầng, không được xem là ba tầng nghiệp vụ thay cho controller–service–repository.

### 1.5.4 Đối chiếu yêu cầu Bootstrap và responsive

Hệ thống chính thức triển khai tầng giao diện người dùng theo kiến trúc Server-Side Rendering (SSR) trên nền tảng Spring Boot kết hợp Spring MVC, Thymeleaf và Bootstrap 5 (phiên bản 5.3.3 tích hợp qua WebJars dependency org.webjars:bootstrap:5.3.3 trong hai ứng dụng độc lập web-portal và web-app).
Giao diện kế thừa hệ thống lưới 12 cột (Grid System), các lớp tiện ích hiện đại (Flexbox, Spacing, Typography) và các thành phần giao diện chuẩn của Bootstrap (Navbar, Cards, Modals, Badges, Alerts, Dropdowns, Tables).
Về tính tương thích thiết bị (Responsive): Hệ thống ứng dụng các breakpoint tiêu chuẩn col-sm, col-md, col-lg kết hợp media queries tùy biến để tự động co giãn và dàn trang linh hoạt trên máy tính để bàn (Desktop), máy tính xách tay (Laptop) và máy tính bảng (Tablet). Đối với màn hình điện thoại di động nhỏ, các chức năng cơ bản (đọc bài, xem tin tức, form đăng nhập/đăng ký) hiển thị tốt, trong khi các giao diện quản trị phức tạp nhiều cột và trình soạn thảo mã nguồn Monaco Editor vẫn đang tiếp tục được tối ưu hóa hiển thị để mang lại trải nghiệm thao tác tốt nhất trên thiết bị màn hình cảm ứng nhỏ.

### 1.5.5 Ma trận minh chứng theo năm tiêu chí rubric

Minh chứng được tổ chức theo năm tiêu chí của rubric: tính thực tiễn và phân công 10 điểm; AI và cộng tác GitHub 20 điểm; cấu trúc dữ liệu, vai trò, giao diện 20 điểm; chất lượng ứng dụng và code 35 điểm; sáng tạo 15 điểm. Tổng là 100 điểm, quy đổi sang thang 10.
Tiêu chí 1 liên hệ mục tiêu và bảng phân công; tiêu chí 2 liên hệ commit và prompt thực tế; tiêu chí 3 liên hệ Chương 3 và giao diện; tiêu chí 4 liên hệ triển khai, kiểm thử và cài đặt; tiêu chí 5 liên hệ các chức năng ngoài chương trình có minh chứng. Mục này chỉ định vị bằng chứng, chưa tự chấm điểm hoặc cộng điểm cho tính năng chưa kiểm chứng.

# CHƯƠNG 2 KHẢO SÁT HIỆN TRẠNG

## 2.1 Mục tiêu và phương pháp khảo sát

### 2.1.1 Phạm vi khảo sát

Khảo sát tập trung vào ba hệ thống Moodle, Google Classroom và Canvas LMS, với các nội dung liên quan trực tiếp đến đề tài: tổ chức khóa học, quản lý người học, giao bài, đánh giá và phản hồi. Việc lựa chọn ba hệ thống giúp nhóm xem xét những cách tổ chức khác nhau mà không cần khảo sát quá nhiều sản phẩm có chức năng tương tự.
Phạm vi hiện tại là khảo sát tài liệu công khai chính thức. Nhóm chưa thực hiện đo hiệu năng, thử nghiệm có kiểm soát hoặc phỏng vấn người dùng. Vì vậy, nhận xét về mức phù hợp là phân tích thiết kế, không phải kết luận định lượng về chất lượng sản phẩm.

### 2.1.2 Tiêu chí so sánh

Các tiêu chí gồm: cách chia nội dung học; phân chia trách nhiệm giảng viên và học viên; giao và nộp bài; quiz và chấm điểm; quản lý kết quả; theo dõi hoàn thành; khả năng tổ chức luồng sử dụng. Với mỗi tiêu chí, nhóm ghi tính năng có căn cứ, điều kiện sử dụng và bài học có thể áp dụng cho LMS.
Không sử dụng các tiêu chí như nhanh hơn, đẹp hơn hoặc dễ dùng hơn nếu chưa có phương pháp đo. Đối với chấm code và AI, tài liệu được chọn chưa đủ để khẳng định toàn bộ khả năng của từng nền tảng; báo cáo không đánh dấu là không hỗ trợ.

### 2.1.3 Nguồn khảo sát và thời điểm ghi nhận

Tài liệu được truy cập ngày 14/10/2026. Nguồn Moodle gồm Teacher quick guide [1], Activities [2] và Activity completion settings của nhánh tài liệu 4.5 [3]. Hai liên kết tổng quát Moodle được chuyển tới nhánh 5.3 tại thời điểm truy cập; nhóm ghi rõ nhánh tài liệu thay vì coi mọi ảnh hoặc thao tác là giống nhau giữa các phiên bản.
Nguồn Google Classroom gồm hướng dẫn tạo assignment [4], tạo quiz assignment [5] và chấm/trả bài [6]. Nguồn Canvas gồm hướng dẫn Modules [7], Assignments [8], tài nguyên bắt đầu cho Instructor [9] và mô tả SpeedGrader trên trang sản phẩm [10]. Danh sách URL đầy đủ nằm ở phần tài liệu tham khảo. Các ảnh khảo sát vẫn cần bổ sung nguồn và thời điểm chụp.

## 2.2 Khảo sát Moodle

### 2.2.1 Tổng quan hệ thống

Moodle tổ chức hoạt động học tập theo course. Theo hướng dẫn giảng viên, một khóa học có thể trình bày nội dung theo chủ đề hoặc tuần; giảng viên phụ trách học liệu và có thể quản lý việc tham gia tùy cấu hình của website [1].
Đối với đề tài, Moodle là nguồn tham khảo cho cách gắn nội dung, hoạt động và quyền truy cập với khóa học. Nhóm chỉ sử dụng các phần đã khảo sát để rút ra yêu cầu thiết kế, không xem cấu hình của một website Moodle là cấu hình chung của mọi đơn vị.

### 2.2.2 Các vai trò và chức năng chính

Teacher chịu trách nhiệm học liệu trong khóa học; người học cần được xác thực và tham gia theo phương thức website cho phép. Teacher thông thường không mặc định có quyền tạo khóa học mới. Hệ thống có nhiều hoạt động như assignment, quiz và forum; assignment cho phép đánh giá bài nộp, forum phục vụ thảo luận [1][2].
Bài học áp dụng là phải tách quyền hệ thống khỏi quyền trong khóa học. EduLearn cho INSTRUCTOR tạo khóa học theo source hiện tại, nên không sao chép nguyên quyền Teacher của Moodle; thay vào đó, nhóm mô tả ma trận quyền riêng và điều kiện sở hữu dữ liệu.

### 2.2.3 Quy trình dạy học và đánh giá

Luồng tham khảo gồm tổ chức course, đưa học liệu và hoạt động vào các phần nội dung, cho người học tham gia, tiếp nhận bài làm và ghi nhận kết quả. Moodle có gradebook ở cấp course [1]. Hoạt động và tài nguyên là hai khái niệm cần phân biệt: một bên yêu cầu người học tương tác, một bên cung cấp nội dung để tiếp cận [2].
Đối với completion, điều kiện có thể gắn với hành động hoặc kết quả của hoạt động tùy thiết lập; việc nộp assignment và việc được chấm là hai thời điểm khác nhau [3]. Vì vậy, EduLearn phải định nghĩa rõ hoàn thành bài học, hoàn thành quiz và kết quả chấm code thay vì dùng chung một trạng thái.

### 2.2.4 Giao diện minh họa

Mục đích minh họa là chỉ ra vị trí học liệu, hoạt động và kết quả. Chưa có ảnh được chèn trong đợt này, nên chưa đưa ra nhận xét về bố cục cụ thể, số lần bấm hoặc trải nghiệm responsive của một website Moodle.

### 2.2.5 Ưu điểm

Ưu điểm liên quan đến đề tài là có nhiều loại hoạt động dạy học trong một course, như giao bài, quiz và thảo luận [2]. Tài liệu completion còn cho thấy tiến độ có thể được xác định bằng các điều kiện khác nhau [3].
Từ góc nhìn thiết kế của nhóm, cách tách học liệu, hoạt động và kết quả giúp mô hình dữ liệu rõ hơn. EduLearn áp dụng nguyên tắc này bằng dữ liệu bài học và tiến độ ở Course Service, dữ liệu quiz và submission ở Assignment Service; đây là lựa chọn của đồ án, không phải sao chép kiến trúc nội bộ Moodle.

### 2.2.6 Hạn chế và bài học áp dụng

Khả năng sử dụng và cấu hình Moodle phụ thuộc quyền được cấp và thiết lập của website. Đối với một đồ án có thời gian giới hạn, tái tạo toàn bộ cơ chế của Moodle sẽ làm phạm vi quá rộng; đây là hạn chế về phạm vi triển khai của nhóm, không phải kết luận Moodle có chất lượng thấp.
Nhóm chọn học theo cấu trúc nội dung, kiểm tra điều kiện tham gia và lưu kết quả rõ ràng. Không khẳng định Moodle thiếu chấm lập trình hoặc AI: việc đánh giá các khả năng đó cần khảo sát thêm plugin, tích hợp và phiên bản cụ thể.

## 2.3 Khảo sát Google Classroom

### 2.3.1 Tổng quan hệ thống

Google Classroom là một hệ thống tham khảo cho quy trình giao và xử lý bài tập. Tài liệu tạo assignment mô tả việc đặt tiêu đề, hướng dẫn, đính kèm học liệu, lựa chọn người nhận và cấu hình điểm hoặc hạn nộp [4].
Điểm nhóm quan tâm là sự liên kết giữa thông tin bài tập và thao tác nộp/chấm/trả bài. Những chức năng được mô tả ở đây thuộc tài liệu đã khảo sát; không mặc định mọi tài khoản hay phiên bản Workspace có cùng tập tính năng.

### 2.3.2 Các vai trò và chức năng chính

Trong luồng assignment, giáo viên tạo và giao yêu cầu, học sinh thực hiện bài, còn giáo viên xem và đánh giá kết quả. Tài liệu cho phép tổ chức assignment bằng thông tin như chủ đề, điểm, hạn nộp và tệp đính kèm [4]. Classroom cũng có quiz assignment gắn với Google Forms [5].
Đối với LMS, nhóm tham khảo cách hiển thị yêu cầu và trạng thái bài tập rõ ràng. Hệ thống của đồ án sử dụng các vai trò STUDENT, INSTRUCTOR và ADMIN riêng; không sử dụng role hay quyền tài khoản Google Classroom làm bằng chứng cho quyền trong source.

### 2.3.3 Quy trình dạy học và đánh giá

Quy trình tham khảo gồm tạo assignment, giao cho học sinh, nhận bài làm, chấm và trả bài. Tài liệu chấm/trả bài nêu rằng học sinh nhận điểm khi bài được trả; giáo viên có thể cung cấp phản hồi cùng kết quả [6]. Quiz assignment có quy trình thiết lập và tùy chọn nhập điểm từ Forms theo điều kiện trong hướng dẫn [5].
Bài học áp dụng là phân biệt bài đang làm, đã nộp và kết quả đã có. Trong EduLearn, trạng thái quiz và submission phải lấy từ dữ liệu thực tế, tránh hiển thị một điểm tạm như điểm chính thức; phần nghiệp vụ này sẽ được đối chiếu source ở Chương 5.

### 2.3.4 Giao diện minh họa

Ảnh sẽ được dùng để giải thích cách đặt yêu cầu và phản hồi kết quả, không làm bằng chứng cho một chức năng chưa thử. Đợt này chưa có ảnh hoặc tài khoản lớp học phục vụ khảo sát thao tác.

### 2.3.5 Ưu điểm

Ưu điểm của luồng được khảo sát là quy trình tạo bài và trả kết quả được tài liệu hóa rõ, có các thông tin giúp tổ chức bài tập như hướng dẫn, tệp, điểm và hạn nộp [4][6]. Với nhóm, đây là mẫu tham khảo phù hợp để xây dựng form và thông báo trạng thái dễ hiểu.
EduLearn có thể áp dụng nguyên tắc trình bày mục tiêu, điều kiện làm bài và kết quả cạnh các thao tác tương ứng. Nhận xét này là định hướng giao diện của đồ án; chưa có khảo sát người dùng để kết luận Classroom dễ dùng hơn các nền tảng còn lại.

### 2.3.6 Hạn chế và bài học áp dụng

Tài liệu quiz assignment cho thấy cần xem xét cả phần cấu hình Classroom và Forms, đồng thời kiểm tra điều kiện nhập điểm [5]. Khi đối chiếu với đồ án, nhóm cần làm rõ quiz được tạo, làm và lưu kết quả ngay trong Assignment Service thay vì giả định cùng một luồng tích hợp.
Không kết luận Classroom không có AI hoặc không thể chấm code. Những khả năng đó có thể liên quan phiên bản hoặc công cụ bổ sung, ngoài phạm vi khảo sát này. Bài học chính là giảm sự mơ hồ của trạng thái và hướng dẫn người dùng từng bước.

## 2.4 Khảo sát Canvas LMS

### 2.4.1 Tổng quan hệ thống

Canvas là hệ thống tham khảo cho tổ chức học tập bằng module. Theo tài liệu, module có thể chứa nhiều loại học liệu và hoạt động, gồm tệp, thảo luận, assignment, quiz và các nội dung khác [7].
Nhóm dùng cách tổ chức này để xem xét mối liên hệ giữa chương mục và hoạt động học. Không lấy module của Canvas làm bằng chứng rằng source EduLearn có đầy đủ cơ chế điều kiện tiên quyết hoặc lộ trình tự động.

### 2.4.2 Các vai trò và chức năng chính

Tài liệu bắt đầu của Canvas mô tả Teacher là người phụ trách tạo nội dung, giảng dạy và quản lý course; quyền có thể khác giữa các tổ chức [9]. Assignment có nhiều hình thức hoạt động được đánh giá, gồm quiz, thảo luận có điểm và bài nộp trực tuyến [8].
Đối với EduLearn, bài học là một vai trò phải được gắn với phạm vi dữ liệu cụ thể. Nhóm sẽ mô tả riêng nội dung học, quiz và bài code, thay vì xem mọi thành phần có điểm là một entity duy nhất.

### 2.4.3 Quy trình dạy học và đánh giá

Luồng tham khảo là tổ chức học liệu thành module, tạo hoạt động đánh giá, cho học viên thực hiện rồi xử lý kết quả. Tài liệu Assignments nêu mối liên hệ của assignment với các phần Grades, Calendar và Syllabus [8]. Trang sản phẩm mô tả SpeedGrader hỗ trợ chấm bài, bình luận và chú thích tài liệu [10].
EduLearn tham khảo việc liên kết hoạt động với kết quả, nhưng không tuyên bố đã triển khai công cụ tương đương SpeedGrader. Quiz của đồ án tính điểm theo câu trả lời; bài lập trình được xử lý bằng judge và test case. Cách chấm cụ thể phải được trình bày theo source.

### 2.4.4 Giao diện minh họa

Việc minh họa nhằm giải thích liên kết giữa nội dung học và đánh giá. Chưa có ảnh của môi trường Canvas trong tài liệu này, nên nhóm chưa đưa kết luận về độ thuận tiện hay số thao tác thực tế.

### 2.4.5 Ưu điểm

Ưu điểm liên quan đến phạm vi đề tài là module tập hợp nhiều loại học liệu và hoạt động [7]; công cụ đánh giá còn được trình bày như nơi tiếp nhận bài và phản hồi [10]. Điều này gợi ý rằng nội dung, thao tác học và kết quả cần liên kết rõ ràng.
EduLearn áp dụng định hướng tổ chức khóa học theo cấu trúc và đưa bài tập vào luồng học tập. Đây là mục tiêu thiết kế; mức độ liên kết trên giao diện thực tế sẽ được minh họa và kiểm chứng ở phần triển khai.

### 2.4.6 Hạn chế và bài học áp dụng

Một môi trường Canvas có thể thay đổi quyền theo tổ chức [9], nên nhóm không suy ra rằng Teacher ở mọi nơi đều có quyền giống nhau. Trong đồ án, điều kiện role và quyền sở hữu phải được kiểm tra bằng code thay vì áp dụng nguyên mô hình Canvas.
Các khả năng nâng cao của Canvas ngoài những tài liệu đã đọc chưa được đánh giá. Nhóm không tuyên bố Canvas thiếu AI, chấm lập trình hoặc tính năng tiến độ. Bài học được lựa chọn là cấu trúc module và sự gắn kết giữa hoạt động với kết quả, phù hợp phạm vi có thể triển khai.

## 2.5 Tổng hợp kết quả khảo sát

### 2.5.1 Bảng so sánh các hệ thống

Bảng 2.1 tổng hợp các đặc điểm đã được tài liệu xác nhận và cách EduLearn lựa chọn triển khai. Cột EduLearn phản ánh sự có mặt của cấu trúc và chức năng trong source, chưa đại diện cho kết quả kiểm thử vận hành. Không dùng ký hiệu có/không cho những tính năng ngoài phạm vi tài liệu khảo sát.

**Bảng 2.1 So sánh các nền tảng quản lý học tập**

| Tiêu chí | Moodle | Google Classroom | Canvas | EduLearn theo source |
|---|---|---|---|---|
| Tổ chức nội dung | Course theo phần/chủ đề [1] | Assignment có chủ đề [4] | Module chứa học liệu [7] | CourseStructure và Lesson |
| Đánh giá | Assignment, quiz [2] | Assignment, quiz với Forms [4][5] | Assignment và quiz [8] | Quiz và bài lập trình |
| Kết quả | Gradebook theo course [1] | Chấm và trả bài [6] | Grades và SpeedGrader [8][10] | Grade, QuizAttempt, Submission |
| Tiến độ | Completion có điều kiện [3] | Chưa đánh giá toàn bộ trong khảo sát | Chưa đánh giá toàn bộ trong khảo sát | LessonProgress; cần kiểm chứng luồng |
| Quyền quản lý | Quyền phụ thuộc cấu hình [1] | Giáo viên giao/chấm bài [4][6] | Teacher; quyền tùy tổ chức [9] | Role và kiểm tra sở hữu tại service |
| Code và AI | Ngoài phạm vi khảo sát | Ngoài phạm vi khảo sát | Ngoài phạm vi khảo sát | Judge0 và chatbot/AI preview; chưa chạy demo |

## 2.5.2 Nhu cầu của học viên và giảng viên

Từ các quy trình tham khảo, nhóm xác định người học cần biết nội dung cần học, điều kiện tham gia, bài tập cần làm và kết quả đã nhận. Giảng viên cần tổ chức nội dung, quản lý người tham gia, xây dựng yêu cầu đánh giá và xem kết quả theo khóa học. Đây là tập nhu cầu được suy ra phục vụ thiết kế, chưa phải kết quả phỏng vấn người dùng.
Đối với thực hành lập trình, đề tài bổ sung nhu cầu soạn code, chạy thử, nộp và xem kết quả test case trong cùng luồng. Các trạng thái xử lý cần rõ ràng để người học phân biệt lỗi biên dịch, kết quả chấm và sự cố dịch vụ; nhóm sẽ xác định chi tiết theo implementation.

## 2.5.3 Các vấn đề EduLearn LMS hướng tới

EduLearn hướng tới tập trung hóa các hoạt động được chọn: tra cứu khóa học, học bài, theo dõi tiến độ, làm quiz và thực hành lập trình. Việc kết hợp này nhằm tạo một luồng sử dụng liên tục trong phạm vi đồ án; chưa có dữ liệu đo để khẳng định tiết kiệm bao nhiêu thời gian hoặc cải thiện kết quả học tập.
Nhóm còn hướng tới ranh giới quyền rõ giữa học viên, giảng viên và Admin; lưu lịch sử bài làm; cung cấp blog và trợ lý hỏi đáp. Các giới hạn về Bootstrap, custom judge và nội dung website lưu cục bộ vẫn được giữ khi kết luận mức hoàn thiện.

## 2.5.4 Định hướng chức năng đề xuất

Định hướng lựa chọn gồm khóa học có cấu trúc, đăng ký tham gia, bài học gắn tiến độ, quiz có lịch sử, bài lập trình có test case, blog tương tác và chatbot hỗ trợ. Mỗi nhóm chức năng được chuyển thành yêu cầu có tác nhân, đầu vào, kết quả và điều kiện kiểm tra, làm cơ sở cho Chương 3.
Những khả năng ngoài source hiện tại chỉ được đưa vào hướng phát triển. Không sử dụng tính năng AI hoặc kiến trúc microservices làm bằng chứng ứng dụng tốt hơn các LMS khảo sát; giá trị của đồ án cần thể hiện bằng nghiệp vụ đúng, mức hoàn thiện, khả năng triển khai và kết quả kiểm thử.

# CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

## 3.1 Phân tích yêu cầu

### 3.1.1 Yêu cầu chức năng phía Guest

Guest là người chưa đăng nhập. Nhóm chức năng công khai gồm trang chủ, giới thiệu, danh sách và chi tiết khóa học, blog công khai, đăng ký, đăng nhập và quên mật khẩu. Các trang công khai không cấp quyền học toàn bộ nội dung hoặc nộp bài.
Yêu cầu xác thực được tách thành đăng ký tài khoản, gửi OTP và xác minh email. Source đăng ký lưu User LOCAL; việc gửi/xác minh OTP là luồng riêng. Cần phân biệt yêu cầu kích hoạt email của môn học với việc authenticate hiện chưa kiểm tra verified.

### 3.1.2 Yêu cầu chức năng phía học viên

Học viên cần tra cứu khóa học, tham gia khóa PUBLIC chưa bị khóa, xem các khóa đã đăng ký, đọc lesson theo quyền, ghi nhận tiến độ, xem lịch và thông báo. Enrollment là căn cứ liên kết người học với khóa học; lesson phải thuộc course được yêu cầu.
Đánh giá gồm quiz có lượt làm, lưu tạm, tiếp tục bài, nộp và xem kết quả theo chính sách; bài code có chạy thử, nộp, chấm bất đồng bộ và lịch sử. Học viên còn có blog và chatbot. Không đồng nhất chạy thử code với nộp chính thức hoặc PENDING với đã chấm xong.

### 3.1.3 Yêu cầu chức năng phía giảng viên

Giảng viên quản lý khóa học và nội dung trong phạm vi được cấp quyền, gồm cấu trúc chương mục, lesson, tài nguyên, học viên, lịch và thông báo. Phần đánh giá gồm ngân hàng câu hỏi, quiz, bài lập trình, test case, nhập dữ liệu và xem kết quả.
Quyền role là bước kiểm tra ban đầu; các thao tác sửa nội dung còn phải đối chiếu owner/instructorId trong service. Khóa và mở khóa khóa học thuộc Admin. Route /manage/assignments đang trỏ AboutUsPage, nên các màn hình /instructor/exercises là căn cứ mô tả chức năng bài tập.

### 3.1.4 Yêu cầu chức năng phía Admin

Admin quản lý người dùng, tài khoản quản trị, khóa học toàn hệ thống, danh mục, blog, hoạt động chấm code và dashboard. Quyền được xác định theo từng controller/service; không giả định Admin có mọi màn hình hoặc được thực hiện mọi endpoint chỉ vì là vai trò cao nhất.
Phần Web Content gồm dữ liệu API và nội dung cục bộ. Danh mục course/blog có API; Inspire Images lưu localStorage; Founders chưa lưu backend. Những phần này được phân loại riêng khi đánh giá yêu cầu đã hoàn thành.

### 3.1.5 Ma trận vai trò và quyền truy cập

Ma trận quyền phải tách truy cập giao diện, quyền API và quyền trên đối tượng dữ liệu. Bộ lọc bảo vệ Spring Security kiểm tra phiên, Bộ lọc kiểm tra vai trò Spring Security giới hạn

nhánh giao diện; backend vẫn phải kiểm tra role và quyền sở hữu. API dùng isAuthenticated không đồng nghĩa chỉ STUDENT được gọi.
Các mức trong Bảng 3.1 được hiểu như sau: Công khai là không cần đăng nhập theo luồng public; Theo phiên là dựa trên người dùng hiện tại; Theo phạm vi là có thêm kiểm tra đối tượng/quyền; Không cấp là không có quyền theo API được đối chiếu. Các giới hạn đã phát hiện được mô tả dưới bảng và trong Use Case.

**Bảng 3.1 Ma trận vai trò và quyền truy cập**

| Nhóm chức năng | Guest | STUDENT | INSTRUCTOR | ADMIN |
|---|---|---|---|---|
| Nội dung công khai | Công khai | Công khai | Công khai | Công khai |
| Hồ sơ và mật khẩu | Không cấp | Theo phiên | Theo phiên | Theo phiên |
| Tham gia khóa học API | Không cấp | Theo phiên | Theo phiên | Theo phiên |
| Đọc lesson | Không cấp | Enrollment | Chủ course hoặc enrollment | Theo quyền Admin |
| Sửa nội dung course | Không cấp | Không cấp | Theo sở hữu | Theo API |
| Khóa/mở course | Không cấp | Không cấp | Không cấp | Được cấp |
| Ngân hàng câu hỏi/test | Không cấp | Không cấp | Role và phạm vi | Role và phạm vi |
| Quiz/code người học | Không cấp | Theo đối tượng | Theo API/service | Theo API/service |
| Blog cá nhân và vote | Chỉ đọc public | Theo tác giả/quy tắc | Theo tác giả/quy tắc | Quyền quản trị theo API |
| AI trực tiếp sinh quiz/test | Không cấp | Không cấp | Được cấp | Không có trong ChatController |
| Tài khoản/category/tag | Chỉ đọc public nếu có | Không CRUD quản trị | Không CRUD quản trị | Được cấp |

### 3.1.6 Yêu cầu phi chức năng

Yêu cầu phi chức năng gồm bảo vệ mật khẩu bằng BCrypt, quản lý phiên bằng JWT/refresh token, kiểm tra dữ liệu đầu vào, kiểm soát truy cập theo role và đối tượng, phản hồi lỗi nhất quán, hiển thị trạng thái tải/rỗng/lỗi và sử dụng được ở các kích thước màn hình cần kiểm thử.
Về vận hành, hệ thống cần cấu hình database, storage, SMTP, OAuth, LLM và Judge0; health check hỗ trợ quan sát trạng thái. Chưa có số liệu để đặt hoặc xác nhận SLA, số người dùng đồng thời, độ trễ tối đa hay tỷ lệ sẵn sàng. Các mục tiêu định lượng chỉ bổ sung khi nhóm thống nhất cách đo và có kết quả.

### 3.1.7 Quy tắc nghiệp vụ và validation

Quy tắc đã đối chiếu: username/email không trùng khi đăng ký; mật khẩu đăng ký tối thiểu tám ký tự và không chứa khoảng trắng theo DTO; OTP sáu chữ số, phân biệt VERIFY_EMAIL/RESET_PASSWORD, có thời hạn và số lần thử theo cấu hình. Hai mật khẩu reset phải khớp.

Enrollment mới chỉ tạo cho course PUBLIC, không LOCKED và chưa có đăng ký trùng. Lesson cần thuộc course; completeLesson kiểm tra enrollment. Lịch mới phải có endTime sau startTime và không bắt đầu trong quá khứ. Quiz phải published, chưa deleted, trong cửa sổ mở và chưa vượt số lượt khi tạo lượt mới; submit kiểm tra owner, câu hỏi và đáp án. Code không trống, tối đa 65536 ký tự và ngôn ngữ được cả engine lẫn problem cho phép. Blog không cho tự vote nội dung của mình; reply không lồng tiếp vào reply.
Các điều kiện trên là mô tả source, chưa chứng minh mọi trường hợp biên đã được kiểm thử. Không ghi quy tắc bình luận sản phẩm tối thiểu 50 ký tự vào blog LMS nếu DTO không có yêu cầu đó.

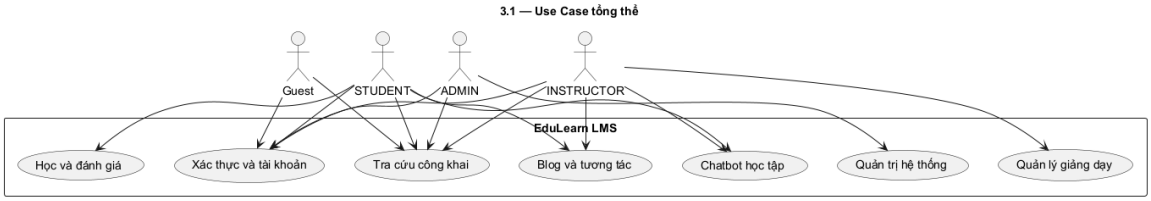
### 3.1.8 Ranh giới dữ liệu và quyền sở hữu

Identity giữ tài khoản và phiên; Course giữ khóa học, enrollment, lesson và tiến độ; Assignment giữ quiz, câu hỏi, điểm và submission; Blog giữ bài viết/bình luận/vote/tag; Chatbot giữ phiên và tin nhắn. ID liên service được tra cứu qua client hoặc lưu tham chiếu, không mặc định là khóa ngoại xuyên database.
Chủ khóa học gắn instructorId; tác giả post/comment và người làm attempt/submission gắn userId. Truy cập đối tượng phải kiểm tra thêm quan hệ course–lesson, quiz–attempt, question–answer và post–comment. Với lịch sử chat, API archive kiểm tra chủ phiên nhưng tuyến getMessages được đọc chưa thể hiện cùng kiểm tra; yêu cầu cô lập dữ liệu cần bổ sung kiểm thử và hoàn thiện nếu thiếu.

## 3.2 Biểu đồ Use Case

### 3.2.1 Use Case tổng thể

Nội dung hình tổng thể: Guest liên hệ tra cứu và xác thực; STUDENT liên hệ enrollment, lesson/progress, quiz/code, blog và chat; INSTRUCTOR liên hệ quản lý course/nội dung/đánh giá và AI preview; ADMIN liên hệ user, khóa course, category/tag, quản trị blog và giám sát. Không có tác nhân MANAGER riêng trong source. Ngoại vi gồm SMTP, Google OAuth/Calendar, Judge0 và LLM. Không dùng một mũi tên kế thừa Admin từ Instructor để suy toàn bộ quyền.

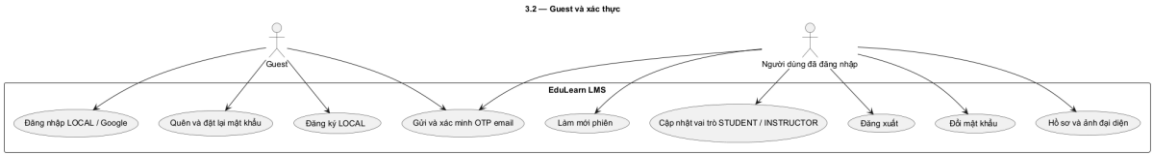



*Hình 3.1: Use Case tổng thể*

Biểu đồ tổng thể sẽ đặt EduLearn LMS trong ranh giới hệ thống, với Guest, STUDENT, INSTRUCTOR và ADMIN ở ngoài. Google OAuth, SMTP, Judge0, Google Calendar và LLM được thể hiện như hệ thống hỗ trợ bên ngoài ở những Use Case liên quan. Gateway, service và database là chi tiết thiết kế, không phải người dùng tác nghiệp.

### 3.2.2 Use Case Guest và xác thực

Nội dung hình xác thực: UC001–UC009 và UC055. SMTP hỗ trợ UC002/UC006; Google hỗ trợ UC004; UC055 là lựa chọn sau OAuth có điều kiện. Đăng nhập là tiền điều kiện của các chức năng bảo vệ, không được include vào mọi thao tác như thể mỗi lần đều nhập lại mật khẩu.

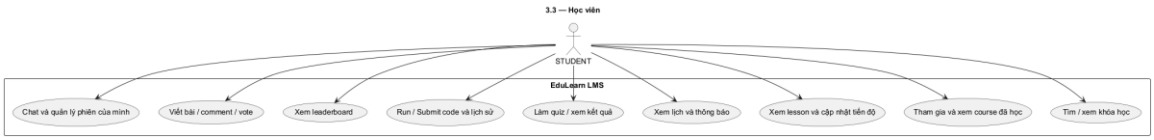



*Hình 3.2: Guest và xác thực*

Nhóm Guest gồm xem nội dung công khai và các luồng xác thực. Đăng ký và xác minh email được tách vì source có API riêng; đăng nhập LOCAL và Google OAuth là hai cách thực hiện khác nhau. Cập nhật vai trò là Use Case bổ sung UC055; rule phải lấy từ UserServiceImpl.updateRole đang được UserController gọi, chưa có giới hạn 30 phút tại tuyến này.
Đăng nhập thường được ghi là tiền điều kiện của Use Case bảo vệ, không thêm include Đăng nhập vào mọi chức năng nếu người dùng không phải đăng nhập lại ở mỗi thao tác.

### 3.2.3 Use Case học viên

Nội dung hình học viên: UC010–UC023, blog cá nhân UC038–UC042 và chat UC044–UC045 theo quyền. Nhánh PUBLIC tra cứu được chia sẻ với Guest. Các API enrollment dùng isAuthenticated cần chú thích rộng hơn nhánh UI STUDENT.

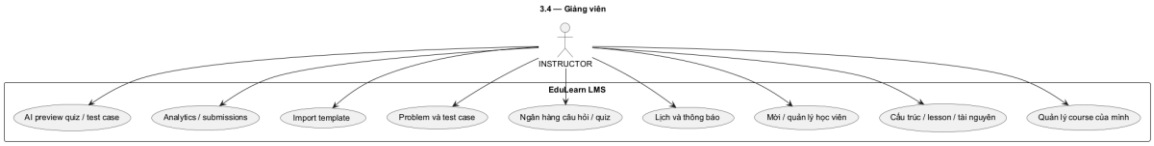



*Hình 3.3: Học viên*

Biểu đồ học viên gồm đăng ký khóa học, học lesson, cập nhật tiến độ, xem lịch/thông báo, làm quiz, xem kết quả, chạy/nộp code và xem lịch sử. Blog và chatbot là nhóm tương tác bổ sung. Những API cho isAuthenticated được ghi chú để không nhầm giới hạn giao diện STUDENT với quyền backend.
Trạng thái xử lý như quiz đang làm hoặc submission PENDING được mô tả trong luồng và sơ đồ hoạt động; không biến mỗi trạng thái thành một tác nhân mới.

### 3.2.4 Use Case giảng viên

Nội dung hình giảng viên: quản lý course/content UC024–UC027, lịch/thông báo UC029–UC030, câu hỏi/quiz/problem/test/results UC031–UC037 và AI UC046–UC047. UC028 khóa/mở course thuộc Admin nên không nối với Instructor. Blog dùng cùng nghiệp vụ tác giả.

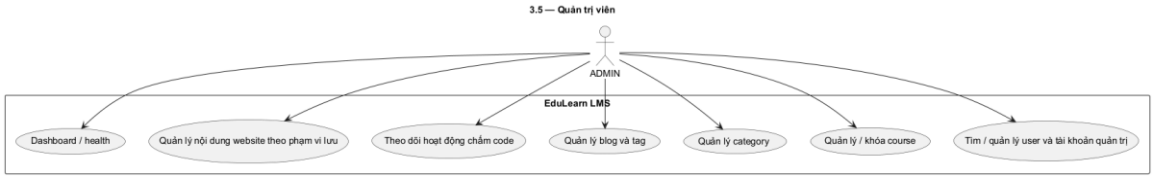



*Hình 3.4: Giảng viên*

Biểu đồ giảng viên gồm quản lý khóa học, cấu trúc, lesson/tài nguyên, học viên, lịch/thông báo, câu hỏi, quiz, problem/test case và xem kết quả. AI preview hỗ trợ biên soạn; thao tác lưu hoặc công bố vẫn là luồng riêng.
Khóa/mở khóa được đặt ở biểu đồ Admin. Quyền Admin ở một số API quản lý nội dung được thể hiện theo ma trận, không dùng quan hệ kế thừa tác nhân nếu chưa chứng minh Admin kế thừa toàn bộ quyền Instructor.

### 3.2.5 Use Case Admin

Nội dung hình Admin: UC028 và UC048–UC054; quyền quản lý nội dung tại một số API được ghi theo ma trận. UC053 phải chú thích Inspire Images là localStorage và Founders chưa lưu backend. Tag CRUD chỉ Admin; luồng AI ChatController không tự cấp Admin quyền INSTRUCTOR.




*Hình 3.5: Quản trị viên*

Biểu đồ Admin gồm quản lý tài khoản, khóa học, danh mục, blog/tag, hoạt động chấm code, dashboard/health và nội dung website. Các mục lưu cục bộ hoặc chưa lưu backend phải có ghi chú phạm vi, không thể hiện như đã có chức năng quản lý dữ liệu tập trung.

### 3.2.6 Quan hệ include và extend

Quan hệ include chỉ dùng khi một hành vi dùng chung luôn được thực hiện trong Use Case cơ sở, chẳng hạn nộp code luôn thực hiện kiểm tra hợp lệ trước lưu/chấm. Quan hệ extend dùng cho hành vi tùy điều kiện và cần nêu điểm mở rộng. Không dùng include/extend thay cho thứ tự màn hình hoặc mũi tên gọi API.
Với AI sinh quiz/test case, có thể mô tả là hành vi hỗ trợ tùy chọn trong quy trình biên soạn, nhưng phải ghi rõ bản xem trước chưa tự lưu/công bố. Gửi OTP và xác minh OTP có tương tác riêng; quan hệ cuối cùng sẽ được lựa chọn và giải thích khi dựng sơ đồ, không mặc định cả hai là một thao tác nguyên tử.

## 3.3 Đặc tả các Use Case

### 3.3.1 Tài khoản và xác thực

• UC001 Đăng ký tài khoản

**Bảng 3.2 Đặc tả UC001 Đăng ký tài khoản**

| Mã và mục tiêu | UC001 Đăng ký tài khoản |
|---|---|
| Tác nhân | Guest |
| Tiền điều kiện | Chưa có username/email trùng; dữ liệu thỏa UserCreationRequest. |
| Luồng chính | 1. Mở đăng ký và nhập thông tin. 2. Gửi request register. 3. Backend kiểm tra trùng username/email và validation. 4. Băm mật khẩu, đặt provider LOCAL và lưu User. 5. Trả thông tin tài khoản. |
| Hậu điều kiện | Tài khoản LOCAL được lưu; đăng ký chưa đồng nghĩa email đã xác minh. |
| Luồng thay thế | Người dùng chuyển sang luồng gửi/xác minh OTP riêng. |
| Ngoại lệ và giới hạn | USER_EXISTED, EMAIL_EXISTED hoặc lỗi DTO. Chưa khẳng định login bị chặn khi verified=false: authenticate hiện kiểm tra active và mật khẩu. |
| Căn cứ source | identity: AuthenticationController, AuthenticationServiceImpl, UserCreationRequest |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC002 Gửi và xác minh OTP email

**Bảng 3.3 Đặc tả UC002 Gửi và xác minh OTP email**

| Mã và mục tiêu | UC002 Gửi và xác minh OTP email |
|---|---|
| Tác nhân | Guest có tài khoản cần xác minh |
| Tiền điều kiện | Email thuộc tài khoản chưa verified; OTP còn hạn, chưa dùng và chưa vượt giới hạn thử. |
| Luồng chính | 1. Nhập email và yêu cầu gửi mã. 2. Backend kiểm tra tài khoản và OTP đang hợp lệ. 3. Tạo mã sáu chữ số, lưu loại VERIFY_EMAIL và gửi SMTP. 4. Nhập OTP. 5. Kiểm tra mã; đánh dấu usedAt và verified=true. |
| Hậu điều kiện | Email được xác minh khi OTP đúng; ghi nhận mã đã sử dụng. |
| Luồng thay thế | Gửi lại khi không còn mã hợp lệ theo điều kiện source; OTP sai tăng số lần thử. |

| Mã và mục tiêu | UC002 Gửi và xác minh OTP email |
|---|---|
| Ngoại lệ và giới hạn | EMAIL_ALREADY_VERIFIED, VERIFICATION_EMAIL_ALREADY_SENT, INVALID_OTP, TOO_MANY_ATTEMPTS; lỗi SMTP cần kiểm thử cấu hình. |
| Căn cứ source | identity: EmailVerificationController, EmailVerificationServiceImpl, OtpAttemptHandler |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC003 Đăng nhập bằng tài khoản

**Bảng 3.4 Đặc tả UC003 Đăng nhập bằng tài khoản**

| Mã và mục tiêu | UC003 Đăng nhập bằng tài khoản |
|---|---|
| Tác nhân | Người có tài khoản LOCAL |
| Tiền điều kiện | Tài khoản tồn tại và active; biết mật khẩu. |
| Luồng chính | 1. Nhập username hoặc email và mật khẩu. 2. Backend tìm User. 3. Kiểm tra active và BCrypt.matches. 4. Cập nhật lastLogin, sinh access/refresh token. 5. Lưu hash refresh token và trả kết quả. |
| Hậu điều kiện | Nhận token và thông tin người dùng để điều hướng theo role. |
| Luồng thay thế | Có thể dùng email thay cho username. |
| Ngoại lệ và giới hạn | INVALID_CREDENTIALS hoặc USER_INACTIVE. Không tự thêm điều kiện email đã verified nếu source chưa kiểm tra. |
| Căn cứ source | identity: AuthenticationServiceImpl.authenticate |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC004 Đăng nhập bằng Google OAuth

• UC004 Đăng nhập bằng Google OAuth

Mã bổ sung UC055 — cập nhật vai trò theo endpoint hiện tại. Tác nhân: người dùng đã đăng nhập; request role thuộc STUDENT/INSTRUCTOR. Luồng chính: PATCH /users/myinfo/role → UserController.updateRole → UserServiceImpl.updateRole → tìm User hiện tại → kiểm tra role khác null → đặt role và roleSelected=true → lưu User và trả UserFullResponse. Endpoint này chưa kiểm tra provider LOCAL, cửa sổ 30 phút hoặc đã chọn trước; không cấp JWT mới trong method. SelectRoleServiceImpl chứa các rule chặt

hơn nhưng chưa tìm thấy controller gọi tới. Cần kiểm thử scope JWT sau đổi role và phạm vi endpoint self role. Ngoại lệ xác định: USER_NOT_EXISTED, INVALID_ROLE_SELECTION và lỗi DTO/quyền theo tuyến API.

**Bảng 3.5 Đặc tả UC004 Đăng nhập bằng Google OAuth**

| **Mã và mục tiêu** | **UC004 Đăng nhập bằng Google OAuth** |
|---|---|
| Tác nhân | Guest hoặc người dùng đăng nhập xã hội |
| Tiền điều kiện | Có cấu hình Google OAuth hợp lệ và thông tin xác thực được provider chấp nhận. |
| Luồng chính | 1. Chọn Google. 2. Thực hiện xác thực và quay về callback. 3. Backend xử lý provider, kiểm tra email và tài khoản. 4. Tạo hoặc cập nhật User theo luồng xã hội. 5. Cấp token và hiển thị giao diện phù hợp. |
| Hậu điều kiện | Có phiên đăng nhập xã hội; trạng thái roleSelected quyết định bước chọn vai trò. |
| Luồng thay thế | Tài khoản xã hội mới đủ điều kiện có thể chọn STUDENT/INSTRUCTOR qua UC055. |
| Ngoại lệ và giới hạn | SOCIAL_PROVIDER_NOT_SUPPORTED, SOCIAL_LOGIN_FAILED, SOCIAL_EMAIL_NOT_PROVIDED, USER_INACTIVE. |
| Căn cứ source | identity: AuthenticationServiceImpl.socialLogin, GoogleOAuthServiceImpl; frontend SocialCallbackPage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC005 Đăng xuất

**Bảng 3.6 Đặc tả UC005 Đăng xuất**

| **Mã và mục tiêu** | **UC005 Đăng xuất** |
|---|---|
| Tác nhân | Người đã đăng nhập |
| Tiền điều kiện | Có phiên và refresh token của mình. |
| Luồng chính | 1. Chọn đăng xuất. 2. Gửi logout theo API. 3. Backend xử lý token theo implementation. 4. Frontend xóa phiên và chuyển về giao diện công khai. |

| Mã và mục tiêu | UC005 Đăng xuất |
|---|---|
| Hậu điều kiện | Phiên frontend được kết thúc; tác động thu hồi token theo source cần mô tả ở Chương 5. |
| Luồng thay thế | Đã mất phiên thì trở về login thay vì giữ giao diện bảo vệ. |
| Ngoại lệ và giới hạn | Token sai/hết hạn hoặc lỗi mạng; không tuyên bố mọi access token lập tức vô hiệu ở mọi service. |
| Căn cứ source | identity: AuthenticationController.logout, AuthenticationServiceImpl.logout; AuthContext |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC006 Quên và đặt lại mật khẩu

**Bảng 3.7 Đặc tả UC006 Quên và đặt lại mật khẩu**

| Mã và mục tiêu | UC006 Quên và đặt lại mật khẩu |
|---|---|
| Tác nhân | Người quên mật khẩu |
| Tiền điều kiện | Biết email; có OTP RESET_PASSWORD hợp lệ khi đặt lại. |
| Luồng chính | 1. Nhập email. 2. Nếu có User, backend vô hiệu OTP reset cũ hợp lệ và gửi mã mới. 3. Nhập OTP, mật khẩu mới và xác nhận. 4. Kiểm tra mã và hai mật khẩu. 5. Đánh dấu OTP dùng rồi và lưu hash mới. |
| Hậu điều kiện | Mật khẩu mới được băm và lưu khi thành công. |
| Luồng thay thế | Email không tồn tại ở bước gửi không tạo OTP; luồng không mặc định tiết lộ tồn tại tài khoản. |
| Ngoại lệ và giới hạn | PASSWORD_NOT_MATCH, INVALID_VERIFICATION_TOKEN, INVALID_OTP, TOO_MANY_ATTEMPTS, USER_NOT_EXISTED ở bước reset. |
| Căn cứ source | identity: EmailVerificationServiceImpl.sendResetPassword/resetPassword |

| Mã và mục tiêu | UC006 Quên và đặt lại mật khẩu |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC007 Làm mới phiên đăng nhập

**Bảng 3.8 Đặc tả UC007 Làm mới phiên đăng nhập**

| Mã và mục tiêu | UC007 Làm mới phiên đăng nhập |
|---|---|
| Tác nhân | Người dùng có refresh token |
| Tiền điều kiện | Refresh token đúng loại, hợp lệ và chưa revoked trong database. |
| Luồng chính | 1. Frontend yêu cầu refresh khi cần. 2. Backend xác minh token và bản ghi hash. 3. Kiểm tra hết hạn/thu hồi. 4. Cấp token theo chính sách rotation implementation. 5. Frontend cập nhật phiên. |
| Hậu điều kiện | Phiên tiếp tục bằng token được trả; không dùng access token thay cho refresh token. |
| Luồng thay thế | Refresh thất bại thì chuyển người dùng đăng nhập lại. |
| Ngoại lệ và giới hạn | INVALID_TOKEN, TOKEN_REVOKED và lỗi thời hạn/token type theo implementation. |
| Căn cứ source | identity: AuthenticationServiceImpl.refreshToken; frontend api.client.js |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC008 Cập nhật hồ sơ và ảnh đại diện

**Bảng 3.9 Đặc tả UC008 Cập nhật hồ sơ và ảnh đại diện**

| Mã và mục tiêu | UC008 Cập nhật hồ sơ và ảnh đại diện |
|---|---|
| Tác nhân | STUDENT, INSTRUCTOR, ADMIN |
| Tiền điều kiện | Đăng nhập; chỉnh hồ sơ của mình; tệp thỏa validation upload. |
| Luồng chính | 1. Mở settings và tải myinfo. 2. Sửa trường hồ sơ. 3. Nếu đổi avatar, lấy URL upload và tải tệp theo luồng lưu trữ. 4. Gửi cập nhật myinfo. 5. Hiển thị dữ liệu mới. |

| Mã và mục tiêu | UC008 Cập nhật hồ sơ và ảnh đại diện |
|---|---|
| Hậu điều kiện | Thông tin User được cập nhật nếu request hợp lệ. |
| Luồng thay thế | Có thể chỉ sửa hồ sơ mà không đổi ảnh. |
| Ngoại lệ và giới hạn | Dữ liệu sai, tệp quá giới hạn, không tìm thấy người dùng hoặc lỗi MinIO. |
| Căn cứ source | identity: UserController, UserServiceImpl, StorageServiceImpl; SettingsPage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC009 Thay đổi mật khẩu

**Bảng 3.10 Đặc tả UC009 Thay đổi mật khẩu**

| Mã và mục tiêu | UC009 Thay đổi mật khẩu |
|---|---|
| Tác nhân | Người dùng có tài khoản phù hợp luồng đổi mật khẩu |
| Tiền điều kiện | Đã đăng nhập; dữ liệu đổi mật khẩu thỏa request. |
| Luồng chính | 1. Mở form đổi mật khẩu. 2. Nhập thông tin theo DTO. 3. Gửi myinfo/password. 4. Service kiểm tra điều kiện mật khẩu và lưu BCrypt hash mới. 5. Thông báo kết quả. |
| Hậu điều kiện | Mật khẩu được cập nhật khi kiểm tra thành công. |
| Luồng thay thế | Hủy thao tác trước khi gửi không thay đổi dữ liệu. |
| Ngoại lệ và giới hạn | Sai mật khẩu/không khớp xác nhận hoặc vi phạm DTO; không suy rằng toàn bộ phiên bị thu hồi. |
| Căn cứ source | identity: UserController.changePassword, UserServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

### 3.3.2 Tra cứu và học tập

• UC010 Xem trang chủ và giới thiệu

**Bảng 3.11 Đặc tả UC010 Xem trang chủ và giới thiệu**

| Mã và mục tiêu | UC010 Xem trang chủ và giới thiệu |
|---|---|
| Tác nhân | Guest và người dùng đăng nhập |
| Tiền điều kiện | Truy cập được frontend. |
| Luồng chính | 1. Mở trang chủ hoặc about-us. 2. Router chọn layout/phân nhánh theo phiên. 3. Hiển thị nội dung công khai và các liên kết. 4. Người dùng chọn khóa học hoặc đăng nhập. |
| Hậu điều kiện | Chỉ đọc nội dung; không tạo enrollment hay quyền học. |
| Luồng thay thế | Người đã đăng nhập có thể được điều hướng theo FallbackRoute. |
| Ngoại lệ và giới hạn | Lỗi tải frontend/API; nội dung founders hiện là hằng trong source. |
| Căn cứ source | frontend: AppRoutes, FallbackRoute, HomePage, AboutUsPage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC011 Tìm kiếm lọc và phân trang khóa học

**Bảng 3.12 Đặc tả UC011 Tìm kiếm lọc và phân trang khóa học**

| Mã và mục tiêu | UC011 Tìm kiếm lọc và phân trang khóa học |
|---|---|
| Tác nhân | Guest và người dùng đăng nhập |
| Tiền điều kiện | Thông số tìm kiếm/lọc/phân trang hợp lệ. |
| Luồng chính | 1. Mở danh sách khóa học. 2. Nhập từ khóa/chọn bộ lọc. 3. Frontend gọi API danh sách. 4. Backend truy vấn và trả Page<CourseResponse>. 5. Hiển thị và chuyển trang. |
| Hậu điều kiện | Có danh sách theo tiêu chí; không thay đổi dữ liệu. |
| Luồng thay thế | Không có kết quả thì hiển thị trạng thái rỗng và cho đổi tiêu chí. |
| Ngoại lệ và giới hạn | Tham số không hợp lệ, lỗi API; phạm vi khóa học hiển thị theo bộ lọc của service. |

| Mã và mục tiêu | UC011 Tìm kiếm lọc và phân trang khóa học |
|---|---|
| Căn cứ source | course: CourseController.getAllCourses, CourseServiceImpl; ListCoursePage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC012 Xem chi tiết khóa học

**Bảng 3.13 Đặc tả UC012 Xem chi tiết khóa học**

| Mã và mục tiêu | UC012 Xem chi tiết khóa học |
|---|---|
| Tác nhân | Guest và người dùng đăng nhập |
| Tiền điều kiện | Có courseId hợp lệ và dữ liệu được phép hiển thị. |
| Luồng chính | 1. Chọn một khóa học. 2. Gọi API chi tiết. 3. Backend lấy Course và dữ liệu bổ sung. 4. Hiển thị mô tả, cấu trúc giới thiệu và thông tin đăng ký. |
| Hậu điều kiện | Người dùng xem thông tin khóa học; không tự có quyền đọc lesson. |
| Luồng thay thế | Đã tham gia thì chuyển sang màn hình khóa học của mình. |
| Ngoại lệ và giới hạn | COURSE_NOT_FOUND hoặc lỗi dữ liệu/phân quyền tùy API. |
| Căn cứ source | course: CourseController.getCourseById; DetailCoursePage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC013 Đăng ký tham gia khóa học

**Bảng 3.14 Đặc tả UC013 Đăng ký tham gia khóa học**

| Mã và mục tiêu | UC013 Đăng ký tham gia khóa học |
|---|---|
| Tác nhân | Người đã đăng nhập; giao diện chính STUDENT |
| Tiền điều kiện | Course phải PUBLIC, không LOCKED; chưa có enrollment của người dùng. |

| Mã và mục tiêu | UC013 Đăng ký tham gia khóa học |
|---|---|
| Luồng chính | 1. Chọn đăng ký tham gia. 2. Gửi courseId. 3. Service lấy userId từ phiên. 4. Kiểm tra trạng thái và đăng ký trùng. 5. Lưu Enrollment ACTIVE và trả kết quả. |
| Hậu điều kiện | Có bản ghi tham gia khóa học. |
| Luồng thay thế | Nếu đã tham gia, giao diện chuyển sang khóa học của mình. |
| Ngoại lệ và giới hạn | COURSE_NOT_FOUND, COURSE_LOCKED, COURSE_NOT_PUBLIC, ALREADY_ENROLLED. API dùng isAuthenticated, không mô tả chỉ STUDENT được gọi. |
| Căn cứ source | course: CourseServiceImpl.enrollCourse |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC014 Xem khóa học đã tham gia

**Bảng 3.15 Đặc tả UC014 Xem khóa học đã tham gia**

| Mã và mục tiêu | UC014 Xem khóa học đã tham gia |
|---|---|
| Tác nhân | Người đã đăng nhập; màn hình STUDENT |
| Tiền điều kiện | Có phiên hợp lệ. |
| Luồng chính | 1. Mở My Courses. 2. Gọi /courses/enrolled. 3. Backend lấy enrollment theo userId hiện tại. 4. Ánh xạ khóa học và trả danh sách. 5. Chọn khóa học để học. |
| Hậu điều kiện | Hiển thị các khóa học gắn với người dùng hiện tại. |
| Luồng thay thế | Không có enrollment thì hiển thị danh sách rỗng. |
| Ngoại lệ và giới hạn | Lỗi phiên hoặc truy vấn; không cho client tự chọn userId để xem danh sách của người khác. |
| Căn cứ source | course: CourseServiceImpl.getEnrolledCourses; MyCoursePage |

| Mã và mục tiêu | UC014 Xem khóa học đã tham gia |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC015 Xem cấu trúc và nội dung bài học

**Bảng 3.16 Đặc tả UC015 Xem cấu trúc và nội dung bài học**

| Mã và mục tiêu | UC015 Xem cấu trúc và nội dung bài học |
|---|---|
| Tác nhân | Học viên đã tham gia; giảng viên chủ khóa học; Admin |
| Tiền điều kiện | Course/lesson tồn tại; lesson thuộc course; thỏa kiểm tra truy cập. |
| Luồng chính | 1. Chọn khóa học và bài. 2. Gọi cấu trúc và chi tiết lesson. 3. Service xác minh quan hệ lesson–course. 4. Kiểm tra Admin/chủ khóa học/enrollment. 5. Hiển thị nội dung. |
| Hậu điều kiện | Nội dung được trả cho người có quyền. |
| Luồng thay thế | Chuyển bài bằng cấu trúc course; không cần tạo lại enrollment. |
| Ngoại lệ và giới hạn | LESSON_NOT_BELONG_TO_COURSE, ACCESS_DENIED, COURSE_NOT_FOUND hoặc lesson không tồn tại. |
| Căn cứ source | course: LessonServiceImpl.getLesson/checkAccessPermission; CourseStructureController |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC016 Cập nhật và theo dõi tiến độ

**Bảng 3.17 Đặc tả UC016 Cập nhật và theo dõi tiến độ**

| Mã và mục tiêu | UC016 Cập nhật và theo dõi tiến độ |
|---|---|
| Tác nhân | Học viên đã tham gia |
| Tiền điều kiện | Lesson thuộc course; có enrollment; dữ liệu hoàn thành hợp lệ. |

| Mã và mục tiêu | UC016 Cập nhật và theo dõi tiến độ |
|---|---|
| Luồng chính | 1. Người học hoàn thành bài theo thao tác giao diện. 2. Gửi completeLesson. 3. Backend kiểm tra enrollment và quan hệ bài học. 4. Lưu/cập nhật LessonProgress. 5. Tải tiến độ khóa học. |
| Hậu điều kiện | Tiến độ gắn user/course/lesson được cập nhật. |
| Luồng thay thế | Đọc lại tiến độ để tiếp tục học; số lần gọi không tự chứng minh đã hiểu nội dung. |
| Ngoại lệ và giới hạn | NOT_ENROLLED, LESSON_NOT_BELONG_TO_COURSE và lỗi dữ liệu. |
| Căn cứ source | course: LessonProgressServiceImpl.completeLesson/getCourseProgress |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC017 Xem lịch học và thông báo

**Bảng 3.18 Đặc tả UC017 Xem lịch học và thông báo**

| Mã và mục tiêu | UC017 Xem lịch học và thông báo |
|---|---|
| Tác nhân | Người có quyền trong khóa học |
| Tiền điều kiện | Đăng nhập; course tồn tại và được phép truy cập. |
| Luồng chính | 1. Mở lịch/thông báo của course. 2. Gọi API tương ứng. 3. Service kiểm tra quyền. 4. Trả danh sách. 5. Nếu có meetingUrl, người học mở liên kết. |
| Hậu điều kiện | Đọc lịch và thông báo; không tạo lịch mới. |
| Luồng thay thế | Danh sách rỗng hoặc chưa có liên kết Meet vẫn hiển thị nội dung khác. |
| Ngoại lệ và giới hạn | ACCESS_DENIED, COURSE_NOT_FOUND hoặc lỗi API. |
| Căn cứ source | course: ScheduleServiceImpl, AnnouncementServiceImpl, ScheduleMapper |

| Mã và mục tiêu | UC017 Xem lịch học và thông báo |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

### 3.3.3 Quiz và bài lập trình

• UC018 Tra cứu bài tập và quiz

**Bảng 3.19 Đặc tả UC018 Tra cứu bài tập và quiz**

| Mã và mục tiêu | UC018 Tra cứu bài tập và quiz |
|---|---|
| Tác nhân | STUDENT; nhóm quản lý có API riêng |
| Tiền điều kiện | Có quyền truy cập khóa học/bài tập tương ứng. |
| Luồng chính | 1. Mở Exercise Hub hoặc quiz/code listing. 2. Chọn course và tiêu chí. 3. Gọi API danh sách theo role. 4. Hiển thị bài được phép truy cập. 5. Chọn bài để xem yêu cầu. |
| Hậu điều kiện | Có danh sách bài tập phù hợp luồng người dùng. |
| Luồng thay thế | Lọc bài hoặc đổi trang; không có bài thì hiển thị rỗng. |
| Ngoại lệ và giới hạn | Không có course/bài hoặc không đủ quyền; không công khai quiz nháp qua luồng student. |
| Căn cứ source | assignment: QuizController.getQuizzesForStudent, ProblemController; ExerciseHubPage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC019 Làm và nộp bài quiz

**Bảng 3.20 Đặc tả UC019 Làm và nộp bài quiz**

| Mã và mục tiêu | UC019 Làm và nộp bài quiz |
|---|---|
| Tác nhân | Người làm quiz theo phiên hiện tại |
| Tiền điều kiện | Quiz chưa deleted, đã published, trong time window và chưa vượt maxAttempts khi tạo lượt mới. |

| Mã và mục tiêu | UC019 Làm và nộp bài quiz |
|---|---|
| Luồng chính | 1. Chọn bắt đầu. 2. Backend start-or-resume, tiếp tục IN_PROGRESS nếu có. 3. Tạo attempt mới khi đủ điều kiện. 4. Người học chọn đáp án và autosave. 5. Gửi submit. 6. Kiểm tra owner/quiz/câu hỏi/đáp án, chấm và lưu trạng thái. |
| Hậu điều kiện | Lưu attempt và kết quả; trạng thái SUBMITTED hoặc EXPIRED theo xử lý thời hạn. |
| Luồng thay thế | F5 lấy lại attempt đang làm; practice có luồng kiểm tra đáp án riêng. |
| Ngoại lệ và giới hạn | QUIZ_NOT_PUBLISHED, QUIZ_NOT_AVAILABLE, MAX_ATTEMPTS_REACHED, ATTEMPT_EXPIRED; đáp án/câu hỏi không thuộc bài bị từ chối. Không tự khẳng định start đã kiểm tra enrollment nếu chưa thấy điều kiện đó. |
| Căn cứ source | assignment: QuizAttemptServiceImpl; QuizAttemptController |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC020 Xem điểm và lịch sử làm quiz

**Bảng 3.21 Đặc tả UC020 Xem điểm và lịch sử làm quiz**

| Mã và mục tiêu | UC020 Xem điểm và lịch sử làm quiz |
|---|---|
| Tác nhân | Chủ lượt làm; người quản lý theo API được cấp |
| Tiền điều kiện | Attempt thuộc quiz; quyền đọc kết quả phù hợp show_result policy. |
| Luồng chính | 1. Mở kết quả hoặc lịch sử. 2. Gửi quizId/attemptId hoặc tiêu chí lịch sử. 3. Backend kiểm tra người yêu cầu. 4. Kiểm tra trạng thái và chính sách xem kết quả. 5. Trả điểm và thông tin được phép. |
| Hậu điều kiện | Đọc kết quả không sửa bài đã nộp. |
| Luồng thay thế | Chưa được phép xem kết quả thì thông báo và chờ điều kiện cho phép. |

| Mã và mục tiêu | UC020 Xem điểm và lịch sử làm quiz |
|---|---|
| Ngoại lệ và giới hạn | ACCESS_DENIED, ATTEMPT_NOT_FOUND, SHOW_RESULT_NOT_ALLOWED. |
| Căn cứ source | assignment: QuizAttemptServiceImpl.getQuizResult/getAttemptHistory |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC021 Soạn chạy thử và nộp bài lập trình

**Bảng 3.22 Đặc tả UC021 Soạn chạy thử và nộp bài lập trình**

| Mã và mục tiêu | UC021 Soạn chạy thử và nộp bài lập trình |
|---|---|
| Tác nhân | Người đã đăng nhập; học viên cần enrollment khi không phải Admin/Instructor |
| Tiền điều kiện | Problem chưa deleted; ngôn ngữ được judge và bài cho phép; code không trống và dài không quá 65536 ký tự. |
| Luồng chính | 1. Mở bài và soạn code. 2. Chọn ngôn ngữ, chạy thử với input. 3. Backend kiểm tra code và quyền. 4. Khi nộp, thực hiện compile preflight. 5. Lưu submission PENDING và gọi chấm bất đồng bộ. 6. Giao diện theo dõi kết quả. |
| Hậu điều kiện | Chạy thử trả output; nộp chính thức tạo submission để chấm. |
| Luồng thay thế | Chạy thử không đồng nghĩa đã nộp; sửa code và nộp lại tạo lượt riêng. |
| Ngoại lệ và giới hạn | INVALID_LANGUAGE, SOURCE_CODE_EMPTY, SOURCE_CODE_TOO_LARGE, NOT_ENROLLED, JUDGE0_TIMEOUT/JUDGE0_ERROR hoặc lỗi biên dịch. |
| Căn cứ source | assignment: SubmissionServiceImpl.submit/runCode, SubmissionJudgingService |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC022 Xem kết quả chấm và lịch sử submission

**Bảng 3.23 Đặc tả UC022 Xem kết quả chấm và lịch sử submission**

| **Mã và mục tiêu** | **UC022 Xem kết quả chấm và lịch sử submission** |
|---|---|
| Tác nhân | Chủ submission; Admin/Instructor theo kiểm tra service |
| Tiền điều kiện | Submission tồn tại; người đọc có quyền. |
| Luồng chính | 1. Mở kết quả/histories. 2. Gọi API. 3. Service kiểm tra dữ liệu và quyền. 4. Trả trạng thái, điểm và thông tin chấm theo response. 5. Với PENDING, giao diện tiếp tục theo dõi. |
| Hậu điều kiện | Hiển thị kết quả được lưu; không coi PENDING là đã chấm xong. |
| Luồng thay thế | Lọc lịch sử hoặc xem latest accepted theo quyền. |
| Ngoại lệ và giới hạn | ACCESS_DENIED, NOT_ENROLLED hoặc submission không tồn tại. |
| Căn cứ source | assignment: SubmissionServiceImpl.getSubmission/getHistory/getLatestAccepted |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC023 Xem bảng xếp hạng

**Bảng 3.24 Đặc tả UC023 Xem bảng xếp hạng**

| **Mã và mục tiêu** | **UC023 Xem bảng xếp hạng** |
|---|---|
| Tác nhân | Người dùng có quyền truy cập màn hình và API leaderboard |
| Tiền điều kiện | Có bài/phạm vi leaderboard hợp lệ. |
| Luồng chính | 1. Mở leaderboard từ khu vực bài tập. 2. Gọi API theo phạm vi. 3. Backend tổng hợp dữ liệu theo implementation. 4. Hiển thị thứ hạng. |
| Hậu điều kiện | Đọc bảng xếp hạng; không sửa điểm gốc. |
| Luồng thay thế | Không có kết quả thì bảng rỗng. |
| Ngoại lệ và giới hạn | Lỗi xác thực, phạm vi không hợp lệ hoặc lỗi tổng hợp; công thức xếp hạng sẽ mô tả từ service ở Chương 5. |

| Mã và mục tiêu | UC023 Xem bảng xếp hạng |
|---|---|
| Căn cứ source | assignment: LeaderboardController/Service; LeaderboardPage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

### 3.3.4 Quản lý giảng dạy

• UC024 Tạo sửa xóa và quản lý khóa học

**Bảng 3.25 Đặc tả UC024 Tạo sửa xóa và quản lý khóa học**

| Mã và mục tiêu | UC024 Tạo sửa xóa và quản lý khóa học |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN |
| Tiền điều kiện | Dữ liệu Course hợp lệ; sửa/xóa thỏa kiểm tra chủ khóa học hoặc Admin. |
| Luồng chính | 1. Mở quản lý khóa học. 2. Chọn tạo/sửa/xóa. 3. Nhập dữ liệu hoặc xác nhận. 4. Backend kiểm tra quyền, trạng thái và dữ liệu. 5. Lưu hoặc xóa theo service. 6. Tải lại danh sách. |
| Hậu điều kiện | Course được cập nhật khi thành công. |
| Luồng thay thế | Hủy trước gửi; giảng viên xem danh sách khóa học của mình. |
| Ngoại lệ và giới hạn | COURSE_NOT_FOUND, NOT_COURSE_OWNER, COURSE_LOCKED hoặc lỗi DTO/liên kết dữ liệu. |
| Căn cứ source | course: CourseController, CourseServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC025 Quản lý chương mục và cấu trúc khóa học

**Bảng 3.26 Đặc tả UC025 Quản lý chương mục và cấu trúc khóa học**

| Mã và mục tiêu | UC025 Quản lý chương mục và cấu trúc khóa học |
|---|---|
| Tác nhân | INSTRUCTOR chủ khóa học; ADMIN |

| Mã và mục tiêu | UC025 Quản lý chương mục và cấu trúc khóa học |
|---|---|
| Tiền điều kiện | Course tồn tại; đủ quyền và không bị khóa với thao tác sửa. |
| Luồng chính | 1. Mở trung tâm khóa học. 2. Tải cấu trúc. 3. Thêm/sửa/xóa hoặc sắp xếp node. 4. API kiểm tra quyền và quan hệ. 5. Lưu CourseStructure. |
| Hậu điều kiện | Cấu trúc MongoDB được cập nhật. |
| Luồng thay thế | Giữ nguyên cấu trúc nếu không xác nhận thao tác. |
| Ngoại lệ và giới hạn | Node/course không tồn tại, không phải chủ hoặc course bị khóa; chi tiết ràng buộc theo DTO/service. |
| Căn cứ source | course: CourseStructureController/Service |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC026 Quản lý bài học và tài nguyên

**Bảng 3.27 Đặc tả UC026 Quản lý bài học và tài nguyên**

| Mã và mục tiêu | UC026 Quản lý bài học và tài nguyên |
|---|---|
| Tác nhân | INSTRUCTOR chủ khóa học; ADMIN |
| Tiền điều kiện | Lesson thuộc course và có nội dung phù hợp lessonType. |
| Luồng chính | 1. Chọn bài học. 2. Sửa nội dung hoặc tải tài nguyên. 3. Gửi cập nhật. 4. Service kiểm tra owner, trạng thái khóa, quan hệ và nội dung. 5. Lưu lesson. |
| Hậu điều kiện | Nội dung lesson được cập nhật. |
| Luồng thay thế | Chỉ đổi một phần dữ liệu theo UpdateLessonRequest. |
| Ngoại lệ và giới hạn | LESSON_NOT_BELONG_TO_COURSE, LESSON_CONTENT_NULL, NOT_COURSE_OWNER, COURSE_LOCKED; lỗi upload. |
| Căn cứ source | course: LessonServiceImpl.updateLesson/validateContent |

| Mã và mục tiêu | UC026 Quản lý bài học và tài nguyên |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC027 Mời và quản lý học viên trong khóa học

**Bảng 3.28 Đặc tả UC027 Mời và quản lý học viên trong khóa học**

| Mã và mục tiêu | UC027 Mời và quản lý học viên trong khóa học |
|---|---|
| Tác nhân | INSTRUCTOR chủ khóa học; ADMIN |
| Tiền điều kiện | Course tồn tại; người học được nhận diện qua Identity. |
| Luồng chính | 1. Xem danh sách học viên. 2. Tìm người dùng/mời theo thông tin đầu vào. 3. Service kiểm tra owner và đăng ký hiện có. 4. Tạo enrollment cho người hợp lệ hoặc loại khỏi course. 5. Hiển thị kết quả. |
| Hậu điều kiện | Danh sách enrollment thay đổi theo từng kết quả xử lý. |
| Luồng thay thế | Danh sách mời có thể chứa người đã tham gia; xử lý theo InviteResultResponse. |
| Ngoại lệ và giới hạn | NOT_COURSE_OWNER, COURSE_NOT_FOUND, không tìm thấy user hoặc enrollment; lỗi Identity. |
| Căn cứ source | course: CourseServiceImpl.inviteStudents/getCourseStudents/removeStudent |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC028 Khóa và mở khóa khóa học bởi Admin

**Bảng 3.29 Đặc tả UC028 Khóa và mở khóa khóa học bởi Admin**

| Mã và mục tiêu | UC028 Khóa và mở khóa khóa học bởi Admin |
|---|---|
| Tác nhân | ADMIN |
| Tiền điều kiện | Course tồn tại; phiên có role ADMIN. |

| Mã và mục tiêu | UC028 Khóa và mở khóa khóa học bởi Admin |
|---|---|
| Luồng chính | 1. Admin tìm khóa học. 2. Chọn khóa/mở khóa. 3. Gửi endpoint lock/unlock. 4. Backend kiểm tra và cập nhật trạng thái. 5. Tải lại thông tin. |
| Hậu điều kiện | Trạng thái quản trị Course thay đổi. |
| Luồng thay thế | Hủy trước gửi không thay đổi trạng thái. |
| Ngoại lệ và giới hạn | Không phải Admin hoặc course không tồn tại; quy tắc chuyển trạng thái theo service. |
| Căn cứ source | course: CourseController.lock/unlock, CourseServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC029 Quản lý lịch học và liên kết Google Meet

**Bảng 3.30 Đặc tả UC029 Quản lý lịch học và liên kết Google Meet**

| Mã và mục tiêu | UC029 Quản lý lịch học và liên kết Google Meet |
|---|---|
| Tác nhân | INSTRUCTOR chủ khóa học; ADMIN |
| Tiền điều kiện | Course chưa LOCKED; thời gian hợp lệ; cấu hình Calendar nếu tạo Meet. |
| Luồng chính | 1. Nhập lịch. 2. Kiểm tra endTime sau startTime; lịch mới không bắt đầu trong quá khứ. 3. Lưu schedule. 4. Hiển thị lịch và meetingUrl của course. |
| Hậu điều kiện | Schedule được lưu; Meet là liên kết cấp course theo source. |
| Luồng thay thế | Sửa/xóa lịch theo API; không mặc định tạo phòng độc lập cho từng lịch. |
| Ngoại lệ và giới hạn | SCHEDULE_TIME_INVALID, NOT_COURSE_OWNER, COURSE_LOCKED, createMeeting có thể trả URL dự phòng với eventId=null khi Calendar lỗi; URL chưa xác nhận có phòng thật. |
| Căn cứ source | course: ScheduleServiceImpl, GoogleMeetServiceImpl, ScheduleMapper |

| Mã và mục tiêu | UC029 Quản lý lịch học và liên kết Google Meet |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC030 Quản lý thông báo khóa học

**Bảng 3.31 Đặc tả UC030 Quản lý thông báo khóa học**

| Mã và mục tiêu | UC030 Quản lý thông báo khóa học |
|---|---|
| Tác nhân | INSTRUCTOR chủ khóa học; ADMIN |
| Tiền điều kiện | Course tồn tại; được phép quản lý thông báo. |
| Luồng chính | 1. Mở thông báo course. 2. Nhập và gửi nội dung hoặc chọn xóa. 3. Backend kiểm tra quyền và dữ liệu. 4. Lưu/xóa announcement. 5. Cập nhật danh sách. |
| Hậu điều kiện | Thông báo được lưu trong course. |
| Luồng thay thế | Người học chỉ đọc qua API được cấp quyền. |
| Ngoại lệ và giới hạn | Course/announcement không tồn tại, không có quyền, validation thất bại. |
| Căn cứ source | course: AnnouncementController/ServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC031 Quản lý ngân hàng câu hỏi và đáp án

**Bảng 3.32 Đặc tả UC031 Quản lý ngân hàng câu hỏi và đáp án**

| Mã và mục tiêu | UC031 Quản lý ngân hàng câu hỏi và đáp án |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN theo quyền nội dung |
| Tiền điều kiện | Có courseId; dữ liệu câu hỏi/đáp án thỏa loại SINGLE, MULTIPLE hoặc TRUE_FALSE. |
| Luồng chính | 1. Mở ngân hàng câu hỏi. 2. Tạo/sửa hoặc tìm câu hỏi. 3. Gửi request. 4. Kiểm tra quyền và quy tắc đáp án. 5. Lưu Question/Answer. |

| Mã và mục tiêu | UC031 Quản lý ngân hàng câu hỏi và đáp án |
|---|---|
| Hậu điều kiện | Ngân hàng câu hỏi được cập nhật. |
| Luồng thay thế | Tái sử dụng câu hỏi khi xây dựng quiz theo liên kết quiz_questions. |
| Ngoại lệ và giới hạn | Câu hỏi không tồn tại, dữ liệu đáp án sai hoặc không đủ quyền. |
| Căn cứ source | assignment: QuestionController, QuestionService, ValidAnswerCount |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC032 Tạo sửa và quản lý quiz

**Bảng 3.33 Đặc tả UC032 Tạo sửa và quản lý quiz**

| Mã và mục tiêu | UC032 Tạo sửa và quản lý quiz |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN theo kiểm tra service |
| Tiền điều kiện | Course và câu hỏi hợp lệ; thỏa kiểm tra quyền và cấu hình quiz. |
| Luồng chính | 1. Tạo/sửa quiz. 2. Chọn câu hỏi, điểm và thời gian. 3. Lưu nháp hoặc sync. 4. Chọn công bố/ẩn khi phù hợp. 5. Backend lưu Quiz và quan hệ câu hỏi. |
| Hậu điều kiện | Quiz và trạng thái published được lưu; khôi phục là thao tác Admin theo API. |
| Luồng thay thế | Giữ nháp để chỉnh sửa; không hiển thị như quiz published cho học viên. |
| Ngoại lệ và giới hạn | Cấu hình sai, câu hỏi không hợp lệ, không đủ quyền, quiz deleted/không tồn tại. |
| Căn cứ source | assignment: QuizController, QuizServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC033 Nhập quiz từ template

**Bảng 3.34 Đặc tả UC033 Nhập quiz từ template**

| Mã và mục tiêu | UC033 Nhập quiz từ template |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN |
| Tiền điều kiện | Tệp theo template được hệ thống chấp nhận; course hợp lệ. |
| Luồng chính | 1. Tải template. 2. Nhập dữ liệu và upload. 3. Backend đọc và kiểm tra. 4. Xem nội dung được xử lý. 5. Tạo quiz qua luồng imported. |
| Hậu điều kiện | Quiz được lưu khi bước nhập/tạo thành công. |
| Luồng thay thế | Chỉnh tệp khi dữ liệu sai; hủy trước tạo không công bố bài. |
| Ngoại lệ và giới hạn | Sai cấu trúc tệp, câu hỏi/đáp án không hợp lệ hoặc lỗi quyền. |
| Căn cứ source | assignment: QuizTemplateController, QuestionImportFileService, QuizController.createQuizFromImportedQuestions |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC034 Tạo sửa và quản lý bài lập trình

**Bảng 3.35 Đặc tả UC034 Tạo sửa và quản lý bài lập trình**

| Mã và mục tiêu | UC034 Tạo sửa và quản lý bài lập trình |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN theo quyền bài |
| Tiền điều kiện | Course và dữ liệu problem hợp lệ; người thao tác có quyền. |
| Luồng chính | 1. Tạo/sửa tiêu đề, đề bài, ngôn ngữ và giới hạn. 2. Gửi API. 3. Backend kiểm tra và lưu. 4. Có thể sao chép problem và test case, ẩn hoặc xóa; restore chỉ Admin. |
| Hậu điều kiện | Problem thay đổi theo thao tác; clone tạo bài mới. |
| Luồng thay thế | Giữ bài chưa công bố; sửa trước cho học viên sử dụng. |
| Ngoại lệ và giới hạn | Problem/course không tồn tại, không đủ quyền hoặc validation sai. |

| Mã và mục tiêu | UC034 Tạo sửa và quản lý bài lập trình |
|---|---|
| Căn cứ source | assignment: ProblemController, ProblemServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC035 Quản lý test case

**Bảng 3.36 Đặc tả UC035 Quản lý test case**

| Mã và mục tiêu | UC035 Quản lý test case |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN |
| Tiền điều kiện | Problem tồn tại; có quyền quản lý test case. |
| Luồng chính | 1. Nhập input/output và thuộc tính test. 2. Chọn tạo đơn/bulk, nhập hoặc sync. 3. API kiểm tra quyền và dữ liệu. 4. Lưu TestCase. 5. Hiển thị danh sách quản lý. |
| Hậu điều kiện | Test case được lưu để judge sử dụng. |
| Luồng thay thế | Preview AI/import trước khi lưu; sửa hoặc xóa test. |
| Ngoại lệ và giới hạn | Problem/test không tồn tại, dữ liệu sai, không đủ quyền. |
| Căn cứ source | assignment: TestCaseController, TestCaseService |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC036 Nhập bài lập trình từ template

**Bảng 3.37 Đặc tả UC036 Nhập bài lập trình từ template**

| Mã và mục tiêu | UC036 Nhập bài lập trình từ template |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN theo API |
| Tiền điều kiện | Tệp nhập đáp ứng định dạng và dữ liệu bài được phép tạo. |

| Mã và mục tiêu | UC036 Nhập bài lập trình từ template |
|---|---|
| Luồng chính | 1. Tải template test case. 2. Chọn problem đã có và file nhập. 3. Gửi POST /problems/{problemId}/testcases/import. 4. Service kiểm tra quyền/resource và điều kiện bài nộp, parse và lưu theo implementation. 5. Đối chiếu BulkTestCaseResponse với danh sách test. PreviewImportPage riêng dùng dữ liệu mẫu và LockedFeature, không coi là preview API hoàn chỉnh. |
| Hậu điều kiện | Test case được xử lý/lưu cho problem hiện có khi API thành công; chưa có bằng chứng chức năng nhập trọn đề bài lập trình qua PreviewImportPage. |
| Luồng thay thế | Hủy preview hoặc chọn lại tệp. |
| Ngoại lệ và giới hạn | Tệp sai, dữ liệu thiếu, lỗi API; chi tiết định dạng lấy từ implementation ở Chương 5. |
| Căn cứ source | assignment: TestCaseController.importTestCases; PreviewImportPage có dữ liệu mẫu và LockedFeature, chưa là luồng preview API hoàn chỉnh |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC037 Xem bảng điểm và thống kê học viên

**Bảng 3.38 Đặc tả UC037 Xem bảng điểm và thống kê học viên**

| Mã và mục tiêu | UC037 Xem bảng điểm và thống kê học viên |
|---|---|
| Tác nhân | INSTRUCTOR và ADMIN theo API thống kê |
| Tiền điều kiện | Có quyền xem dữ liệu khóa học/học viên tương ứng. |
| Luồng chính | 1. Mở ExerciseResults hoặc dashboard. 2. Chọn phạm vi. 3. Gọi gradebook/analytics. 4. Backend kiểm tra và tổng hợp. 5. Hiển thị kết quả. |
| Hậu điều kiện | Đọc dữ liệu điểm/thống kê; không mặc định có quyền sửa điểm. |

| Mã và mục tiêu | UC037 Xem bảng điểm và thống kê học viên |
|---|---|
| Luồng thay thế | Lọc dữ liệu hoặc xuất nếu có implementation; không tự thêm chức năng xuất chưa xác nhận. |
| Ngoại lệ và giới hạn | Không đủ quyền, course không tồn tại hoặc lỗi dịch vụ phụ thuộc. |
| Căn cứ source | assignment: InstructorAssignmentAnalyticsController và LeaderboardController; GradeBookController hiện chưa có endpoint; ExerciseResultsPage |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

### 3.3.5 Blog và tương tác

• UC038 Tìm kiếm và xem bài viết

**Bảng 3.39 Đặc tả UC038 Tìm kiếm và xem bài viết**

| Mã và mục tiêu | UC038 Tìm kiếm và xem bài viết |
|---|---|
| Tác nhân | Guest và người dùng đăng nhập |
| Tiền điều kiện | Bài công khai theo trạng thái và route public. |
| Luồng chính | 1. Mở Blog Hub. 2. Tìm/lọc bài. 3. Gọi public listing. 4. Chọn slug bài. 5. Backend trả nội dung được công bố. |
| Hậu điều kiện | Đọc nội dung public. |
| Luồng thay thế | Không có kết quả thì đổi tiêu chí. |
| Ngoại lệ và giới hạn | POST_NOT_FOUND hoặc bài chưa công bố; không dùng public route để đọc nháp. |
| Căn cứ source | blog: PostController.public, PostServiceImpl.getPosts/getPostBySlug |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC039 Tạo sửa xóa bài viết cá nhân

**Bảng 3.40 Đặc tả UC039 Tạo sửa xóa bài viết cá nhân**

| Mã và mục tiêu | UC039 Tạo sửa xóa bài viết cá nhân |
|---|---|
| Tác nhân | Người đã đăng nhập; tác giả; ADMIN có quyền quản trị |
| Tiền điều kiện | Bài sửa/xóa thuộc tác giả hoặc người thao tác là Admin; dữ liệu hợp lệ. |
| Luồng chính | 1. Mở form bài viết. 2. Nhập nội dung và tệp. 3. Gửi tạo/sửa hoặc xác nhận xóa. 4. Backend kiểm tra title và quyền sở hữu. 5. Lưu dữ liệu/xóa tài nguyên theo service. |
| Hậu điều kiện | Post được tạo/cập nhật hoặc xóa. |
| Luồng thay thế | Giữ nháp theo trạng thái được hỗ trợ; hủy form. |
| Ngoại lệ và giới hạn | POST_TITLE_EXISTED, NOT_POST_OWNER, bài không tồn tại, lỗi upload. |
| Căn cứ source | blog: PostServiceImpl.createPost/updatePost/deletePost/checkPostOwner |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC040 Quản lý bài viết cá nhân

**Bảng 3.41 Đặc tả UC040 Quản lý bài viết cá nhân**

| Mã và mục tiêu | UC040 Quản lý bài viết cá nhân |
|---|---|
| Tác nhân | Người đã đăng nhập |
| Tiền điều kiện | Phiên xác định tác giả hiện tại. |
| Luồng chính | 1. Mở My Posts. 2. Gọi /posts/me với bộ lọc. 3. Backend lấy authorId từ phiên. 4. Trả danh sách phân trang. 5. Chọn bài để sửa/xem. |
| Hậu điều kiện | Danh sách chỉ gắn với tác giả hiện tại theo truy vấn. |
| Luồng thay thế | Không có bài hoặc lọc trạng thái khác. |
| Ngoại lệ và giới hạn | Mất phiên hoặc lỗi API; không lấy userId tùy ý từ client. |
| Căn cứ source | blog: PostServiceImpl.getMyPosts; MyBlogPostsPage |

| Mã và mục tiêu | UC040 Quản lý bài viết cá nhân |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC041 Bình luận và trả lời bình luận

**Bảng 3.42 Đặc tả UC041 Bình luận và trả lời bình luận**

| Mã và mục tiêu | UC041 Bình luận và trả lời bình luận |
|---|---|
| Tác nhân | Người đã đăng nhập |
| Tiền điều kiện | Post published; parent comment nếu có phải thuộc post và không phải reply lồng thêm. |
| Luồng chính | 1. Nhập bình luận/reply. 2. Gửi API. 3. Backend kiểm tra post và parent. 4. Lưu comment. 5. Khi sửa/xóa, kiểm tra owner theo service. |
| Hậu điều kiện | Comment được lưu hoặc cập nhật/xóa theo quyền. |
| Luồng thay thế | Reply vào comment gốc; không cho reply của reply theo source. |
| Ngoại lệ và giới hạn | POST_NOT_PUBLISHED, COMMENT_NOT_BELONG_TO_POST, CANNOT_REPLY_TO_REPLY, NOT_COMMENT_OWNER. |
| Căn cứ source | blog: CommentServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC042 Bình chọn bài viết và bình luận

**Bảng 3.43 Đặc tả UC042 Bình chọn bài viết và bình luận**

| Mã và mục tiêu | UC042 Bình chọn bài viết và bình luận |
|---|---|
| Tác nhân | Người đã đăng nhập |
| Tiền điều kiện | Bài published; comment thuộc post; không bình chọn nội dung của mình. |
| Luồng chính | 1. Chọn giá trị vote. 2. Gửi post/comment vote. 3. Backend kiểm tra tác giả và quan hệ. 4. Xử lý bản ghi vote theo implementation. 5. Trả kết quả. |

| Mã và mục tiêu | UC042 Bình chọn bài viết và bình luận |
|---|---|
| Hậu điều kiện | Vote được xử lý theo dữ liệu người dùng và nội dung. |
| Luồng thay thế | Đổi hoặc bỏ vote theo cơ chế service; không tạo trùng tùy ý. |
| Ngoại lệ và giới hạn | CANNOT_VOTE_OWN_POST/COMMENT, POST_NOT_PUBLISHED, COMMENT_NOT_BELONG_TO_POST. |
| Căn cứ source | blog: VoteServiceImpl.votePost/voteComment |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC043 Quản lý tag và tệp đính kèm

**Bảng 3.44 Đặc tả UC043 Quản lý tag và tệp đính kèm**

| Mã và mục tiêu | UC043 Quản lý tag và tệp đính kèm |
|---|---|
| Tác nhân | ADMIN quản lý tag; người viết upload theo quyền |
| Tiền điều kiện | Tag/tệp hợp lệ; có quyền API tương ứng. |
| Luồng chính | 1. Admin tạo/sửa/xóa tag. 2. Người viết chọn tag và tệp trong luồng bài viết. 3. Backend kiểm tra dữ liệu và upload. 4. Lưu tag/liên kết hoặc tài nguyên. |
| Hậu điều kiện | Tag và post_tags được lưu; tệp theo luồng MinIO. |
| Luồng thay thế | Đọc tag công khai; upload không tự cấp quyền CRUD tag. |
| Ngoại lệ và giới hạn | Không có quyền, tag không tồn tại, tệp sai hoặc lỗi storage. |
| Căn cứ source | blog: TagController/Service, PostController, MinioService |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

### 3.3.6 Chatbot và hỗ trợ AI

• UC044 Hỏi đáp với chatbot học tập

**Bảng 3.45 Đặc tả UC044 Hỏi đáp với chatbot học tập**

| Mã và mục tiêu | UC044 Hỏi đáp với chatbot học tập |
|---|---|
| Tác nhân | Người đã đăng nhập |
| Tiền điều kiện | Cấu hình LLM hợp lệ; request phù hợp context và hạn mức. |
| Luồng chính | 1. Nhập câu hỏi. 2. Gửi chat. 3. Backend kiểm tra request/hạn mức, xây context và lịch sử. 4. Gọi provider. 5. Trả nội dung theo SSE và cập nhật lịch sử theo service. |
| Hậu điều kiện | Nhận phản hồi từng phần; lưu lịch sử theo luồng đã triển khai. |
| Luồng thay thế | Người dùng bắt đầu câu hỏi khác hoặc phiên mới; phản hồi cần được người dùng đánh giá. |
| Ngoại lệ và giới hạn | Rate limit, context/provider không khả dụng, lỗi kết nối/stream; không cam kết AI luôn đúng. |
| Căn cứ source | chatbot: ChatController, ChatServiceImpl, ContextBuilderServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC045 Xem lịch sử và lưu trữ phiên chat

**Bảng 3.46 Đặc tả UC045 Xem lịch sử và lưu trữ phiên chat**

| Mã và mục tiêu | UC045 Xem lịch sử và lưu trữ phiên chat |
|---|---|
| Tác nhân | Người đã đăng nhập |
| Tiền điều kiện | Có phiên chat và quyền theo API hiện tại. |
| Luồng chính | 1. Tải danh sách phiên theo userId. 2. Chọn sessionId và đọc tin nhắn. 3. Chọn New Chat/archive. 4. Backend archiveSessionByUser kiểm tra chủ phiên. |
| Hậu điều kiện | Phiên được lưu trữ khi chủ phiên thực hiện thành công. |
| Luồng thay thế | Lần chat sau tạo phiên mới theo luồng hệ thống. |

| Mã và mục tiêu | UC045 Xem lịch sử và lưu trữ phiên chat |
|---|---|
| Ngoại lệ và giới hạn | SESSION_NOT_OWNER khi archive. getMessages/getMessageResponses chưa thấy kiểm tra owner trong tuyến đã đọc; không tuyên bố đã cô lập lịch sử hoàn toàn, cần kiểm thử truy cập chéo. |
| Căn cứ source | chatbot: ChatController.getMessages/archiveSession, ChatMessageServiceImpl, ChatSessionServiceImpl |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC046 Sinh quiz bằng AI

**Bảng 3.47 Đặc tả UC046 Sinh quiz bằng AI**

| Mã và mục tiêu | UC046 Sinh quiz bằng AI |
|---|---|
| Tác nhân | INSTRUCTOR qua ChatController; Admin/Instructor qua luồng assignment AI preview tương ứng |
| Tiền điều kiện | Request nội dung và cấu hình LLM hợp lệ; chưa mặc định công bố quiz. |
| Luồng chính | 1. Nhập yêu cầu sinh quiz. 2. Backend kiểm tra role/hạn mức. 3. Xây prompt và gọi LLM. 4. Parse/kiểm tra kết quả. 5. Giảng viên xem và sửa. 6. Lưu bằng luồng tạo quiz riêng. |
| Hậu điều kiện | Trả nội dung gợi ý; quiz chỉ được lưu khi thực hiện luồng lưu. |
| Luồng thay thế | Sửa tay hoặc sinh lại nếu nội dung không phù hợp. |
| Ngoại lệ và giới hạn | GENERATE_QUIZ_FAILED, lỗi provider/hạn mức hoặc định dạng response. |
| Căn cứ source | chatbot: ChatController.generateQuiz, GenerateServiceImpl; assignment QuizAiPreviewService |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC047 Sinh test case bằng AI

**Bảng 3.48 Đặc tả UC047 Sinh test case bằng AI**

| Mã và mục tiêu | UC047 Sinh test case bằng AI |
|---|---|
| Tác nhân | INSTRUCTOR qua ChatController; Admin/Instructor qua test case AI preview |
| Tiền điều kiện | Problem hoặc mô tả đầu vào hợp lệ; LLM khả dụng. |
| Luồng chính | 1. Chọn bài/nhập mô tả. 2. Yêu cầu sinh test. 3. Backend xây ngữ cảnh và gọi LLM. 4. Kiểm tra response. 5. Người phụ trách rà soát input/output. 6. Lưu qua API test case. |
| Hậu điều kiện | Có test case gợi ý; chưa tự trở thành test chính thức. |
| Luồng thay thế | Sửa đầu ra hoặc dùng test nhập tay. |
| Ngoại lệ và giới hạn | PROBLEM_NOT_FOUND, TESTCASE_TITLE_BLANK, TESTCASE_DESCRIPTION_BLANK, GENERATE_TESTCASE_FAILED hoặc lỗi dịch vụ. |
| Căn cứ source | chatbot: GenerateServiceImpl.generateTestCase; assignment TestCaseController.ai preview |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

## 3.3.7 Quản trị hệ thống

• UC048 Quản lý người dùng và vai trò

**Bảng 3.49 Đặc tả UC048 Quản lý người dùng và vai trò**

| Mã và mục tiêu | UC048 Quản lý người dùng và vai trò |
|---|---|
| Tác nhân | ADMIN |
| Tiền điều kiện | Có phiên Admin; dữ liệu user và quyền quản trị hợp lệ. |
| Luồng chính | 1. Tìm/lọc người dùng. 2. Xem thông tin. 3. Cập nhật hoặc đổi active theo API. 4. Backend kiểm tra request và lưu. 5. Tải lại danh sách. |
| Hậu điều kiện | User thay đổi theo thao tác đã được API hỗ trợ. |

| Mã và mục tiêu | UC048 Quản lý người dùng và vai trò |
|---|---|
| Luồng thay thế | Quản lý tài khoản Admin có luồng tạo/sửa/đổi mật khẩu/trạng thái riêng. |
| Ngoại lệ và giới hạn | User không tồn tại, dữ liệu trùng/sai hoặc không đủ quyền; không tự thêm cấp quyền tùy ý ngoài API. |
| Căn cứ source | identity: AdminUserController, AdminUserServiceImpl; UserManagement/AdminManagement |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC049 Quản lý khóa học toàn hệ thống

**Bảng 3.50 Đặc tả UC049 Quản lý khóa học toàn hệ thống**

| Mã và mục tiêu | UC049 Quản lý khóa học toàn hệ thống |
|---|---|
| Tác nhân | ADMIN |
| Tiền điều kiện | Course tồn tại; quyền quản trị hợp lệ. |
| Luồng chính | 1. Tìm và xem khóa học. 2. Chọn thao tác quản trị được giao diện hỗ trợ. 3. Gọi API course, gồm lock/unlock. 4. Kiểm tra quyền và cập nhật. 5. Tải lại. |
| Hậu điều kiện | Dữ liệu/trạng thái course được cập nhật khi thành công. |
| Luồng thay thế | Đọc dữ liệu hoặc hủy thao tác. |
| Ngoại lệ và giới hạn | Course không tồn tại, lỗi API, dữ liệu sai. |
| Căn cứ source | course: CourseController/Service; admin CourseManagement |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC050 Quản lý danh mục khóa học

**Bảng 3.51 Đặc tả UC050 Quản lý danh mục khóa học**

| Mã và mục tiêu | UC050 Quản lý danh mục khóa học |
|---|---|
| Tác nhân | ADMIN; Guest/người dùng đọc danh mục theo API public |
| Tiền điều kiện | Dữ liệu category hợp lệ; sửa/xóa category phải tồn tại. |

| Mã và mục tiêu | UC050 Quản lý danh mục khóa học |
|---|---|
| Luồng chính | 1. Mở Course Categories. 2. Tạo/sửa/xóa. 3. Gửi API categories. 4. Backend validation và lưu. 5. Cập nhật giao diện. |
| Hậu điều kiện | Category thay đổi theo API. |
| Luồng thay thế | Người không quản trị chỉ đọc danh sách. |
| Ngoại lệ và giới hạn | Không phải Admin, category không tồn tại hoặc lỗi ràng buộc/DTO. |
| Căn cứ source | course: CategoryController/Service; CourseCategoriesTab |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC051 Quản lý nội dung blog

**Bảng 3.52 Đặc tả UC051 Quản lý nội dung blog**

| Mã và mục tiêu | UC051 Quản lý nội dung blog |
|---|---|
| Tác nhân | ADMIN |
| Tiền điều kiện | Có quyền quản trị Post/Tag và dữ liệu hợp lệ. |
| Luồng chính | 1. Mở Blog Management. 2. Lọc bài theo trạng thái/tác giả/tiêu chí được hỗ trợ. 3. Xem hoặc thực hiện thao tác. 4. Service xử lý theo quyền Admin. 5. Cập nhật danh sách. |
| Hậu điều kiện | Dữ liệu blog thay đổi theo thao tác thực tế; không tự thêm quy trình duyệt nhiều bước. |
| Luồng thay thế | Quản lý tag qua khu vực nội dung website. |
| Ngoại lệ và giới hạn | Post/tag không tồn tại, dữ liệu sai hoặc lỗi lưu tệp. |
| Căn cứ source | blog: PostController.getAllPostsForAdmin, PostServiceImpl; BlogMgmt/BlogCategoriesTab |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC052 Theo dõi hoạt động chấm bài lập trình

**Bảng 3.53 Đặc tả UC052 Theo dõi hoạt động chấm bài lập trình**

| Mã và mục tiêu | UC052 Theo dõi hoạt động chấm bài lập trình |
|---|---|
| Tác nhân | ADMIN |
| Tiền điều kiện | Có quyền gọi /admin/code-judge/activity. |
| Luồng chính | 1. Mở quản lý Code Judge. 2. Gọi API activity với tiêu chí. 3. Backend tổng hợp hoạt động. 4. Hiển thị dữ liệu trả về. |
| Hậu điều kiện | Theo dõi hoạt động; không mặc định có quyền chạy lại hay sửa điểm. |
| Luồng thay thế | Dữ liệu rỗng hoặc lọc phạm vi khác. |
| Ngoại lệ và giới hạn | Không đủ quyền, lỗi dịch vụ/database. |
| Căn cứ source | assignment: AdminCodeJudgeActivityController; CodeJudgeMgmt |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC053 Quản lý nội dung website

**Bảng 3.54 Đặc tả UC053 Quản lý nội dung website**

| Mã và mục tiêu | UC053 Quản lý nội dung website |
|---|---|
| Tác nhân | ADMIN |
| Tiền điều kiện | Mở trang quản lý nội dung website. |
| Luồng chính | 1. Chọn tab. 2. Danh mục khóa học/blog gọi API. 3. Ảnh Inspire lưu localStorage. 4. Founders sửa state; nút lưu thông báo nội dung đặt cứng. 5. Hiển thị kết quả tương ứng. |
| Hậu điều kiện | Chỉ dữ liệu API có lưu backend; ảnh Inspire lưu riêng trình duyệt; Founders chưa lưu backend. |
| Luồng thay thế | Khôi phục giá trị mặc định theo giao diện. |
| Ngoại lệ và giới hạn | localStorage đầy/lỗi; lỗi API danh mục; chưa có persistence Founders, không coi thông báo là lưu thành công. |
| Căn cứ source | frontend: WebContentMgmt và bốn tab con |

| Mã và mục tiêu | UC053 Quản lý nội dung website |
|---|---|
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

• UC054 Theo dõi dashboard và sức khỏe dịch vụ

**Bảng 3.55 Đặc tả UC054 Theo dõi dashboard và sức khỏe dịch vụ**

| Mã và mục tiêu | UC054 Theo dõi dashboard và sức khỏe dịch vụ |
|---|---|
| Tác nhân | ADMIN trên dashboard |
| Tiền điều kiện | Các API/dashboard và endpoint health truy cập được. |
| Luồng chính | 1. Mở dashboard. 2. Giao diện gọi dữ liệu tổng hợp và health. 3. Nhận trạng thái dịch vụ. 4. Hiển thị thống kê và lỗi nếu có. |
| Hậu điều kiện | Đọc số liệu và trạng thái thời điểm kiểm tra. |
| Luồng thay thế | Một service lỗi có thể hiển thị trạng thái không sẵn sàng; không suy số liệu thiếu là bằng không. |
| Ngoại lệ và giới hạn | Timeout, mất kết nối, thiếu quyền hoặc lỗi service; health không chứng minh toàn bộ nghiệp vụ đúng. |
| Căn cứ source | frontend: HomePageAdmin, dashboard.api; backend actuator và health endpoints |
| Trạng thái xác minh | Đặc tả từ source tại commit 4288b95; chưa chạy kiểm thử chức năng. Kịch bản nghiệm thu: luồng chính thành công, dữ liệu sai, thiếu quyền và ngoại lệ nêu trên. |

# 3.4 Biểu đồ hoạt động

## 3.4.1 Đăng ký và xác minh OTP

Bắt đầu → nhập đăng ký → dữ liệu hợp lệ? Không: hiển thị lỗi và sửa. Có: username/email đã tồn tại? Có: từ chối. Không: băm và lưu User → yêu cầu gửi OTP → gửi SMTP → nhập OTP → mã còn hạn/chưa dùng/chưa vượt số lần thử? Không: lỗi. Có: mã đúng? Không: tăng attempts. Có: ghi usedAt và verified → kết thúc. Đăng ký và xác minh là hai luồng, không tự coi tài khoản vừa lưu là đã xác minh.

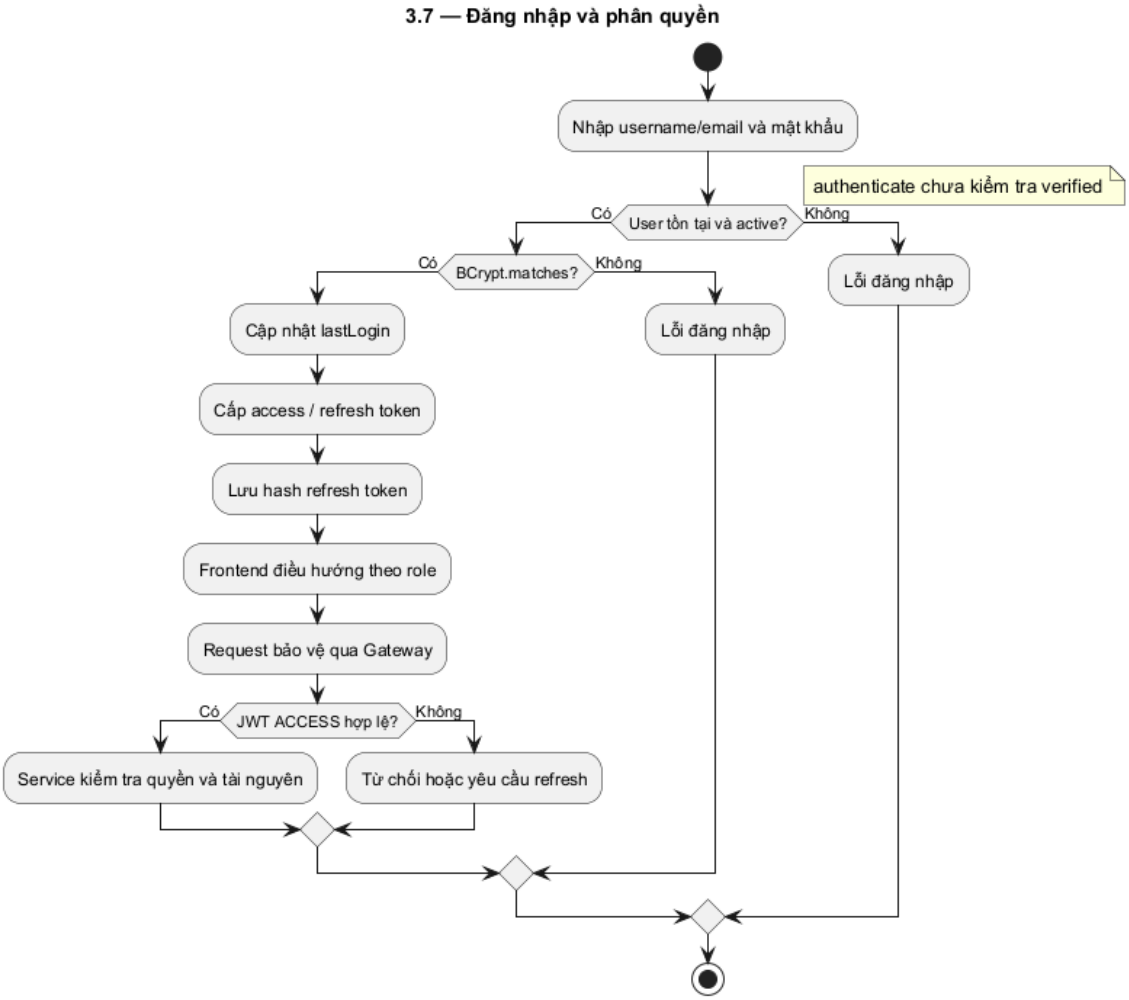
### 3.6 — Đăng ký và xác minh OTP



*Hình 3.6: Đăng ký và xác minh OTP*

### 3.4.2 Đăng nhập và phân quyền

Nhập username/email và mật khẩu → tìm User → active? → BCrypt.matches? → cập nhật lastLogin và cấp token → frontend điều hướng theo role. Mỗi nhánh sai trả lỗi phù hợp. Hiện authenticate chưa có điều kiện verified; cần thể hiện đúng source. Gateway kiểm tra token cho request bảo vệ; quyền đối tượng tiếp tục được kiểm tra tại service.

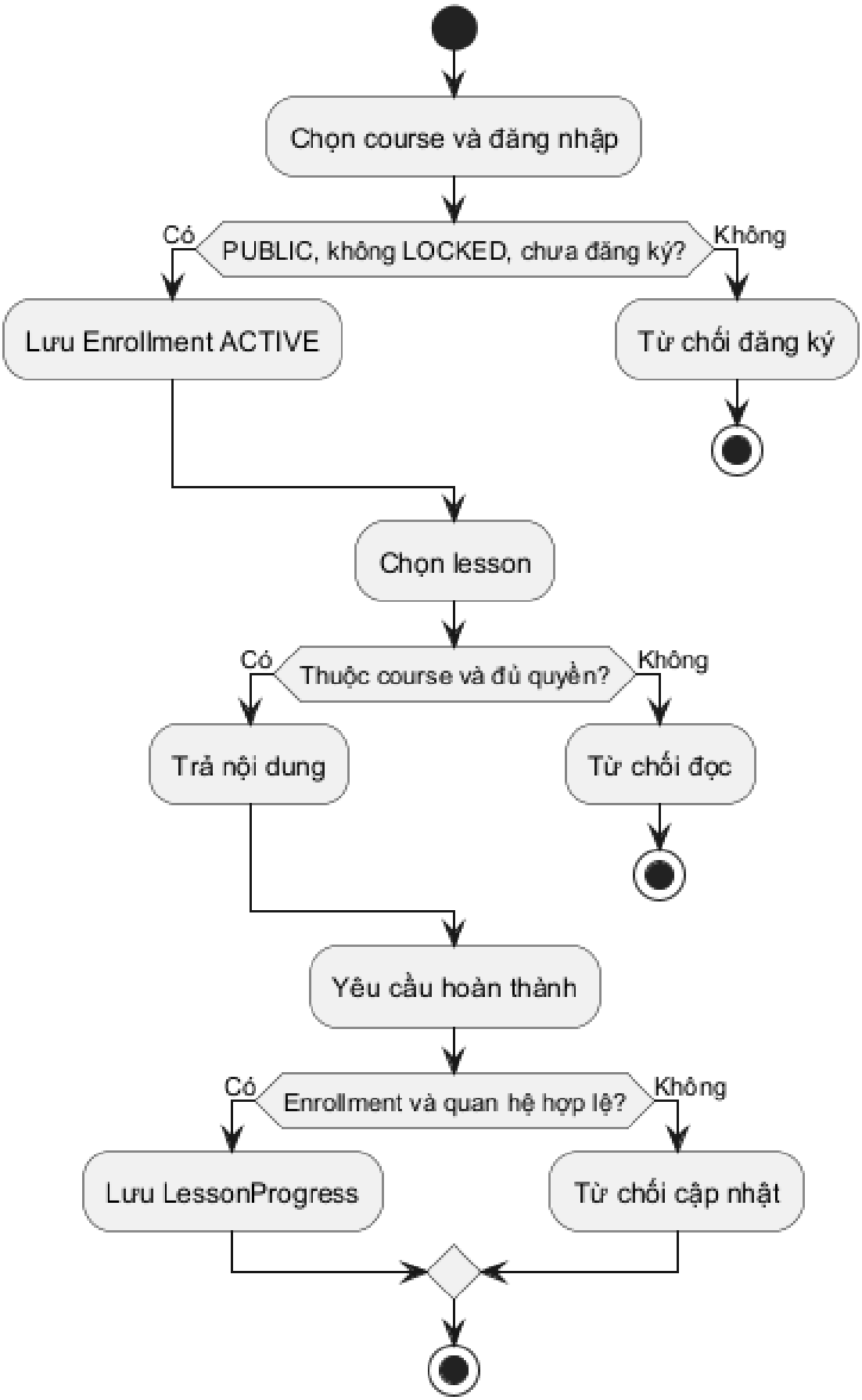



*Hình 3.7: Đăng nhập và phân quyền*

### 3.4.3 Đăng ký khóa học và học bài

Xem course → đăng nhập → course PUBLIC, không LOCKED và chưa enrollment? → lưu enrollment ACTIVE → chọn lesson → lesson thuộc course và đủ quyền? → trả nội dung → hoàn thành bài → kiểm tra enrollment/quan hệ → lưu LessonProgress. Nhánh từ chối không được chuyển thẳng tới cập nhật tiến độ.

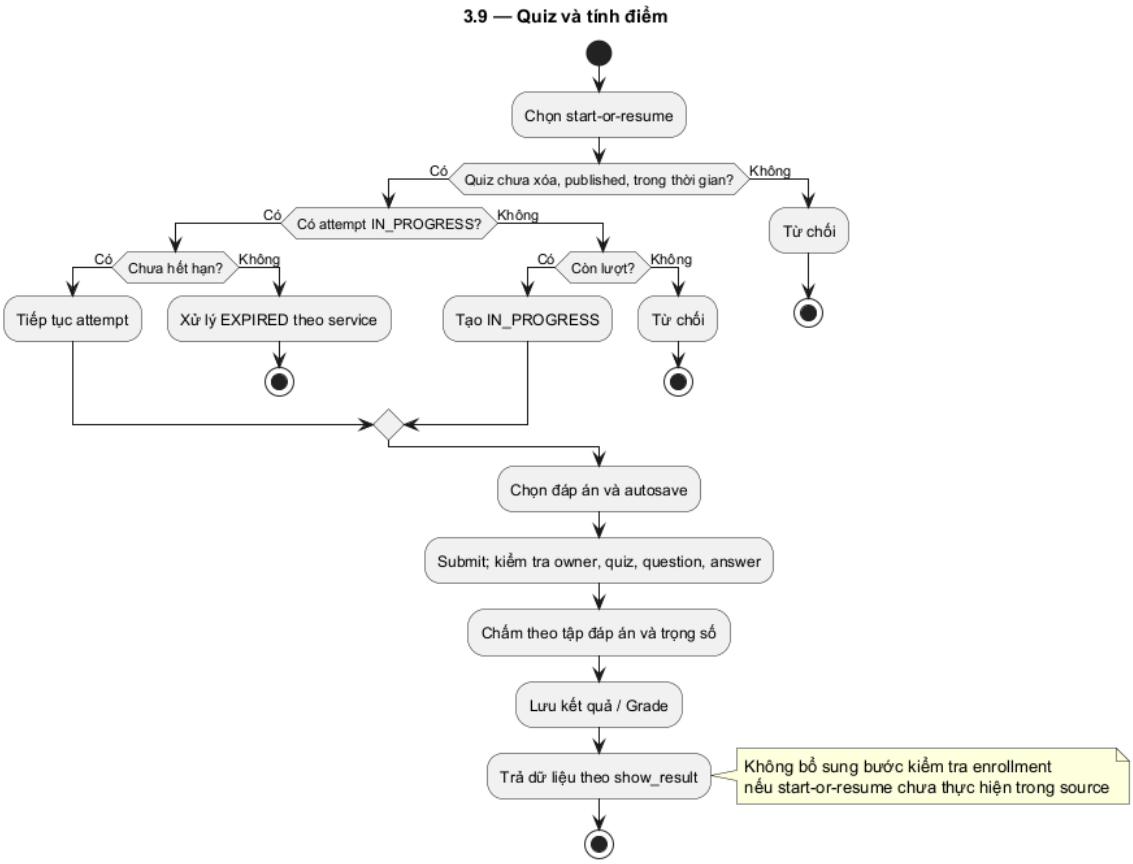
### 3.8 — Enrollment, lesson và tiến độ



*Hình 3.8: Enrollment lesson và tiến độ*

### 3.4.4 Làm quiz và tính điểm

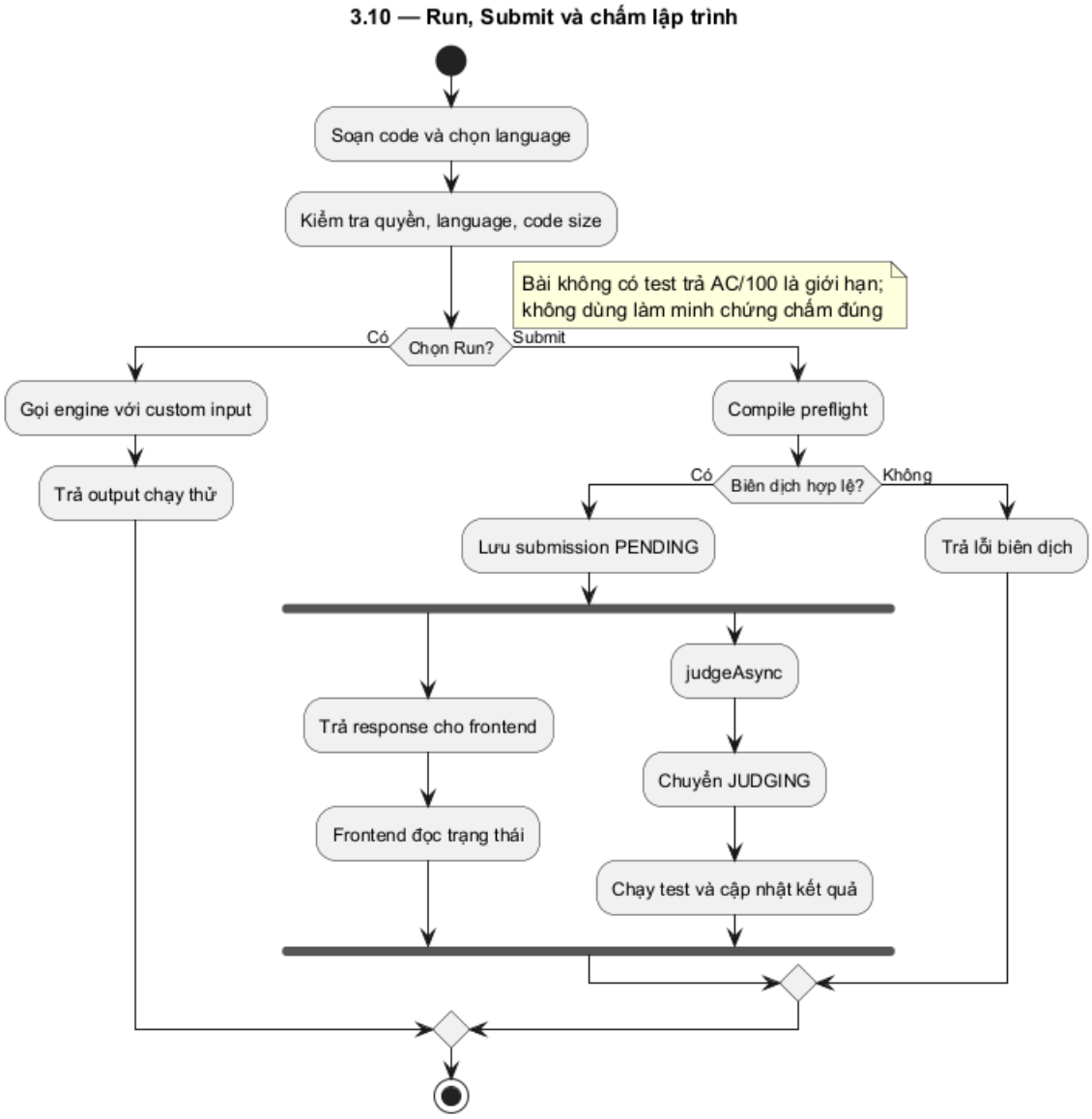
Chọn quiz → chưa deleted, published và thời gian mở? → có attempt IN_PROGRESS? Có: tiếp tục nếu chưa hết hạn. Không: kiểm tra maxAttempts và tạo lượt → làm bài/autosave → submit → kiểm tra owner và đáp án → tính điểm → lưu SUBMITTED hoặc EXPIRED theo điều kiện → xem kết quả theo show_result. Cần vẽ nhánh hết giờ riêng; không khẳng định start đã kiểm tra enrollment nếu chưa thấy trong service.




*Hình 3.9: Quiz và tính điểm*

### 3.4.5 Chạy thử nộp bài và chấm lập trình

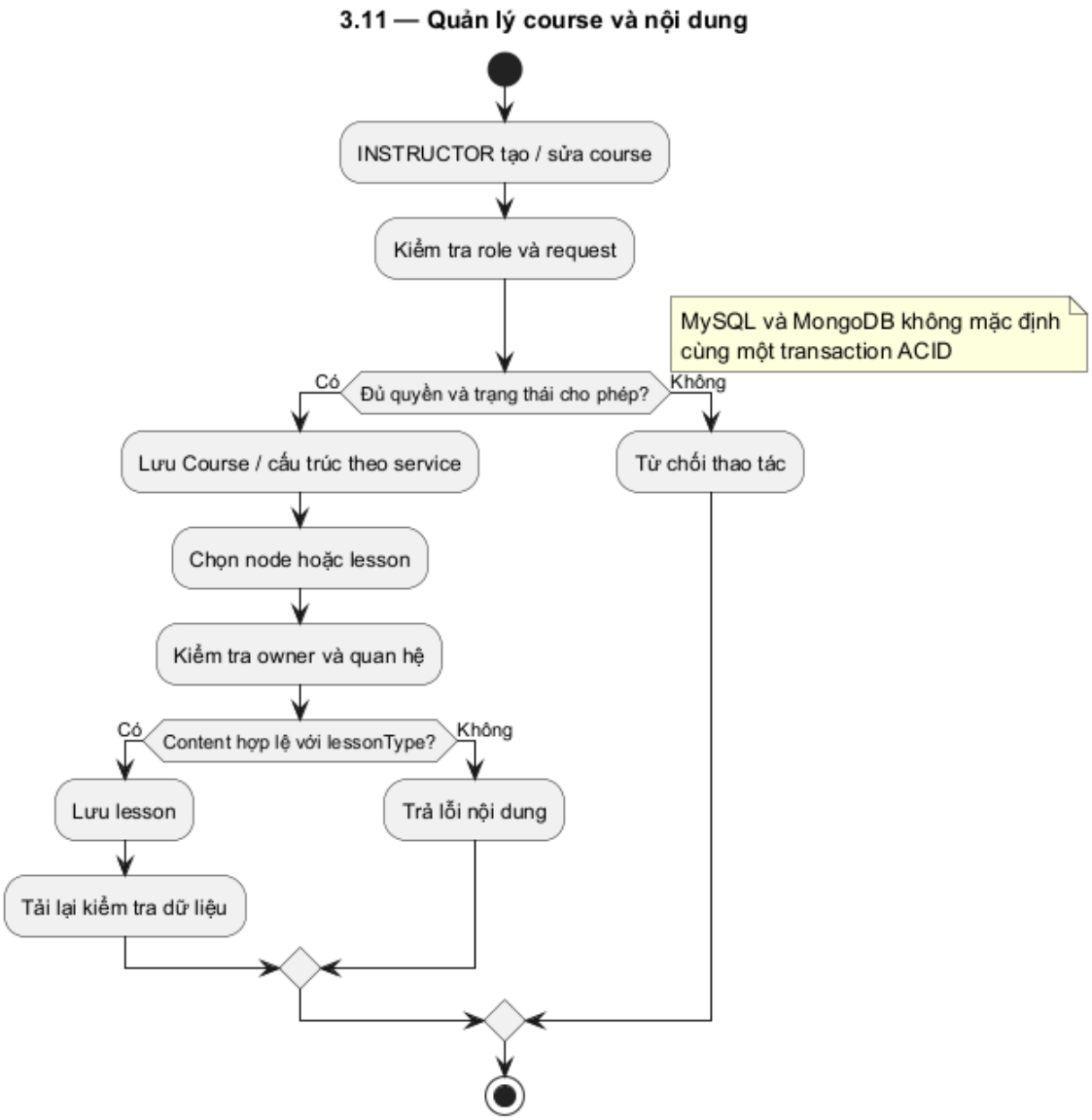
Soạn code → kiểm tra ngôn ngữ/kích thước/quyền → chọn run hay submit. Run: gọi engine và nhận output. Submit: compile preflight → lưu PENDING → gọi judgeAsync → trả response → tác vụ chấm cập nhật kết quả → frontend đọc kết quả. Nhánh compile lỗi/timeout đi về thông báo; không vẽ chạy thử thành submission chính thức.




*Hình 3.10: Run Submit và chấm lập trình*

### 3.4.6 Tạo khóa học và quản lý nội dung

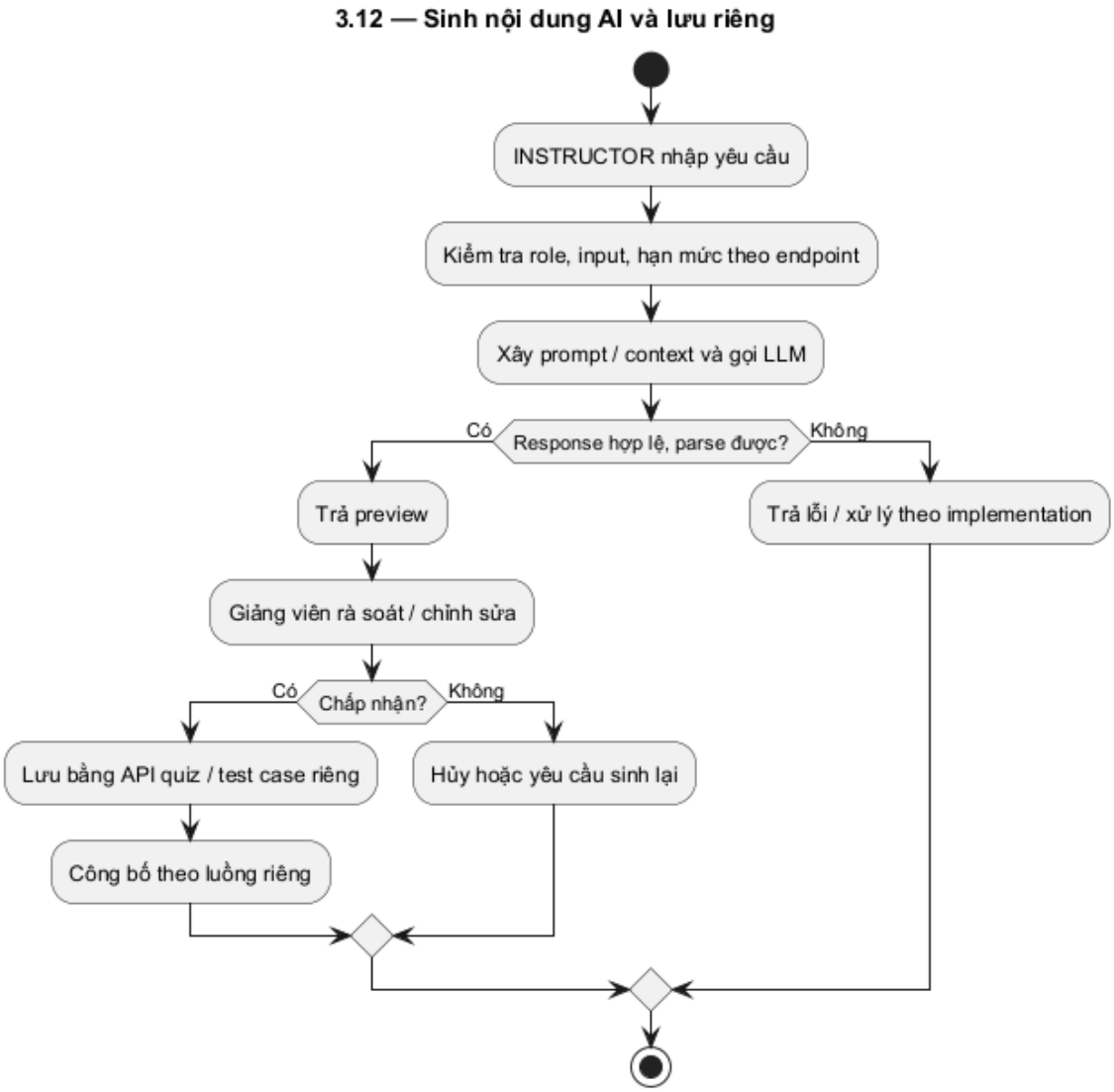
Tạo Course → kiểm tra role và request → lưu thông tin/cấu trúc theo service → chọn node hoặc lesson → kiểm tra owner và trạng thái khóa → nhập nội dung đúng lessonType → lưu → kiểm tra hiển thị. Nhánh course LOCKED hoặc sai owner trả lỗi. Không mô tả thao tác ở MySQL và MongoDB là một transaction ACID chung nếu source không có cơ chế đó.




*Hình 3.11: Quản lý course và nội dung*

### 3.4.7 Sinh nội dung bằng AI

Nhập yêu cầu sinh nội dung → kiểm tra role/hạn mức/input → xây prompt/context → gọi LLM → kết quả parse được? Không: lỗi hoặc retry theo implementation. Có: trả preview → giảng viên rà soát/sửa → chấp nhận? Không: hủy/sinh lại. Có: lưu qua API quiz/test case → công bố theo luồng riêng. Không coi nội dung LLM là đáp án chuẩn khi chưa kiểm tra.



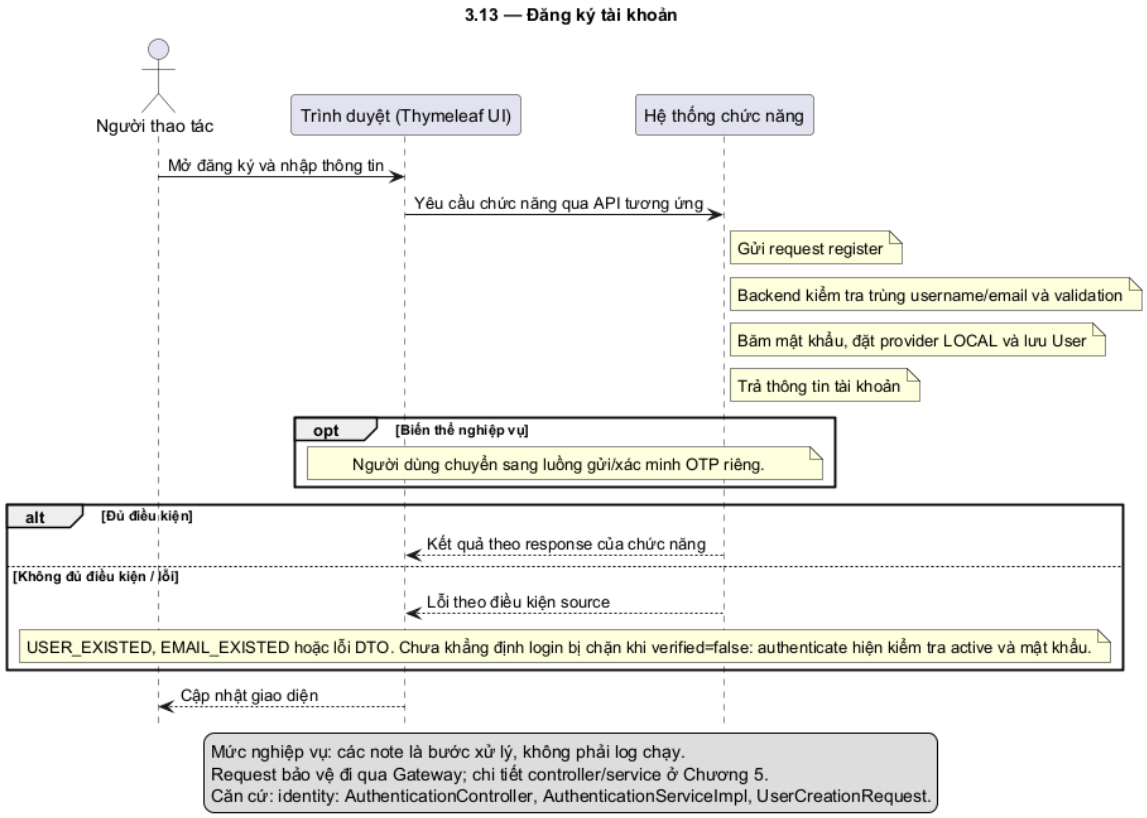

*Hình 3.12: Sinh nội dung AI và lưu riêng*

# 3.5 Biểu đồ tuần tự

## 3.5.1 Nhóm tài khoản và xác thực

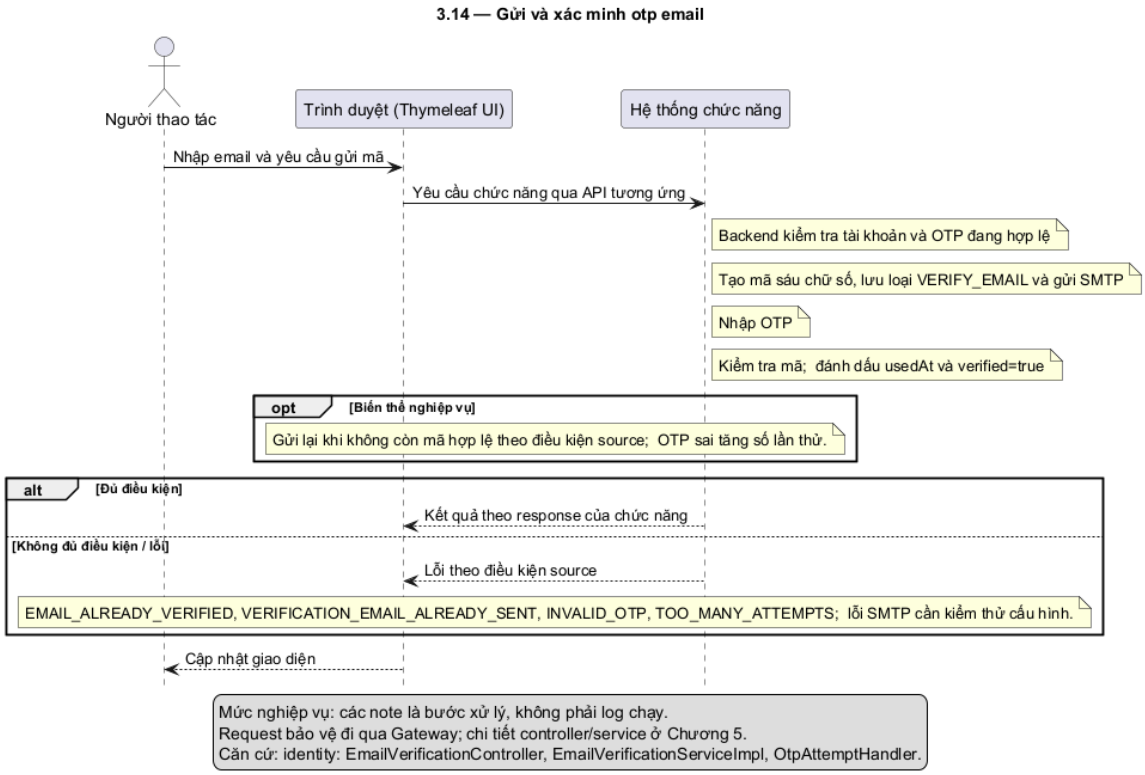
• Sequence Diagram đăng ký tài khoản
Nội dung Hình 3.13: Sequence Diagram đăng ký tài khoản.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.

Luồng nghiệp vụ: 1. Mở đăng ký và nhập thông tin. 2. Gửi request register. 3. Backend kiểm tra trùng username/email và validation. 4. Băm mật khẩu, đặt provider LOCAL và lưu User. 5. Trả thông tin tài khoản.
Nhánh thay thế: Người dùng chuyển sang luồng gửi/xác minh OTP riêng.
Nhánh lỗi: USER_EXISTED, EMAIL_EXISTED hoặc lỗi DTO. Chưa khẳng định login bị chặn khi verified=false: authenticate hiện kiểm tra active và mật khẩu.
Căn cứ: identity: AuthenticationController, AuthenticationServiceImpl, UserCreationRequest.



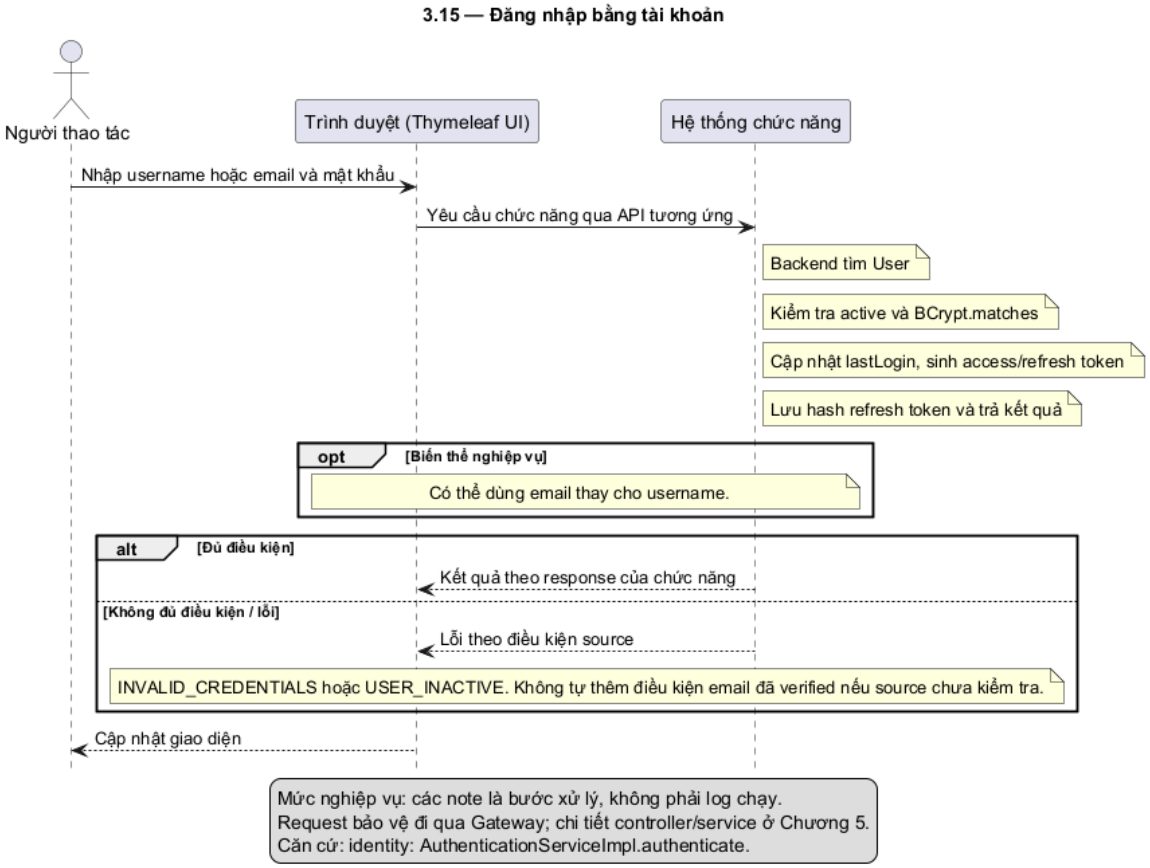

*Hình 3.13: Đăng ký tài khoản*

• Sequence Diagram gửi và xác minh otp email
Nội dung Hình 3.14: Sequence Diagram gửi và xác minh otp email.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Tách gửi OTP và xác nhận OTP thành hai request. Luồng gửi có lưu bản ghi mã và gọi EmailService/SMTP; luồng xác nhận tìm mã theo loại/thời hạn, kiểm tra số lần thử, đánh dấu usedAt và cập nhật User. Không vẽ SMTP như hệ thống trực tiếp đổi mật khẩu.
Luồng nghiệp vụ: 1. Nhập email và yêu cầu gửi mã. 2. Backend kiểm tra tài khoản và OTP đang hợp lệ. 3. Tạo mã sáu chữ số, lưu loại VERIFY_EMAIL và gửi SMTP. 4. Nhập OTP. 5. Kiểm tra mã; đánh dấu usedAt và verified=true.
Nhánh thay thế: Gửi lại khi không còn mã hợp lệ theo điều kiện source; OTP sai tăng số lần thử.
Nhánh lỗi: EMAIL_ALREADY_VERIFIED, VERIFICATION_EMAIL_ALREADY_SENT, INVALID_OTP, TOO_MANY_ATTEMPTS; lỗi SMTP cần kiểm thử cấu hình.
Căn cứ: identity: EmailVerificationController, EmailVerificationServiceImpl, OtpAttemptHandler.



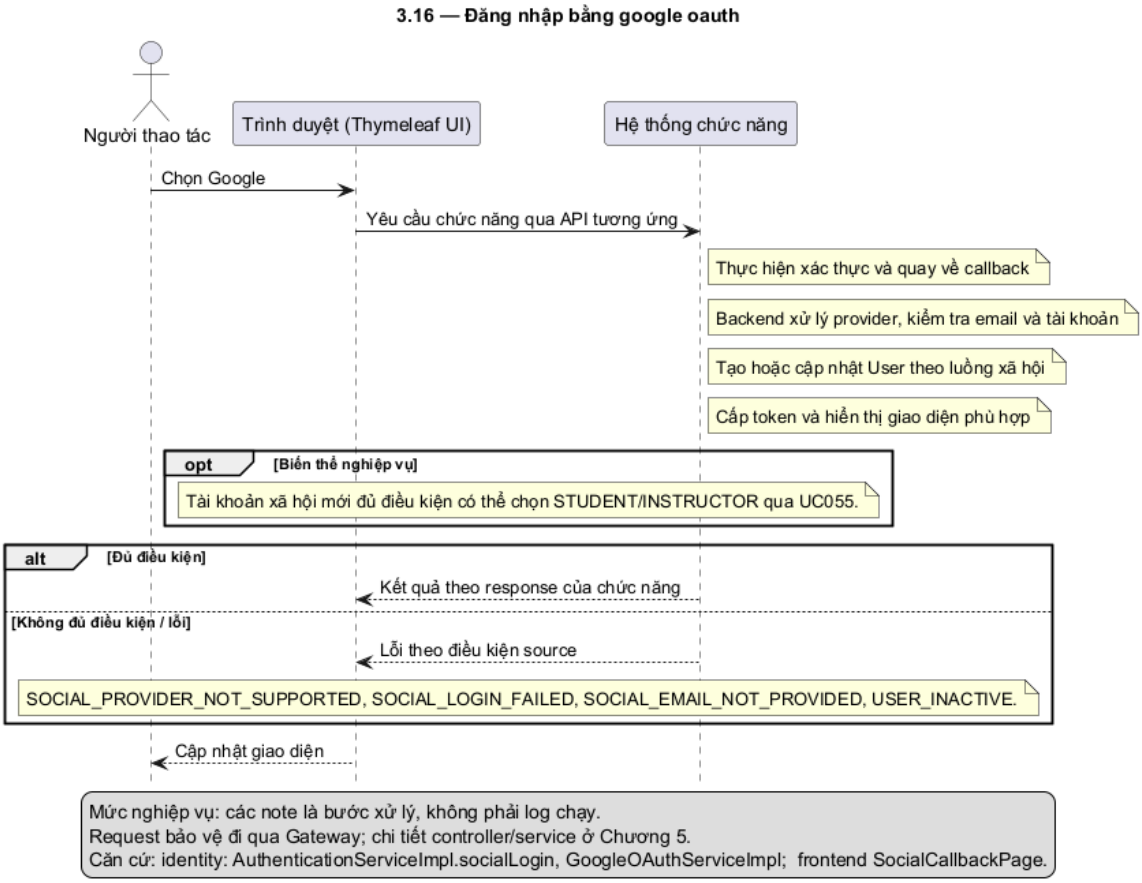

*Hình 3.14: Gửi và xác minh otp email*

• Sequence Diagram đăng nhập bằng tài khoản
Nội dung Hình 3.15: Sequence Diagram đăng nhập bằng tài khoản.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Nhập username hoặc email và mật khẩu. 2. Backend tìm User. 3. Kiểm tra active và BCrypt.matches. 4. Cập nhật lastLogin, sinh access/refresh token. 5. Lưu hash refresh token và trả kết quả.
Nhánh thay thế: Có thể dùng email thay cho username.
Nhánh lỗi: INVALID_CREDENTIALS hoặc USER_INACTIVE. Không tự thêm điều kiện email đã verified nếu source chưa kiểm tra.
Căn cứ: identity: AuthenticationServiceImpl.authenticate.



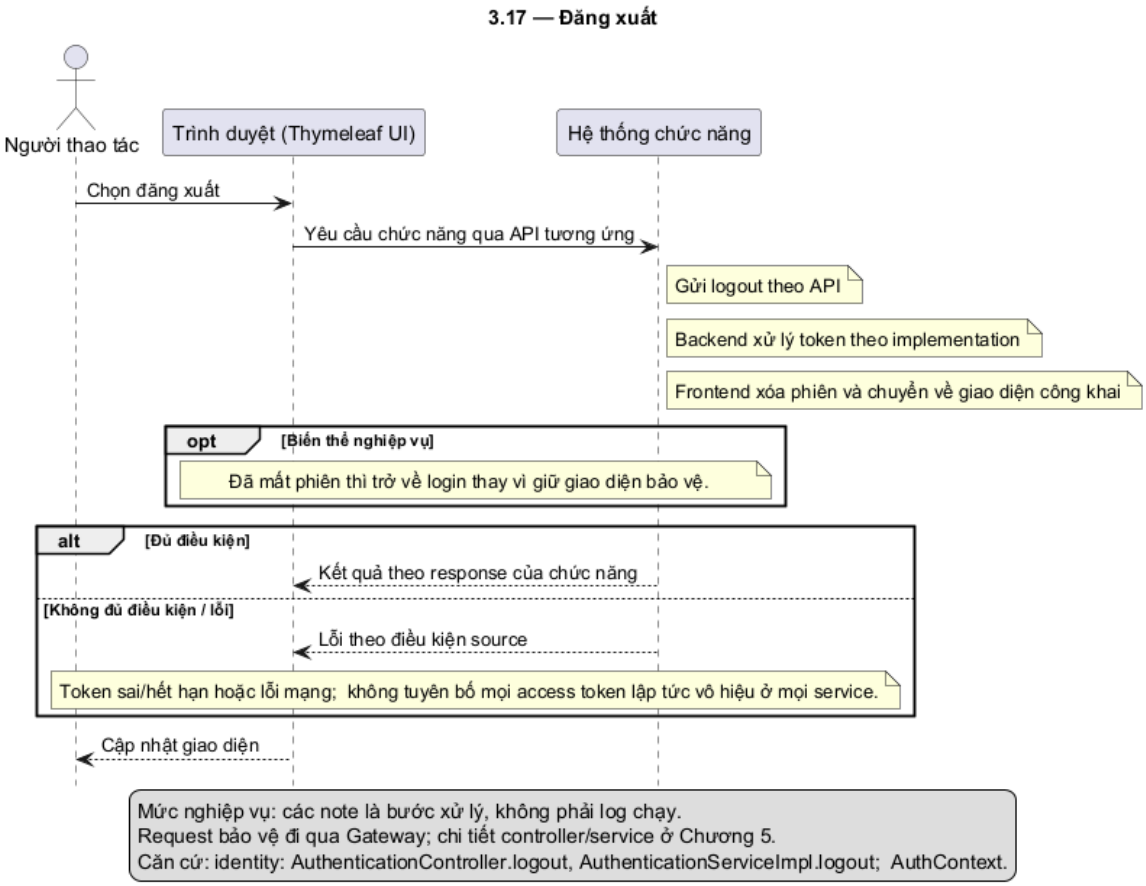

*Hình 3.15: Đăng nhập bằng tài khoản*

• Sequence Diagram đăng nhập bằng google oauth
Nội dung Hình 3.16: Sequence Diagram đăng nhập bằng google oauth.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn Google. 2. Thực hiện xác thực và quay về callback. 3. Backend xử lý provider, kiểm tra email và tài khoản. 4. Tạo hoặc cập nhật User theo luồng xã hội. 5. Cấp token và hiển thị giao diện phù hợp.
Nhánh thay thế: Tài khoản xã hội mới đủ điều kiện có thể chọn STUDENT/INSTRUCTOR qua UC055.
Nhánh lỗi: SOCIAL_PROVIDER_NOT_SUPPORTED, SOCIAL_LOGIN_FAILED, SOCIAL_EMAIL_NOT_PROVIDED, USER_INACTIVE.
Căn cứ: identity: AuthenticationServiceImpl.socialLogin, GoogleOAuthServiceImpl; frontend SocialCallbackPage.



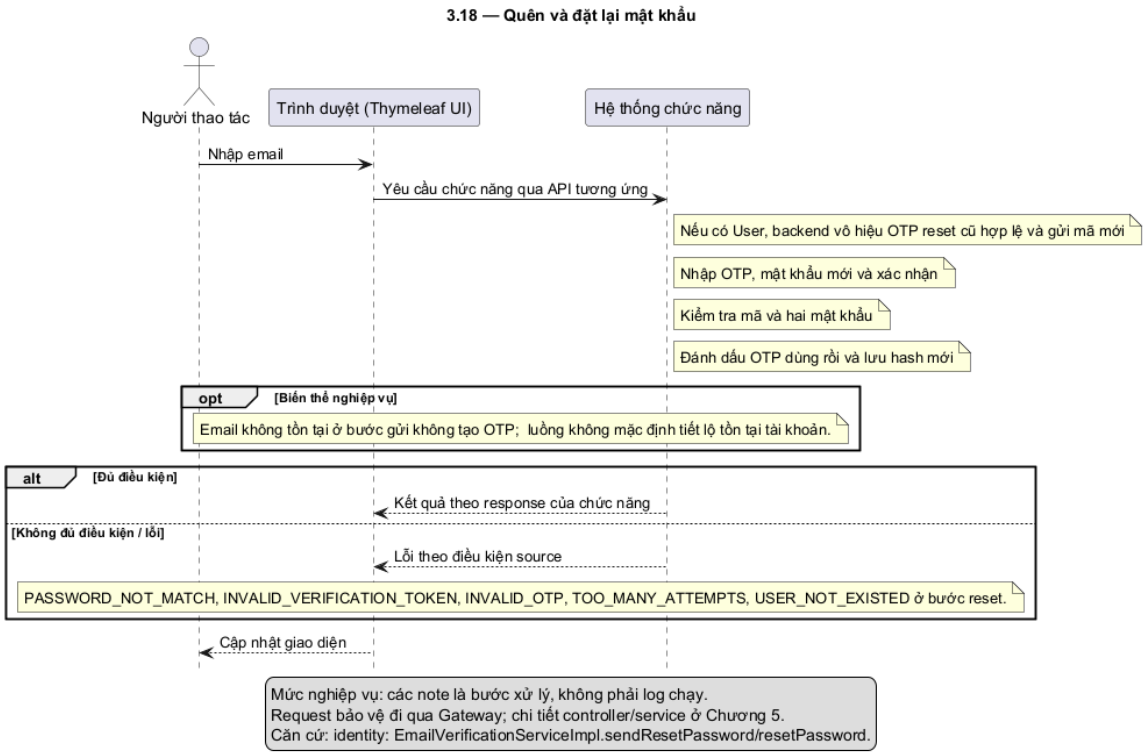

*Hình 3.16: Đăng nhập bằng google oauth*

• Sequence Diagram đăng xuất
Nội dung Hình 3.17: Sequence Diagram đăng xuất.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn đăng xuất. 2. Gửi logout theo API. 3. Backend xử lý token theo implementation. 4. Frontend xóa phiên và chuyển về giao diện công khai.
Nhánh thay thế: Đã mất phiên thì trở về login thay vì giữ giao diện bảo vệ.
Nhánh lỗi: Token sai/hết hạn hoặc lỗi mạng; không tuyên bố mọi access token lập tức vô hiệu ở mọi service.
Căn cứ: identity: AuthenticationController.logout, AuthenticationServiceImpl.logout; Spring Session & Security Context.



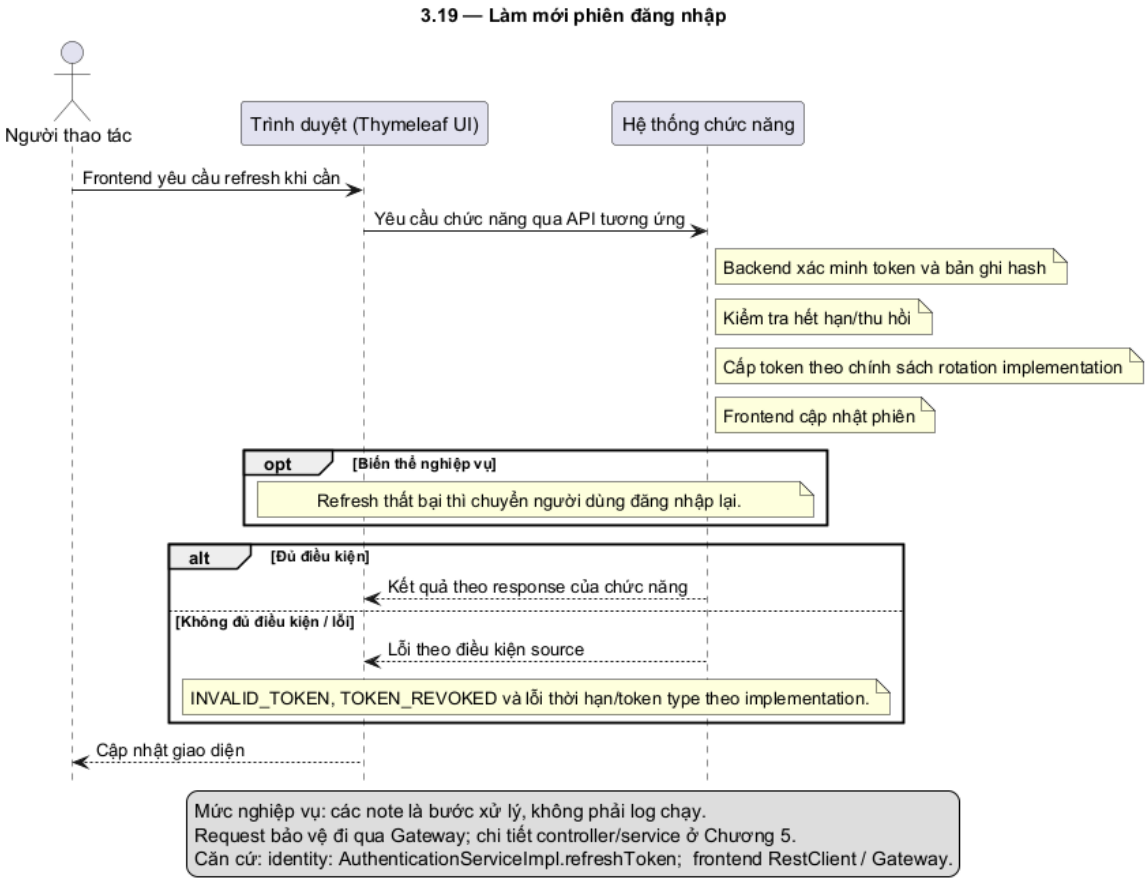

*Hình 3.17: Đăng xuất*

• Sequence Diagram quên và đặt lại mật khẩu
Nội dung Hình 3.18: Sequence Diagram quên và đặt lại mật khẩu.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Tách gửi OTP và xác nhận OTP thành hai request. Luồng gửi có lưu bản ghi mã và gọi EmailService/SMTP; luồng xác nhận tìm mã theo loại/thời hạn, kiểm tra số lần thử, đánh dấu usedAt và cập nhật User. Không vẽ SMTP như hệ thống trực tiếp đổi mật khẩu.
Luồng nghiệp vụ: 1. Nhập email. 2. Nếu có User, backend vô hiệu OTP reset cũ hợp lệ và gửi mã mới. 3. Nhập OTP, mật khẩu mới và xác nhận. 4. Kiểm tra mã và hai mật khẩu. 5. Đánh dấu OTP dùng rồi và lưu hash mới.
Nhánh thay thế: Email không tồn tại ở bước gửi không tạo OTP; luồng không mặc định tiết lộ tồn tại tài khoản.
Nhánh lỗi: PASSWORD_NOT_MATCH, INVALID_VERIFICATION_TOKEN, INVALID_OTP, TOO_MANY_ATTEMPTS, USER_NOT_EXISTED ở bước reset.
Căn cứ: identity: EmailVerificationServiceImpl.sendResetPassword/resetPassword.



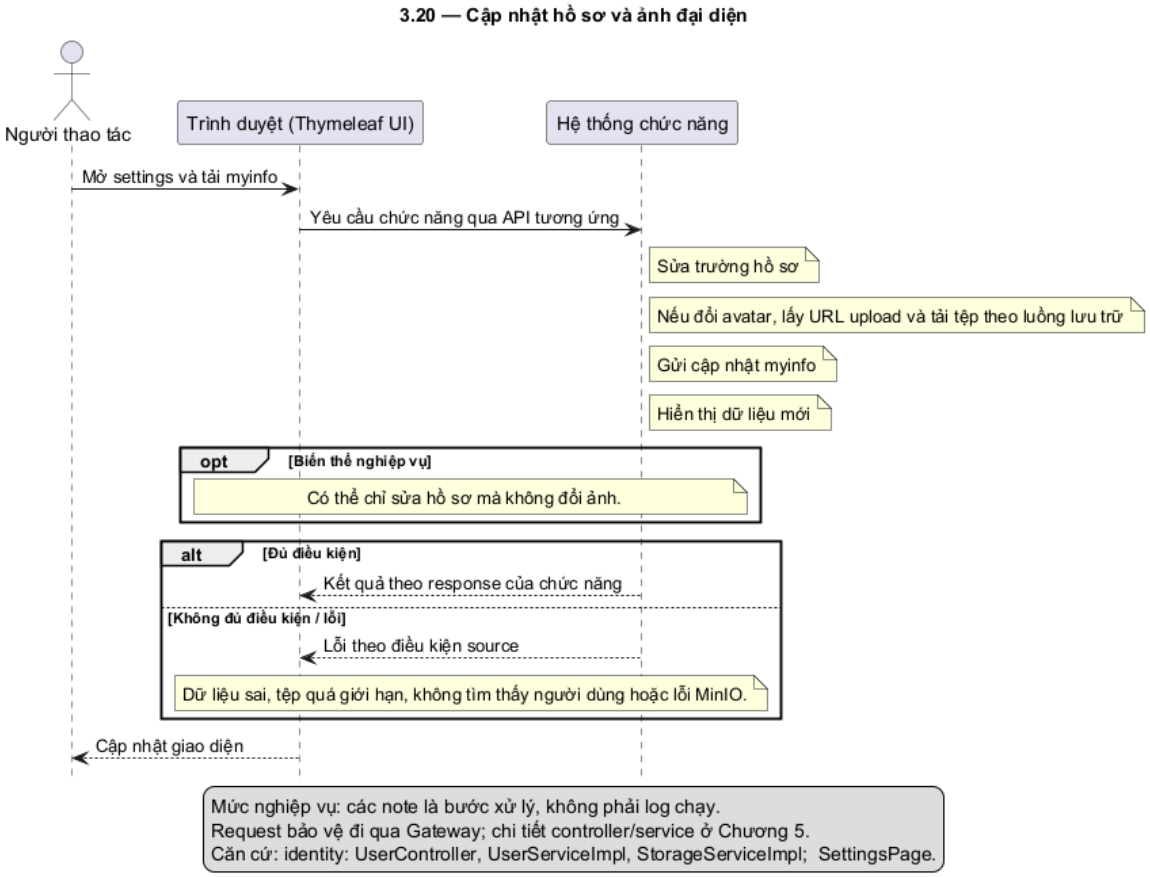

*Hình 3.18: Quên và đặt lại mật khẩu*

• Sequence Diagram làm mới phiên đăng nhập
Nội dung Hình 3.19: Sequence Diagram làm mới phiên đăng nhập.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Frontend yêu cầu refresh khi cần. 2. Backend xác minh token và bản ghi hash. 3. Kiểm tra hết hạn/thu hồi. 4. Cấp token theo chính sách rotation implementation. 5. Frontend cập nhật phiên.
Nhánh thay thế: Refresh thất bại thì chuyển người dùng đăng nhập lại.
Nhánh lỗi: INVALID_TOKEN, TOKEN_REVOKED và lỗi thời hạn/token type theo implementation.
Căn cứ: identity: AuthenticationServiceImpl.refreshToken; frontend Spring MVC Form Action & RestClient.



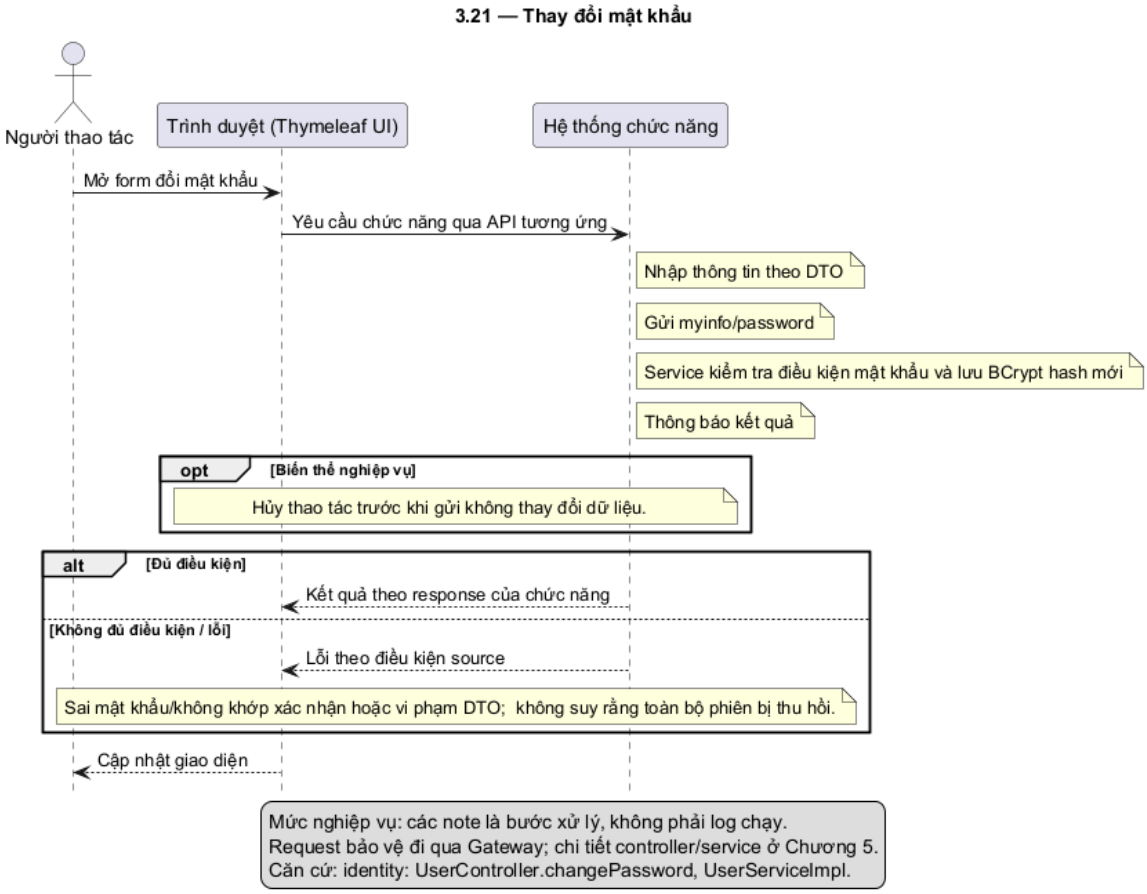

*Hình 3.19: Làm mới phiên đăng nhập*

• Sequence Diagram cập nhật hồ sơ và ảnh đại diện
Nội dung Hình 3.20: Sequence Diagram cập nhật hồ sơ và ảnh đại diện.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở settings và tải myinfo. 2. Sửa trường hồ sơ. 3. Nếu đổi avatar, lấy URL upload và tải tệp theo luồng lưu trữ. 4. Gửi cập nhật myinfo. 5. Hiển thị dữ liệu mới.
Nhánh thay thế: Có thể chỉ sửa hồ sơ mà không đổi ảnh.
Nhánh lỗi: Dữ liệu sai, tệp quá giới hạn, không tìm thấy người dùng hoặc lỗi MinIO.
Căn cứ: identity: UserController, UserServiceImpl, StorageServiceImpl; SettingsPage.




*Hình 3.20: Cập nhật hồ sơ và ảnh đại diện*

• Sequence Diagram thay đổi mật khẩu
Nội dung Hình 3.21: Sequence Diagram thay đổi mật khẩu.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở form đổi mật khẩu. 2. Nhập thông tin theo DTO. 3. Gửi myinfo/password. 4. Service kiểm tra điều kiện mật khẩu và lưu BCrypt hash mới. 5. Thông báo kết quả.
Nhánh thay thế: Hủy thao tác trước khi gửi không thay đổi dữ liệu.
Nhánh lỗi: Sai mật khẩu/không khớp xác nhận hoặc vi phạm DTO; không suy rằng toàn bộ phiên bị thu hồi.
Căn cứ: identity: UserController.changePassword, UserServiceImpl.




*Hình 3.21: Thay đổi mật khẩu*

## 3.5.2 Nhóm tra cứu và học tập

• Sequence Diagram xem trang chủ và giới thiệu

Nội dung Hình 3.22: Sequence Diagram xem trang chủ và giới thiệu.

Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.

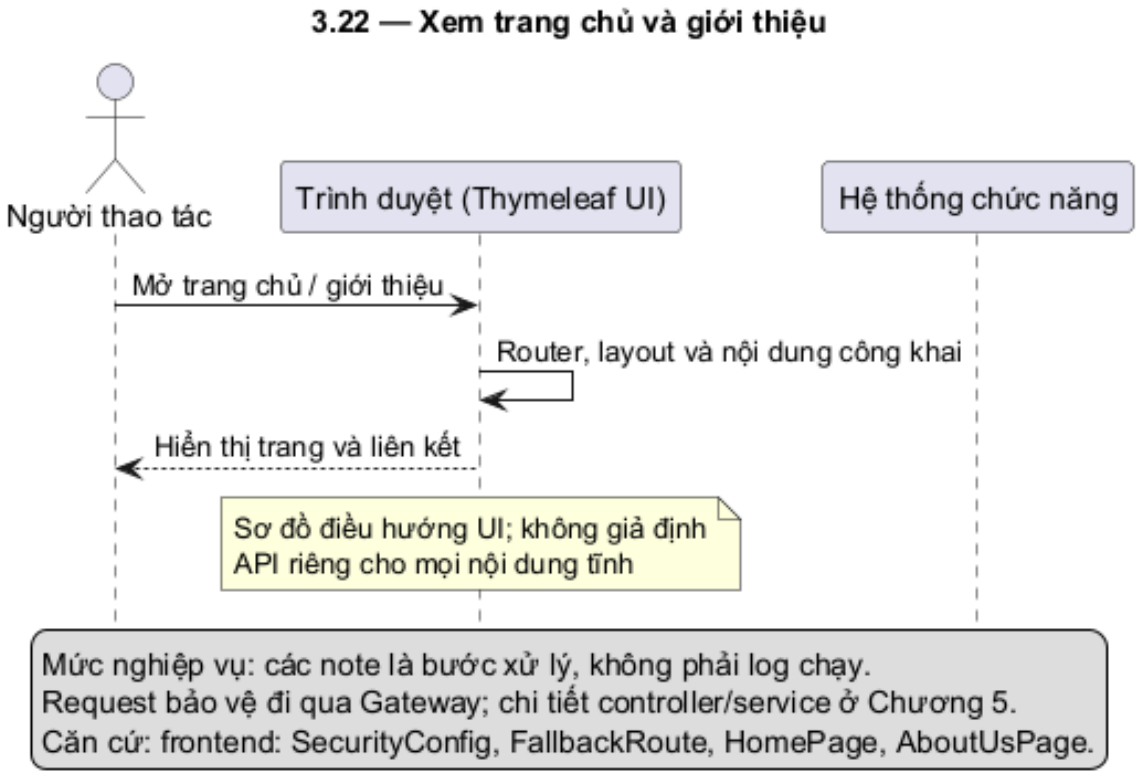
Tách nhánh xử lý cục bộ frontend khỏi nhánh gọi API. Thành phần cấu hình tĩnh/state/localStorage không được vẽ mũi tên lưu database. Với dữ liệu danh mục, vẽ nhánh API đến service/repository tương ứng.

Luồng nghiệp vụ: 1. Mở trang chủ hoặc about-us. 2. Router chọn layout/phân nhánh theo phiên. 3. Hiển thị nội dung công khai và các liên kết. 4. Người dùng chọn khóa học hoặc đăng nhập.

Nhánh thay thế: Người đã đăng nhập có thể được điều hướng theo FallbackRoute.

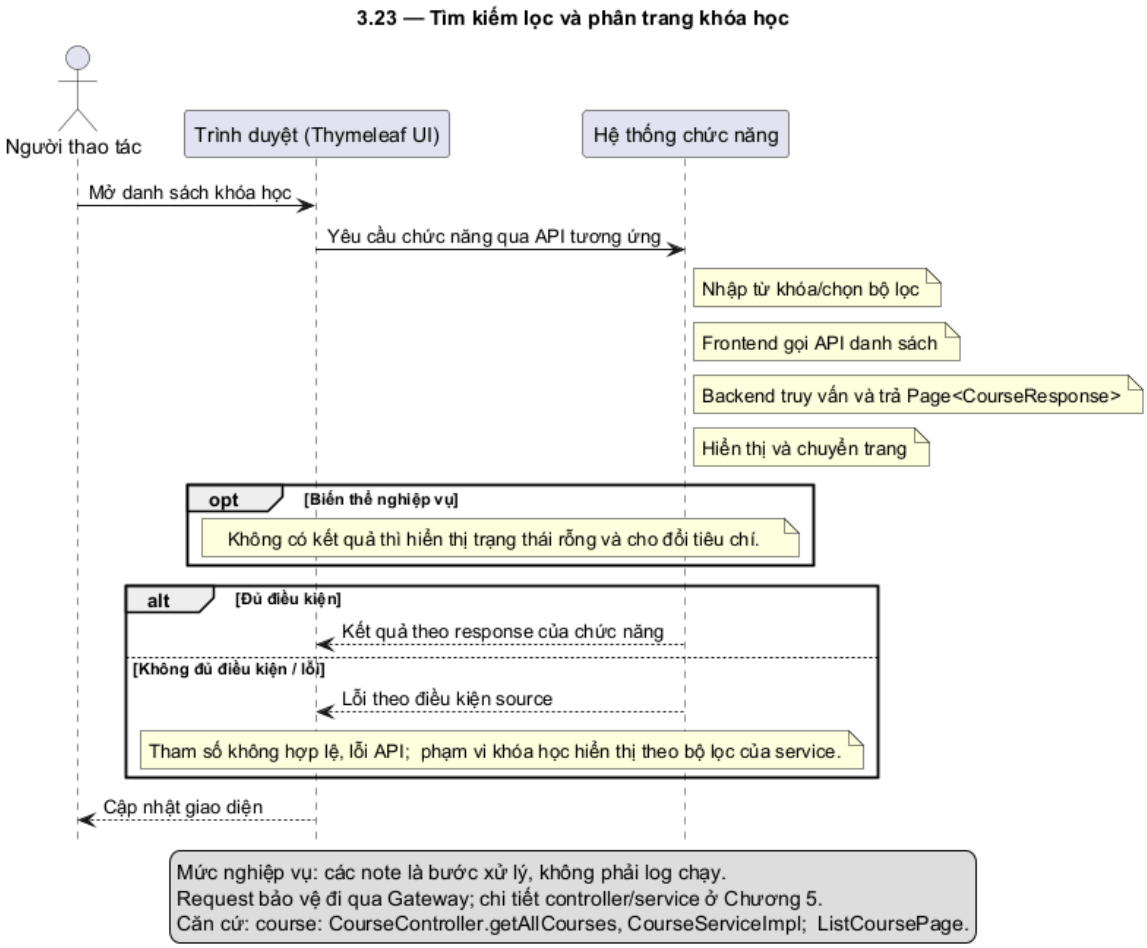
Nhánh lỗi: Lỗi tải frontend/API; nội dung founders hiện là hằng trong source.

Căn cứ: frontend: AppRoutes, FallbackRoute, HomePage, AboutUsPage.



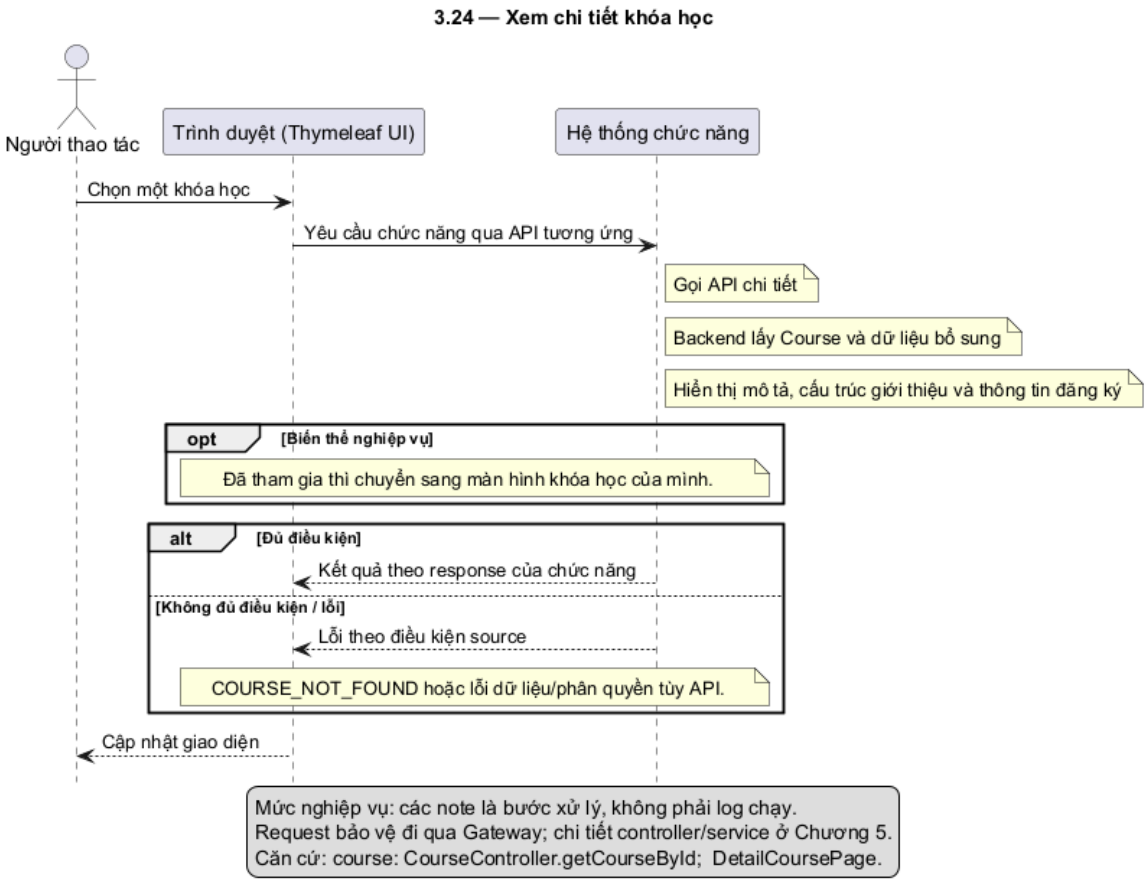

*Hình 3.22: Xem trang chủ và giới thiệu*

• Sequence Diagram tìm kiếm lọc và phân trang khóa học
Nội dung Hình 3.23: Sequence Diagram tìm kiếm lọc và phân trang khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở danh sách khóa học. 2. Nhập từ khóa/chọn bộ lọc. 3. Frontend gọi API danh sách. 4. Backend truy vấn và trả Page<CourseResponse>. 5. Hiển thị và chuyển trang.
Nhánh thay thế: Không có kết quả thì hiển thị trạng thái rỗng và cho đổi tiêu chí.
Nhánh lỗi: Tham số không hợp lệ, lỗi API; phạm vi khóa học hiển thị theo bộ lọc của service.
Căn cứ: course: CourseController.getAllCourses, CourseServiceImpl; ListCoursePage.



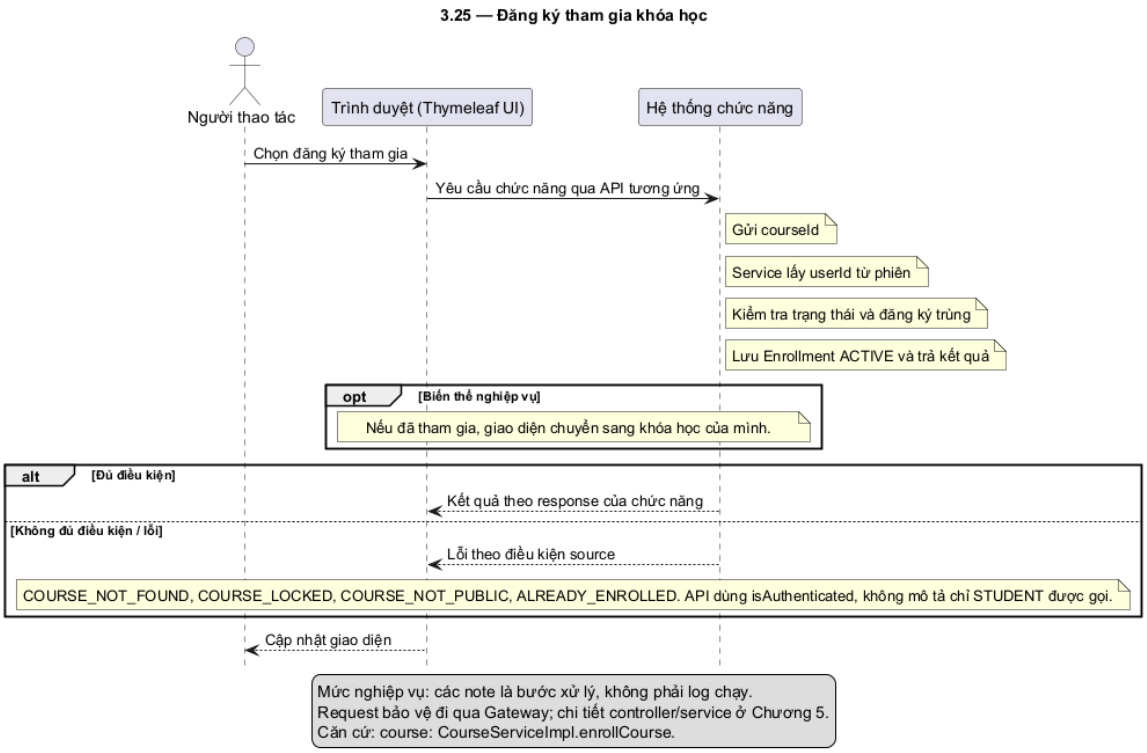

*Hình 3.23: Tìm kiếm lọc và phân trang khóa học*

• Sequence Diagram xem chi tiết khóa học
Nội dung Hình 3.24: Sequence Diagram xem chi tiết khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn một khóa học. 2. Gọi API chi tiết. 3. Backend lấy Course và dữ liệu bổ sung. 4. Hiển thị mô tả, cấu trúc giới thiệu và thông tin đăng ký.
Nhánh thay thế: Đã tham gia thì chuyển sang màn hình khóa học của mình.
Nhánh lỗi: COURSE_NOT_FOUND hoặc lỗi dữ liệu/phân quyền tùy API.
Căn cứ: course: CourseController.getCourseById; DetailCoursePage.



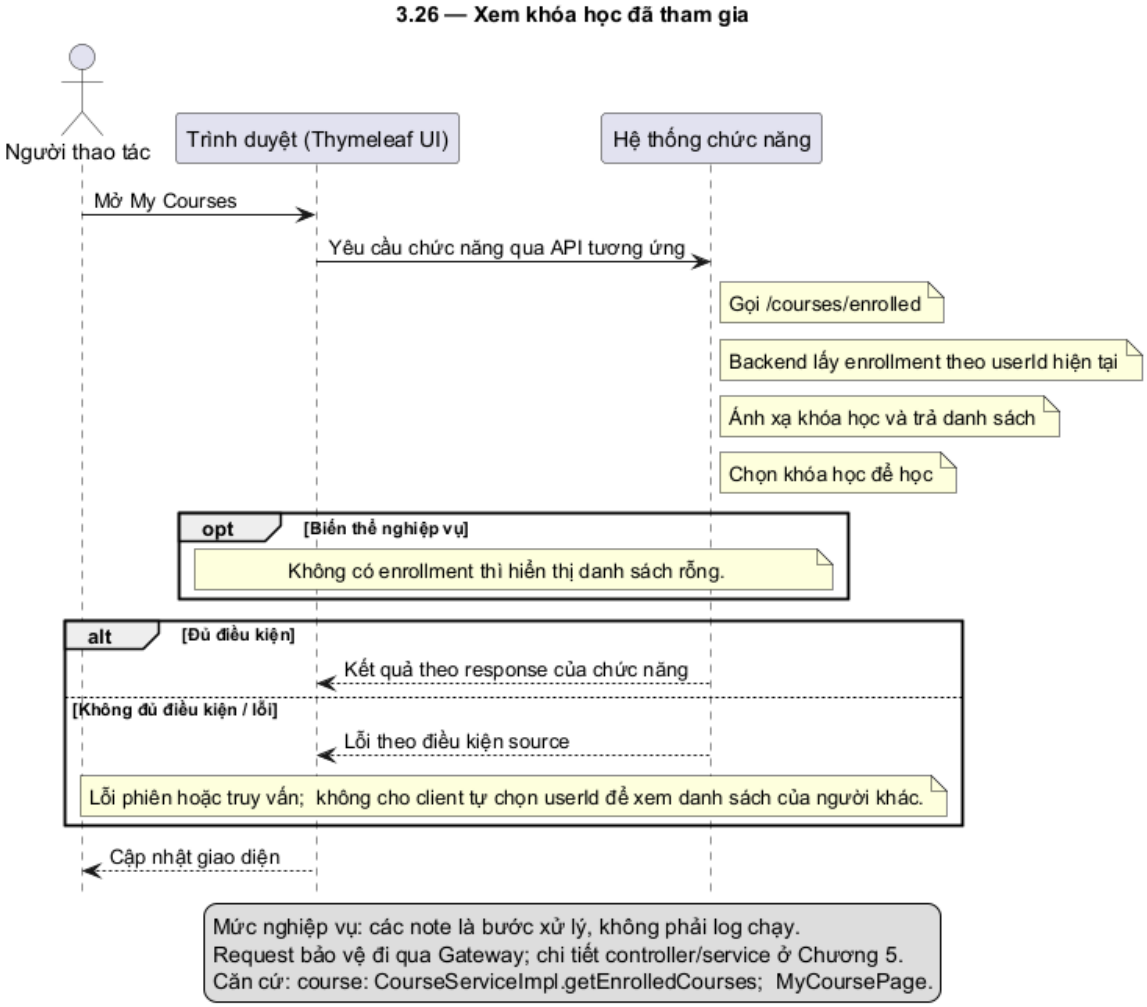

*Hình 3.24: Xem chi tiết khóa học*

• Sequence Diagram đăng ký tham gia khóa học
Nội dung Hình 3.25: Sequence Diagram đăng ký tham gia khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn đăng ký tham gia. 2. Gửi courseId. 3. Service lấy userId từ phiên. 4. Kiểm tra trạng thái và đăng ký trùng. 5. Lưu Enrollment ACTIVE và trả kết quả.
Nhánh thay thế: Nếu đã tham gia, giao diện chuyển sang khóa học của mình.
Nhánh lỗi: COURSE_NOT_FOUND, COURSE_LOCKED, COURSE_NOT_PUBLIC, ALREADY_ENROLLED. API dùng isAuthenticated, không mô tả chỉ STUDENT được gọi.
Căn cứ: course: CourseServiceImpl.enrollCourse.



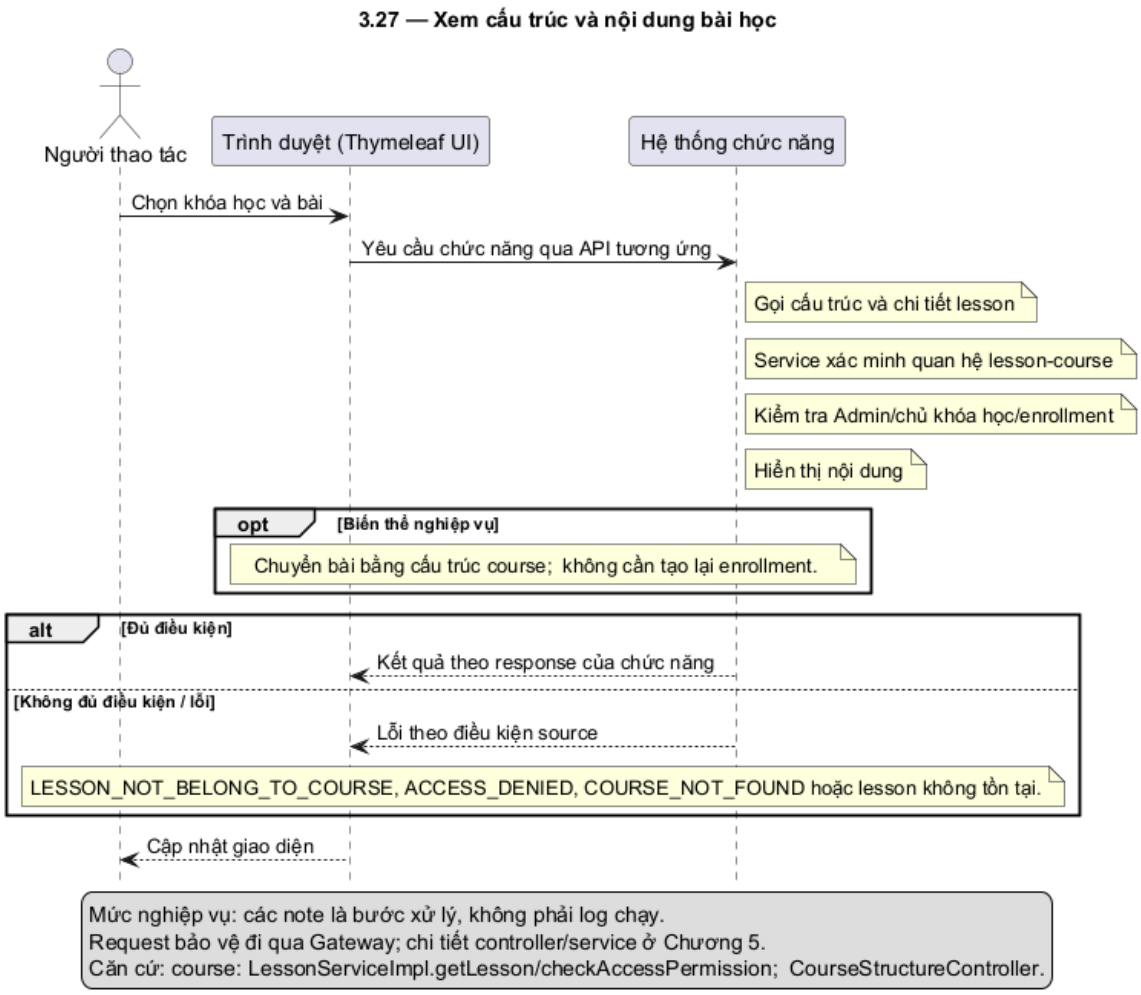

*Hình 3.25: Đăng ký tham gia khóa học*

• Sequence Diagram xem khóa học đã tham gia
Nội dung Hình 3.26: Sequence Diagram xem khóa học đã tham gia.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở My Courses. 2. Gọi /courses/enrolled. 3. Backend lấy enrollment theo userId hiện tại. 4. Ánh xạ khóa học và trả danh sách. 5. Chọn khóa học để học.
Nhánh thay thế: Không có enrollment thì hiển thị danh sách rỗng.
Nhánh lỗi: Lỗi phiên hoặc truy vấn; không cho client tự chọn userId để xem danh sách của người khác.
Căn cứ: course: CourseServiceImpl.getEnrolledCourses; MyCoursePage.



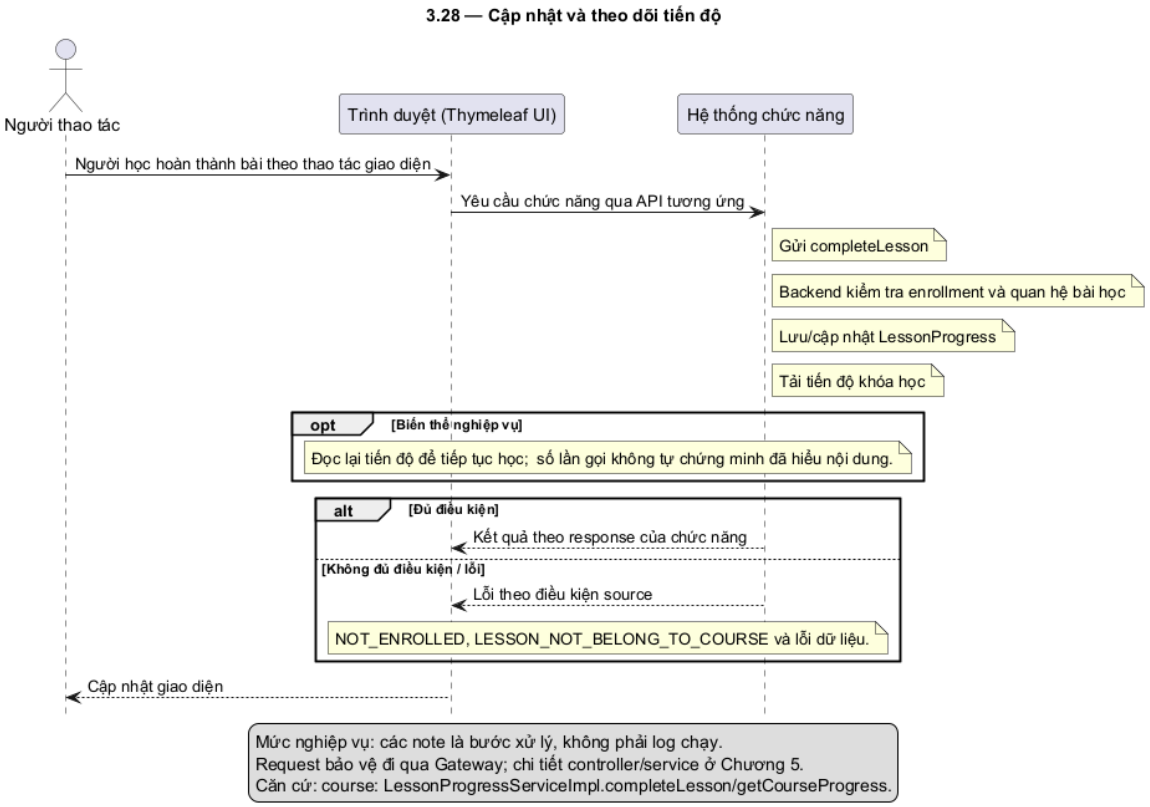

*Hình 3.26: Xem khóa học đã tham gia*

• Sequence Diagram xem cấu trúc và nội dung bài học
Nội dung Hình 3.27: Sequence Diagram xem cấu trúc và nội dung bài học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn khóa học và bài. 2. Gọi cấu trúc và chi tiết lesson. 3. Service xác minh quan hệ lesson–course. 4. Kiểm tra Admin/chủ khóa học/enrollment. 5. Hiển thị nội dung.
Nhánh thay thế: Chuyển bài bằng cấu trúc course; không cần tạo lại enrollment.
Nhánh lỗi: LESSON_NOT_BELONG_TO_COURSE, ACCESS_DENIED, COURSE_NOT_FOUND hoặc lesson không tồn tại.
Căn cứ: course: LessonServiceImpl.getLesson/checkAccessPermission; CourseStructureController.



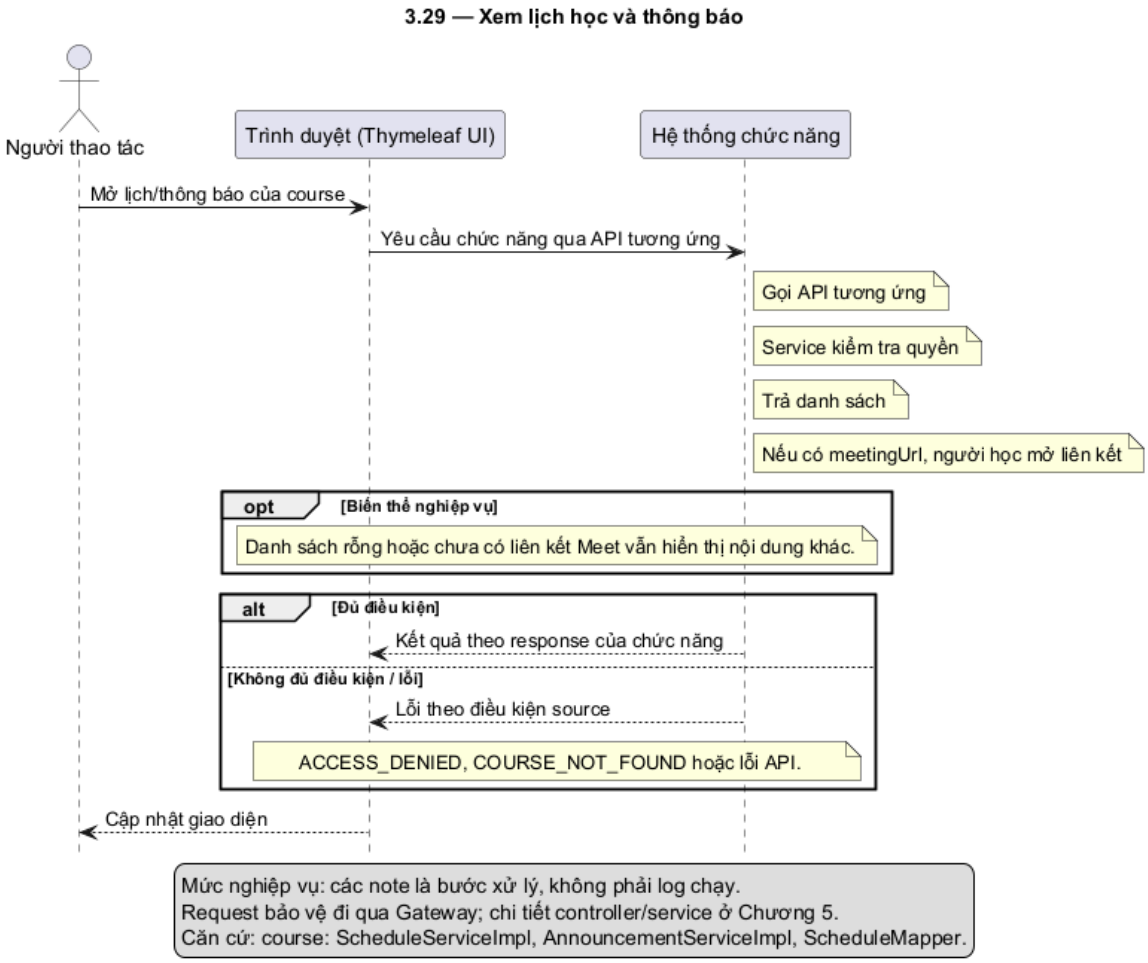

*Hình 3.27: Xem cấu trúc và nội dung bài học*

• Sequence Diagram cập nhật và theo dõi tiến độ
Nội dung Hình 3.28: Sequence Diagram cập nhật và theo dõi tiến độ.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Người học hoàn thành bài theo thao tác giao diện. 2. Gửi completeLesson. 3. Backend kiểm tra enrollment và quan hệ bài học. 4. Lưu/cập nhật LessonProgress. 5. Tải tiến độ khóa học.
Nhánh thay thế: Đọc lại tiến độ để tiếp tục học; số lần gọi không tự chứng minh đã hiểu nội dung.
Nhánh lỗi: NOT_ENROLLED, LESSON_NOT_BELONG_TO_COURSE và lỗi dữ liệu.
Căn cứ: course: LessonProgressServiceImpl.completeLesson/getCourseProgress.




*Hình 3.28: Cập nhật và theo dõi tiến độ*

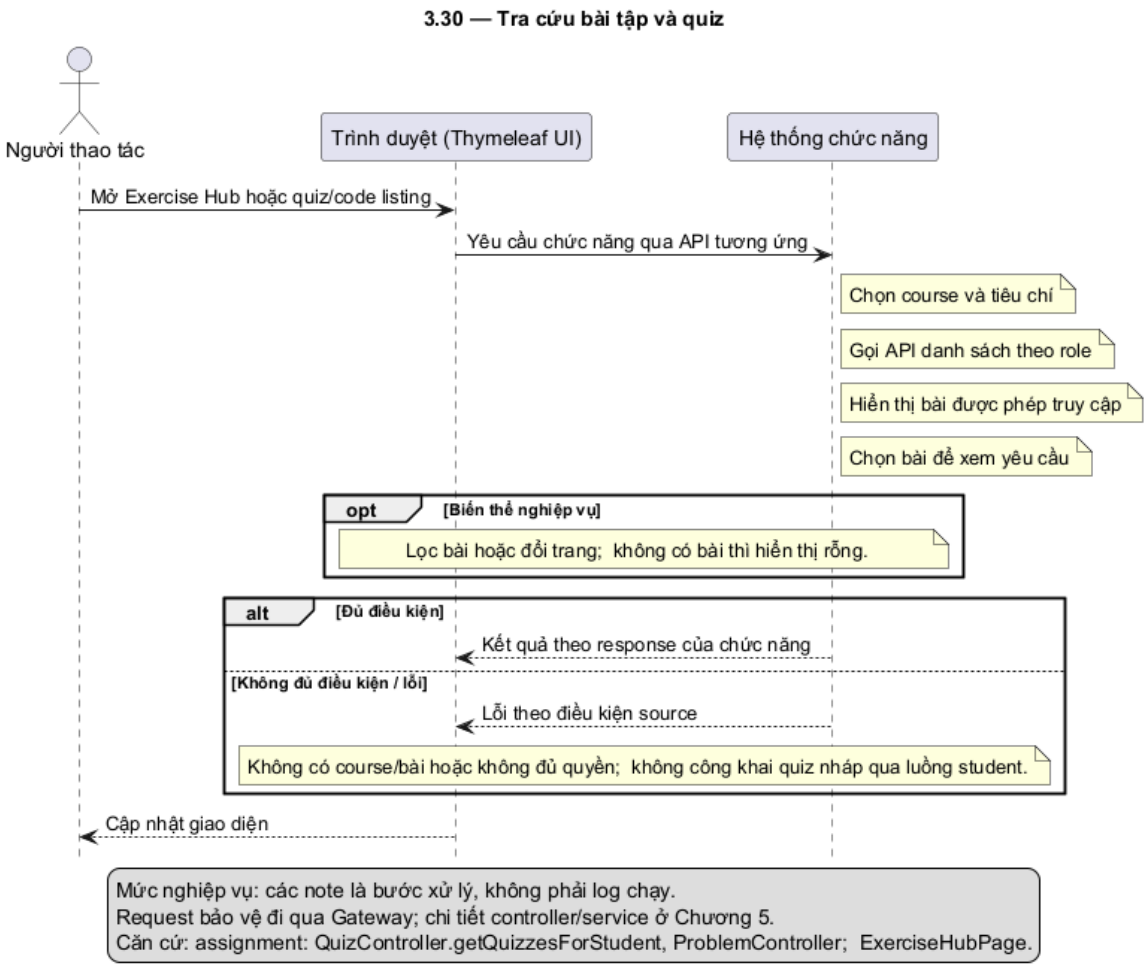
• Sequence Diagram xem lịch học và thông báo
Nội dung Hình 3.29: Sequence Diagram xem lịch học và thông báo.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở lịch/thông báo của course. 2. Gọi API tương ứng. 3. Service kiểm tra quyền. 4. Trả danh sách. 5. Nếu có meetingUrl, người học mở liên kết.
Nhánh thay thế: Danh sách rỗng hoặc chưa có liên kết Meet vẫn hiển thị nội dung khác.
Nhánh lỗi: ACCESS_DENIED, COURSE_NOT_FOUND hoặc lỗi API.
Căn cứ: course: ScheduleServiceImpl, AnnouncementServiceImpl, ScheduleMapper.




*Hình 3.29: Xem lịch học và thông báo*

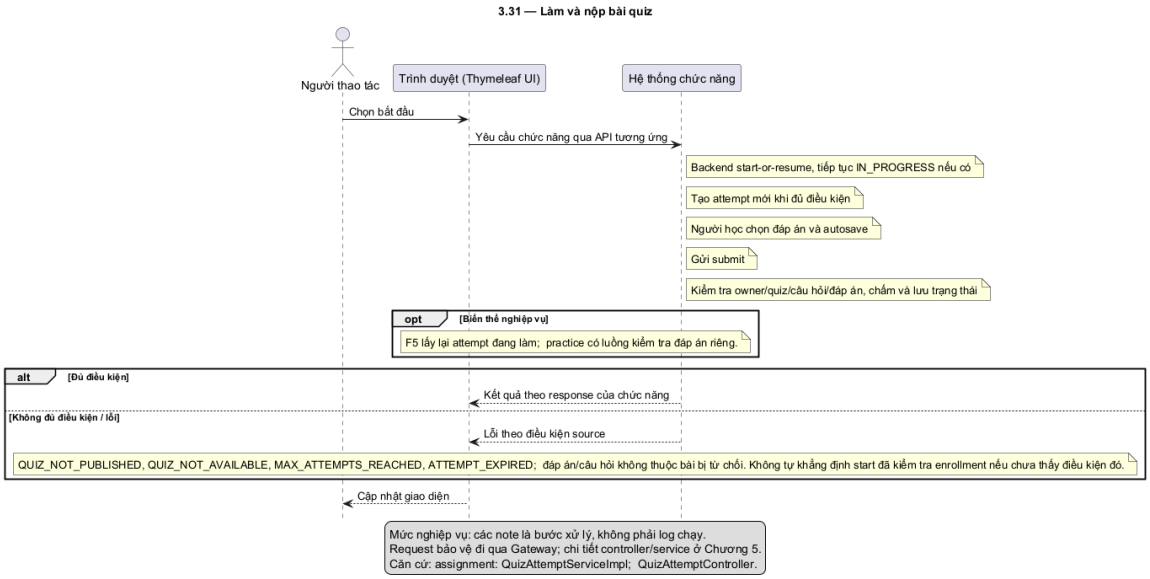
## 3.5.3 Nhóm quiz và bài lập trình

• Sequence Diagram tra cứu bài tập và quiz
Nội dung Hình 3.30: Sequence Diagram tra cứu bài tập và quiz.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở Exercise Hub hoặc quiz/code listing. 2. Chọn course và tiêu chí. 3. Gọi API danh sách theo role. 4. Hiển thị bài được phép truy cập. 5. Chọn bài để xem yêu cầu.
Nhánh thay thế: Lọc bài hoặc đổi trang; không có bài thì hiển thị rỗng.
Nhánh lỗi: Không có course/bài hoặc không đủ quyền; không công khai quiz nháp qua luồng student.
Căn cứ: assignment: QuizController.getQuizzesForStudent, ProblemController; ExerciseHubPage.




*Hình 3.30: Tra cứu bài tập và quiz*

• Sequence Diagram làm và nộp bài quiz
Nội dung Hình 3.31: Sequence Diagram làm và nộp bài quiz.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn bắt đầu. 2. Backend start-or-resume, tiếp tục IN_PROGRESS nếu có. 3. Tạo attempt mới khi đủ điều kiện. 4. Người học chọn đáp án và autosave. 5. Gửi submit. 6. Kiểm tra owner/quiz/câu hỏi/đáp án, chấm và lưu trạng thái.
Nhánh thay thế: F5 lấy lại attempt đang làm; practice có luồng kiểm tra đáp án riêng.
Nhánh lỗi: QUIZ_NOT_PUBLISHED, QUIZ_NOT_AVAILABLE, MAX_ATTEMPTS_REACHED, ATTEMPT_EXPIRED; đáp án/câu hỏi không thuộc bài bị từ chối. Không tự khẳng định start đã kiểm tra enrollment nếu chưa thấy điều kiện đó.
Căn cứ: assignment: QuizAttemptServiceImpl; QuizAttemptController.




*Hình 3.31: Làm và nộp bài quiz*

• Sequence Diagram xem điểm và lịch sử làm quiz

Nội dung Hình 3.32: Sequence Diagram xem điểm và lịch sử làm quiz.

Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.

Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.

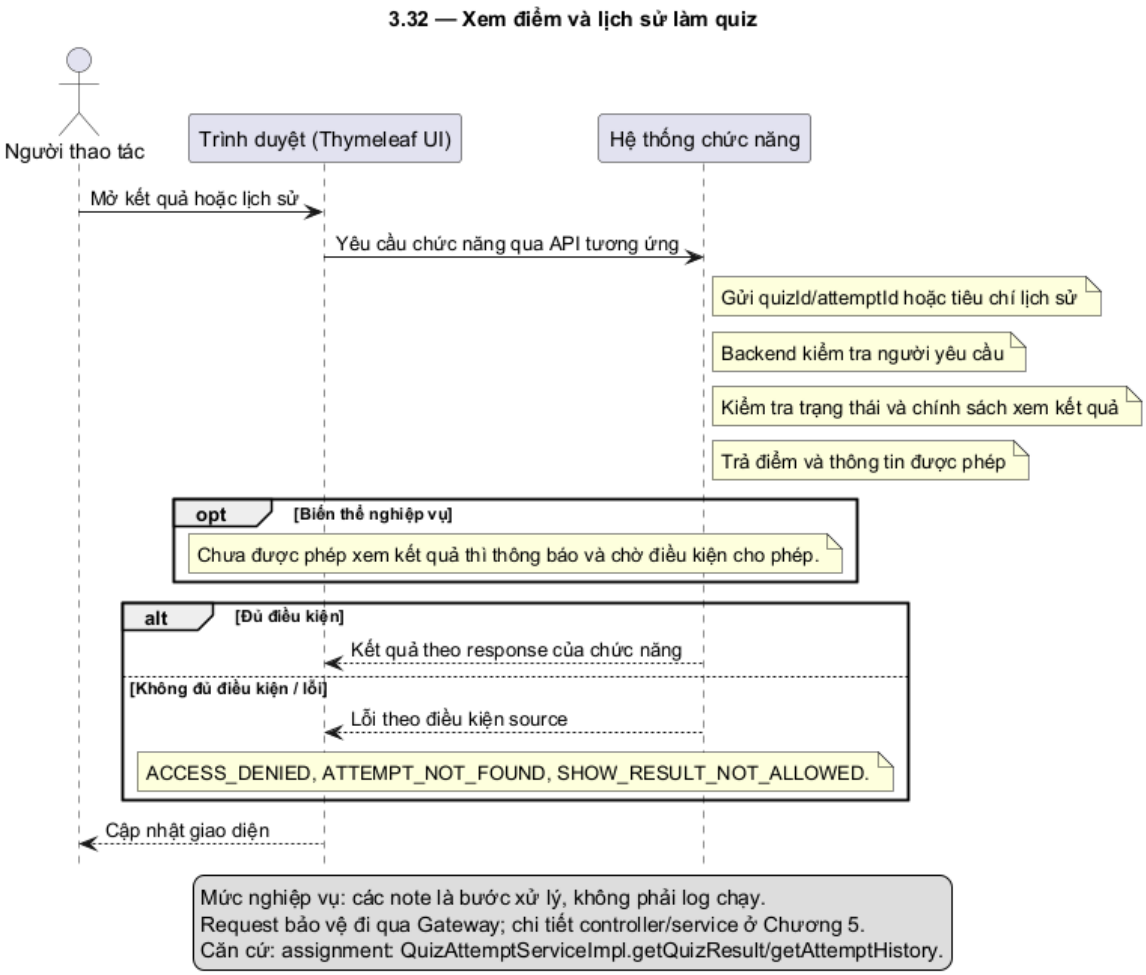
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.

Luồng nghiệp vụ: 1. Mở kết quả hoặc lịch sử. 2. Gửi quizId/attemptId hoặc tiêu chí lịch sử. 3. Backend kiểm tra người yêu cầu. 4. Kiểm tra trạng thái và chính sách xem kết quả. 5. Trả điểm và thông tin được phép.

Nhánh thay thế: Chưa được phép xem kết quả thì thông báo và chờ điều kiện cho phép.

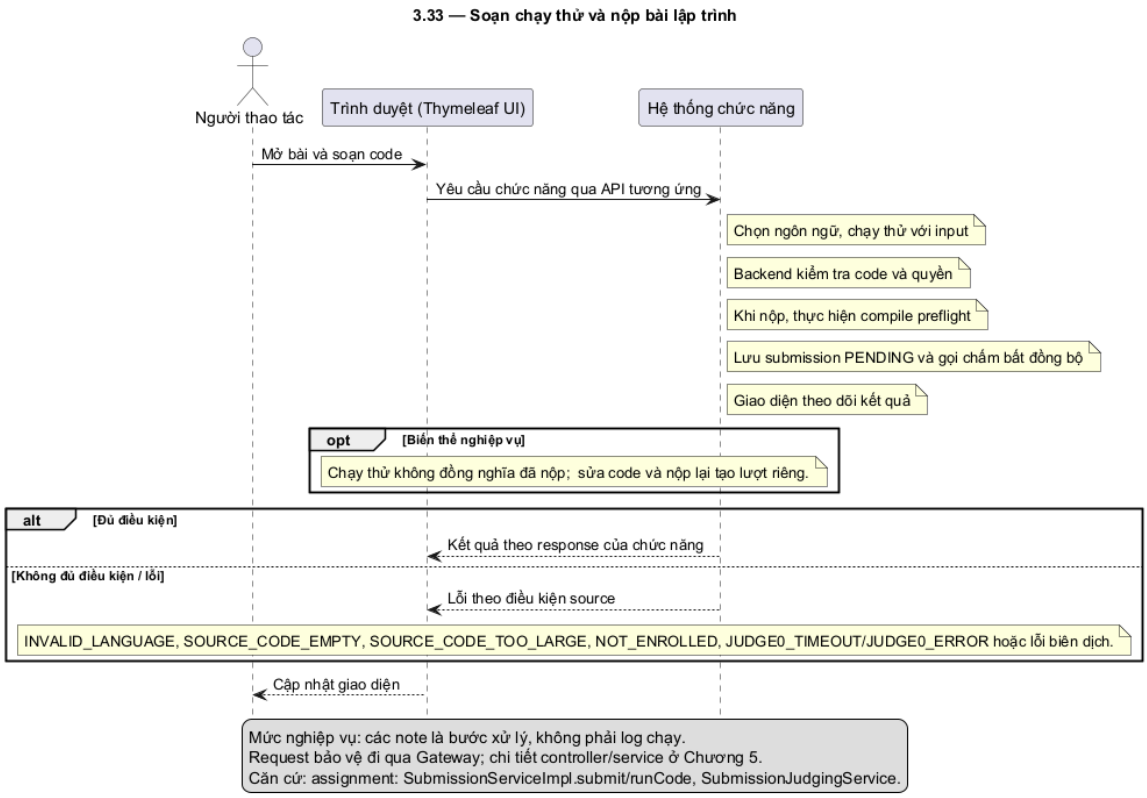
Nhánh lỗi: ACCESS_DENIED, ATTEMPT_NOT_FOUND, SHOW_RESULT_NOT_ALLOWED.

Căn cứ: assignment: QuizAttemptServiceImpl.getQuizResult/getAttemptHistory.



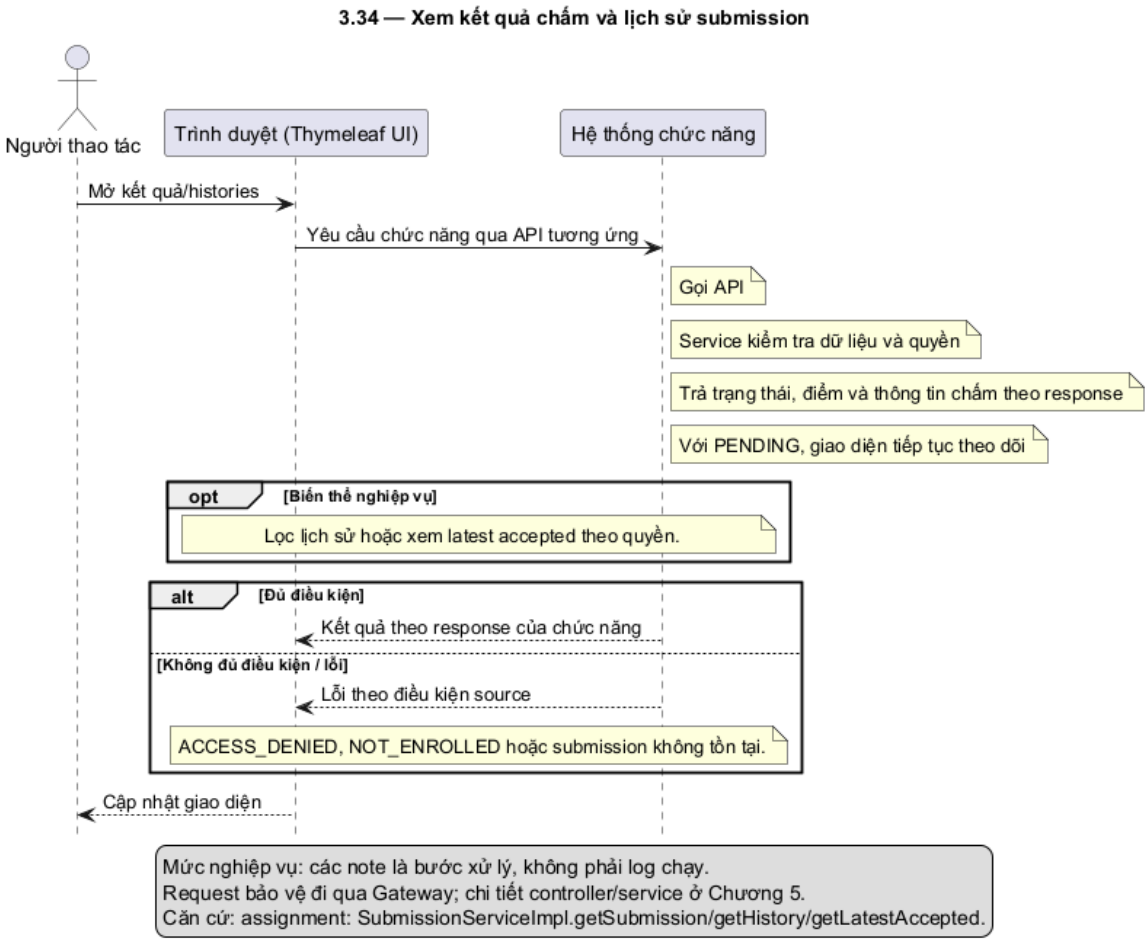

*Hình 3.32: Xem điểm và lịch sử làm quiz*

• Sequence Diagram soạn chạy thử và nộp bài lập trình
Nội dung Hình 3.33: Sequence Diagram soạn chạy thử và nộp bài lập trình.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Phân biệt runCode với submit: runCode kiểm tra dữ liệu/quyền và gọi engine trả output; submit kiểm tra và compile preflight, lưu submission PENDING, gọi judgeAsync rồi trả response. Tác vụ chấm chạy bất đồng bộ; lần đọc kết quả sau đó là request riêng, không vẽ như submit chờ chấm tất cả test mới trả.
Luồng nghiệp vụ: 1. Mở bài và soạn code. 2. Chọn ngôn ngữ, chạy thử với input. 3. Backend kiểm tra code và quyền. 4. Khi nộp, thực hiện compile preflight. 5. Lưu submission PENDING và gọi chấm bất đồng bộ. 6. Giao diện theo dõi kết quả.
Nhánh thay thế: Chạy thử không đồng nghĩa đã nộp; sửa code và nộp lại tạo lượt riêng.
Nhánh lỗi: INVALID_LANGUAGE, SOURCE_CODE_EMPTY, SOURCE_CODE_TOO_LARGE, NOT_ENROLLED, JUDGE0_TIMEOUT/JUDGE0_ERROR hoặc lỗi biên dịch.
Căn cứ: assignment: SubmissionServiceImpl.submit/runCode, SubmissionJudgingService.



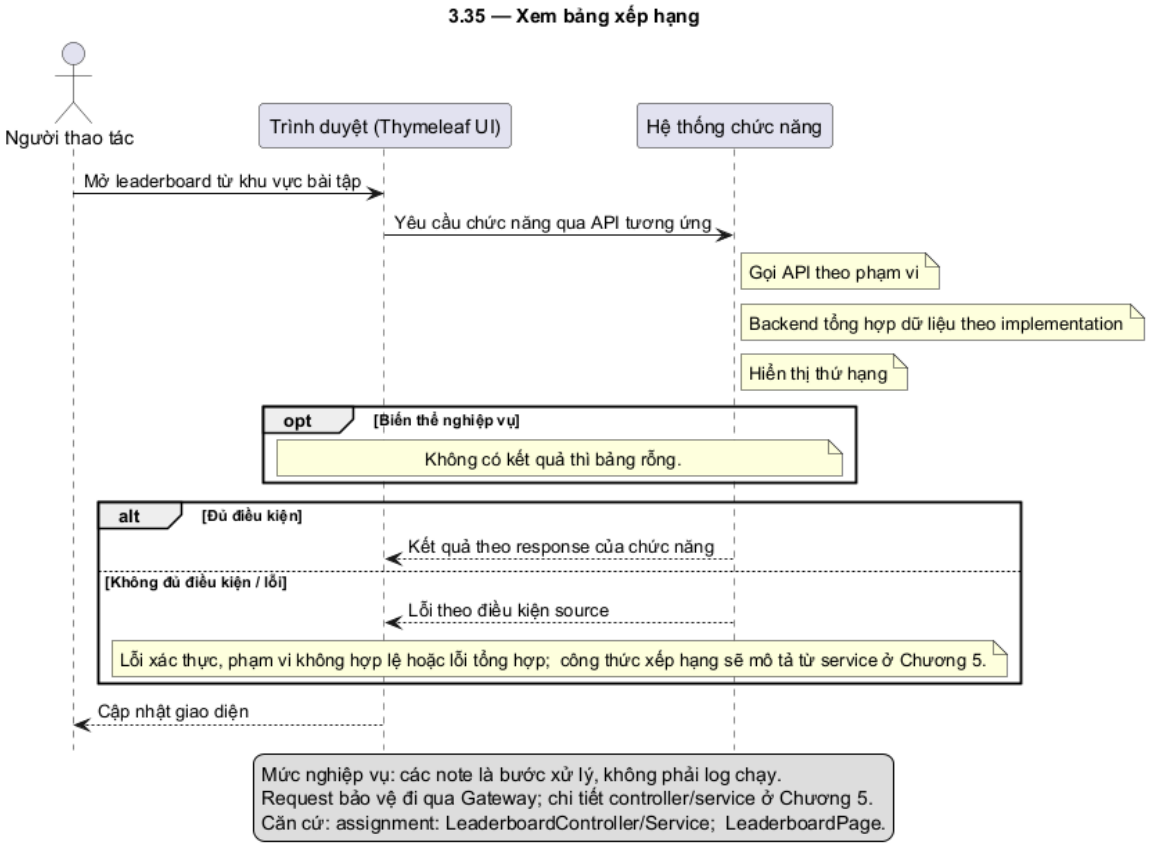

*Hình 3.33: Soạn chạy thử và nộp bài lập trình*

• Sequence Diagram xem kết quả chấm và lịch sử submission
Nội dung Hình 3.34: Sequence Diagram xem kết quả chấm và lịch sử submission.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở kết quả/histories. 2. Gọi API. 3. Service kiểm tra dữ liệu và quyền. 4. Trả trạng thái, điểm và thông tin chấm theo response. 5. Với PENDING, giao diện tiếp tục theo dõi.
Nhánh thay thế: Lọc lịch sử hoặc xem latest accepted theo quyền.
Nhánh lỗi: ACCESS_DENIED, NOT_ENROLLED hoặc submission không tồn tại.
Căn cứ: assignment: SubmissionServiceImpl.getSubmission/getHistory/getLatestAccepted.




*Hình 3.34: Xem kết quả chấm và lịch sử submission*

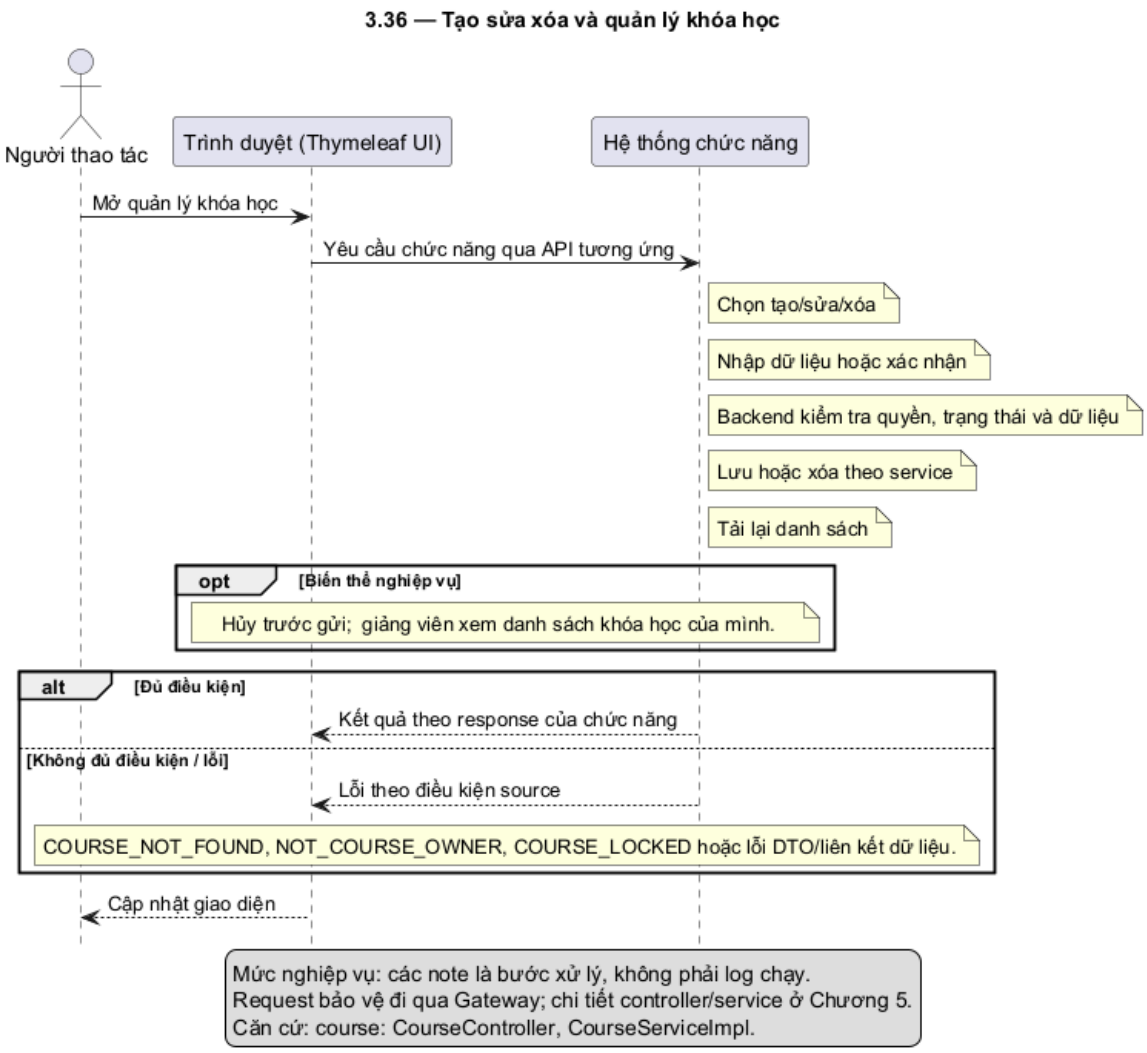
• Sequence Diagram xem bảng xếp hạng
Nội dung Hình 3.35: Sequence Diagram xem bảng xếp hạng.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở leaderboard từ khu vực bài tập. 2. Gọi API theo phạm vi. 3. Backend tổng hợp dữ liệu theo implementation. 4. Hiển thị thứ hạng.
Nhánh thay thế: Không có kết quả thì bảng rỗng.
Nhánh lỗi: Lỗi xác thực, phạm vi không hợp lệ hoặc lỗi tổng hợp; công thức xếp hạng sẽ mô tả từ service ở Chương 5.
Căn cứ: assignment: LeaderboardController/Service; LeaderboardPage.




*Hình 3.35: Xem bảng xếp hạng*

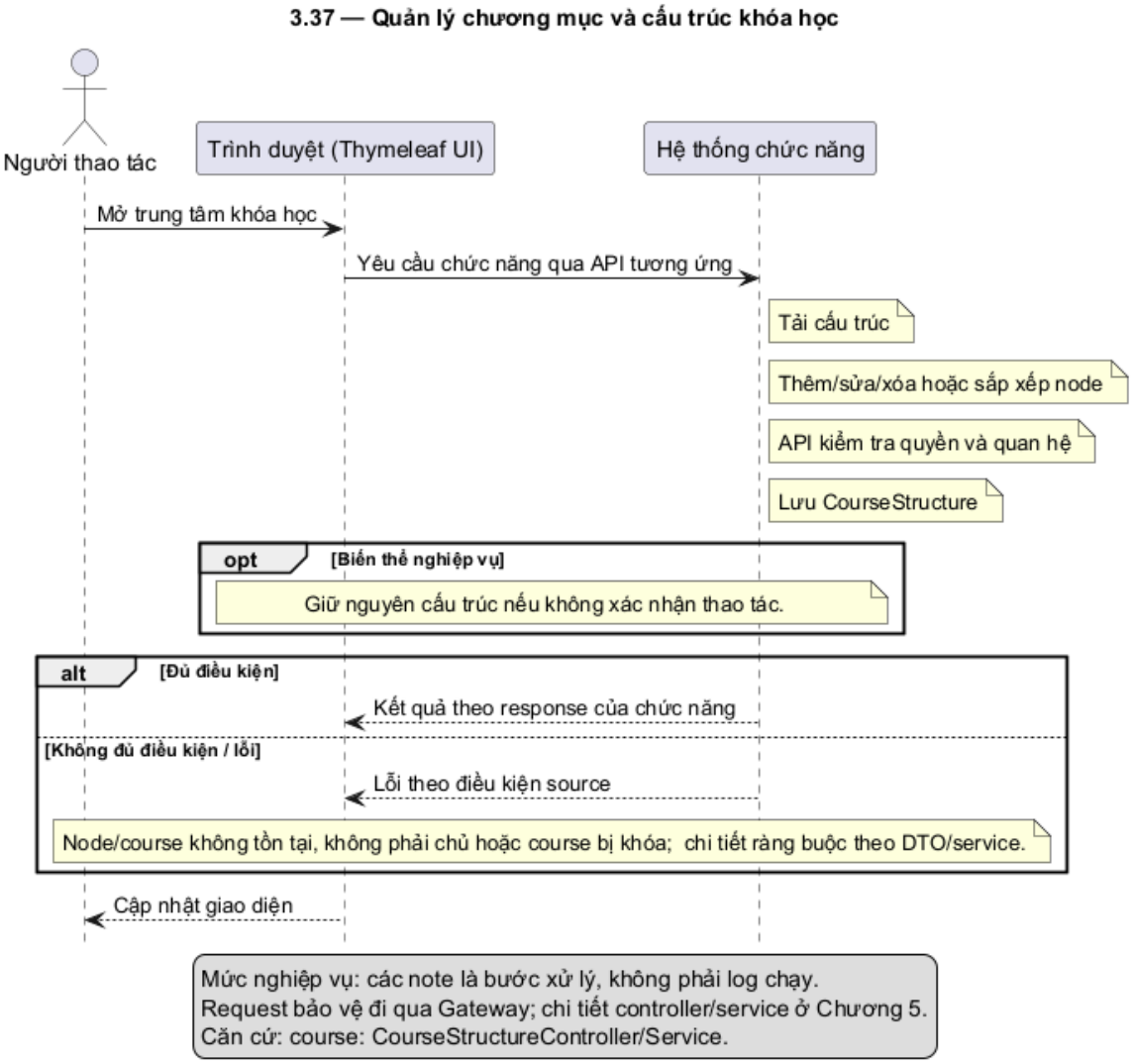
## 3.5.4 Nhóm quản lý giảng dạy

• Sequence Diagram tạo sửa xóa và quản lý khóa học
Nội dung Hình 3.36: Sequence Diagram tạo sửa xóa và quản lý khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở quản lý khóa học. 2. Chọn tạo/sửa/xóa. 3. Nhập dữ liệu hoặc xác nhận. 4. Backend kiểm tra quyền, trạng thái và dữ liệu. 5. Lưu hoặc xóa theo service. 6. Tải lại danh sách.
Nhánh thay thế: Hủy trước gửi; giảng viên xem danh sách khóa học của mình.
Nhánh lỗi: COURSE_NOT_FOUND, NOT_COURSE_OWNER, COURSE_LOCKED hoặc lỗi DTO/liên kết dữ liệu.
Căn cứ: course: CourseController, CourseServiceImpl.



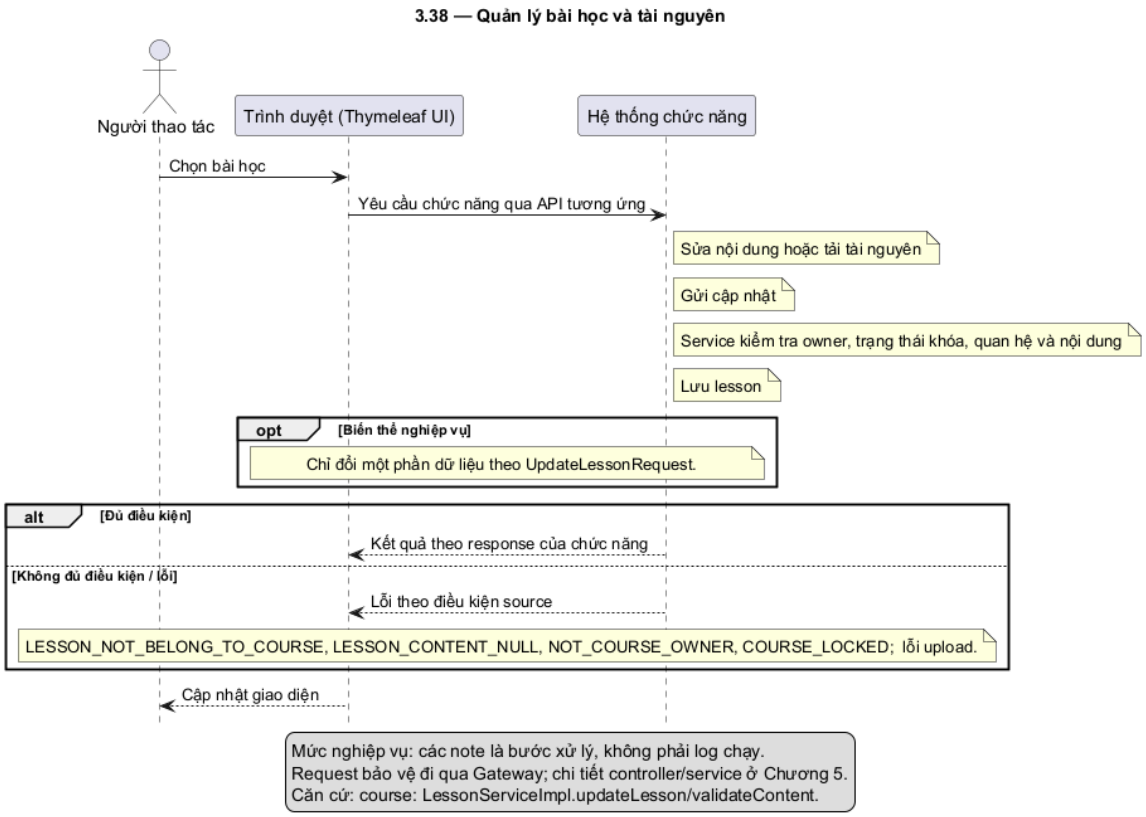

*Hình 3.36: Tạo sửa xóa và quản lý khóa học*

• Sequence Diagram quản lý chương mục và cấu trúc khóa học
Nội dung Hình 3.37: Sequence Diagram quản lý chương mục và cấu trúc khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở trung tâm khóa học. 2. Tải cấu trúc. 3. Thêm/sửa/xóa hoặc sắp xếp node. 4. API kiểm tra quyền và quan hệ. 5. Lưu CourseStructure.
Nhánh thay thế: Giữ nguyên cấu trúc nếu không xác nhận thao tác.
Nhánh lỗi: Node/course không tồn tại, không phải chủ hoặc course bị khóa; chi tiết ràng buộc theo DTO/service.
Căn cứ: course: CourseStructureController/Service.



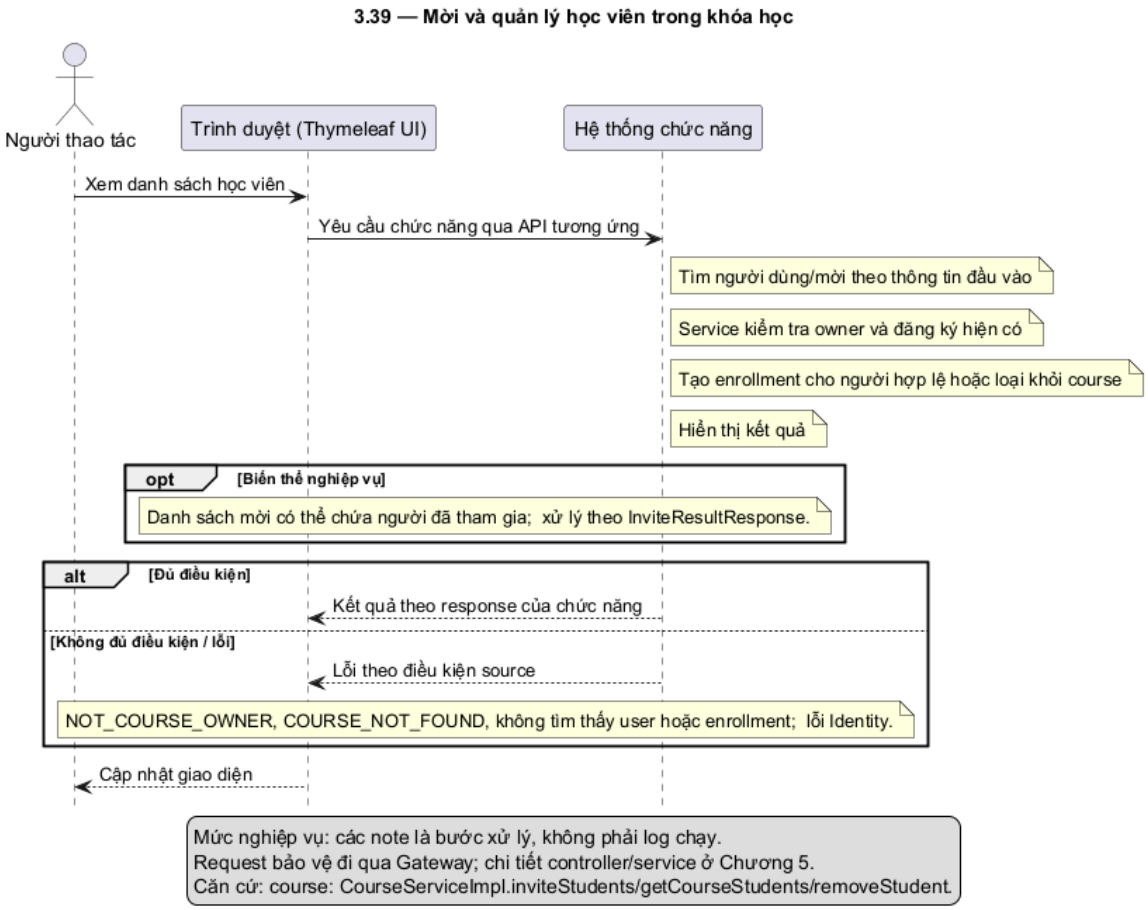

*Hình 3.37: Quản lý chương mục và cấu trúc khóa học*

• Sequence Diagram quản lý bài học và tài nguyên
Nội dung Hình 3.38: Sequence Diagram quản lý bài học và tài nguyên.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn bài học. 2. Sửa nội dung hoặc tải tài nguyên. 3. Gửi cập nhật. 4. Service kiểm tra owner, trạng thái khóa, quan hệ và nội dung. 5. Lưu lesson.
Nhánh thay thế: Chỉ đổi một phần dữ liệu theo UpdateLessonRequest.
Nhánh lỗi: LESSON_NOT_BELONG_TO_COURSE, LESSON_CONTENT_NULL, NOT_COURSE_OWNER, COURSE_LOCKED; lỗi upload.
Căn cứ: course: LessonServiceImpl.updateLesson/validateContent.



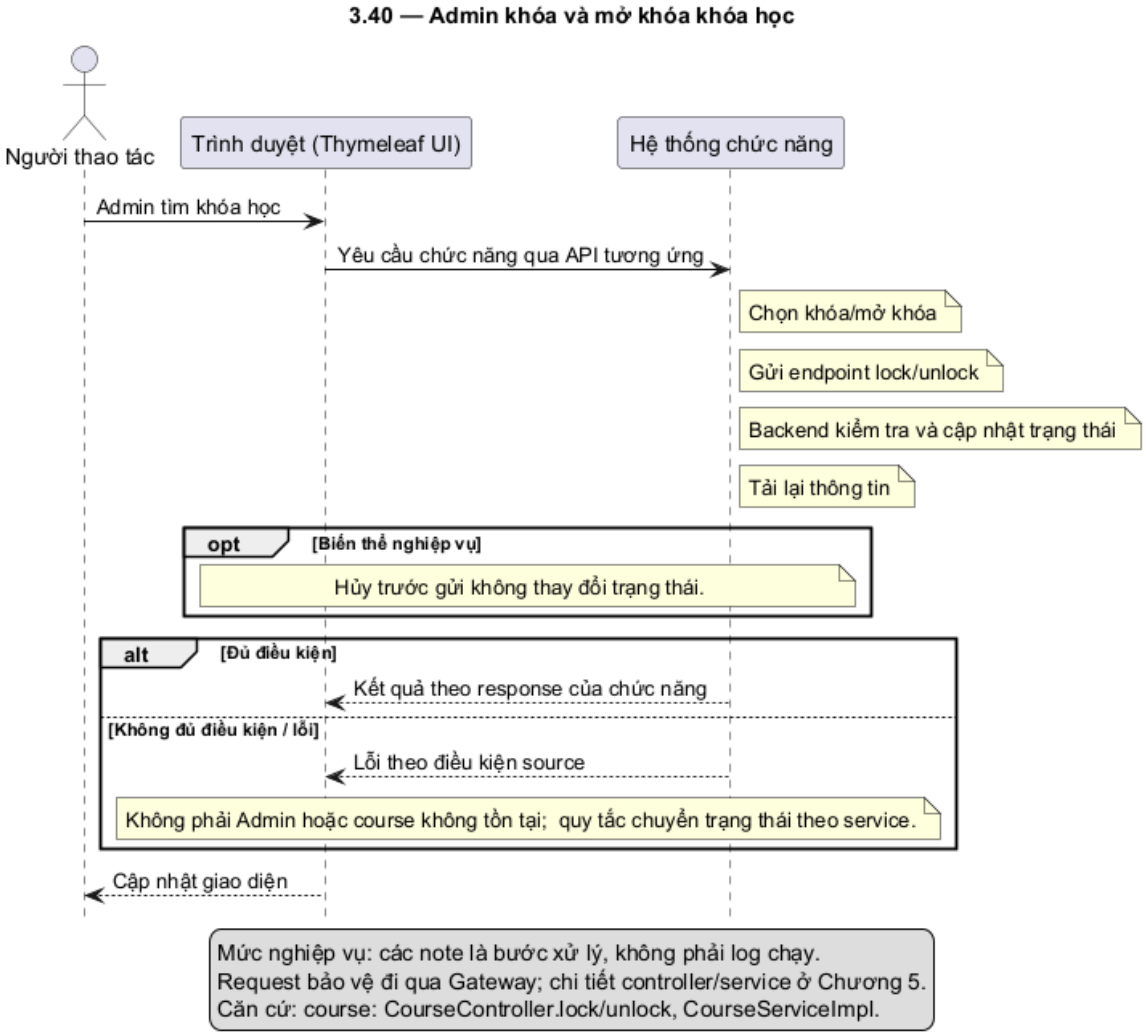

*Hình 3.38: Quản lý bài học và tài nguyên*

• Sequence Diagram mời và quản lý học viên trong khóa học
Nội dung Hình 3.39: Sequence Diagram mời và quản lý học viên trong khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Xem danh sách học viên. 2. Tìm người dùng/mời theo thông tin đầu vào. 3. Service kiểm tra owner và đăng ký hiện có. 4. Tạo enrollment cho người hợp lệ hoặc loại khỏi course. 5. Hiển thị kết quả.
Nhánh thay thế: Danh sách mời có thể chứa người đã tham gia; xử lý theo InviteResultResponse.
Nhánh lỗi: NOT_COURSE_OWNER, COURSE_NOT_FOUND, không tìm thấy user hoặc enrollment; lỗi Identity.
Căn cứ: course: CourseServiceImpl.inviteStudents/getCourseStudents/removeStudent.



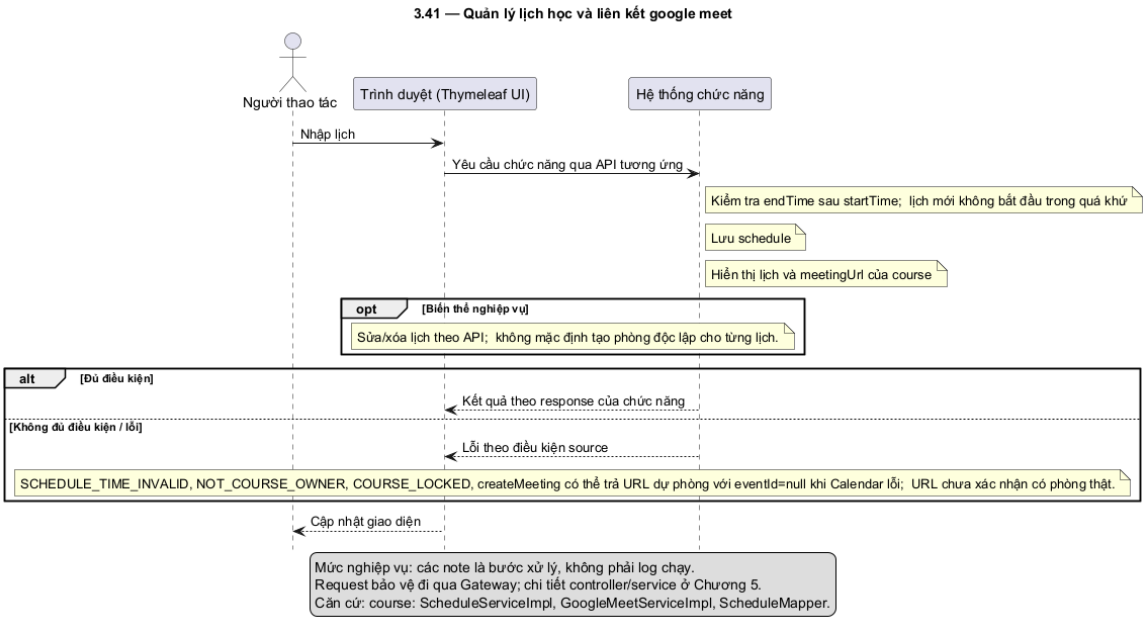

*Hình 3.39: Mời và quản lý học viên trong khóa học*

• Sequence Diagram Admin khóa và mở khóa khóa học
Nội dung Hình 3.40: Sequence Diagram Admin khóa và mở khóa khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Admin tìm khóa học. 2. Chọn khóa/mở khóa. 3. Gửi endpoint lock/unlock. 4. Backend kiểm tra và cập nhật trạng thái. 5. Tải lại thông tin.
Nhánh thay thế: Hủy trước gửi không thay đổi trạng thái.
Nhánh lỗi: Không phải Admin hoặc course không tồn tại; quy tắc chuyển trạng thái theo service.
Căn cứ: course: CourseController.lock/unlock, CourseServiceImpl.



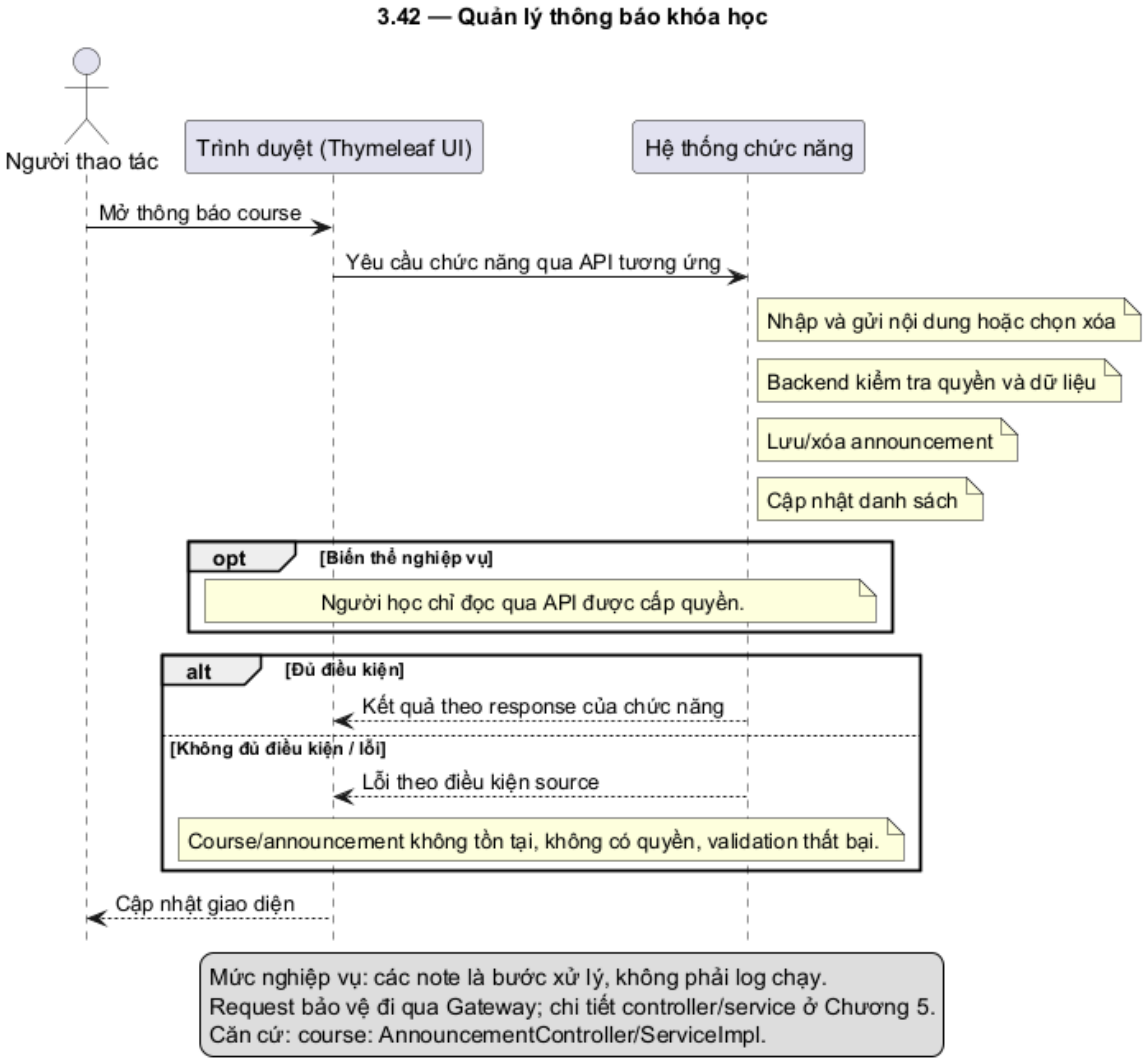

*Hình 3.40: Admin khóa và mở khóa khóa học*

• Sequence Diagram quản lý lịch học và liên kết google meet
Nội dung Hình 3.41: Sequence Diagram quản lý lịch học và liên kết google meet.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Nhập lịch. 2. Kiểm tra endTime sau startTime; lịch mới không bắt đầu trong quá khứ. 3. Lưu schedule. 4. Hiển thị lịch và meetingUrl của course.
Nhánh thay thế: Sửa/xóa lịch theo API; không mặc định tạo phòng độc lập cho từng lịch.
Nhánh lỗi: SCHEDULE_TIME_INVALID, NOT_COURSE_OWNER, COURSE_LOCKED, createMeeting có thể trả URL dự phòng với eventId=null khi Calendar lỗi; URL chưa xác nhận có phòng thật.
Căn cứ: course: ScheduleServiceImpl, GoogleMeetServiceImpl, ScheduleMapper.



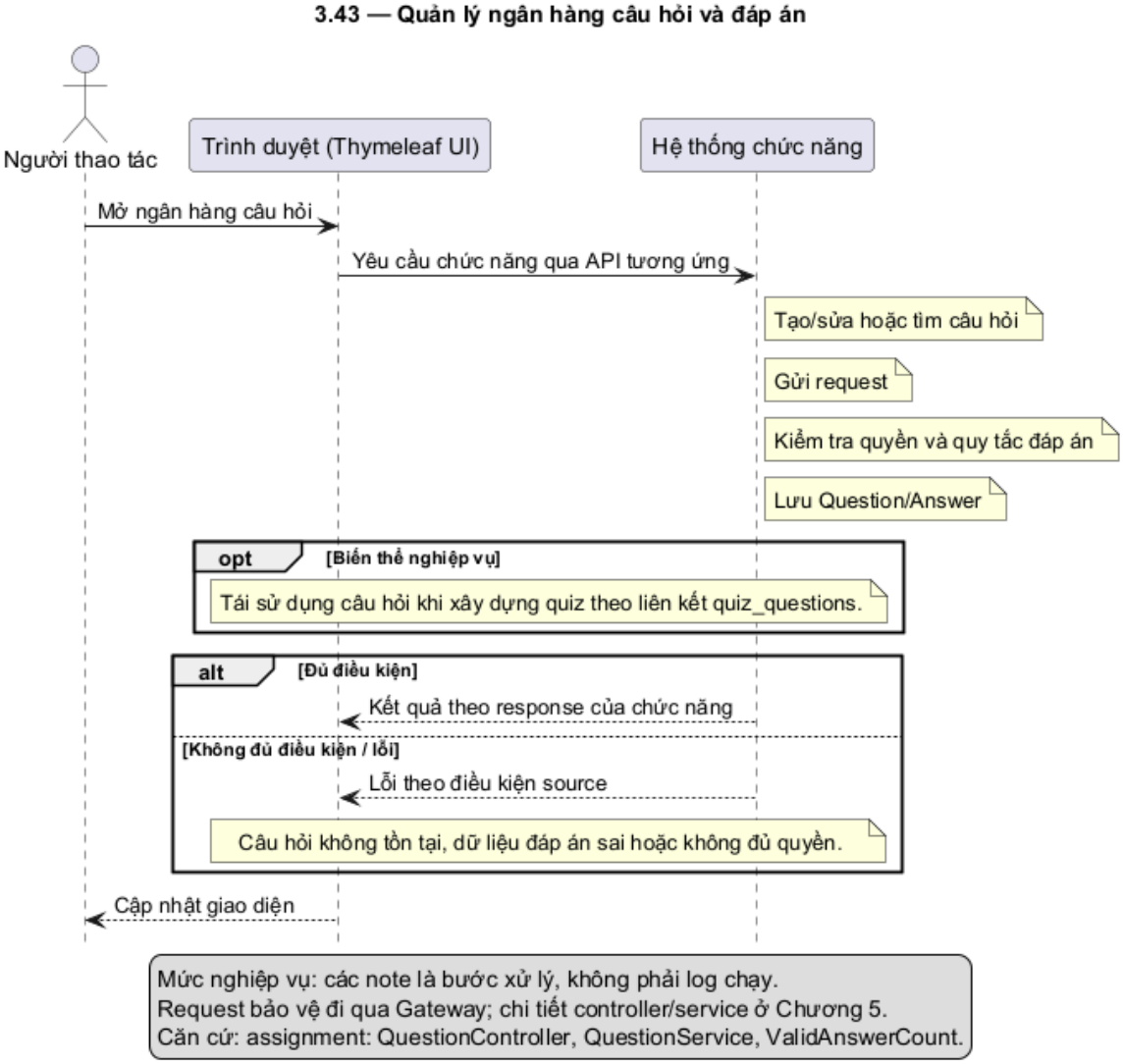

*Hình 3.41: Quản lý lịch học và liên kết google meet*

• Sequence Diagram quản lý thông báo khóa học
Nội dung Hình 3.42: Sequence Diagram quản lý thông báo khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở thông báo course. 2. Nhập và gửi nội dung hoặc chọn xóa. 3. Backend kiểm tra quyền và dữ liệu. 4. Lưu/xóa announcement. 5. Cập nhật danh sách.
Nhánh thay thế: Người học chỉ đọc qua API được cấp quyền.
Nhánh lỗi: Course/announcement không tồn tại, không có quyền, validation thất bại.
Căn cứ: course: AnnouncementController/ServiceImpl.



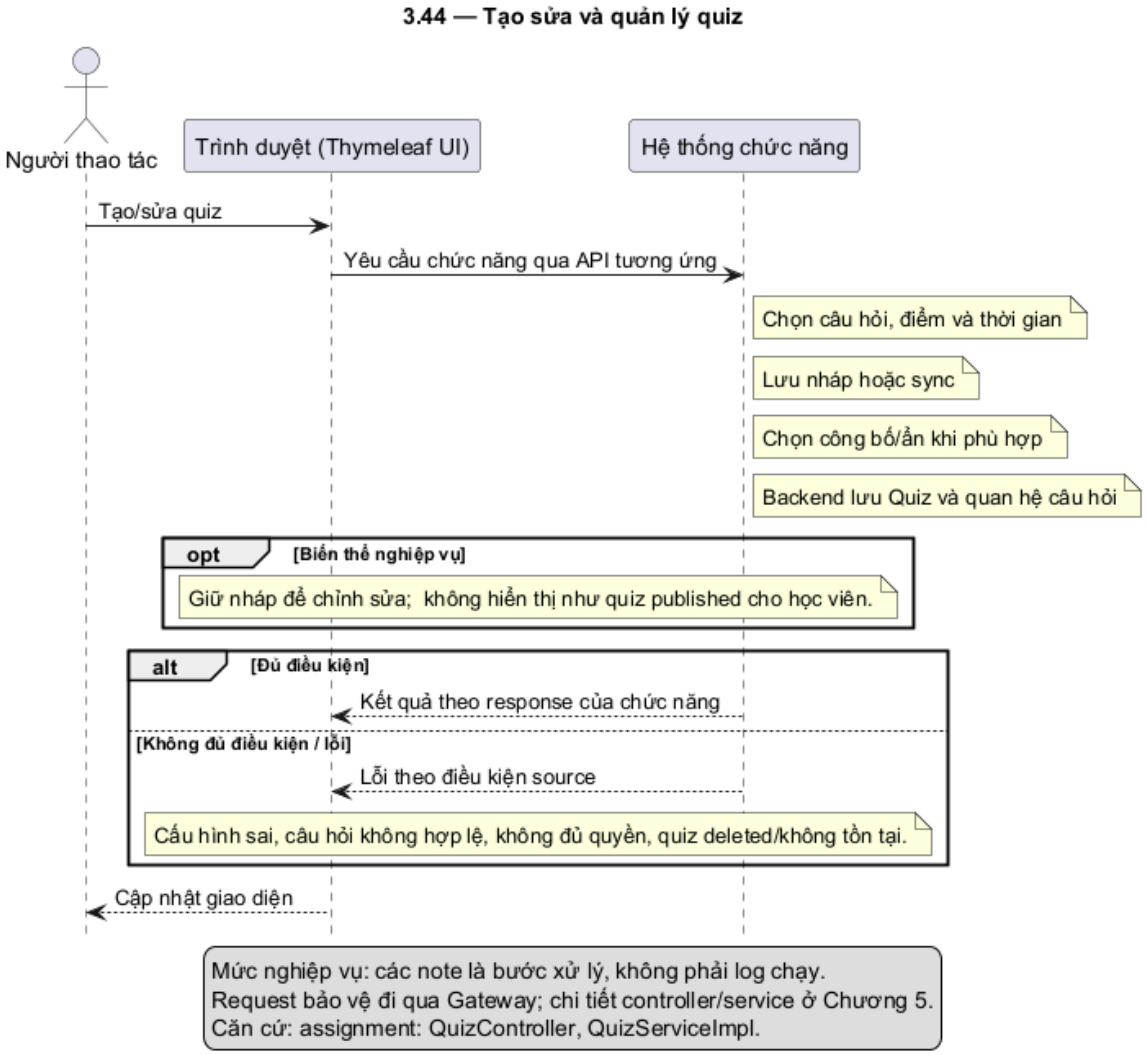

*Hình 3.42: Quản lý thông báo khóa học*

• Sequence Diagram quản lý ngân hàng câu hỏi và đáp án
Nội dung Hình 3.43: Sequence Diagram quản lý ngân hàng câu hỏi và đáp án.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở ngân hàng câu hỏi. 2. Tạo/sửa hoặc tìm câu hỏi. 3. Gửi request. 4. Kiểm tra quyền và quy tắc đáp án. 5. Lưu Question/Answer.
Nhánh thay thế: Tái sử dụng câu hỏi khi xây dựng quiz theo liên kết quiz_questions.
Nhánh lỗi: Câu hỏi không tồn tại, dữ liệu đáp án sai hoặc không đủ quyền.
Căn cứ: assignment: QuestionController, QuestionService, ValidAnswerCount.



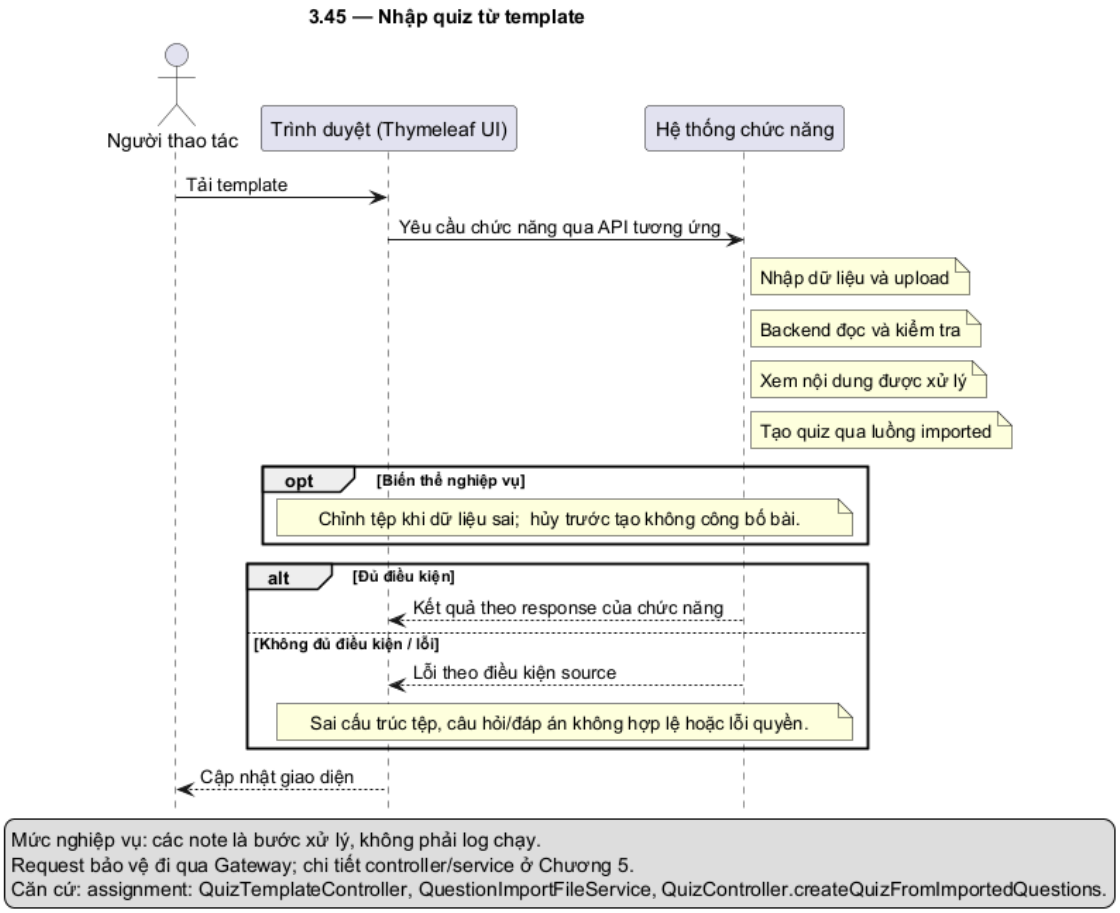

*Hình 3.43: Quản lý ngân hàng câu hỏi và đáp án*

• Sequence Diagram tạo sửa và quản lý quiz
Nội dung Hình 3.44: Sequence Diagram tạo sửa và quản lý quiz.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tạo/sửa quiz. 2. Chọn câu hỏi, điểm và thời gian. 3. Lưu nháp hoặc sync. 4. Chọn công bố/ẩn khi phù hợp. 5. Backend lưu Quiz và quan hệ câu hỏi.
Nhánh thay thế: Giữ nháp để chỉnh sửa; không hiển thị như quiz published cho học viên.
Nhánh lỗi: Cấu hình sai, câu hỏi không hợp lệ, không đủ quyền, quiz deleted/không tồn tại.
Căn cứ: assignment: QuizController, QuizServiceImpl.



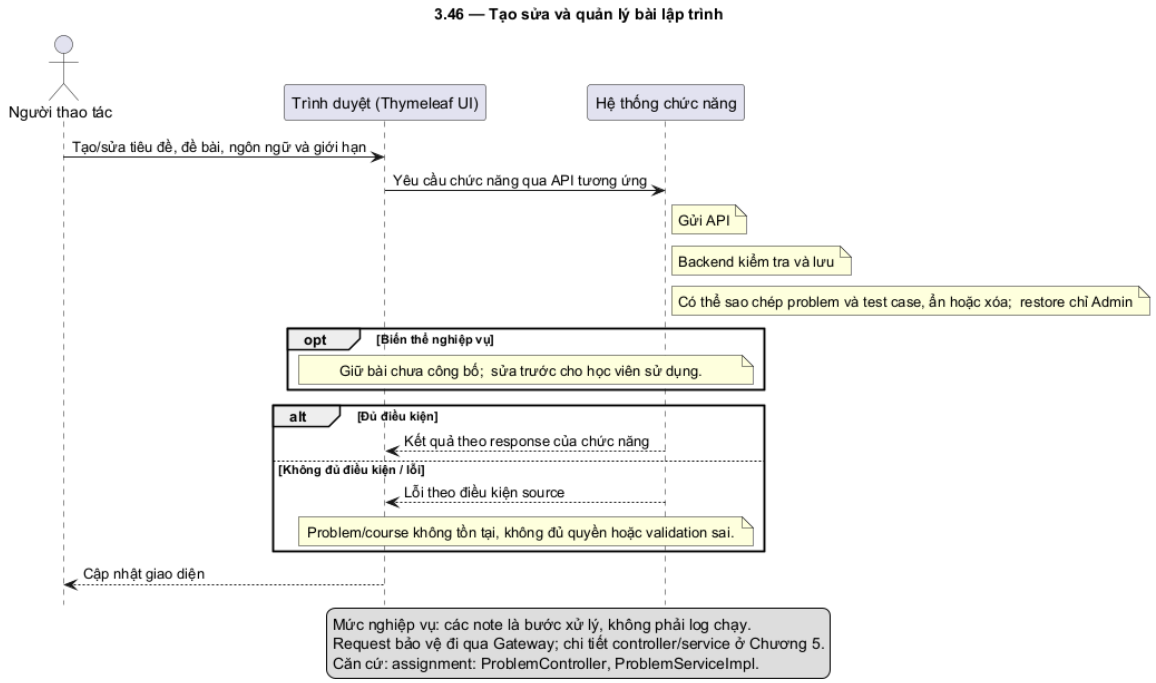

*Hình 3.44: Tạo sửa và quản lý quiz*

• Sequence Diagram nhập quiz từ template
Nội dung Hình 3.45: Sequence Diagram nhập quiz từ template.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tải template. 2. Nhập dữ liệu và upload. 3. Backend đọc và kiểm tra. 4. Xem nội dung được xử lý. 5. Tạo quiz qua luồng imported.
Nhánh thay thế: Chỉnh tệp khi dữ liệu sai; hủy trước tạo không công bố bài.
Nhánh lỗi: Sai cấu trúc tệp, câu hỏi/đáp án không hợp lệ hoặc lỗi quyền.
Căn cứ: assignment: QuizTemplateController, QuestionImportFileService, QuizController.createQuizFromImportedQuestions.




*Hình 3.45: Nhập quiz từ template*

• Sequence Diagram tạo sửa và quản lý bài lập trình
Nội dung Hình 3.46: Sequence Diagram tạo sửa và quản lý bài lập trình.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tạo/sửa tiêu đề, đề bài, ngôn ngữ và giới hạn. 2. Gửi API. 3. Backend kiểm tra và lưu. 4. Có thể sao chép problem và test case, ẩn hoặc xóa; restore chỉ Admin.
Nhánh thay thế: Giữ bài chưa công bố; sửa trước cho học viên sử dụng.
Nhánh lỗi: Problem/course không tồn tại, không đủ quyền hoặc validation sai.
Căn cứ: assignment: ProblemController, ProblemServiceImpl.




*Hình 3.46: Tạo sửa và quản lý bài lập trình*

• Sequence Diagram quản lý test case

Nội dung Hình 3.47: Sequence Diagram quản lý test case.

Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.

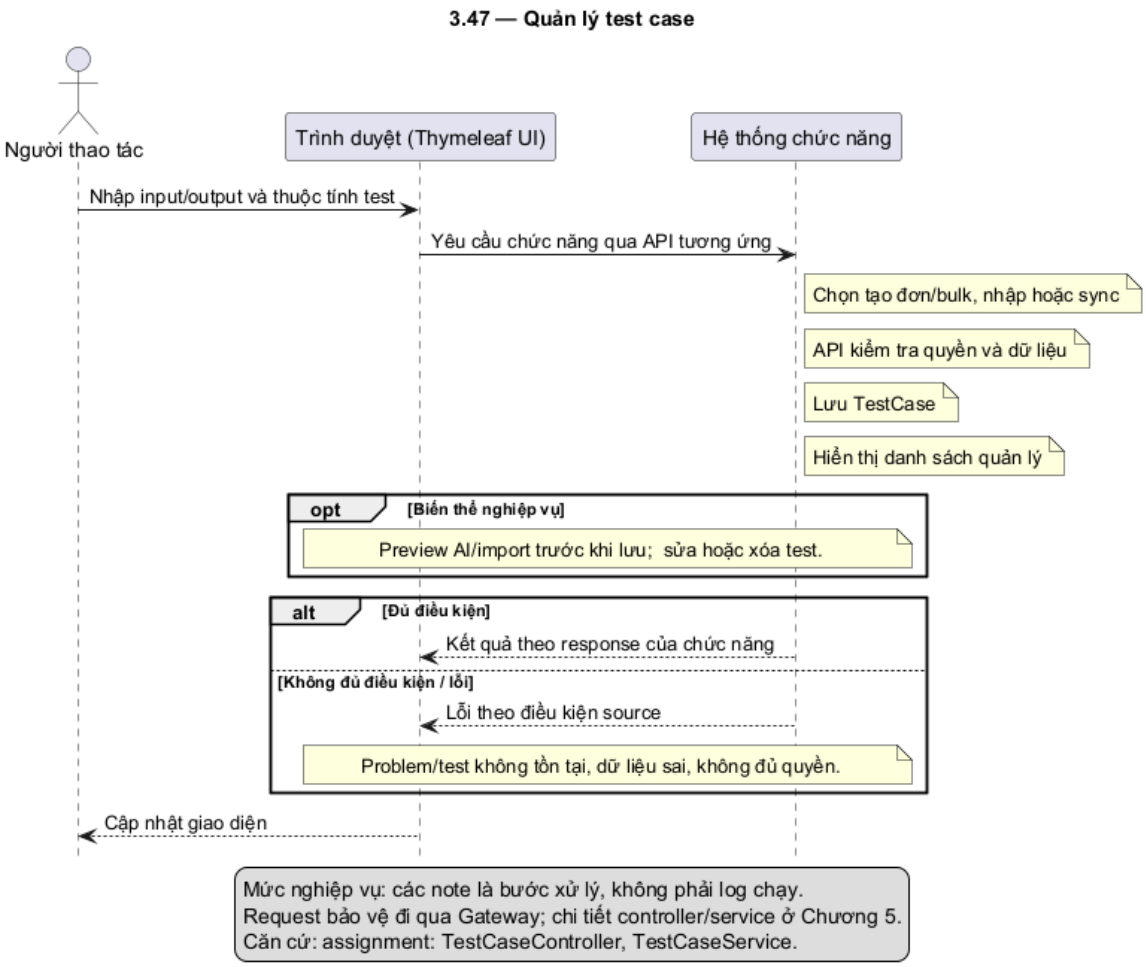
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.

Luồng nghiệp vụ: 1. Nhập input/output và thuộc tính test. 2. Chọn tạo đơn/bulk, nhập hoặc sync. 3. API kiểm tra quyền và dữ liệu. 4. Lưu TestCase. 5. Hiển thị danh sách quản lý.

Nhánh thay thế: Preview AI/import trước khi lưu; sửa hoặc xóa test.

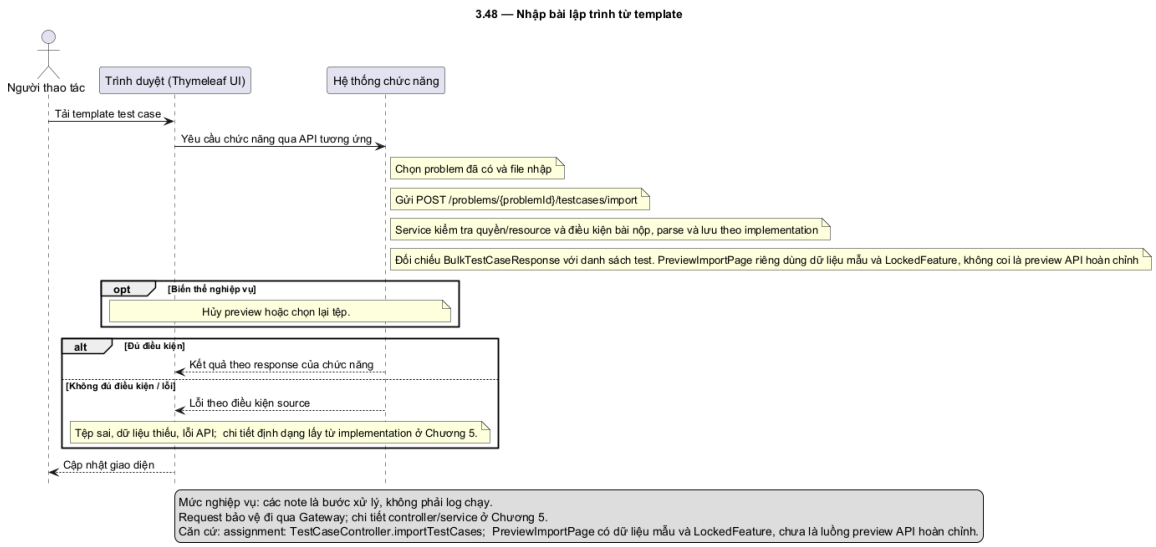
Nhánh lỗi: Problem/test không tồn tại, dữ liệu sai, không đủ quyền.

Căn cứ: assignment: TestCaseController, TestCaseService.



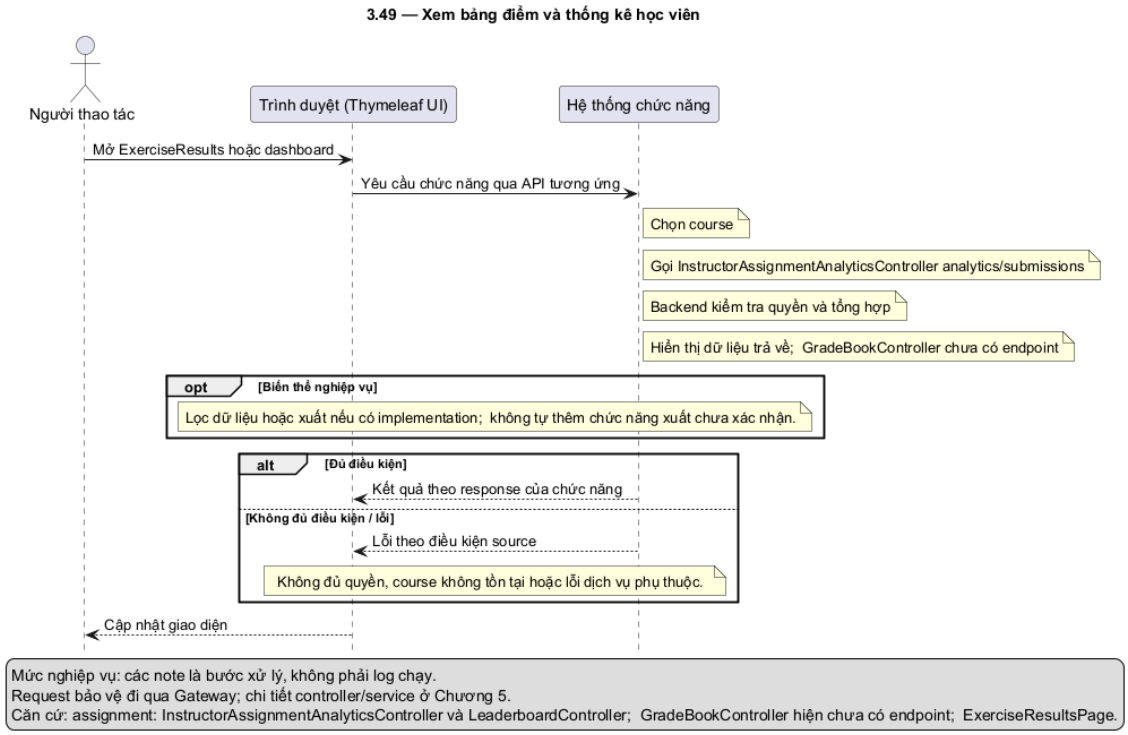

*Hình 3.47: Quản lý test case*

• Sequence Diagram nhập bài lập trình từ template
Nội dung Hình 3.48: Sequence Diagram nhập bài lập trình từ template.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tải template test case. 2. Chọn problem đã có và file nhập. 3. Gửi POST /problems/{problemId}/testcases/import. 4. Service kiểm tra quyền/resource và điều kiện bài nộp, parse và lưu theo implementation. 5. Đối chiếu BulkTestCaseResponse với danh sách test. PreviewImportPage riêng dùng dữ liệu mẫu và LockedFeature, không coi là preview API hoàn chỉnh.
Nhánh thay thế: Hủy preview hoặc chọn lại tệp.
Nhánh lỗi: Tệp sai, dữ liệu thiếu, lỗi API; chi tiết định dạng lấy từ implementation ở Chương 5.
Căn cứ: assignment: TestCaseController.importTestCases; PreviewImportPage có dữ liệu mẫu và LockedFeature, chưa là luồng preview API hoàn chỉnh.




*Hình 3.48: Nhập bài lập trình từ template*

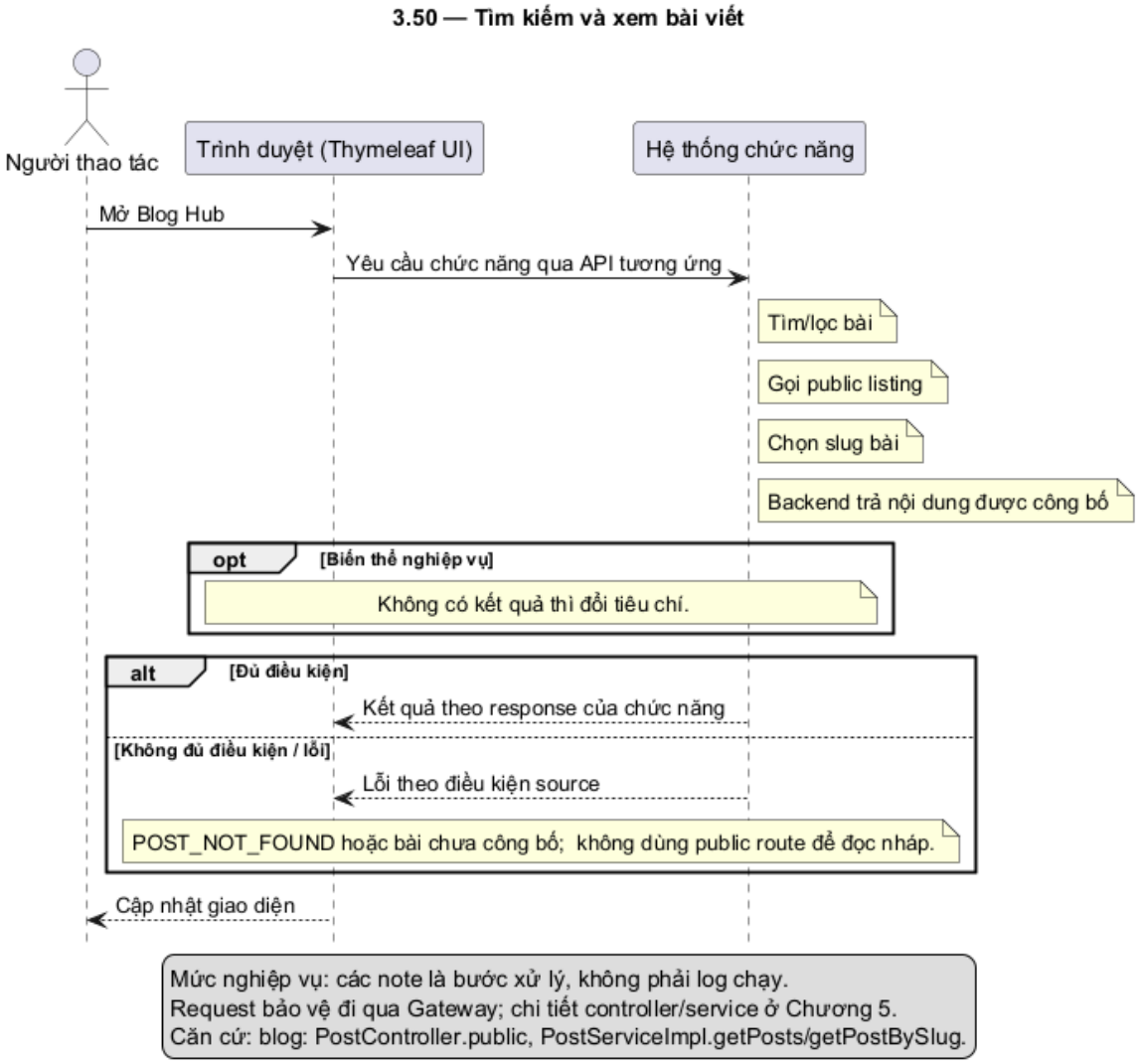
• Sequence Diagram xem bảng điểm và thống kê học viên
Nội dung Hình 3.49: Sequence Diagram xem bảng điểm và thống kê học viên.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở ExerciseResults hoặc dashboard. 2. Chọn course và bộ lọc. 3. Gọi InstructorAssignmentAnalyticsController (analytics/submissions) hoặc API leaderboard tương ứng. 4. Backend kiểm tra quyền và tổng hợp. 5. Hiển thị dữ liệu trả về.
GradeBookController hiện chưa có endpoint, không đưa vào luồng gọi thực tế.
Nhánh thay thế: Lọc dữ liệu hoặc xuất nếu có implementation; không tự thêm chức năng xuất chưa xác nhận.
Nhánh lỗi: Không đủ quyền, course không tồn tại hoặc lỗi dịch vụ phụ thuộc.
Căn cứ: assignment: InstructorAssignmentAnalyticsController và LeaderboardController; GradeBookController hiện chưa có endpoint; ExerciseResultsPage.




*Hình 3.49: Xem bảng điểm và thống kê học viên*

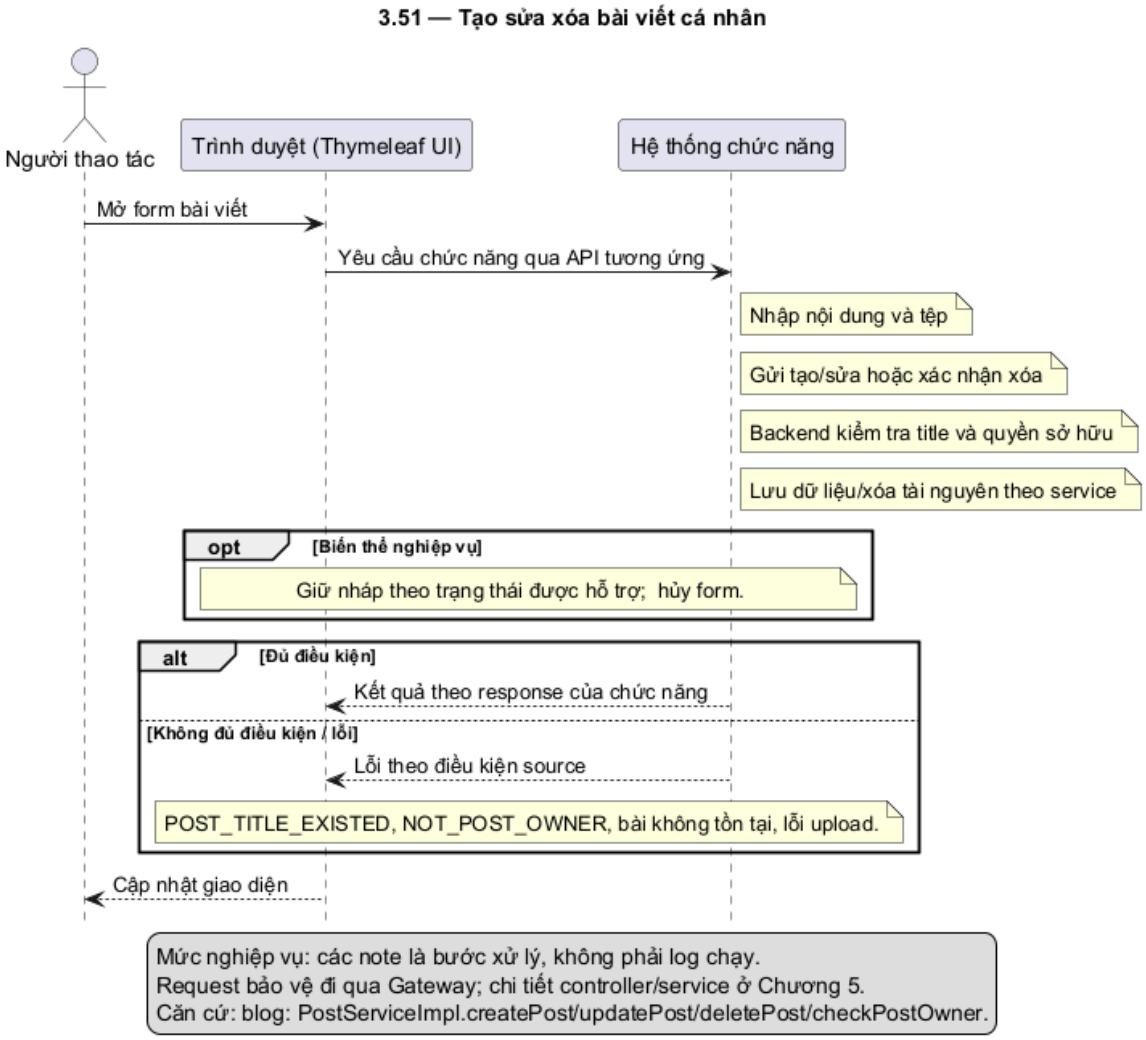
## 3.5.5 Nhóm blog và tương tác

• Sequence Diagram tìm kiếm và xem bài viết
Nội dung Hình 3.50: Sequence Diagram tìm kiếm và xem bài viết.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở Blog Hub. 2. Tìm/lọc bài. 3. Gọi public listing. 4. Chọn slug bài. 5. Backend trả nội dung được công bố.
Nhánh thay thế: Không có kết quả thì đổi tiêu chí.
Nhánh lỗi: POST_NOT_FOUND hoặc bài chưa công bố; không dùng public route để đọc nháp.
Căn cứ: blog: PostController.public, PostServiceImpl.getPosts/getPostBySlug.



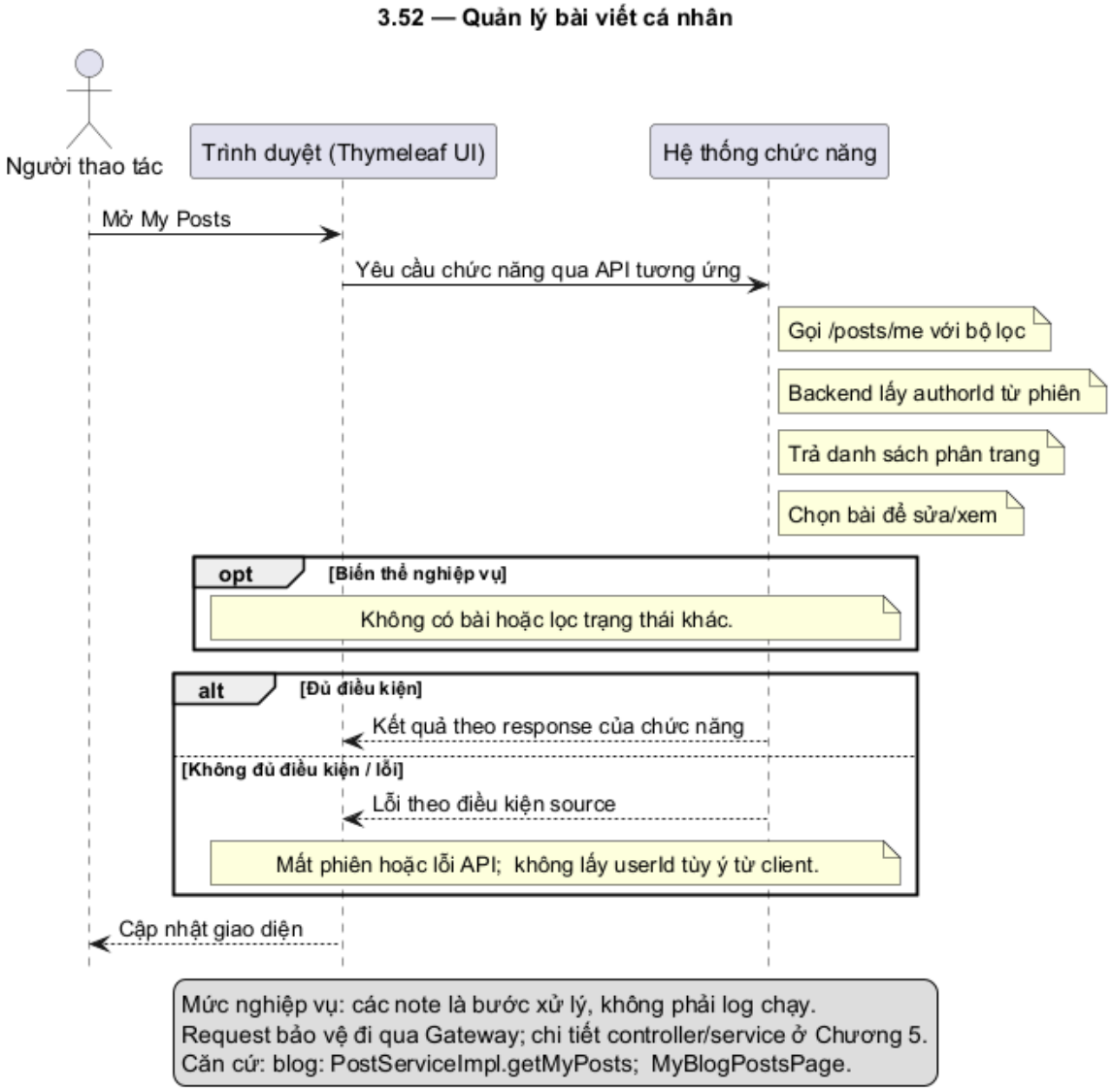

*Hình 3.50: Tìm kiếm và xem bài viết*

• Sequence Diagram tạo sửa xóa bài viết cá nhân
Nội dung Hình 3.51: Sequence Diagram tạo sửa xóa bài viết cá nhân.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái. Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở form bài viết. 2. Nhập nội dung và tệp. 3. Gửi tạo/sửa hoặc xác nhận xóa. 4. Backend kiểm tra title và quyền sở hữu. 5. Lưu dữ liệu/xóa tài nguyên theo service.
Nhánh thay thế: Giữ nháp theo trạng thái được hỗ trợ; hủy form.
Nhánh lỗi: POST_TITLE_EXISTED, NOT_POST_OWNER, bài không tồn tại, lỗi upload.
Căn cứ: blog: PostServiceImpl.createPost/updatePost/deletePost/checkPostOwner.



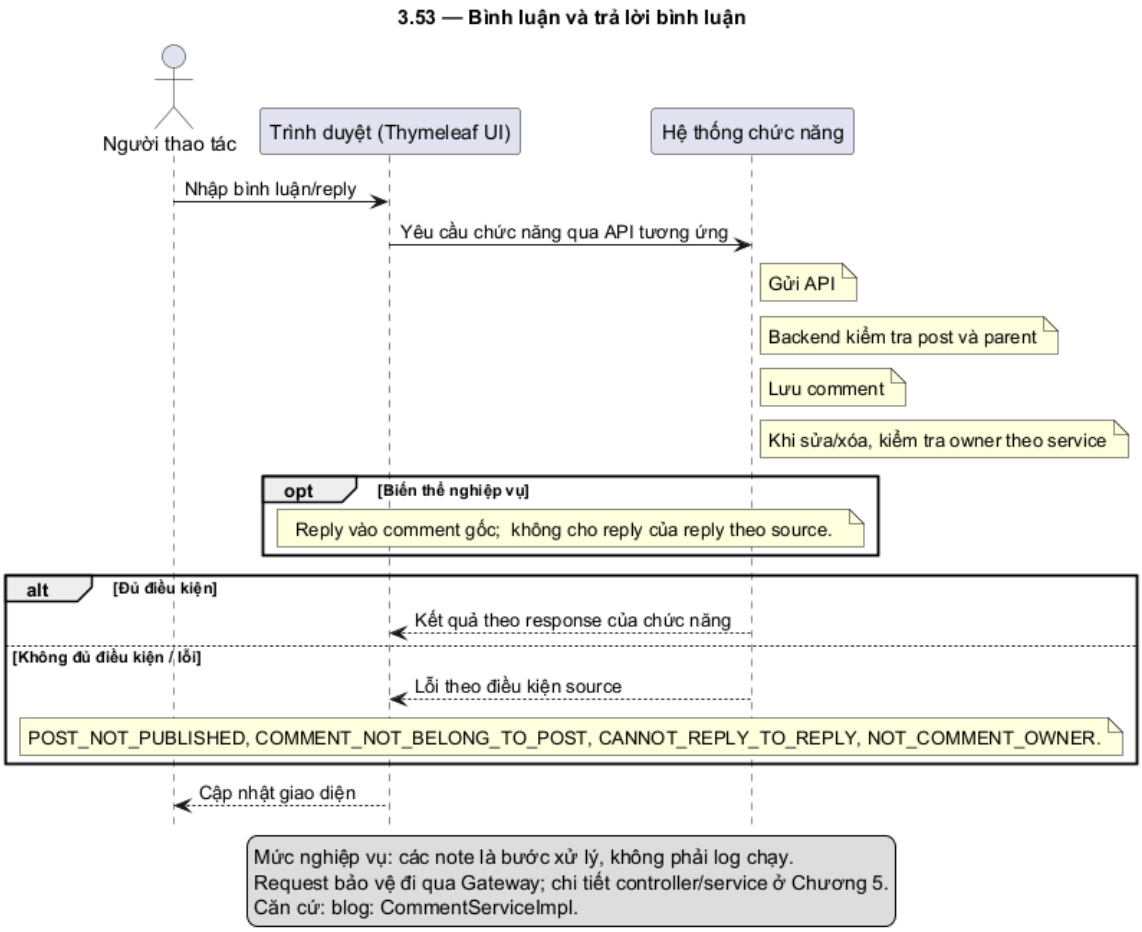

*Hình 3.51: Tạo sửa xóa bài viết cá nhân*

• Sequence Diagram quản lý bài viết cá nhân
Nội dung Hình 3.52: Sequence Diagram quản lý bài viết cá nhân.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở My Posts. 2. Gọi /posts/me với bộ lọc. 3. Backend lấy authorId từ phiên. 4. Trả danh sách phân trang. 5. Chọn bài để sửa/xem.
Nhánh thay thế: Không có bài hoặc lọc trạng thái khác.
Nhánh lỗi: Mất phiên hoặc lỗi API; không lấy userId tùy ý từ client.
Căn cứ: blog: PostServiceImpl.getMyPosts; MyBlogPostsPage.



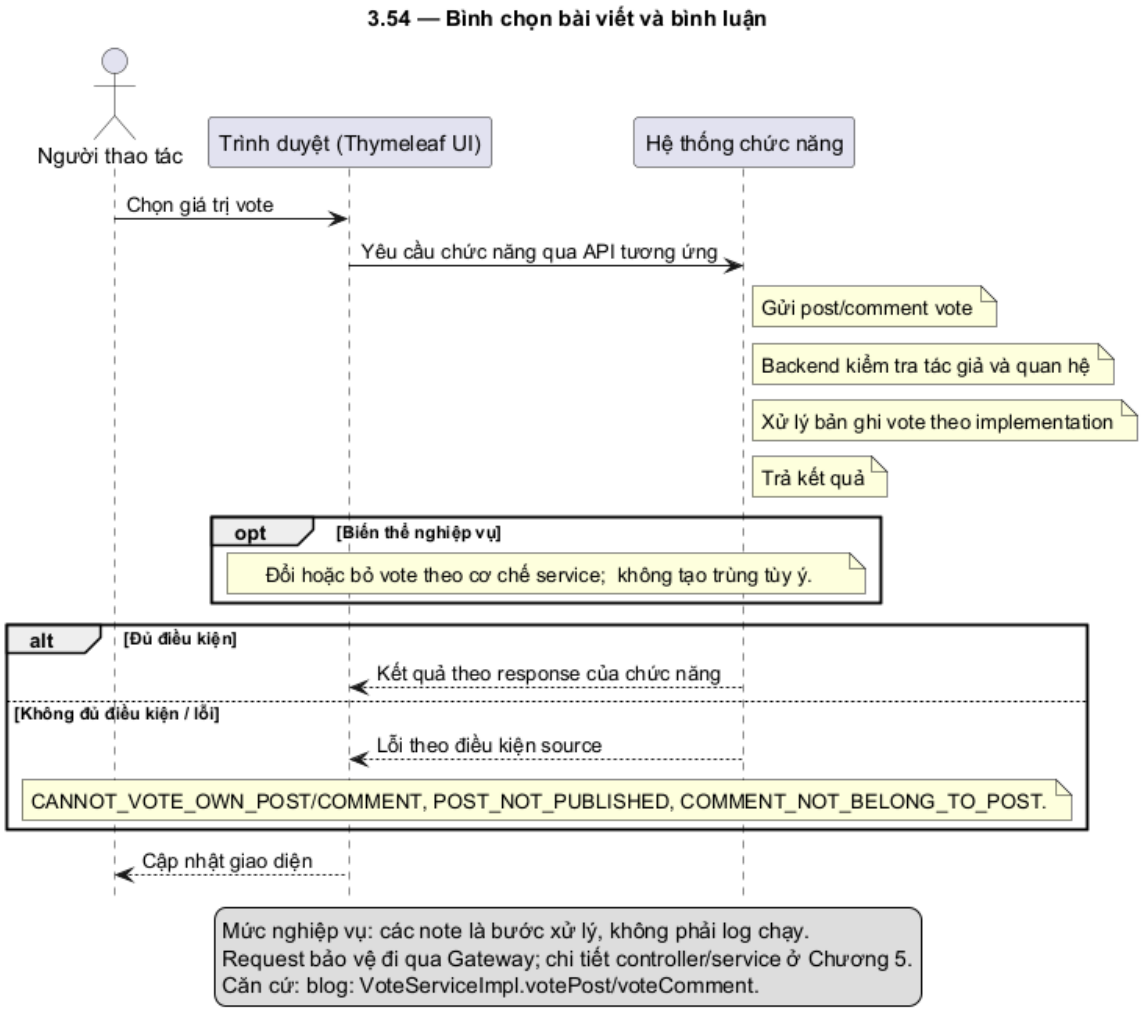

*Hình 3.52: Quản lý bài viết cá nhân*

• Sequence Diagram bình luận và trả lời bình luận
Nội dung Hình 3.53: Sequence Diagram bình luận và trả lời bình luận.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Nhập bình luận/reply. 2. Gửi API. 3. Backend kiểm tra post và parent. 4. Lưu comment. 5. Khi sửa/xóa, kiểm tra owner theo service.
Nhánh thay thế: Reply vào comment gốc; không cho reply của reply theo source.
Nhánh lỗi: POST_NOT_PUBLISHED, COMMENT_NOT_BELONG_TO_POST, CANNOT_REPLY_TO_REPLY, NOT_COMMENT_OWNER.
Căn cứ: blog: CommentServiceImpl.



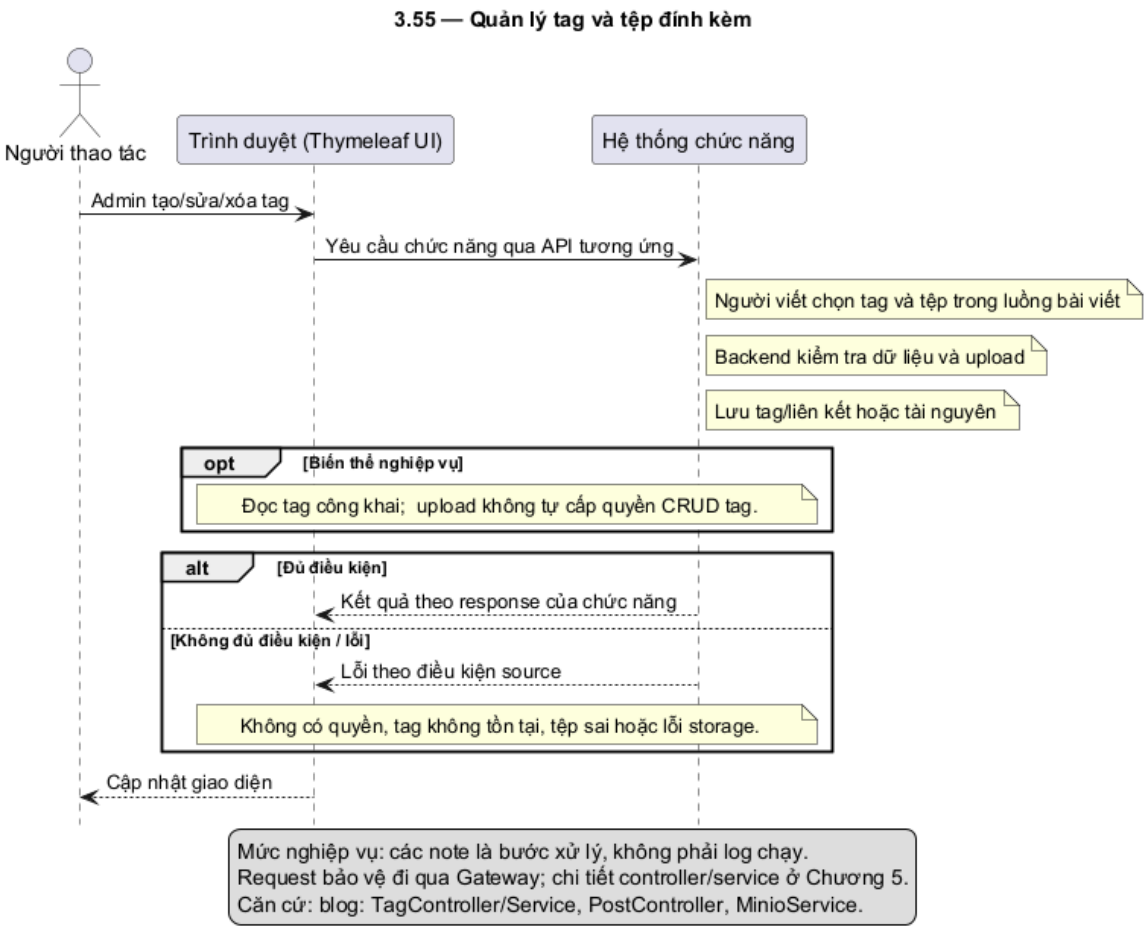

*Hình 3.53: Bình luận và trả lời bình luận*

• Sequence Diagram bình chọn bài viết và bình luận
Nội dung Hình 3.54: Sequence Diagram bình chọn bài viết và bình luận.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn giá trị vote. 2. Gửi post/comment vote. 3. Backend kiểm tra tác giả và quan hệ. 4. Xử lý bản ghi vote theo implementation. 5. Trả kết quả.
Nhánh thay thế: Đổi hoặc bỏ vote theo cơ chế service; không tạo trùng tùy ý.
Nhánh lỗi: CANNOT_VOTE_OWN_POST/COMMENT, POST_NOT_PUBLISHED, COMMENT_NOT_BELONG_TO_POST.
Căn cứ: blog: VoteServiceImpl.votePost/voteComment.




*Hình 3.54: Bình chọn bài viết và bình luận*

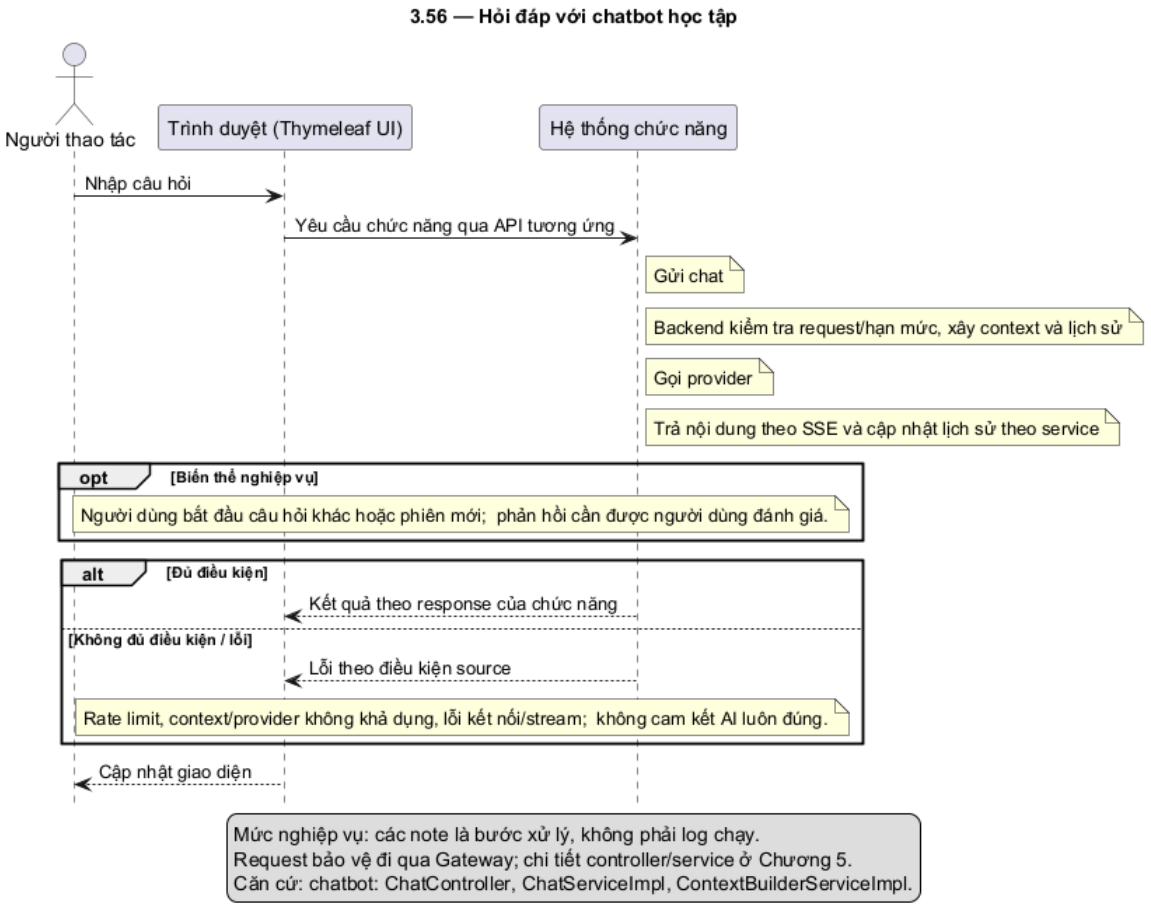
• Sequence Diagram quản lý tag và tệp đính kèm
Nội dung Hình 3.55: Sequence Diagram quản lý tag và tệp đính kèm.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Admin tạo/sửa/xóa tag. 2. Người viết chọn tag và tệp trong luồng bài viết. 3. Backend kiểm tra dữ liệu và upload. 4. Lưu tag/liên kết hoặc tài nguyên.
Nhánh thay thế: Đọc tag công khai; upload không tự cấp quyền CRUD tag.
Nhánh lỗi: Không có quyền, tag không tồn tại, tệp sai hoặc lỗi storage.
Căn cứ: blog: TagController/Service, PostController, MinioService.




*Hình 3.55: Quản lý tag và tệp đính kèm*

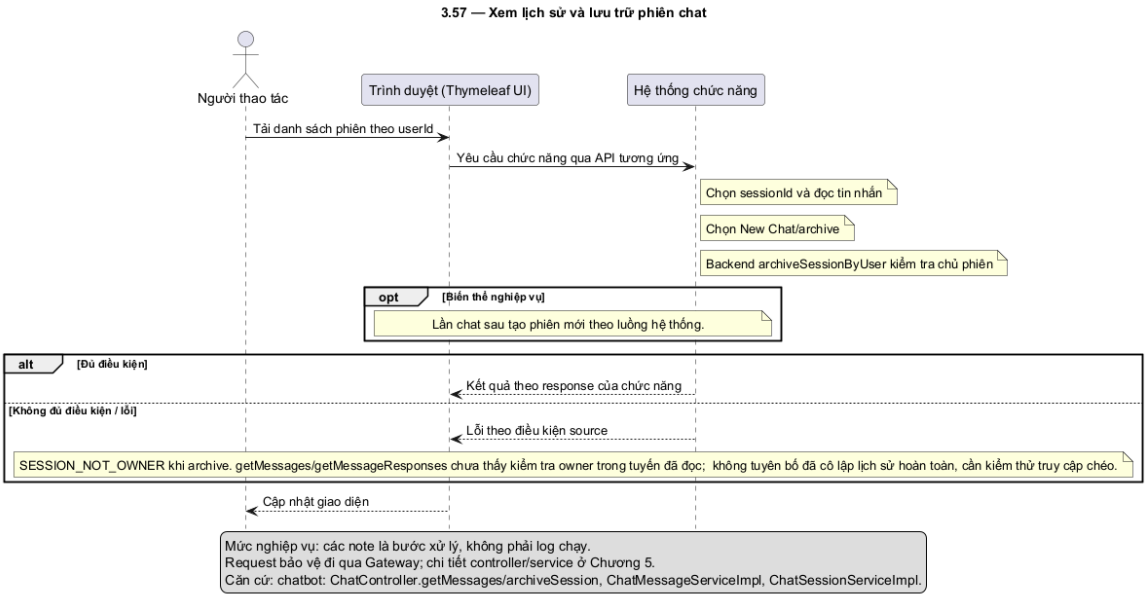
## 3.5.6 Nhóm chatbot và hỗ trợ ai

• Sequence Diagram hỏi đáp với chatbot học tập
Nội dung Hình 3.56: Sequence Diagram hỏi đáp với chatbot học tập.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thể hiện request chat, kiểm tra hạn mức, xây context/lịch sử và gọi provider; phản hồi đi qua kênh SSE tới frontend thành nhiều phần. Không vẽ WebSocket hoặc coi từng phần stream là một request REST mới.
Luồng nghiệp vụ: 1. Nhập câu hỏi. 2. Gửi chat. 3. Backend kiểm tra request/hạn mức, xây context và lịch sử. 4. Gọi provider. 5. Trả nội dung theo SSE và cập nhật lịch sử theo service.
Nhánh thay thế: Người dùng bắt đầu câu hỏi khác hoặc phiên mới; phản hồi cần được người dùng đánh giá.
Nhánh lỗi: Rate limit, context/provider không khả dụng, lỗi kết nối/stream; không cam kết AI luôn đúng.
Căn cứ: chatbot: ChatController, ChatServiceImpl, ContextBuilderServiceImpl.



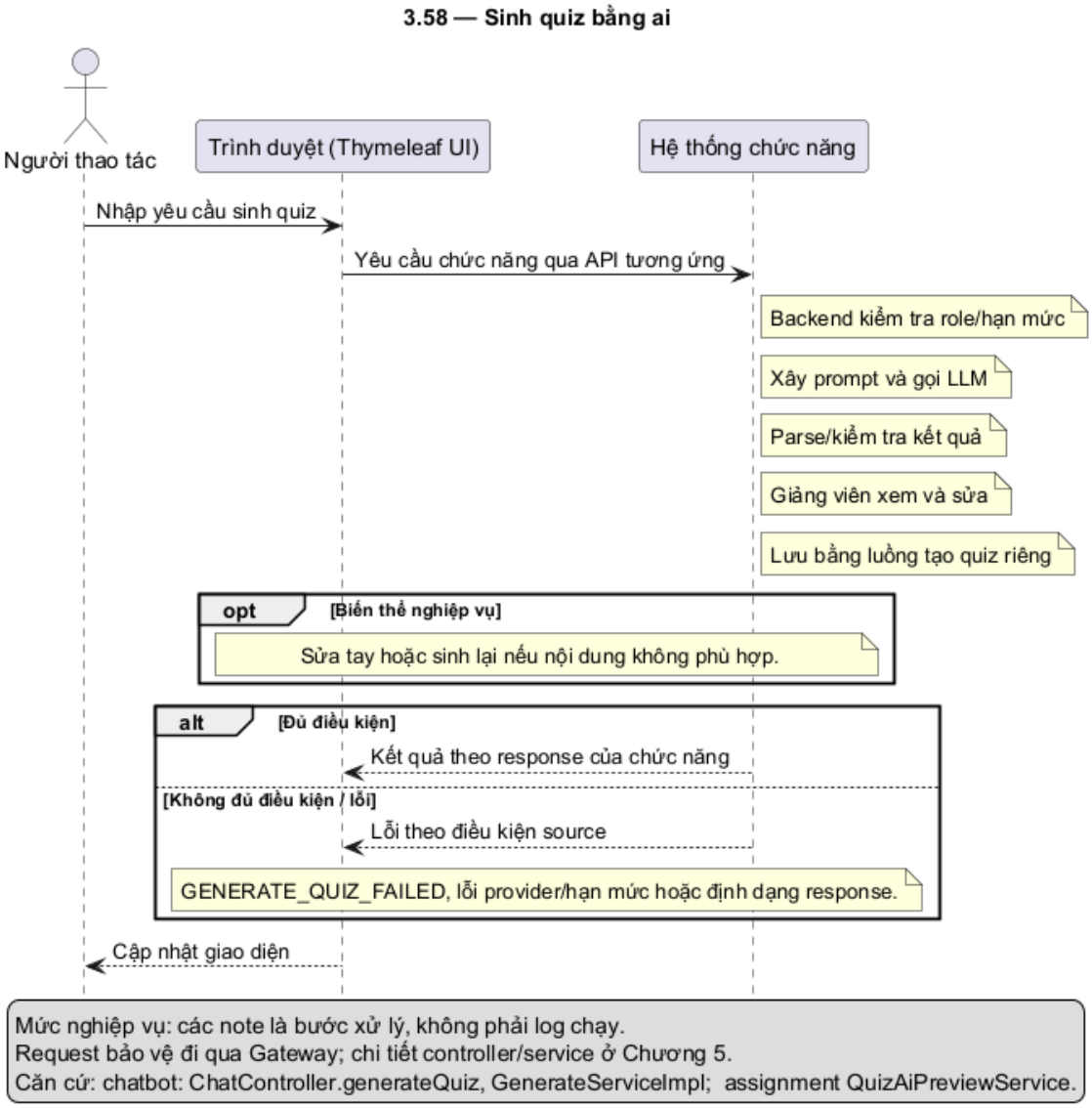

*Hình 3.56: Hỏi đáp với chatbot học tập*

• Sequence Diagram xem lịch sử và lưu trữ phiên chat
Nội dung Hình 3.57: Sequence Diagram xem lịch sử và lưu trữ phiên chat.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tải danh sách phiên theo userId. 2. Chọn sessionId và đọc tin nhắn. 3. Chọn New Chat/archive. 4. Backend archiveSessionByUser kiểm tra chủ phiên.
Nhánh thay thế: Lần chat sau tạo phiên mới theo luồng hệ thống.
Nhánh lỗi: SESSION_NOT_OWNER khi archive. getMessages/getMessageResponses chưa thấy kiểm tra owner trong tuyến đã đọc; không tuyên bố đã cô lập lịch sử hoàn toàn, cần kiểm thử truy cập chéo.
Căn cứ: chatbot: ChatController.getMessages/archiveSession, ChatMessageServiceImpl, ChatSessionServiceImpl.




*Hình 3.57: Xem lịch sử và lưu trữ phiên chat*

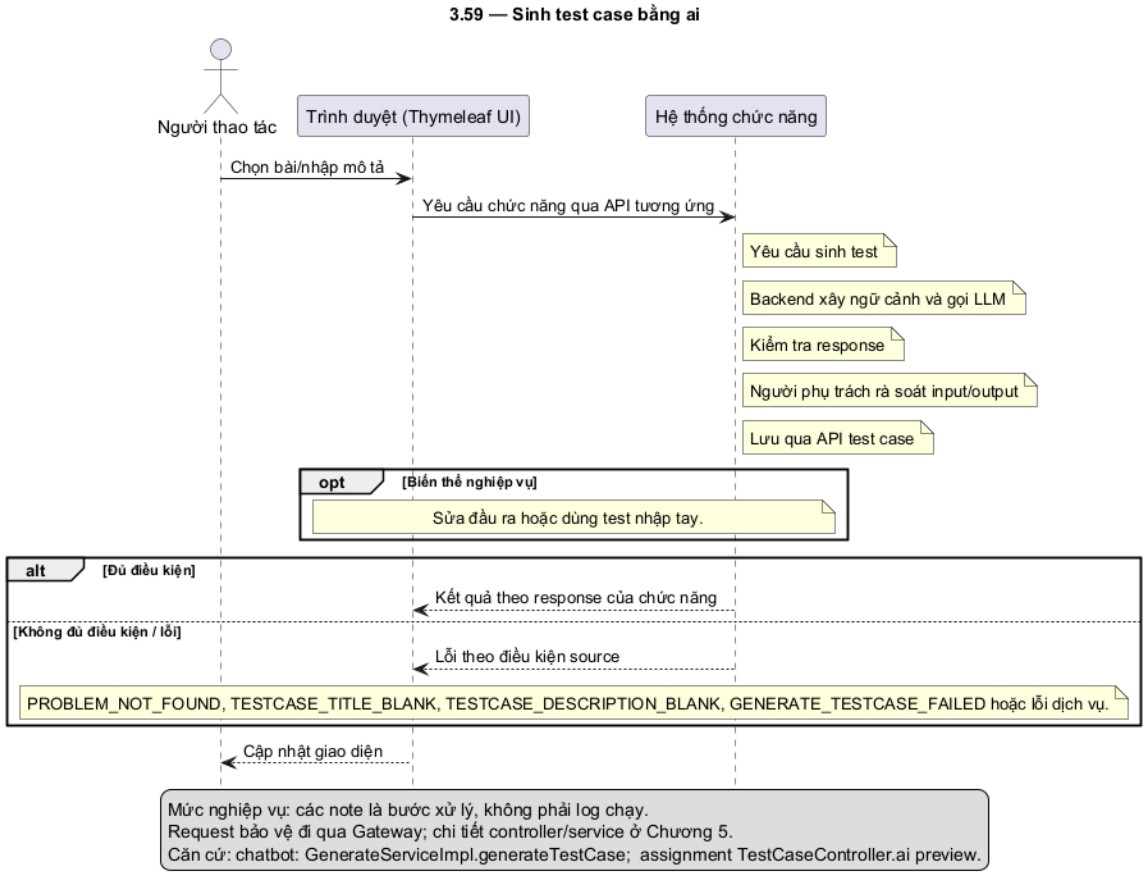
• Sequence Diagram sinh quiz bằng ai
Nội dung Hình 3.58: Sequence Diagram sinh quiz bằng ai.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Nhập yêu cầu sinh quiz. 2. Backend kiểm tra role/hạn mức. 3. Xây prompt và gọi LLM. 4. Parse/kiểm tra kết quả. 5. Giảng viên xem và sửa. 6. Lưu bằng luồng tạo quiz riêng.
Nhánh thay thế: Sửa tay hoặc sinh lại nếu nội dung không phù hợp.
Nhánh lỗi: GENERATE_QUIZ_FAILED, lỗi provider/hạn mức hoặc định dạng response.
Căn cứ: chatbot: ChatController.generateQuiz, GenerateServiceImpl; assignment QuizAiPreviewService.




*Hình 3.58: Sinh quiz bằng ai*

• Sequence Diagram sinh test case bằng ai
Nội dung Hình 3.59: Sequence Diagram sinh test case bằng ai.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Chọn bài/nhập mô tả. 2. Yêu cầu sinh test. 3. Backend xây ngữ cảnh và gọi LLM. 4. Kiểm tra response. 5. Người phụ trách rà soát input/output. 6. Lưu qua API test case.
Nhánh thay thế: Sửa đầu ra hoặc dùng test nhập tay.
Nhánh lỗi: PROBLEM_NOT_FOUND, TESTCASE_TITLE_BLANK, TESTCASE_DESCRIPTION_BLANK, GENERATE_TESTCASE_FAILED hoặc lỗi dịch vụ.

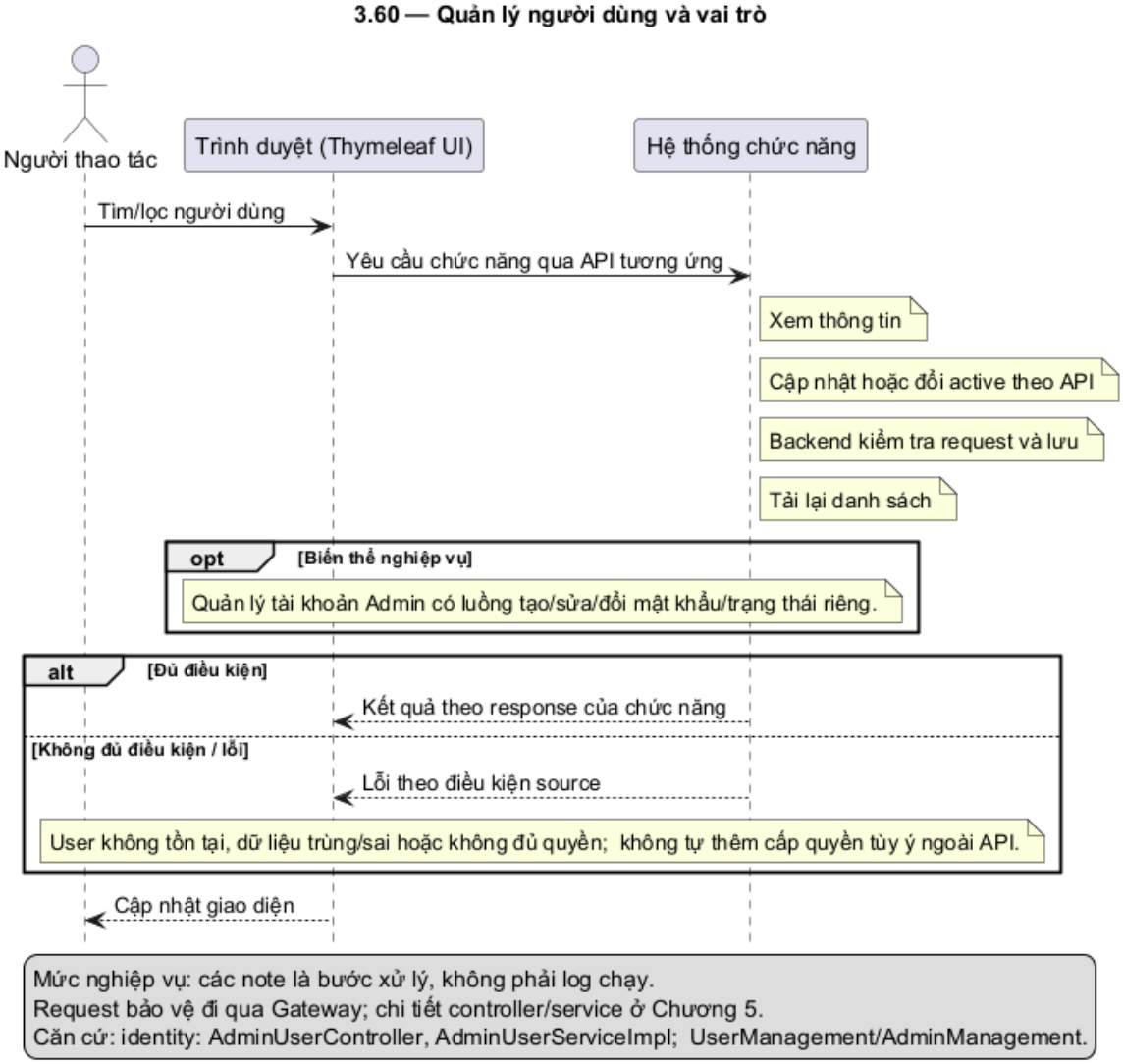
Căn cứ: chatbot: GenerateServiceImpl.generateTestCase; assignment TestCaseController.ai preview.




*Hình 3.59: Sinh test case bằng ai*

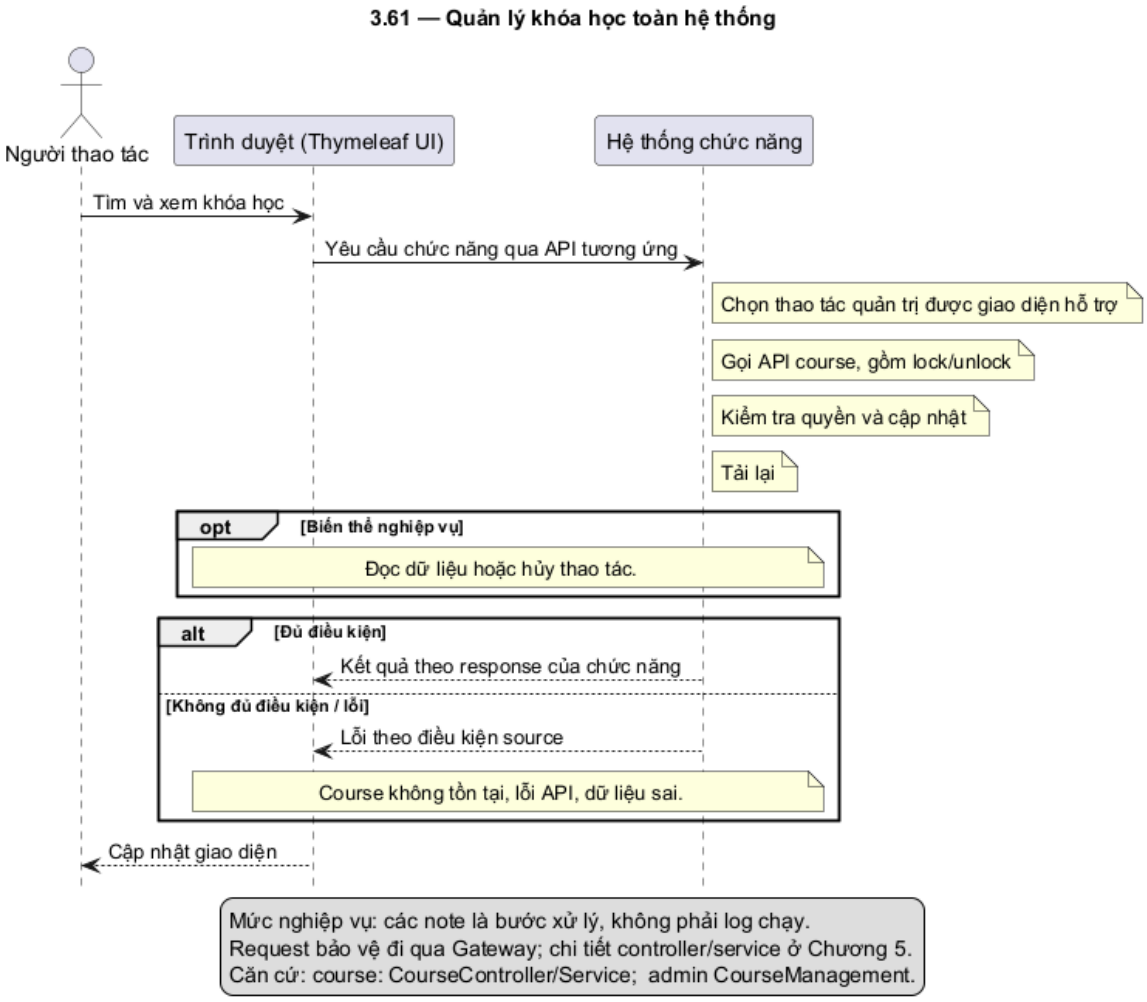
## 3.5.7 Nhóm quản trị hệ thống

• Sequence Diagram quản lý người dùng và vai trò
Nội dung Hình 3.60: Sequence Diagram quản lý người dùng và vai trò.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tìm/lọc người dùng. 2. Xem thông tin. 3. Cập nhật hoặc đổi active theo API. 4. Backend kiểm tra request và lưu. 5. Tải lại danh sách.
Nhánh thay thế: Quản lý tài khoản Admin có luồng tạo/sửa/đổi mật khẩu/trạng thái riêng.
Nhánh lỗi: User không tồn tại, dữ liệu trùng/sai hoặc không đủ quyền; không tự thêm cấp quyền tùy ý ngoài API.
Căn cứ: identity: AdminUserController, AdminUserServiceImpl; UserManagement/AdminManagement.



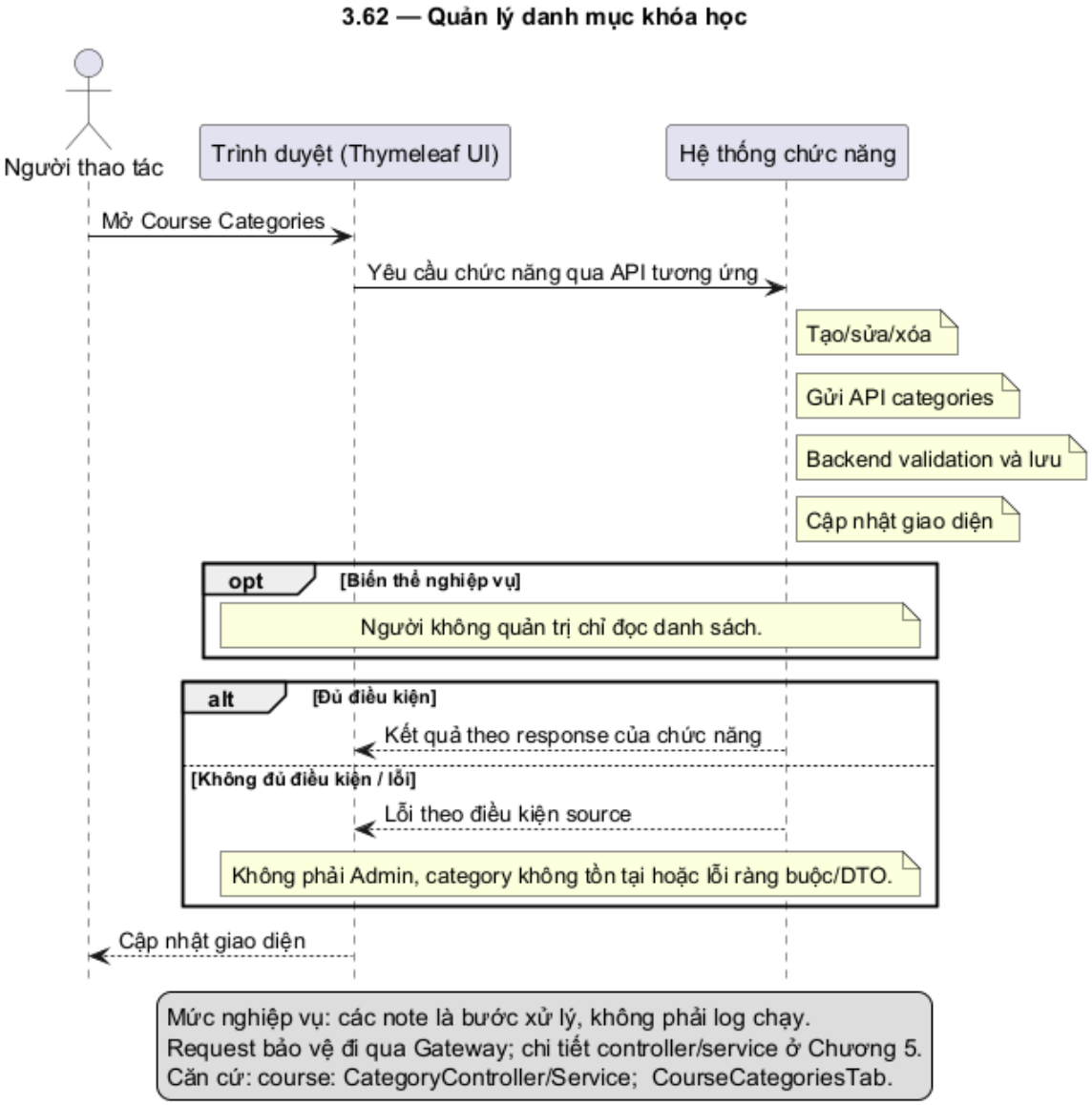

*Hình 3.60: Quản lý người dùng và vai trò*

• Sequence Diagram quản lý khóa học toàn hệ thống
Nội dung Hình 3.61: Sequence Diagram quản lý khóa học toàn hệ thống.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Tìm và xem khóa học. 2. Chọn thao tác quản trị được giao diện hỗ trợ. 3. Gọi API course, gồm lock/unlock. 4. Kiểm tra quyền và cập nhật. 5. Tải lại.
Nhánh thay thế: Đọc dữ liệu hoặc hủy thao tác.
Nhánh lỗi: Course không tồn tại, lỗi API, dữ liệu sai.
Căn cứ: course: CourseController/Service; admin CourseManagement.




*Hình 3.61: Quản lý khóa học toàn hệ thống*

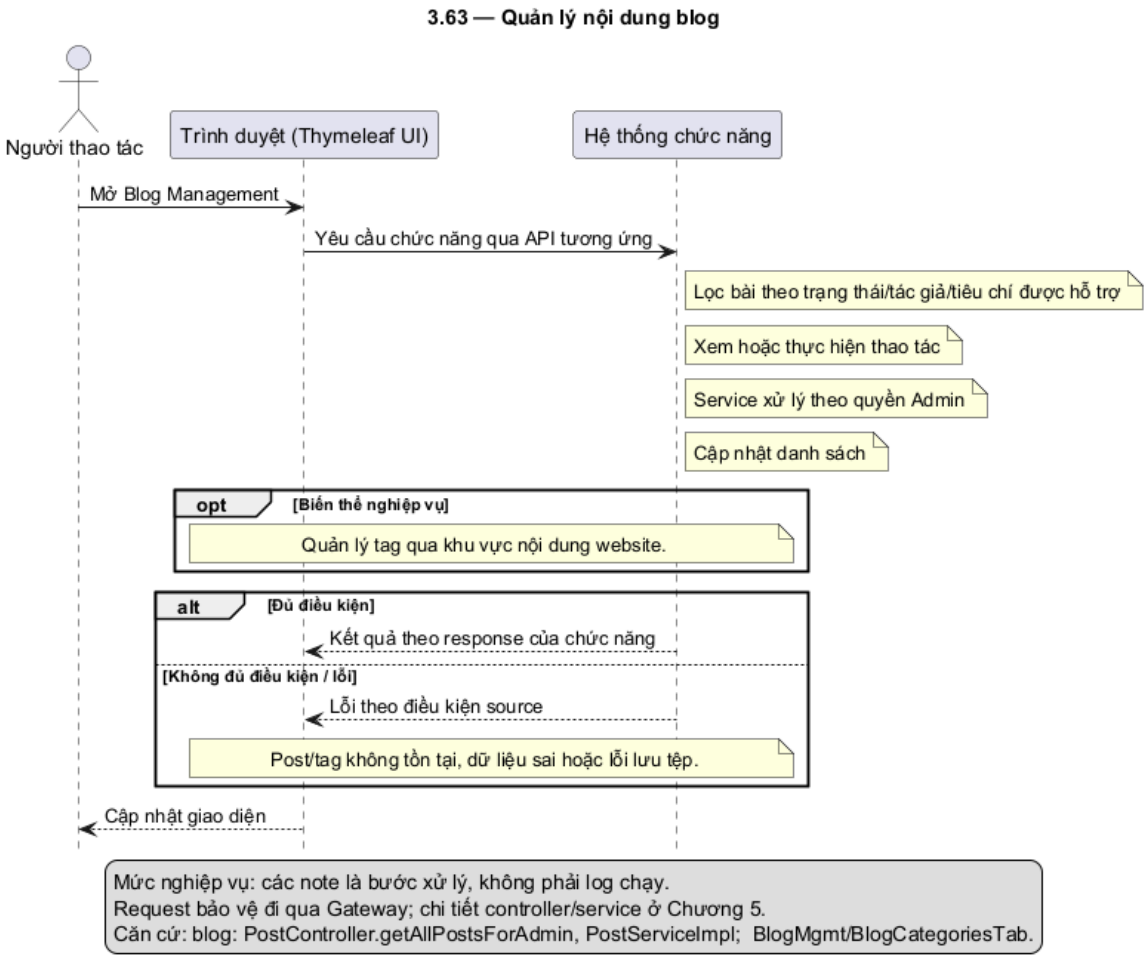
• Sequence Diagram quản lý danh mục khóa học
Nội dung Hình 3.62: Sequence Diagram quản lý danh mục khóa học.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở Course Categories. 2. Tạo/sửa/xóa. 3. Gửi API categories. 4. Backend validation và lưu. 5. Cập nhật giao diện.
Nhánh thay thế: Người không quản trị chỉ đọc danh sách.
Nhánh lỗi: Không phải Admin, category không tồn tại hoặc lỗi ràng buộc/DTO.
Căn cứ: course: CategoryController/Service; CourseCategoriesTab.




*Hình 3.62: Quản lý danh mục khóa học*

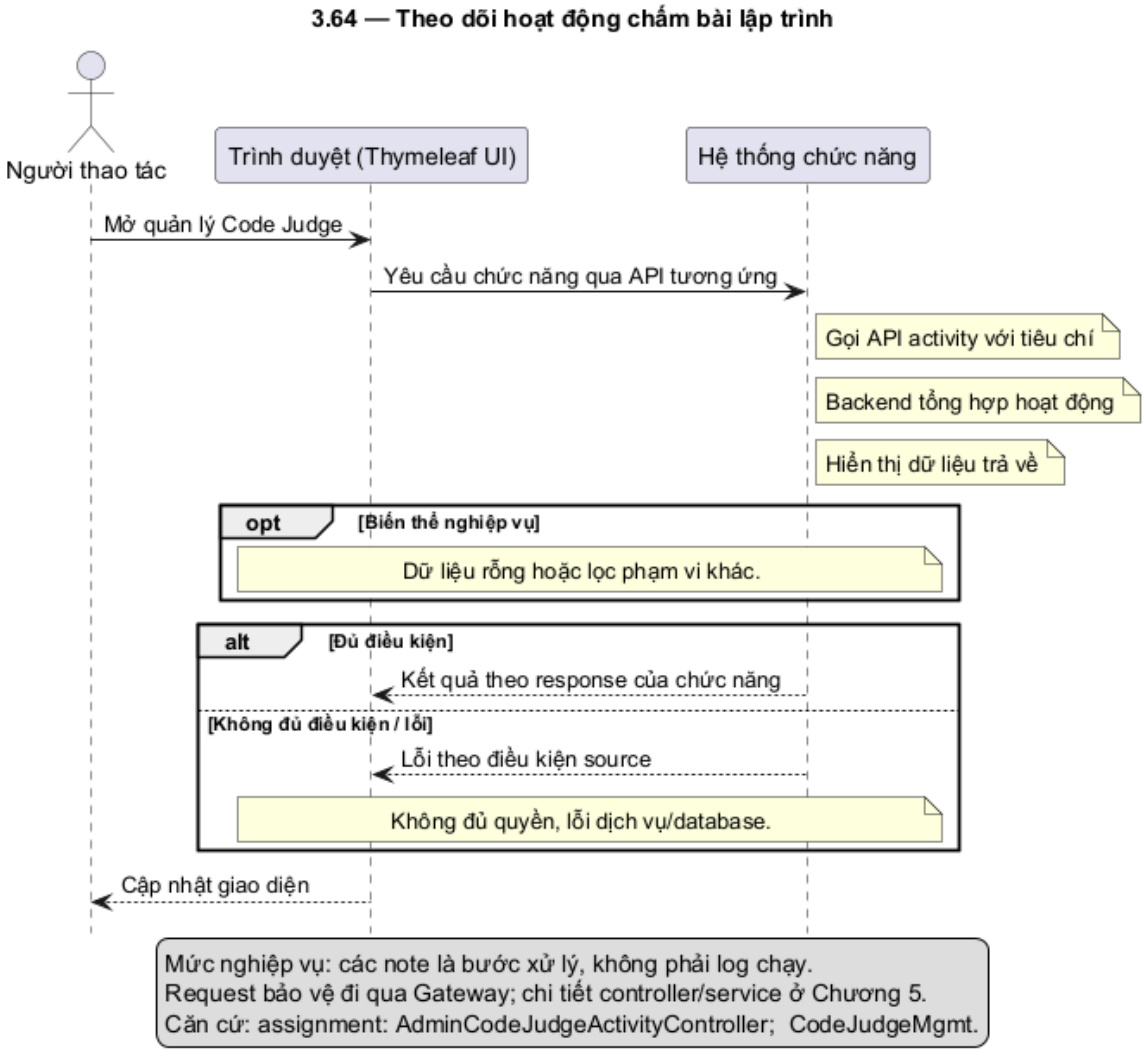
• Sequence Diagram quản lý nội dung blog
Nội dung Hình 3.63: Sequence Diagram quản lý nội dung blog.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở Blog Management. 2. Lọc bài theo trạng thái/tác giả/tiêu chí được hỗ trợ. 3. Xem hoặc thực hiện thao tác. 4. Service xử lý theo quyền Admin. 5. Cập nhật danh sách.
Nhánh thay thế: Quản lý tag qua khu vực nội dung website.
Nhánh lỗi: Post/tag không tồn tại, dữ liệu sai hoặc lỗi lưu tệp.

Căn cứ: blog: PostController.getAllPostsForAdmin, PostServiceImpl; BlogMgmt/BlogCategoriesTab.



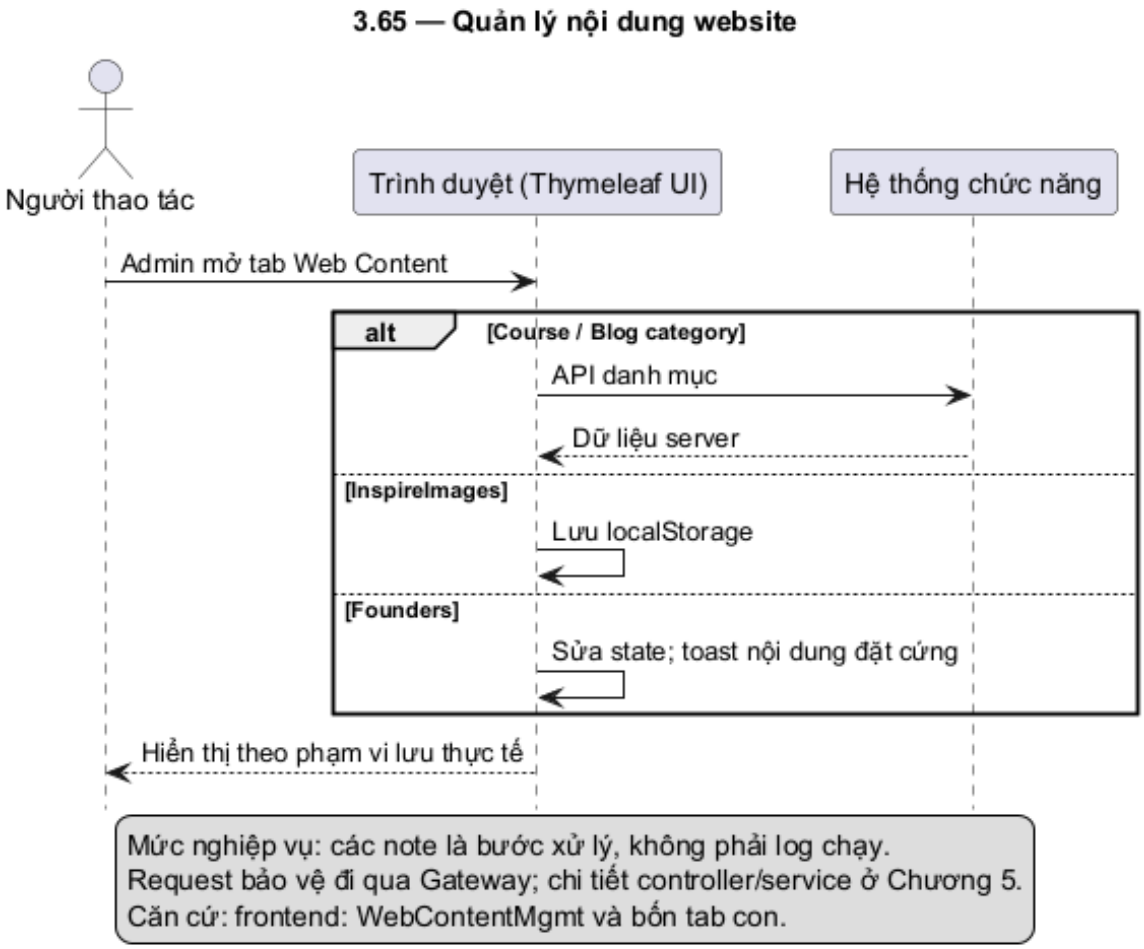

*Hình 3.63: Quản lý nội dung blog*

• Sequence Diagram theo dõi hoạt động chấm bài lập trình
Nội dung Hình 3.64: Sequence Diagram theo dõi hoạt động chấm bài lập trình.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở quản lý Code Judge. 2. Gọi API activity với tiêu chí. 3. Backend tổng hợp hoạt động. 4. Hiển thị dữ liệu trả về.
Nhánh thay thế: Dữ liệu rỗng hoặc lọc phạm vi khác.
Nhánh lỗi: Không đủ quyền, lỗi dịch vụ/database.
Căn cứ: assignment: AdminCodeJudgeActivityController; CodeJudgeMgmt.



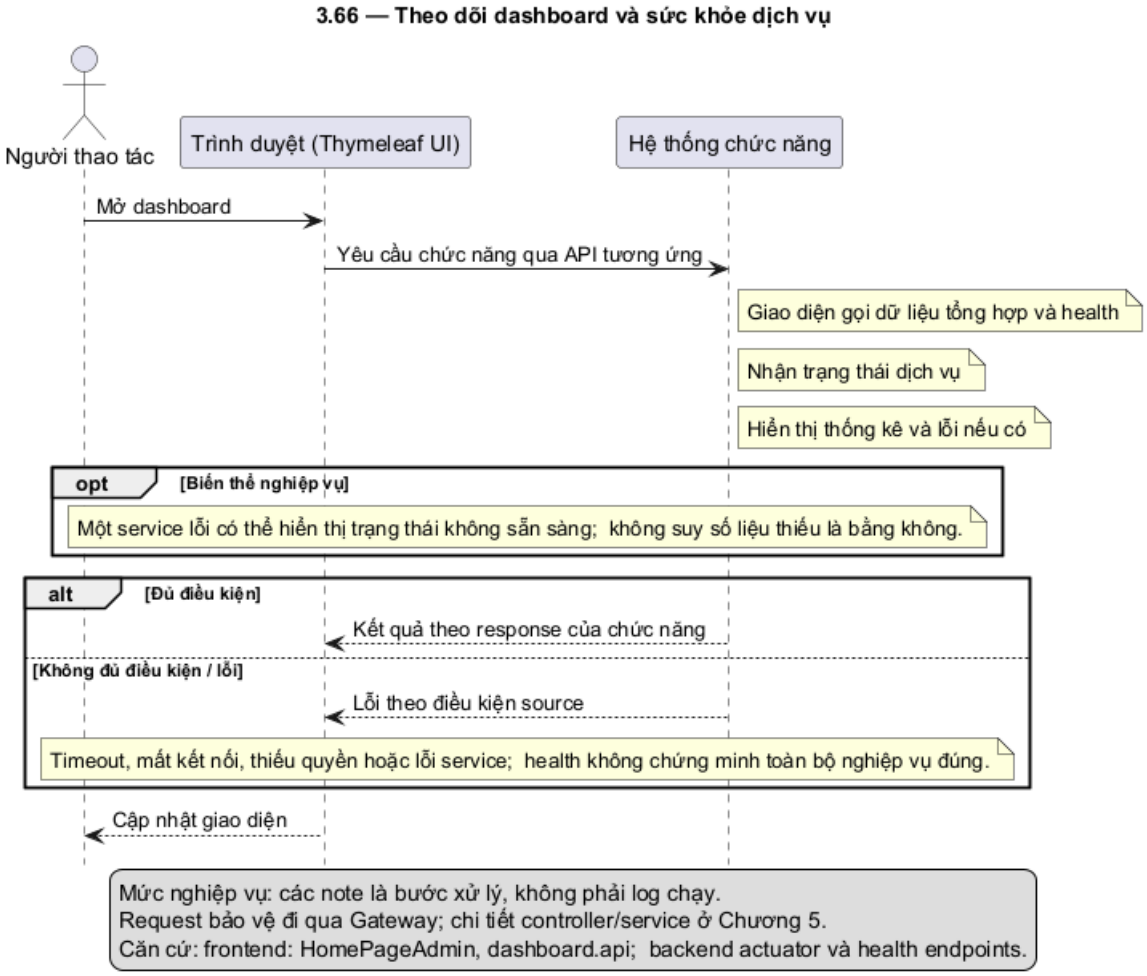

*Hình 3.64: Theo dõi hoạt động chấm bài lập trình*

• Sequence Diagram quản lý nội dung website
Nội dung Hình 3.65: Sequence Diagram quản lý nội dung website.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Tách nhánh xử lý cục bộ frontend khỏi nhánh gọi API. Thành phần cấu hình tĩnh/state/localStorage không được vẽ mũi tên lưu database. Với dữ liệu danh mục, vẽ nhánh API đến service/repository tương ứng.
Luồng nghiệp vụ: 1. Chọn tab. 2. Danh mục khóa học/blog gọi API. 3. Ảnh Inspire lưu localStorage. 4. Founders sửa state; nút lưu thông báo nội dung đặt cứng. 5. Hiển thị kết quả tương ứng.
Nhánh thay thế: Khôi phục giá trị mặc định theo giao diện.
Nhánh lỗi: localStorage đầy/lỗi; lỗi API danh mục; chưa có persistence Founders, không coi thông báo là lưu thành công.
Căn cứ: frontend: WebContentMgmt và bốn tab con.




*Hình 3.65: Quản lý nội dung website*

• Sequence Diagram theo dõi dashboard và sức khỏe dịch vụ
Nội dung Hình 3.66: Sequence Diagram theo dõi dashboard và sức khỏe dịch vụ.
Thành phần: Người dùng, Trình duyệt (Thymeleaf UI), API Gateway, controller/service nghiệp vụ và thành phần dữ liệu hoặc dịch vụ ngoài liên quan.
Thứ tự tương tác cần thể hiện: người dùng thao tác trên giao diện Thymeleaf; frontend gửi request qua Gateway; controller nhận và kiểm tra dữ liệu; service xử lý điều kiện nghiệp vụ; repository hoặc client phụ thuộc trả kết quả; API phản hồi để frontend cập nhật trạng thái.
Đây là mô hình tương tác ở mức chức năng; tên phương thức và thứ tự chi tiết sẽ được đối chiếu trong phần giải thích implementation.
Luồng nghiệp vụ: 1. Mở dashboard. 2. Giao diện gọi dữ liệu tổng hợp và health. 3. Nhận trạng thái dịch vụ. 4. Hiển thị thống kê và lỗi nếu có.
Nhánh thay thế: Một service lỗi có thể hiển thị trạng thái không sẵn sàng; không suy số liệu thiếu là bằng không.
Nhánh lỗi: Timeout, mất kết nối, thiếu quyền hoặc lỗi service; health không chứng minh toàn bộ nghiệp vụ đúng.
Căn cứ: frontend: HomePageAdmin, dashboard.api; backend actuator và health endpoints.




*Hình 3.66: Theo dõi dashboard và sức khỏe dịch vụ*

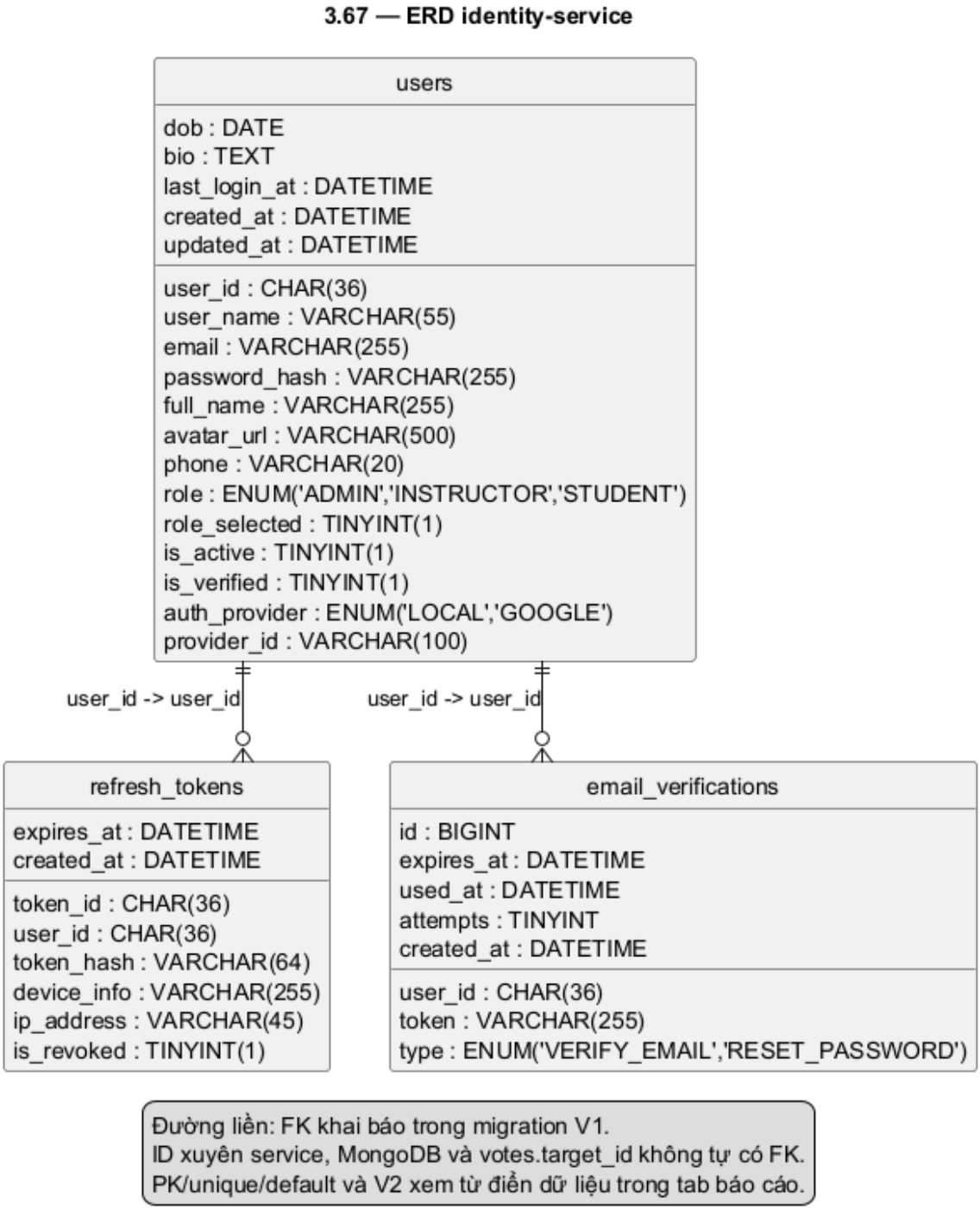
# 3.6 Thiết kế dữ liệu

## 3.6.1 Phân chia dữ liệu theo service

MySQL lưu dữ liệu quan hệ của Identity, Course, Assignment và Blog; MongoDB lưu cấu trúc/nội dung lesson, submission và lịch sử chat. Redis giữ cache và bộ đếm có thời hạn; MinIO giữ object. Các model trong báo cáo được đối chiếu migration V1/V2 và entity/document ở commit 4288b95; chưa phải introspection database đang chạy.

## 3.6.2 ERD Identity Service

ERD Identity: users (1) — (0..n) refresh_tokens qua user_id; users (1) — (0..n) email_verifications qua user_id. Đây là các FK thật trong migration. users.user_name/email và refresh_tokens.token_hash có unique. email_verifications.token cũng unique theo V1; không bỏ ràng buộc này trong mô hình dù mã OTP ngắn có nguy cơ trùng. Hình cần thể hiện PK/FK và những unique quan trọng.

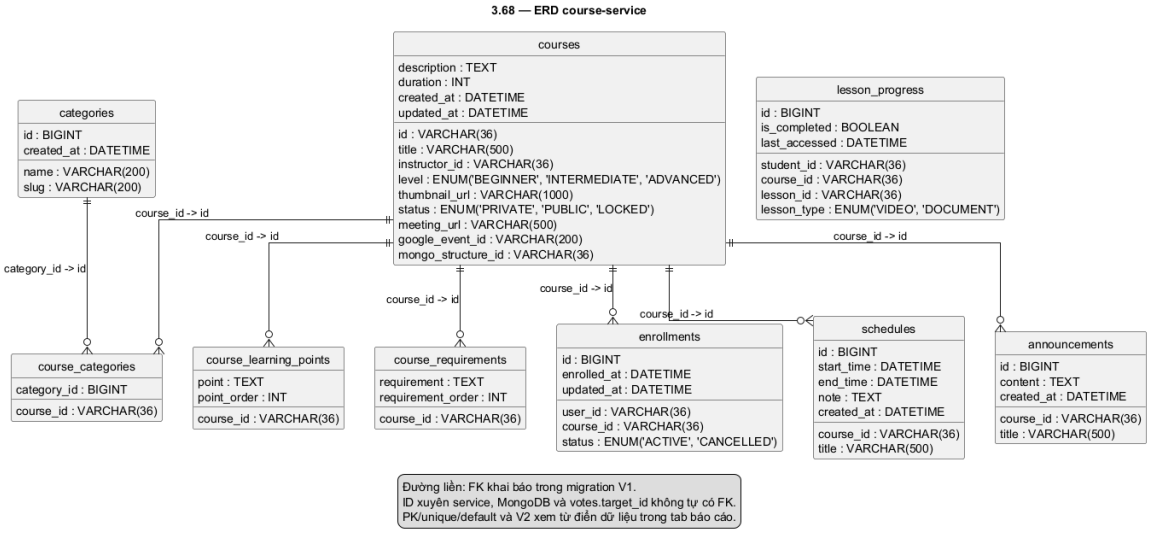



*Hình 3.67: ERD identity-service*

## 3.6.3 ERD Course Service

ERD Course: courses (1) — (0..n) enrollments, schedules, announcements, course_learning_points và course_requirements; courses–categories là N–N qua course_categories. FK và ON DELETE CASCADE lấy từ V1. enrollments có unique(user_id,course_id). lesson_progress có unique(student_id,lesson_id), nhưng V1 không khai báo FK đến course/lesson/user. instructor_id, user_id và student_id tham chiếu

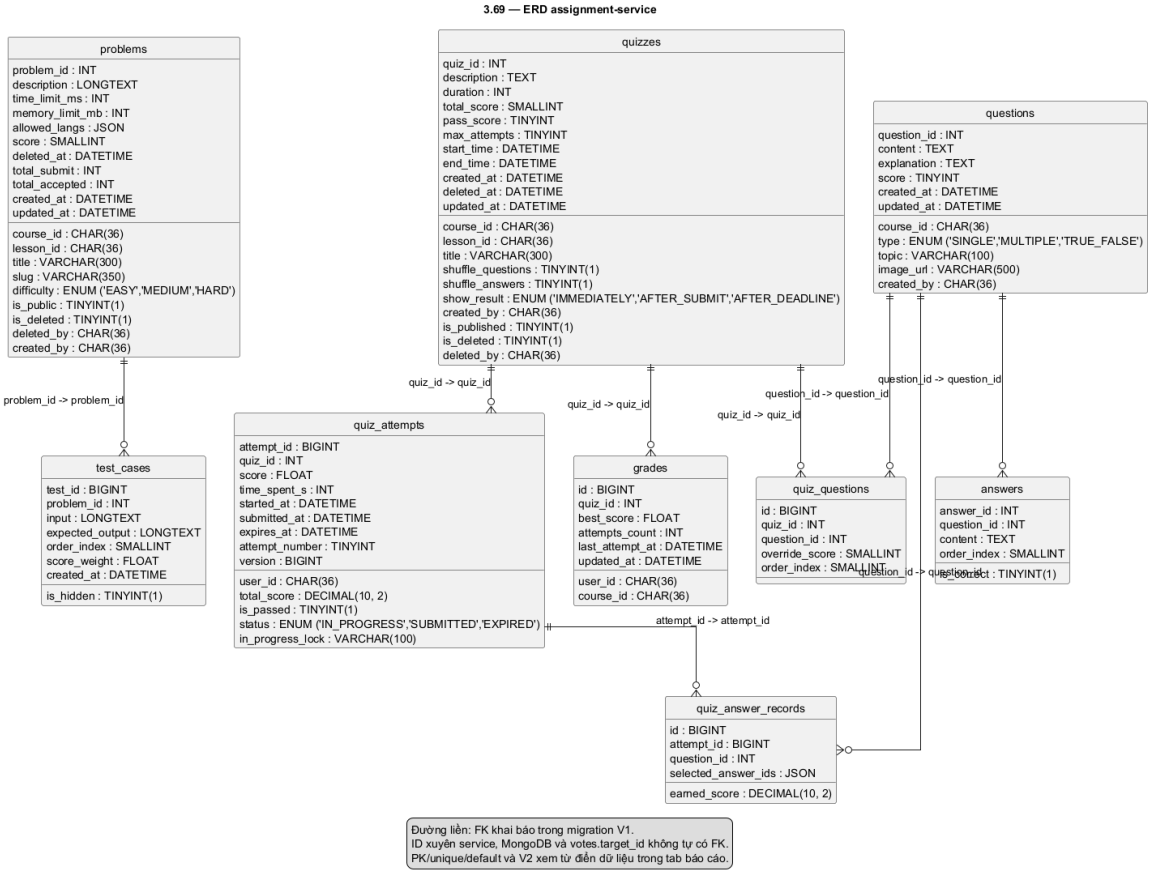
Identity về mặt nghiệp vụ; không vẽ chúng thành FK MySQL xuyên service. mongo_structure_id là unique nhưng là tham chiếu MongoDB.




*Hình 3.68: ERD course-service*

## 3.6.4 ERD Assignment Service

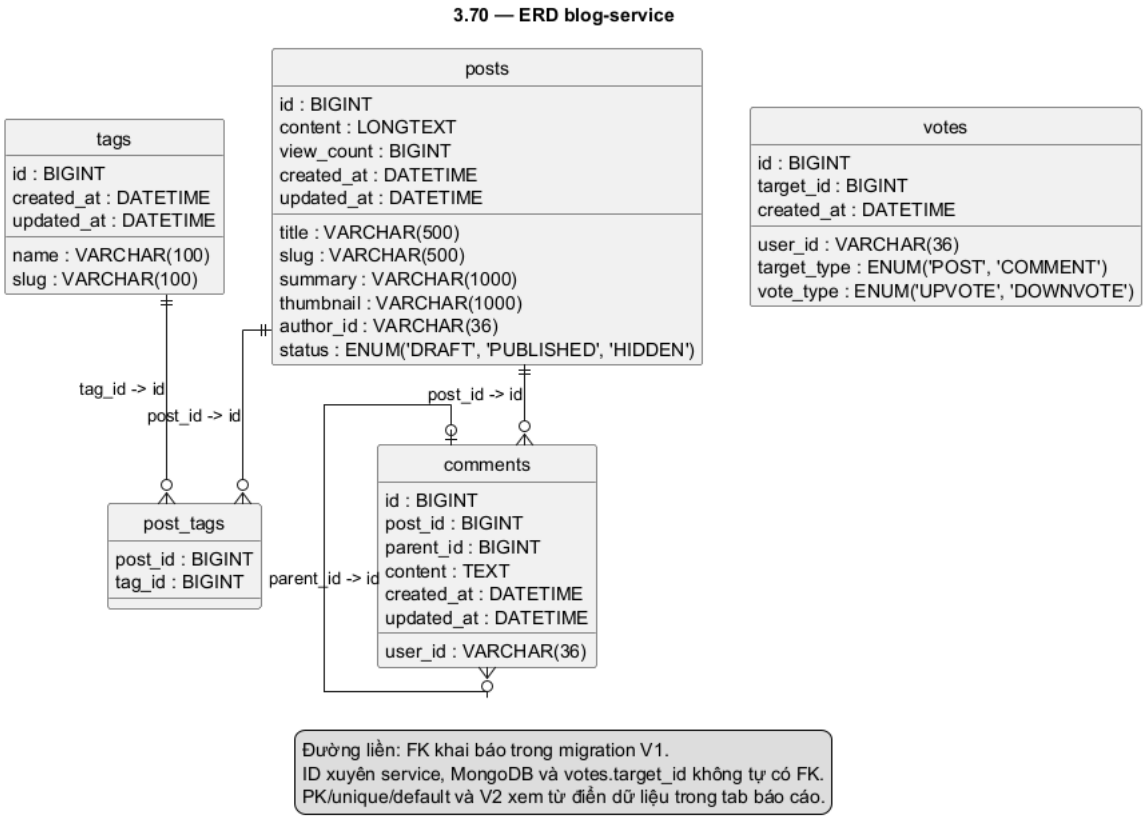
ERD Assignment: problems (1) — (0..n) test_cases; questions (1) — (0..n) answers; quizzes–questions là N–N qua quiz_questions; quizzes (1) — (0..n) quiz_attempts và grades; quiz_attempts (1) — (0..n) quiz_answer_records; mỗi record liên kết một question. quiz_questions unique(quiz_id,question_id), records unique(attempt_id,question_id), grades unique(user_id,quiz_id). V2 tạo unique trên in_progress_lock để chặn hai lượt IN_PROGRESS cùng user/quiz. course_id, lesson_id, user_id và created_by là ID tham chiếu nghiệp vụ; submission nằm ở MongoDB.




*Hình 3.69: ERD assignment-service*

## 3.6.5 ERD Blog Service

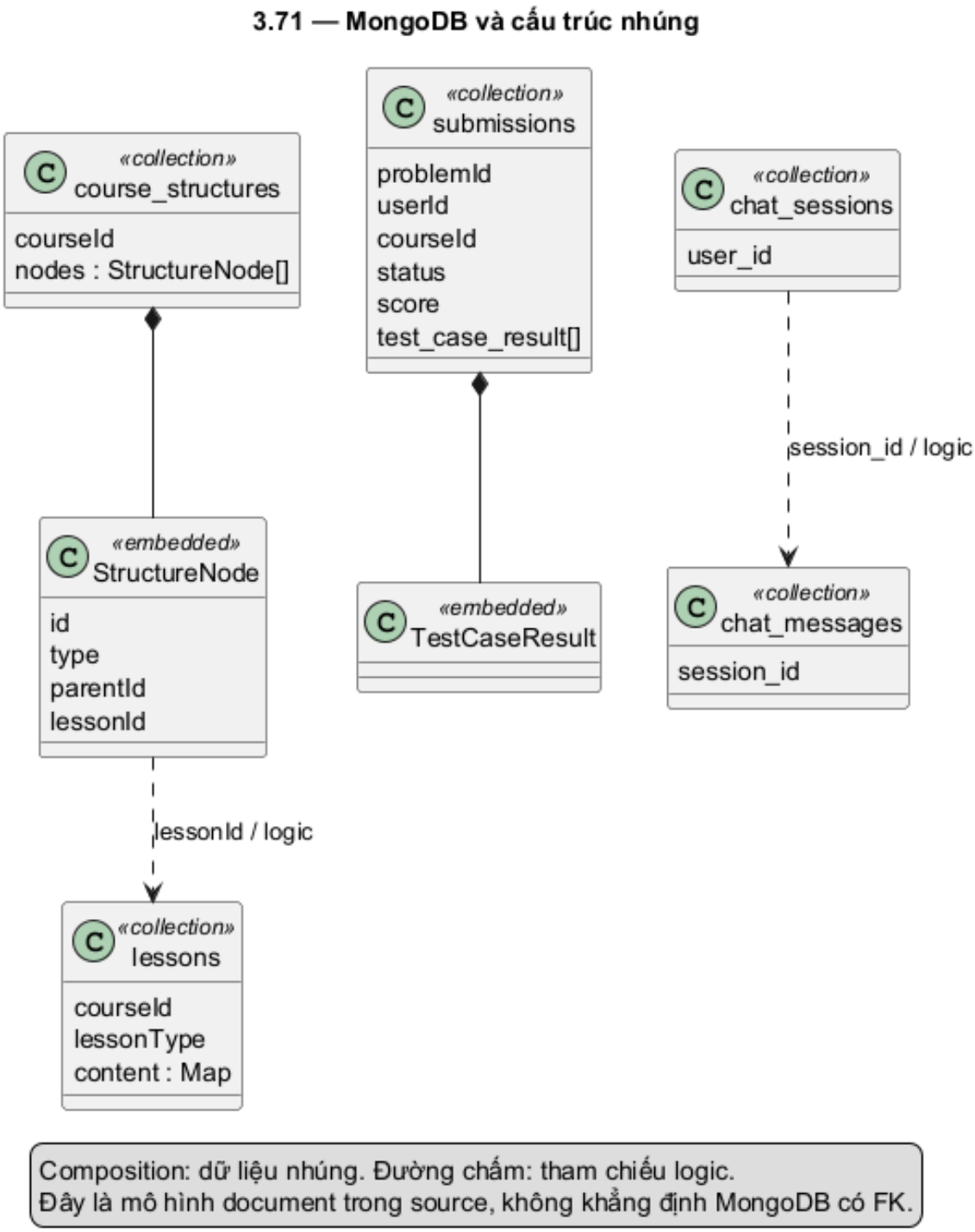
ERD Blog: posts–tags là N–N qua post_tags; posts (1) — (0..n) comments; comments có parent_id nullable tham chiếu chính bảng comments. Service chỉ cho một tầng reply dù FK tự tham chiếu có thể biểu diễn nhiều tầng. votes unique(user_id,target_id,target_type), target_type phân biệt POST/COMMENT; V1 không tạo FK target_id đến cả hai bảng. author_id/user_id tham chiếu Identity logic. Phải phân biệt đường FK với đường tham chiếu đa hình.




*Hình 3.70: ERD blog-service*

## 3.6.6 Mô hình document MongoDB

Mô hình document: course_structures chứa danh sách StructureNode và courseId; node có id, type, title, order, parentId và lessonId. lessons chứa courseId, lessonType và content kiểu Map. submissions chứa ID problem/user/course, code, trạng thái, điểm, thời gian/bộ nhớ và danh sách test_case_result nhúng. chat_sessions và chat_messages liên hệ qua session_id, cùng user_id tham chiếu Identity. MongoDB không tự bảo đảm FK như MySQL; kiểm tra quan hệ phụ thuộc service.

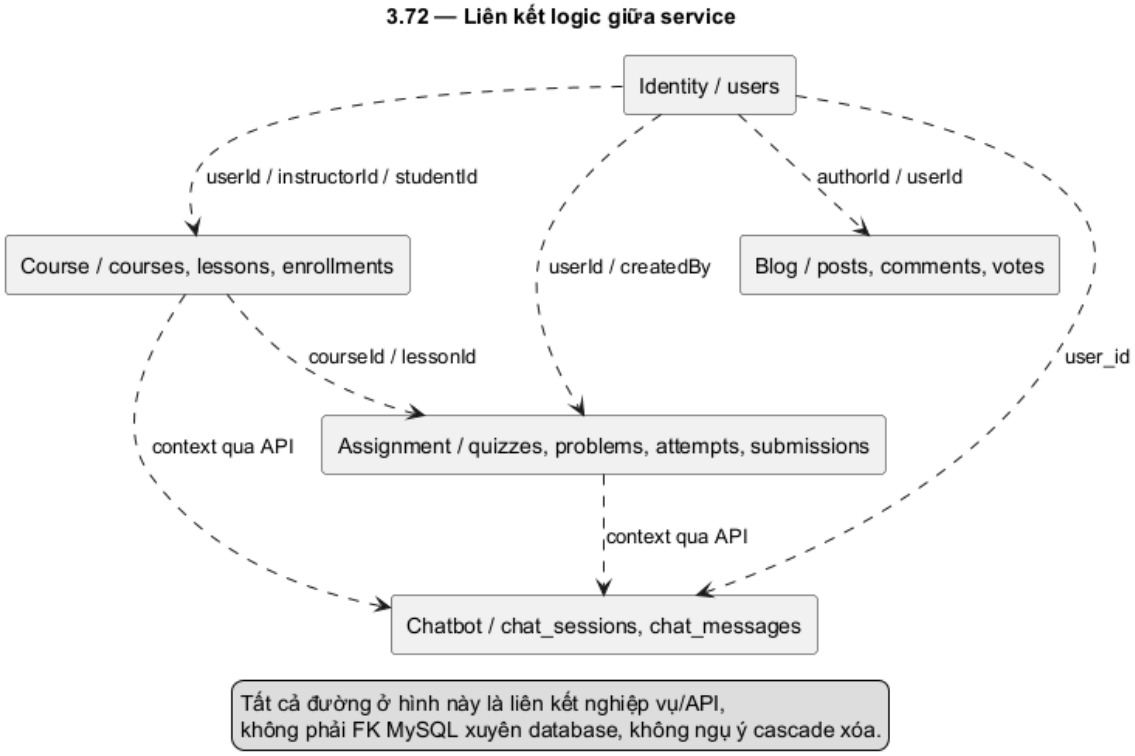



*Hình 3.71: MongoDB và cấu trúc nhúng*

### 3.6.7 Liên kết dữ liệu giữa các service

Các service trao đổi qua API nội bộ/OpenFeign và lưu ID của đối tượng ở service khác. Mô hình liên kết tổng quát cần thể hiện Identity.userId liên hệ enrollment, instructor, author, attempt/submission và chat; Course.courseId liên hệ quiz/problem/lesson/submission; Problem.problemId liên hệ submissions. Các đường này ký hiệu tham chiếu logic. Không

suy rằng xóa User hoặc Course sẽ tự cascade ở mọi database; phải đối chiếu API xóa và cơ chế dọn dữ liệu cụ thể.




*Hình 3.72: Liên kết logic giữa service*

### 3.6.8 Ràng buộc khóa chỉ mục và tính nhất quán

Khóa chính, unique và FK được lấy từ V1; index truy vấn bổ sung từ V2. Course có index theo instructor/status/title và thời gian; Assignment có index theo course, tác giả, trạng thái và unique lượt quiz đang làm; Blog có index theo tác giả/trạng thái/tương tác; Identity có index truy vấn user và token. Những ràng buộc được khai báo trong source chưa chứng minh migration đã áp dụng trong môi trường demo. Kiểm tra schema thực tế và log Flyway cần bổ sung khi nhóm chạy ứng dụng.

## 3.7 Từ điển dữ liệu

### 3.7.1 Identity Service MySQL

• users

Căn cứ: identity-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: Khóa/unique nằm trực tiếp trên dòng khai báo trường..

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.56 Từ điển dữ liệu users**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| user_id | CHAR(36) | NOT NULL PRIMARY KEY |
| user_name | VARCHAR(55) | NOT NULL UNIQUE |
| email | VARCHAR(255) | NOT NULL UNIQUE |
| password_hash | VARCHAR(255) | NOT NULL |
| full_name | VARCHAR(255) | NOT NULL |
| dob | DATE | NULL |
| avatar_url | VARCHAR(500) | NULL |
| bio | TEXT | NULL |
| phone | VARCHAR(20) | NULL |
| role | ENUM('ADMIN','INSTRUCTOR','STUDENT') | NOT NULL |
| role_selected | TINYINT(1) | NOT NULL DEFAULT 0 |
| is_active | TINYINT(1) | NOT NULL DEFAULT 1 |
| is_verified | TINYINT(1) | NOT NULL DEFAULT 0 |
| auth_provider | ENUM('LOCAL','GOOGLE') | NOT NULL DEFAULT 'LOCAL' |
| provider_id | VARCHAR(100) | NULL |
| last_login_at | DATETIME | NULL |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |
| updated_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |

• refresh_tokens

Căn cứ: identity-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT fk_refresh_token_user FOREIGN KEY (user_id) REFERENCES users (user_id).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.57 Từ điển dữ liệu refresh_tokens**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| token_id | CHAR(36) | NOT NULL PRIMARY KEY |

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| user_id | CHAR(36) | NOT NULL |
| token_hash | VARCHAR(64) | NOT NULL UNIQUE |
| device_info | VARCHAR(255) | NULL |
| ip_address | VARCHAR(45) | NULL |
| expires_at | DATETIME | NOT NULL |
| is_revoked | TINYINT(1) | NOT NULL DEFAULT 0 |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |

• email_verifications
Căn cứ: identity-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT fk_email_verif_user FOREIGN KEY (user_id) REFERENCES users (user_id).
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.58 Từ điển dữ liệu email_verifications**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| user_id | CHAR(36) | NOT NULL |
| token | VARCHAR(255) | NOT NULL UNIQUE |
| type | ENUM('VERIFY_EMAIL','RESET_PASSWORD') | NOT NULL |
| expires_at | DATETIME | NOT NULL |
| used_at | DATETIME | NULL |
| attempts | TINYINT | NOT NULL DEFAULT 0 |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |

### 3.7.2 Course Service MySQL

• categories
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_categories PRIMARY KEY (id); CONSTRAINT uq_categories_name UNIQUE (name); CONSTRAINT uq_categories_slug UNIQUE (slug).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.59 Từ điển dữ liệu categories**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| name | VARCHAR(200) | NOT NULL |
| slug | VARCHAR(200) | NULL |
| created_at | DATETIME | NOT NULL |

• courses

Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_courses PRIMARY KEY (id); CONSTRAINT uq_courses_mongo_structure UNIQUE (mongo_structure_id).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.60 Từ điển dữ liệu courses**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | VARCHAR(36) | NOT NULL |
| title | VARCHAR(500) | NOT NULL |
| description | TEXT | NULL |
| instructor_id | VARCHAR(36) | NOT NULL |
| duration | INT | NULL |
| level | ENUM('BEGINNER', 'INTERMEDIATE', 'ADVANCED') | NULL |
| thumbnail_url | VARCHAR(1000) | NULL |
| status | ENUM('PRIVATE', 'PUBLIC', 'LOCKED') | NOT NULL DEFAULT 'PRIVATE' |
| meeting_url | VARCHAR(500) | NULL |
| google_event_id | VARCHAR(200) | NULL |
| mongo_structure_id | VARCHAR(36) | NOT NULL |
| created_at | DATETIME | NOT NULL |
| updated_at | DATETIME | NOT NULL |

• enrollments

Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_enrollments PRIMARY KEY (id); CONSTRAINT uq_enrollments_user_course UNIQUE (user_id, course_id); CONSTRAINT fk_enrollments_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.61 Từ điển dữ liệu enrollments**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| user_id | VARCHAR(36) | NOT NULL |
| course_id | VARCHAR(36) | NOT NULL |
| status | ENUM('ACTIVE', 'CANCELLED') | NOT NULL DEFAULT 'ACTIVE' |
| enrolled_at | DATETIME | NOT NULL |
| updated_at | DATETIME | NOT NULL |

• lesson_progress
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_lesson_progress PRIMARY KEY (id); CONSTRAINT uq_student_lesson UNIQUE (student_id, lesson_id).
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.62 Từ điển dữ liệu lesson_progress**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| student_id | VARCHAR(36) | NOT NULL |
| course_id | VARCHAR(36) | NOT NULL |
| lesson_id | VARCHAR(36) | NOT NULL |
| lesson_type | ENUM('VIDEO', 'DOCUMENT') | NOT NULL |
| is_completed | BOOLEAN | DEFAULT FALSE |
| last_accessed | DATETIME | NULL |

• schedules
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_schedules PRIMARY KEY (id); CONSTRAINT

fk_sch_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.63 Từ điển dữ liệu schedules**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| course_id | VARCHAR(36) | NOT NULL |
| title | VARCHAR(500) | NOT NULL |
| start_time | DATETIME | NOT NULL |
| end_time | DATETIME | NOT NULL |
| note | TEXT | NULL |
| created_at | DATETIME | NOT NULL |

• announcements
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_announcements PRIMARY KEY (id); CONSTRAINT fk_ann_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.64 Từ điển dữ liệu announcements**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| course_id | VARCHAR(36) | NOT NULL |
| title | VARCHAR(500) | NOT NULL |
| content | TEXT | NOT NULL |
| created_at | DATETIME | NOT NULL |

• course_categories
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_course_categories PRIMARY KEY (course_id, category_id); CONSTRAINT fk_cc_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE; CONSTRAINT fk_cc_category FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.65 Từ điển dữ liệu course_categories**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| course_id | VARCHAR(36) | NOT NULL |
| category_id | BIGINT | NOT NULL |

• course_learning_points
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_course_learning_points PRIMARY KEY (course_id, point_order); CONSTRAINT fk_course_learning_points_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.66 Từ điển dữ liệu course_learning_points**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| course_id | VARCHAR(36) | NOT NULL |
| point | TEXT | NULL |
| point_order | INT | NOT NULL |

• course_requirements
Căn cứ: course-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_course_requirements PRIMARY KEY (course_id, requirement_order); CONSTRAINT fk_course_requirements_course FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.67 Từ điển dữ liệu course_requirements**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| course_id | VARCHAR(36) | NOT NULL |
| requirement | TEXT | NULL |
| requirement_order | INT | NOT NULL |

### 3.7.3 Assignment Service MySQL

• questions
Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: Khóa/unique nằm trực tiếp trên dòng khai báo trường..
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.68 Từ điển dữ liệu questions**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| question_id | INT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| course_id | CHAR(36) | NOT NULL COMMENT 'Ref → Course Service' |
| content | TEXT | NOT NULL |
| type | ENUM ('SINGLE','MULTIPLE','TRUE_FALSE') | NOT NULL |
| topic | VARCHAR(100) | NULL |
| explanation | TEXT | NULL |
| score | TINYINT | NOT NULL DEFAULT 10 |
| image_url | VARCHAR(500) | NULL |
| created_by | CHAR(36) | NOT NULL |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |
| updated_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP |

• answers

Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT fk_answer_question FOREIGN KEY (question_id) REFERENCES questions (question_id) ON DELETE CASCADE.

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.69 Từ điển dữ liệu answers**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| answer_id | INT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| question_id | INT | NOT NULL |
| content | TEXT | NOT NULL |
| is_correct | TINYINT(1) | NOT NULL DEFAULT 0 |
| order_index | SMALLINT | NOT NULL DEFAULT 0 |

• quizzes
Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: Khóa/unique nằm trực tiếp trên dòng khai báo trường..
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.70 Từ điển dữ liệu quizzes**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| quiz_id | INT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| course_id | CHAR(36) | NOT NULL |
| lesson_id | CHAR(36) | NULL |
| title | VARCHAR(300) | NOT NULL |
| description | TEXT | NULL |
| duration | INT | NULL |
| total_score | SMALLINT | NOT NULL DEFAULT 100 |
| pass_score | TINYINT | NOT NULL DEFAULT 50 |
| max_attempts | TINYINT | NOT NULL DEFAULT 0 |
| shuffle_questions | TINYINT(1) | NOT NULL DEFAULT 1 |
| shuffle_answers | TINYINT(1) | NOT NULL DEFAULT 1 |
| show_result | ENUM ('IMMEDIATELY','AFTER_SUBMIT','AFTER_DEADLINE') | NOT NULL DEFAULT 'AFTER_SUBMIT' |
| start_time | DATETIME | NULL |
| end_time | DATETIME | NULL |
| created_by | CHAR(36) | NOT NULL |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |
| is_published | TINYINT(1) | NOT NULL DEFAULT 0 |
| is_deleted | TINYINT(1) | NOT NULL DEFAULT 0 |
| deleted_at | DATETIME | NULL |
| deleted_by | CHAR(36) | NULL |

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| updated_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP |

• quiz_questions

Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT uq_quiz_question UNIQUE (quiz_id, question_id); CONSTRAINT fk_qq_quiz FOREIGN KEY (quiz_id) REFERENCES quizzes (quiz_id) ON DELETE CASCADE; CONSTRAINT fk_qq_question FOREIGN KEY (question_id) REFERENCES questions (question_id).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.71 Từ điển dữ liệu quiz_questions**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| quiz_id | INT | NOT NULL |
| question_id | INT | NOT NULL |
| override_score | SMALLINT | NULL COMMENT 'NULL = dùng score gốc của question' |
| order_index | SMALLINT | NOT NULL DEFAULT 0 |

• quiz_attempts

Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT fk_attempt_quiz FOREIGN KEY (quiz_id) REFERENCES quizzes (quiz_id).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.72 Từ điển dữ liệu quiz_attempts**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| attempt_id | BIGINT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| quiz_id | INT | NOT NULL |
| user_id | CHAR(36) | NOT NULL |
| score | FLOAT | NOT NULL DEFAULT 0 |

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| total_score | DECIMAL(10, 2) | NOT NULL |
| is_passed | TINYINT(1) | NOT NULL DEFAULT 0 |
| time_spent_s | INT | NULL |
| status | ENUM ('IN_PROGRESS','SUBMITTED','EXPIRED') | NOT NULL DEFAULT 'IN_PROGRESS' |
| started_at | DATETIME | NOT NULL |
| submitted_at | DATETIME | NULL |
| expires_at | DATETIME | NULL |
| attempt_number | TINYINT | NOT NULL DEFAULT 1 |
| in_progress_lock | VARCHAR(100) | GENERATED ALWAYS AS (IF(status = 'IN_PROGRESS', CONCAT(user_id, ':', quiz_id), NULL)) VIRTUAL |
| version | BIGINT | NOT NULL DEFAULT 0 |

• quiz_answer_records

Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT uq_record UNIQUE (attempt_id, question_id); CONSTRAINT fk_record_attempt FOREIGN KEY (attempt_id) REFERENCES quiz_attempts (attempt_id) ON DELETE CASCADE; CONSTRAINT fk_record_question FOREIGN KEY (question_id) REFERENCES questions (question_id).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.73 Từ điển dữ liệu quiz_answer_records**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| attempt_id | BIGINT | NOT NULL |
| question_id | INT | NOT NULL |
| selected_answer_ids | JSON | NOT NULL |
| earned_score | DECIMAL(10, 2) | NOT NULL DEFAULT 0 |

• problems

Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: Khóa/unique nằm trực tiếp trên dòng khai báo trường..
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.74 Từ điển dữ liệu problems**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| problem_id | INT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| course_id | CHAR(36) | NOT NULL |
| lesson_id | CHAR(36) | NOT NULL |
| title | VARCHAR(300) | NOT NULL |
| slug | VARCHAR(350) | NOT NULL UNIQUE |
| description | LONGTEXT | NOT NULL |
| difficulty | ENUM ('EASY','MEDIUM','HARD') | NOT NULL |
| time_limit_ms | INT | NOT NULL DEFAULT 2000 |
| memory_limit_mb | INT | NOT NULL DEFAULT 256 |
| allowed_langs | JSON | NOT NULL |
| score | SMALLINT | NOT NULL DEFAULT 100 |
| is_public | TINYINT(1) | NOT NULL DEFAULT 0 |
| is_deleted | TINYINT(1) | NOT NULL DEFAULT 0 |
| deleted_at | DATETIME | NULL |
| deleted_by | CHAR(36) | NULL |
| total_submit | INT | NOT NULL DEFAULT 0 |
| total_accepted | INT | NOT NULL DEFAULT 0 |
| created_by | CHAR(36) | NOT NULL |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |
| updated_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP |

• test_cases

Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT fk_tc_problem FOREIGN KEY (problem_id) REFERENCES problems (problem_id) ON DELETE CASCADE.
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.75 Từ điển dữ liệu test_cases**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| test_id | BIGINT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| problem_id | INT | NOT NULL |
| input | LONGTEXT | NOT NULL |
| expected_output | LONGTEXT | NOT NULL |
| is_hidden | TINYINT(1) | NOT NULL DEFAULT 1 |
| order_index | SMALLINT | NOT NULL DEFAULT 0 |
| score_weight | FLOAT | NOT NULL DEFAULT 1.0 |
| created_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP |

• grades
Căn cứ: assignment-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT uq_user_quiz UNIQUE (user_id, quiz_id); CONSTRAINT fk_grade_quiz FOREIGN KEY (quiz_id) REFERENCES quizzes (quiz_id).
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.76 Từ điển dữ liệu grades**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT PRIMARY KEY |
| user_id | CHAR(36) | NOT NULL |
| course_id | CHAR(36) | NOT NULL |
| quiz_id | INT | NOT NULL |
| best_score | FLOAT | NOT NULL DEFAULT 0 |
| attempts_count | INT | NOT NULL DEFAULT 0 |
| last_attempt_at | DATETIME | NULL |

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| updated_at | DATETIME | NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP |

### 3.7.4 Blog Service MySQL

• posts
Căn cứ: blog-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_posts PRIMARY KEY (id); CONSTRAINT uq_posts_slug UNIQUE (slug).
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.77 Từ điển dữ liệu posts**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| title | VARCHAR(500) | NOT NULL |
| slug | VARCHAR(500) | NOT NULL |
| summary | VARCHAR(1000) | NULL |
| content | LONGTEXT | NOT NULL |
| thumbnail | VARCHAR(1000) | NULL |
| author_id | VARCHAR(36) | NOT NULL |
| status | ENUM('DRAFT', 'PUBLISHED', 'HIDDEN') | NOT NULL DEFAULT 'DRAFT' |
| view_count | BIGINT | NOT NULL DEFAULT 0 |
| created_at | DATETIME | NOT NULL |
| updated_at | DATETIME | NOT NULL |

• tags
Căn cứ: blog-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_tags PRIMARY KEY (id); CONSTRAINT uq_tags_name UNIQUE (name); CONSTRAINT uq_tags_slug UNIQUE (slug).
Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.78 Từ điển dữ liệu tags**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| name | VARCHAR(100) | NOT NULL |
| slug | VARCHAR(100) | NOT NULL |
| created_at | DATETIME | NOT NULL |
| updated_at | DATETIME | NOT NULL |

• comments

Căn cứ: blog-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_comments PRIMARY KEY (id); CONSTRAINT fk_cmt_post FOREIGN KEY (post_id) REFERENCES posts (id) ON DELETE CASCADE; CONSTRAINT fk_cmt_parent FOREIGN KEY (parent_id) REFERENCES comments (id) ON DELETE CASCADE.

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.79 Từ điển dữ liệu comments**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |
| post_id | BIGINT | NOT NULL |
| user_id | VARCHAR(36) | NOT NULL |
| parent_id | BIGINT | NULL |
| content | TEXT | NOT NULL |
| created_at | DATETIME | NOT NULL |
| updated_at | DATETIME | NOT NULL |

• votes

Căn cứ: blog-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_votes PRIMARY KEY (id); CONSTRAINT uq_votes_user_target UNIQUE (user_id, target_id, target_type).

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.80 Từ điển dữ liệu votes**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| id | BIGINT | NOT NULL AUTO_INCREMENT |

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| user_id | VARCHAR(36) | NOT NULL |
| target_id | BIGINT | NOT NULL |
| target_type | ENUM('POST', 'COMMENT') | NOT NULL |
| vote_type | ENUM('UPVOTE', 'DOWNVOTE') | NOT NULL |
| created_at | DATETIME | NOT NULL |

• post_tags liên kết bài viết và tag

Căn cứ: blog-service/src/main/resources/db/migration/V1__initial_schema.sql. Các ràng buộc cấp bảng: CONSTRAINT pk_post_tags PRIMARY KEY (post_id, tag_id); CONSTRAINT fk_pt_post FOREIGN KEY (post_id) REFERENCES posts (id) ON DELETE CASCADE; CONSTRAINT fk_pt_tag FOREIGN KEY (tag_id) REFERENCES tags (id) ON DELETE CASCADE.

Bảng dữ liệu dưới đây chép kiểu và ràng buộc từ DDL; validation ứng dụng có thể chặt hơn và được trình bày ở Chương 5.

**Bảng 3.81 Từ điển dữ liệu post_tags liên kết bài viết và tag**

| Trường | Kiểu dữ liệu SQL | Ràng buộc/giá trị mặc định |
|---|---|---|
| post_id | BIGINT | NOT NULL |
| tag_id | BIGINT | NOT NULL |

### 3.7.5 MongoDB

• course_structures

Căn cứ document/entity: CourseStructure.java và StructureNode.java. Kiểu dưới đây là kiểu Java và ánh xạ @Field; không coi là SQL DDL hoặc schema validator MongoDB đã triển khai.

**Bảng 3.82 Từ điển dữ liệu course_structures**

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| _id | String id | Định danh document theo @Id |
| courseId | String | Tham chiếu logic khóa học MySQL |
| nodes | List<StructureNode> | Danh sách nhúng; mặc định ArrayList rỗng |
| updatedAt | LocalDateTime | @LastModifiedDate |
| nodes[].id | String | ID node |

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| nodes[].type | NodeType | FOLDER/LESSON; JSON dùng folder/lesson |
| nodes[].title | String | Tên node |
| nodes[].order | Integer | Thứ tự |
| nodes[].parentId | String | Null nếu root; quan hệ logic giữa node |
| nodes[].lessonId | String | Tham chiếu lesson khi node là lesson |

• lessons

Căn cứ document/entity: Lesson.java và LessonType.java. Kiểu dưới đây là kiểu Java và ánh xạ @Field; không coi là SQL DDL hoặc schema validator MongoDB đã triển khai.

**Bảng 3.83 Từ điển dữ liệu lessons**

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| _id | String id | @Id |
| courseId | String | ID course logic |
| lessonType | LessonType | VIDEO hoặc DOCUMENT; API dùng giá trị lowercase |
| content | Map<String,Object> | Nội dung linh hoạt; mặc định HashMap rỗng; kiểm tra tại LessonServiceImpl |
| createdAt | LocalDateTime | @CreatedDate |
| updatedAt | LocalDateTime | @LastModifiedDate |

• submissions

Căn cứ document/entity: Submission.java. Kiểu dưới đây là kiểu Java và ánh xạ @Field; không coi là SQL DDL hoặc schema validator MongoDB đã triển khai.

**Bảng 3.84 Từ điển dữ liệu submissions**

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| _id | String id | Tên trường Java id; cần xác minh mapping thực tế khi chạy Mongo |
| problem_id | Integer | Tham chiếu problems.problem_id logic |
| user_id | String | Tham chiếu Identity; @Indexed |
| course_id | String | Tham chiếu Course |
| language | String | Ngôn ngữ bài nộp |

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| source_code | String | Mã nguồn |
| status | String | PENDING/JUDGING và trạng thái kết quả theo service |
| score | Integer | Điểm; kiểm tra khởi tạo thực tế trong submit |
| exec_time_ms | Integer | Thời gian thực thi ms |
| memory_used_kb | Integer | Bộ nhớ KB |
| test_case_result | List<TestCaseResult> | Danh sách document nhúng |
| judge_raw_response | Object | Phản hồi engine linh hoạt |
| compile_error | String | Lỗi biên dịch; có thể null |
| judge0_token | String | Token dùng đọc kết quả Judge0 |
| submitted_at | Instant | Thời điểm nộp |
| judge_at | Instant | Thời điểm chấm |
| engine_used | String | Engine sử dụng |
| test_case_result[].test_id | Long | ID test case logic |
| test_case_result[].status | String | AC/WA/TLE/MLE/RE/CE/IE |
| test_case_result[].time_ms | Integer | Thời gian test |
| test_case_result[].memory_kb | Integer | Bộ nhớ test |
| test_case_result[].is_hidden | Boolean | Căn cứ giới hạn dữ liệu trả học viên |
| test_case_result[].output_snippet | String | Đoạn output; chú thích source nêu 200 ký tự, cần đối chiếu code tạo dữ liệu |

• chat_sessions

Căn cứ document/entity: ChatSession.java. Kiểu dưới đây là kiểu Java và ánh xạ @Field; không coi là SQL DDL hoặc schema validator MongoDB đã triển khai.

**Bảng 3.85 Từ điển dữ liệu chat_sessions**

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| _id | String id | @Id |
| user_id | String | Chủ phiên |

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| context_type | ContextType | GENERAL/PROBLEM/COURSE/GENERATE |
| context_ref_id | String | ID đối tượng ngữ cảnh; nullable theo comment source |
| title | String | Tiêu đề phiên |
| status | SessionStatus | ACTIVE/ARCHIVED |
| total_messages | int | Số tin nhắn |
| last_message_at | LocalDateTime | Thời gian tin cuối |
| created_at | LocalDateTime | Thời gian tạo |

• chat_messages

Căn cứ document/entity: ChatMessage.java. Kiểu dưới đây là kiểu Java và ánh xạ @Field; không coi là SQL DDL hoặc schema validator MongoDB đã triển khai.

**Bảng 3.86 Từ điển dữ liệu chat_messages**

| Trường lưu trữ | Kiểu khai báo Java | Ý nghĩa và ánh xạ |
|---|---|---|
| _id | String id | @Id |
| session_id | String | Liên kết session logic |
| user_id | String | @Indexed |
| role | MessageRole | USER/MODEL |
| content | String | Nội dung đầy đủ |
| token_count | int | Ước lượng token theo code |
| created_at | LocalDateTime | Thời điểm tạo |

### 3.7.6 Redis và MinIO

• Cấu trúc khóa cache và rate limit

Redis cache dùng khóa chat:session:{sessionId}:messages, kiểu list. TTL mặc định trong @Value là 24 giờ; số tin cache mặc định 20, có thể đổi bằng cấu hình. Cache đọc/lưu lỗi thì service bỏ qua hoặc quay về MongoDB. Rate limit dùng rate:chatbot:{userId}:{yyyy-MM-dd}, TTL đến nửa đêm theo thời gian server; Lua kiểm tra/tăng bộ đếm nguyên tử. Redis lỗi tại RateLimiterServiceImpl hiện fail-open; không mô tả hệ thống luôn chặn hạn mức khi Redis không sẵn sàng.

• Bucket và quy ước lưu tài nguyên

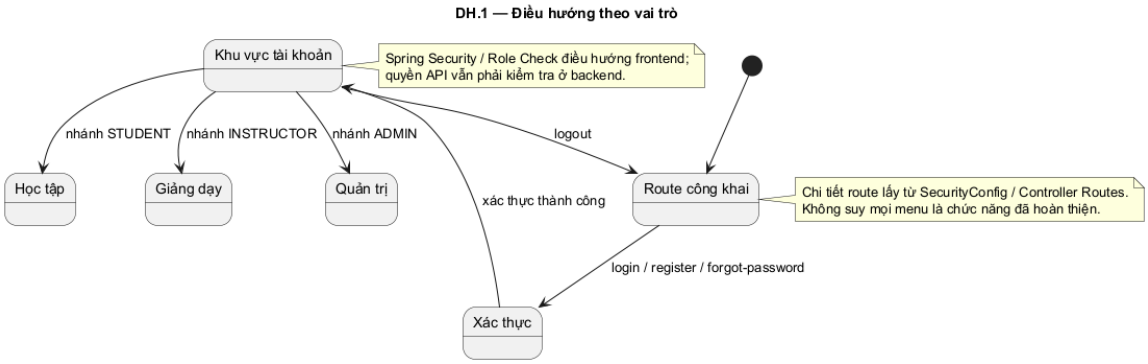
Bucket lấy từ minio.bucket-name của từng service. ObjectName dùng folder, UUID và phần mở rộng ở Course/Blog; Identity có kiểm tra prefix của user khi xóa avatar. Database lưu URL/định danh tài nguyên, không lưu toàn bộ tệp như cột nhị phân trong các bảng đã rà

soát. Tên bucket thực tế và quyền truy cập cần lấy từ cấu hình môi trường demo; báo cáo không đưa secret/access key vào hình minh chứng.

# 3.8 Thiết kế giao diện

## 3.8.1 Sơ đồ điều hướng tổng thể

Cơ chế điều hướng hệ thống được kiểm soát hoàn toàn ở tầng máy chủ thông qua Spring Security kết hợp Thymeleaf layout. Hệ thống phân tách các khu vực giao diện rõ rệt: Tuyến công khai cho khách (Guest) và đăng nhập/đăng ký; Khu vực học tập cho sinh viên (Student); Studio quản lý giảng dạy cho giảng viên (Instructor); và Trung tâm điều hành cho quản trị viên (Admin). Mọi request được xác thực và phân quyền nghiêm ngặt trước khi tải tài nguyên.

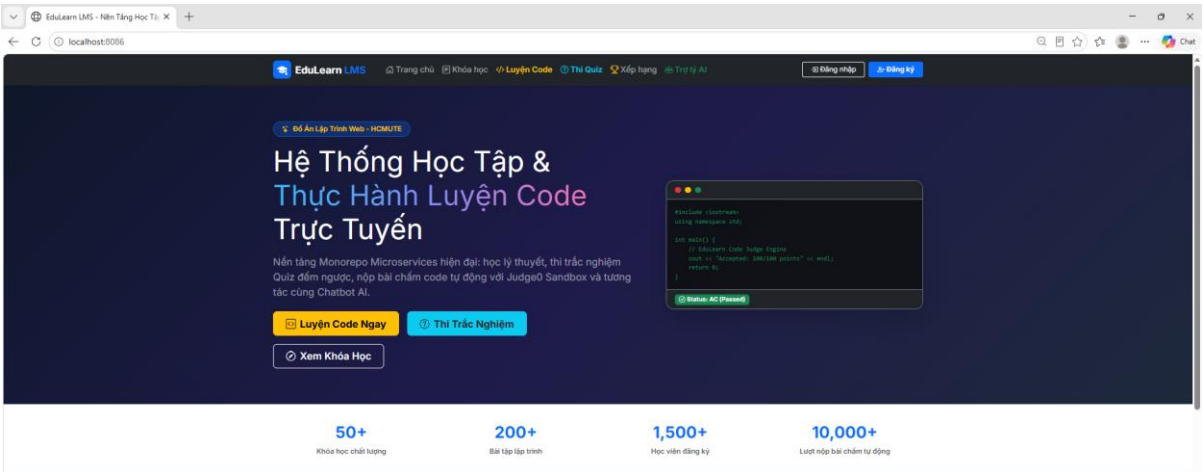



*Hình DH.1: Sơ đồ điều hướng tổng thể theo vai trò (Guest, Student, Instructor, Admin)*

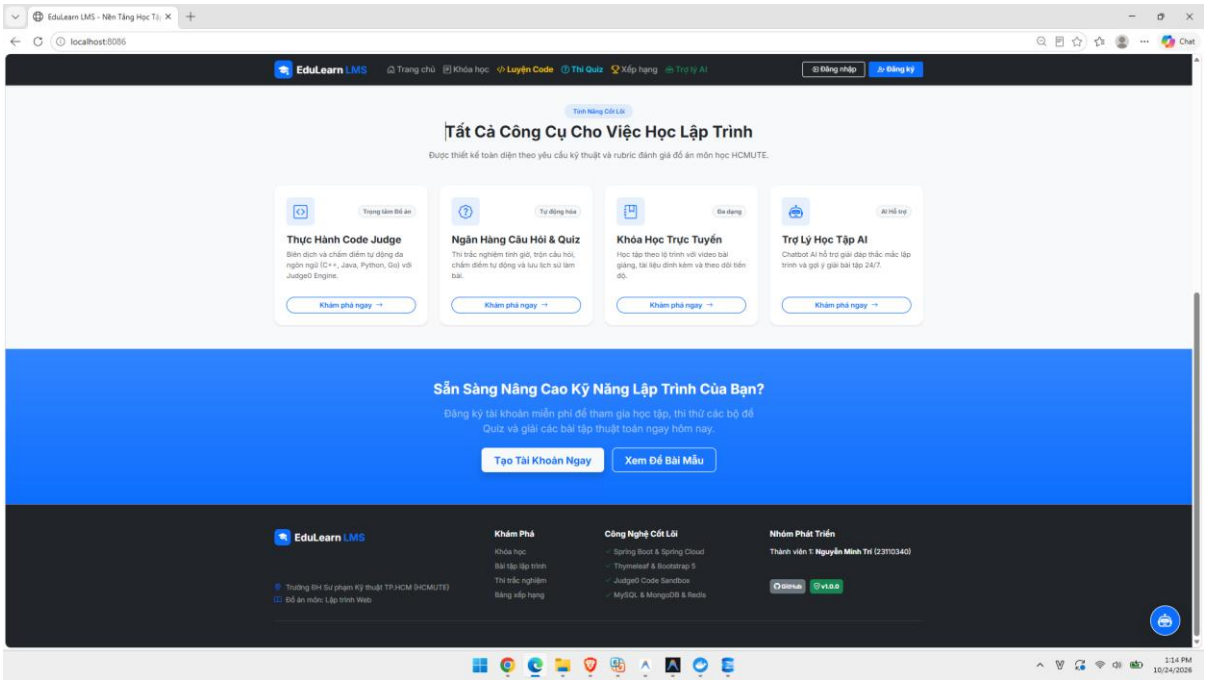
## 3.8.2 Giao diện Guest

Guest truy cập trang chủ, about-us, list-course/detail và blog public; các nút học/làm bài dẫn vào luồng xác thực hoặc enrollment tương ứng. Thiết kế cần hiển thị dữ liệu công khai rõ và không để URL chi tiết tự cấp quyền đọc lesson.

Giao diện Guest do nhóm cung cấp ngày 24/10/2026, đang mở tại localhost:8086. Thanh điều hướng có các mục Trang chủ, Khóa học, Luyện Code, Thi Quiz, Xếp hạng, Trợ lý AI và hai nút Đăng nhập/Đăng ký.




*Hình 3.73 Trang chủ khi chưa đăng nhập — khu vực giới thiệu*




*Hình 3.74 Trang chủ — nhóm chức năng, lời mời đăng ký và chân trang*

Phần đầu giới thiệu nền tảng học tập và luyện lập trình; các thẻ bên dưới giới thiệu Code Judge, ngân hàng câu hỏi/quiz, khóa học và trợ lý AI. Những con số như số khóa học, học viên, lượt nộp và trạng thái AC trong khung minh họa là nội dung hiển thị, chưa được đối chiếu dữ liệu hoặc kết quả chấm thật. Chân trang ghi Thymeleaf & Bootstrap 5; riêng ảnh này chưa đủ để xác nhận mã nguồn/cấu hình đã chuyển đổi. Danh sách và chi tiết khóa học được minh họa tại Hình 5.3–5.4.

### 3.8.3 Giao diện học viên

Giao diện học viên gồm dashboard, My Courses, màn hình nội dung/tiến độ, Exercise Hub, quiz/code, kết quả/lịch sử, leaderboard, blog, settings và chatbot. Mỗi màn hình cần tách dữ liệu đang tải, rỗng, lỗi và không có quyền.
Ở màn hình làm bài cần phân biệt lưu tạm, nộp và kết quả; editor có Run/Submit riêng. [Cần ảnh thật của các luồng, kèm viewport để kiểm tra layout.]

### 3.8.4 Giao diện giảng viên

Giao diện giảng viên gồm home/dashboard, quản lý course/create/edit/center, cấu trúc lesson, học viên, lịch/thông báo và khu vực bài tập. Form quiz và code có thiết lập riêng, không dùng một form giống nhau để che khác biệt dữ liệu.
AI/import phải có thông tin đủ để rà soát trước lưu. PreviewImportPage đang locked và có dữ liệu mẫu cần chú thích. [Cần ảnh course center, quiz editor, code/test case và kết quả học viên.]

### 3.8.5 Giao diện Admin

Giao diện Admin gồm home, UserManagement, AdminManagement, CourseManagement, BlogMgmt, CodeJudgeMgmt và WebContentMgmt theo source. Thao tác quản trị cần hiển thị đối tượng, trạng thái và xác nhận khi ảnh hưởng dữ liệu.

Web Content có API category, localStorage Inspire và Founders chưa persist; thiết kế hiện có không được mô tả như đã quản lý nội dung tập trung. [Cần ảnh thật từng khu vực và phản hồi lỗi/không quyền.]

### 3.8.6 Giao diện xác thực và cài đặt tài khoản

AuthPage xử lý đăng ký/đăng nhập; ForgotPasswordPage xử lý yêu cầu và đặt lại mật khẩu; SocialCallbackPage hoàn tất OAuth. SettingsPage quản lý profile/avatar/password theo các API myinfo. Dữ liệu user hiện tại lấy từ phiên, không để người dùng nhập userId khác làm chủ hồ sơ.
Form cần thông báo đúng trường, trạng thái gửi và lỗi provider/OTP. [Cần ảnh dữ liệu hợp lệ/sai và trạng thái lưu, che thông tin nhạy cảm.]

### 3.8.7 Trạng thái tải lỗi rỗng và responsive

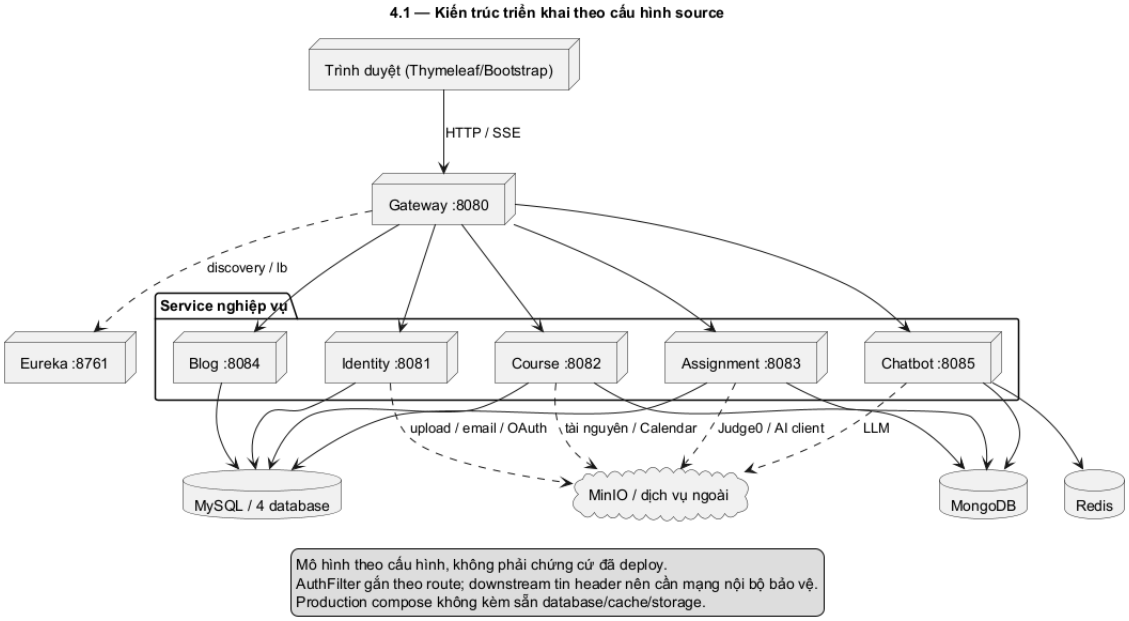
Các trạng thái chung gồm loader, toast, trang lỗi, danh sách rỗng và chuyển hướng khi mất quyền. Với bảng/sidebars/editor cần kiểm tra mobile/tablet/desktop theo Chương6; không coi CSS có media query là kết luận responsive đã đạt.
Nhóm bổ sung ảnh cùng màn hình ở nhiều viewport. Chưa tạo ảnh hoặc mockup, chưa kết luận chất lượng thị giác hay khả năng truy cập đã được kiểm chứng.

# CHƯƠNG 4 KIẾN TRÚC VÀ CÔNG NGHỆ

## 4.1 Kiến trúc tổng thể

### 4.1.1 Sơ đồ triển khai frontend gateway và microservices

Ứng dụng có Tầng giao diện Thymeleaf (web-portal / web-app)/Vite, API Gateway, Eureka Server và năm service nghiệp vụ: Identity, Course, Assignment, Blog, Chatbot. Request nghiệp vụ từ frontend được cấu hình đi tới Gateway; các route phân chia theo tiền tố /api/identity, /api/course, /api/assignment, /api/blog và /api/chatbot. Riêng upload presigned và một số health URL có luồng riêng theo cấu hình.
MySQL phục vụ dữ liệu quan hệ; MongoDB phục vụ lesson/cấu trúc, submission và chat; Redis phục vụ cache/hạn mức; MinIO lưu tệp. docker-compose.prod.yml dùng địa chỉ hostname nội bộ cho service. Sơ đồ cần tách mạng client, thành phần được công bố ra ngoài và mạng dịch vụ/database; không suy rằng source đã chứng minh toàn bộ firewall/HTTPS đang vận hành.




*Hình 4.1: Kiến trúc triển khai hệ thống EduLearn LMS theo cấu hình source thực tế*

### 4.1.2 Trách nhiệm của từng service

Identity quản lý tài khoản, xác thực, OTP, profile và API quản trị user. Course quản lý danh mục, khóa học, cấu trúc, lesson, enrollment, lịch, thông báo và tiến độ. Assignment quản lý ngân hàng câu hỏi, quiz/attempt/grade, problem/test case, submission và chấm code. Blog quản lý post/comment/vote/tag. Chatbot xây dựng ngữ cảnh, hỏi đáp, lịch sử và sinh nội dung hỗ trợ.
Gateway chịu trách nhiệm định tuyến và xác thực token ở các route bảo vệ; Registry cung cấp đăng ký/tra cứu service. Hai thành phần này không lưu thay dữ liệu nghiệp vụ. Ranh giới module giúp phân chia trách nhiệm, nhưng có thêm chi phí cấu hình, giao tiếp và xử lý

lỗi phụ thuộc; báo cáo chưa có đo đạc để kết luận microservices nhanh hơn một ứng dụng đơn khối.

**Bảng 4.1 Thành phần kiến trúc và trách nhiệm**

| Thành phần | Vai trò | Dữ liệu hoặc phụ thuộc |
|---|---|---|
| service-registry | Đăng ký và tra cứu instance | Eureka; không giữ dữ liệu học tập |
| api-gateway | Route, xác thực JWT, fallback | Eureka, signer key và cấu hình route |
| identity-service | Tài khoản, OTP, phiên, profile, Admin user | MySQL, SMTP, Google OAuth, MinIO |
| course-service | Course, lesson, enrollment, lịch, tiến độ | MySQL, MongoDB, MinIO, Calendar, API nội bộ |
| assignment-service | Quiz, code, điểm và chấm bài | MySQL, MongoDB, Judge0, Course/Chatbot API |
| blog-service | Post, comment, tag và vote | MySQL, MinIO, Identity API |
| chatbot-service | Ngữ cảnh, lịch sử, chat và sinh nội dung | MongoDB, Redis, LLM, Course/Assignment API |

### 4.1.3 Luồng request qua API Gateway

Ví dụ request xem khóa đã tham gia: Giao diện web-portal gửi request đến /api/course/courses/enrolled kèm thông tin phiên xác thực; route course-private-read-shortcuts có ưu tiên trước route public. StripPrefix=1 bỏ phần /api, AuthFilter xác minh phiên/token và gắn header nhận dạng X-User-Id; controller trong Course tiếp nhận đường dẫn dưới context /course, service lấy userId và truy vấn bảng enrollment.

Các route công khai bỏ Authorization và header nhận dạng theo cấu hình tương ứng. Như vậy, chỉ biết tiền tố URL chưa đủ xác định request có AuthFilter: cần xét Path, Method, order và các shortcut. Response quay về frontend để cập nhật state; trình duyệt không cần tải lại toàn bộ trang cho mỗi thao tác.

### 4.1.4 Eureka và service discovery

service-registry chạy Eureka để các ứng dụng đăng ký tên và địa chỉ. Gateway dùng URI lb://identity-service và các tên tương ứng để tìm instance; cấu hình có fetch-registry=true và register-with-eureka=true. Eureka mặc định theo source dùng cổng 8761, có thể đổi bằng cấu hình.

Việc đăng ký thành công cho thấy instance được biết đến, chưa chứng minh mọi API nghiệp vụ hoạt động. Cần phân biệt service discovery với health check. Trong triển khai nhiều instance, cần kiểm tra dữ liệu và tác vụ nền dùng chung; báo cáo không mặc định source đã có cơ chế đồng bộ phân tán cho mọi tác vụ.

### 4.1.5 Giao tiếp nội bộ với OpenFeign

Các interface @FeignClient biểu diễn API phụ thuộc giữa service. Ví dụ Course.IdentityClient kiểm tra user tồn tại, lấy thông tin và tìm bằng email;

Course.AssignmentClient lấy số bài tập đơn lẻ hoặc theo batch. Assignment và Chatbot có client tới những service cung cấp thông tin phục vụ chấm bài hoặc ngữ cảnh.
Một số FeignClient khai báo url=${services...url}; khi URL được cấu hình, không được mô tả tất cả lời gọi nội bộ đều tự đi qua Eureka hoặc Gateway. Fallback cũng khác nhau: có phương thức ném EXTERNAL_SERVICE_UNAVAILABLE, có phương thức trả null hoặc Map rỗng. Giao diện cần phân biệt dữ liệu thiếu do dịch vụ lỗi với kết quả thực sự bằng không.

## 4.2 Tổ chức source code và mô hình ba tầng

### 4.2.1 Cấu trúc monorepo

Repository chứa ứng dụng dưới Learning-Management-System-main. backend/pom.xml là Maven parent khai báo bảy module; frontend là ứng dụng npm riêng. Ngoài source còn có Dockerfile theo module, các Compose local/dependency/production, cấu hình Nginx, README và API_REFERENCE.md.
Mô hình monorepo ở đây nghĩa là các thành phần được quản lý trong một repository; không đồng nghĩa backend và frontend chạy chung một process. Khi build backend, Maven xử lý module Java; khi build frontend, Vite xử lý bundle trình duyệt. Phân tích báo cáo dùng commit 4288b95 để cố định căn cứ.

### 4.2.2 Controller và tầng giao tiếp

Controller định nghĩa endpoint, HTTP method, DTO nhận/trả và các điều kiện như @Valid hoặc @PreAuthorize. Ví dụ CourseController tiếp nhận đăng ký khóa học; QuizAttemptController tiếp nhận start/resume, autosave, submit và đọc kết quả. Controller chuyển yêu cầu nghiệp vụ sang service.
Không phải mọi controller tự kiểm tra toàn bộ quyền. Một số rule đặt ở SecurityConfig hoặc service; vì vậy, đánh giá bảo mật phải theo cả tuyến xử lý. Controller không nên được mô tả như nơi trực tiếp chấm mọi test code hoặc tự chứa toàn bộ truy vấn database.

### 4.2.3 Service và tầng nghiệp vụ

Service thực hiện quy tắc nghiệp vụ và phối hợp repository/client.
CourseServiceImpl.enrollCourse kiểm tra course PUBLIC, không LOCKED và chưa đăng ký trùng. QuizAttemptServiceImpl quản lý vòng đời lượt quiz; SubmissionServiceImpl kiểm tra code/ngôn ngữ, compile preflight và tạo PENDING trước gọi chấm bất đồng bộ.
Các transaction được áp dụng ở những luồng source khai báo; không suy @Transactional tạo transaction chung giữa MySQL, MongoDB, SMTP và API ngoài. Service còn có helper kiểm tra chủ khóa học/tác giả/lượt làm; đây là phần cần giải thích khi bảo vệ về quyền trên dữ liệu.

### 4.2.4 Repository và tầng truy cập dữ liệu

Repository cung cấp thao tác persistence theo công nghệ từng loại dữ liệu. JPA repository xử lý bảng MySQL như Course, Enrollment, QuizAttempt, Post; Mongo repository xử lý CourseStructure, Lesson, Submission và chat. Các truy vấn theo user/course/status phục vụ danh sách và thống kê.

Repository không thay cho validation HTTP hoặc kiểm tra quyền nghiệp vụ. Ràng buộc database như unique enrollment hay unique lượt IN_PROGRESS là lớp bảo đảm dữ liệu quan trọng, nhưng lỗi vi phạm cần được service/exception handler chuyển thành phản hồi phù hợp.

### 4.2.5 DTO mapper entity document và validator

DTO giới hạn dữ liệu request/response; entity biểu diễn persistence JPA; document biểu diễn dữ liệu MongoDB. Mapper dùng để chuyển đổi, với MapStruct được cấu hình annotation processor ở parent POM. Validator bổ sung kiểm tra như tuổi hoặc số lượng đáp án theo loại câu hỏi.
Không trả entity trực tiếp được xem là mục tiêu phân tách dữ liệu, nhưng phải kiểm tra từng response thực tế để xác nhận không lộ dữ liệu nhạy cảm. DTO validation và độ rộng SQL có thể khác nhau; báo cáo ghi đúng từng tầng thay vì lấy giới hạn cột làm giới hạn form.

### 4.2.6 Cấu trúc frontend theo feature

Frontend tổ chức theo feature: auth, courses, assignment, blog, dashboard, settings và aboutus. Mỗi feature chứa những phần pages/components/styles hoặc cấu trúc con phù hợp. Việc đặt tên student/teacher trong thư mục là cách tổ chức giao diện; role trong code là STUDENT/INSTRUCTOR/ADMIN.
Cấu trúc này giúp tìm một chức năng theo nghiệp vụ, nhưng không tạo ranh giới bảo mật. Code trong trình duyệt vẫn là code client; mọi quyền thay đổi dữ liệu phải được backend xác nhận.

### 4.2.7 Routes layouts context hooks services và utils

routes xác định đường dẫn và bảo vệ giao diện; layouts tổ chức khung theo role; context quản lý trạng thái như Spring Session & Security Context; hooks đóng gói hành vi dùng lại; services bọc API; utils xử lý các phép biến đổi/validation tiện ích. configurations/env.js cung cấp địa chỉ và timeout.
Ví dụ một trang gọi courseApi, courseApi dùng apiClient, apiClient gắn token và xử lý refresh. Trang nhận dữ liệu rồi render component. Chỉ có route đến một component chưa chứng minh chức năng đã hoàn chỉnh; /manage/assignments hiện trỏ AboutUsPage là trường hợp cần ghi đúng phạm vi.

## 4.3 Công nghệ backend

### 4.3.1 Java Maven và Spring Boot

backend/pom.xml khai báo Java 21, Spring Boot parent 3.3.5 và Spring Cloud BOM 2023.0.1. Maven quản lý dependency, build module và annotation processor. Phiên bản này là cấu hình của source, không phải khuyến nghị phiên bản mới nhất.
Spring Boot cung cấp cấu hình và khởi động service; các dependency khác nằm trong từng module POM. Việc POM có dependency chưa chứng minh mọi chức năng của thư viện đều được dùng. Báo cáo chỉ giải thích những cơ chế gắn với implementation.

### 4.3.2 Spring Cloud Gateway và Eureka

Gateway dùng Spring Cloud Gateway với filter phản ứng trên ServerWebExchange, Mono và DataBuffer. Khi token sai, AuthFilter tạo response JSON theo API reactive, thay vì dùng HttpServletResponse. Các service nghiệp vụ sử dụng cấu hình HTTP riêng.
Eureka và URI lb:// phục vụ việc tìm địa chỉ service ở Gateway. Route public/private và ưu tiên route là nội dung cốt lõi cần giải thích. Không gọi AuthFilter là GlobalFilter áp dụng tuyệt đối cho mọi route: class thực tế kế thừa AbstractGatewayFilterFactory và được gắn bằng tên trong YAML.

### 4.3.3 Spring Security JWT và Google OAuth

Spring Security ở service thiết lập public endpoints, security context và method security. Gateway dùng JwtUtil kiểm tra chữ ký/hạn và claim type=ACCESS; Identity tạo JWT bằng HS512 và quản lý refresh token. Google OAuth là phương án xác thực xã hội riêng.
Phân quyền service dựa vào authorities từ header và @PreAuthorize/kiểm tra owner. Cần phân biệt xác thực danh tính với kiểm tra quyền thực hiện hành động. Source có những giới hạn đã ghi trong Chương 3, như điều kiện verified ở login và tuyến đọc lịch sử chat.

### 4.3.4 JPA Hibernate và Spring Data MongoDB

Spring Data JPA/Hibernate ánh xạ entity và quan hệ với MySQL; Spring Data MongoDB lưu document linh hoạt. Course dùng MySQL cho metadata/enrollment và MongoDB cho cấu trúc/lesson. Assignment kết hợp bảng quan hệ với document submission chứa kết quả test nhúng.
Đây là lựa chọn persistence của đồ án, không phải mỗi request đều phải dùng cả hai database. Liên kết giữa hai nơi lưu bằng ID không tự có FK hoặc cascade. Khi xóa hoặc thay đổi cấu trúc cần kiểm tra cơ chế nghiệp vụ và xử lý dữ liệu còn lại.

### 4.3.5 MySQL MongoDB và Flyway

MySQL schema được khai báo qua migration Flyway V1, còn V2 bổ sung index và unique phục vụ truy vấn. Chương 3 đã mô tả 26 bảng theo migration. MongoDB có năm collection được đối chiếu document/entity; schema linh hoạt không có nghĩa service được bỏ validation.
Flyway hỗ trợ quản lý các thay đổi schema theo thứ tự, nhưng cần log/version table của môi trường demo để xác nhận migration đã áp dụng. Không dùng schema source thay cho ảnh hoặc kết quả kiểm tra database đang chạy.

### 4.3.6 Redis và MinIO

RedisCacheServiceImpl cache tin nhắn bằng list với khóa chat:session:{sessionId}:messages; TTL và giới hạn số tin lấy từ cấu hình.
RateLimiterServiceImpl dùng bộ đếm theo user/ngày và Lua để kiểm tra/tăng nguyên tử. Khi Redis cache lỗi, có luồng quay về MongoDB; rate limit hiện fail-open.
MinIO lưu object cho ảnh/tài nguyên, database giữ URL hoặc định danh. Một số luồng dùng presigned URL để client upload trực tiếp. Bucket, endpoint, URL public và quyền truy cập phải phù hợp môi trường; không coi URL đã cấp là bằng chứng tệp đã tải lên thành công.

### 4.3.7 Resilience4j và cơ chế fallback

Gateway có CircuitBreaker và TimeLimiter của Resilience4j. Cấu hình mặc định gồm cửa sổ 10 lời gọi, tối thiểu 5 lời gọi, ngưỡng lỗi 50%, mở 10 giây và 3 lời gọi khi half-open; các giá trị có thể đổi bằng biến môi trường.
Fallback chuyển request tới endpoint /fallback/<service>. Cơ chế này giúp phản hồi khi phụ thuộc lỗi, không sửa dữ liệu sai hoặc bảo đảm dịch vụ luôn sẵn sàng. Thời gian và threshold trong YAML là cấu hình, chưa phải số liệu thực nghiệm.

## 4.4 Công nghệ giao diện người dùng và cơ chế bảo mật phía máy chủ

### 4.4.1 Kiến trúc Server-Side Rendering với Spring Boot và Thymeleaf

Tầng hiển thị người dùng của EduLearn LMS được xây dựng hoàn toàn theo mô hình Server-Side Rendering (SSR) sử dụng Spring Boot Web Starter và Thymeleaf template engine. Cấu trúc frontend được phân tách rõ ràng thành hai cổng dịch vụ độc lập:
• web-portal (Cổng học tập & luyện thi cho Sinh viên, cổng HTTP 8086): Phụ trách hiển thị danh mục khóa học, bài học video, phòng làm bài trắc nghiệm, trình soạn thảo mã nguồn Monaco Editor và cửa sổ AI Chatbot tương tác thời gian thực.
• web-app (Cổng quản trị & đào tạo cho Giảng viên và Admin, cổng HTTP 8088): Phụ trách xác thực tập trung, studio tạo khóa học, quản lý cây bài giảng, quản lý học viên, quản trị người dùng và kiểm duyệt bài viết Blog.
Mô hình Server-Side Rendering giúp toàn bộ mã HTML được kết xuất và nhúng dữ liệu trực tiếp tại máy chủ trước khi truyền về trình duyệt. Thư mục mã nguồn giao diện đặt tại src/main/resources/templates/ với cấu trúc module hóa cao độ: layout dùng chung (main-layout.html), các thành phần giao diện tái sử dụng (navbar.html, footer.html, sidebar.html) nhúng qua cú pháp th:replace và th:fragment. Biểu mẫu dữ liệu sử dụng th:action, th:object và th:field, giúp tự động liên kết (data binding) chặt chẽ giữa Model của Spring MVC và form HTML.

### 4.4.2 Kiểm soát truy cập và phân quyền theo vai trò với Spring Security

Toàn bộ cơ chế điều hướng và phân quyền truy cập được quản lý chặt chẽ ở tầng máy chủ thông qua Spring Security (SecurityConfig.java). Khác với các ứng dụng Single Page Application (SPA) thuần túy phụ thuộc vào định tuyến phía client (như Bộ lọc bảo vệ Spring Security hay Spring Session & Security Context), hệ thống sử dụng SecurityFilterChain để kiểm soát nghiêm ngặt từng URL pattern ngay khi request chạm tới server:
• Tuyến công khai (permitAll): Tuyến trang chủ (/), xem danh mục khóa học (/courses), diễn đàn Blog (/blog/**), tài nguyên tĩnh (/css/**, /js/**, /webjars/**).
• Phân hệ học tập: Yêu cầu xác thực và sở hữu quyền STUDENT (/student/**, /learning/**).
• Phân hệ giảng dạy: Yêu cầu quyền INSTRUCTOR (/instructor/**) để truy cập studio bài giảng và quản lý đề thi.
• Phân hệ quản trị: Yêu cầu đặc quyền ADMIN (/admin/**) để giám sát người dùng, phân quyền và kiểm duyệt dữ liệu.
Khi người dùng chưa đăng nhập cố gắng truy cập các tài nguyên bảo vệ, Spring Security tự động chặn request và chuyển hướng về trang /login thông qua SavedRequestAwareAuthenticationSuccessHandler, lưu lại URL dự định truy cập để điều

hướng chính xác sau khi đăng nhập thành công. Cơ chế phân quyền phía máy chủ đảm bảo rằng người dùng không thể can thiệp hoặc vượt mặt kiểm tra quyền bằng cách chỉnh sửa trạng thái JavaScript ở trình duyệt.

### 4.4.3 Quản lý phiên làm việc, Cookie bảo mật và hạn chế rủi ro an ninh mạng

Cơ chế xác thực của ứng dụng kết hợp giữa phiên máy chủ (Spring Session) và mã thông báo bảo mật JWT. Cấu hình bảo mật tại SecurityConfig.java thiết lập DefaultCookieSerializer với cookieName="EDUSESSION", useHttpOnlyCookie=true, và sameSite="Lax". Đồng thời, cơ chế bảo vệ Cross-Site Request Forgery (CSRF) được kích hoạt qua csrf(Customizer.withDefaults()), tự động nhúng CSRF Token vào tất cả biểu mẫu POST/PUT của Thymeleaf.
Phân tích an ninh mạng: Việc gắn cờ HttpOnly cho cookie phiên giúp ngăn chặn mã JavaScript độc hại trên trang đọc được giá trị cookie, từ đó phòng ngừa hiệu quả kỹ thuật đánh cắp mã phiên (Session Hijacking) qua tấn công Cross-Site Scripting (XSS). Tuy nhiên, báo cáo khẳng định một cách khoa học rằng: Cờ HttpOnly KHÔNG THỂ loại bỏ hoàn toàn mọi rủi ro của XSS. Nếu kẻ tấn công chèn được mã kịch bản độc hại vào trang web, script đó vẫn có thể âm thầm thực hiện các thao tác giả mạo (thao tác DOM, kích hoạt HTTP request dưới danh nghĩa người dùng đang đăng nhập). Vì vậy, hệ thống thiết lập cơ chế phòng thủ theo chiều sâu: kết hợp cờ HttpOnly, cờ SameSite=Lax (ngăn chặn gửi cookie tự động trong truy vấn cross-site), mã xác thực CSRF Token bắt buộc trên mỗi biểu mẫu nhạy cảm, và cơ chế tự động mã hóa ký tự (HTML escaping) của Thymeleaf engine khi hiển thị dữ liệu người dùng.

### 4.4.4 Tích hợp trình soạn thảo mã nguồn Monaco Editor

Phân hệ làm bài tập lập trình trực tuyến trên web-portal tích hợp Monaco Editor — lõi biên tập mã nguồn danh tiếng của Visual Studio Code. Monaco Editor cung cấp trải nghiệm lập trình chuyên nghiệp ngay trên nền web với tính năng làm nổi bật cú pháp (syntax highlighting), gợi ý mã thông minh (IntelliSense) và đánh số dòng trực quan cho các ngôn ngữ lập trình phổ biến: Java, Python, C++, C, JavaScript.
Phân định vai trò hệ thống: Monaco Editor chỉ đóng vai trò thu nhận văn bản mã nguồn ở tầng trình duyệt. Toàn bộ quy trình biên dịch, khởi tạo môi trường thực thi (sandbox), nạp dữ liệu đầu vào và kiểm tra đối chiếu bộ test case hoàn toàn do Judge Engine độc lập ở backend đảm trách. Giao diện web-portal tách biệt rõ ràng giữa thao tác 'Chạy thử' (Run Code với dữ liệu mẫu) và 'Nộp bài' (Submit Code để chấm điểm chính thức).

### 4.4.5 Hệ thống lưới Bootstrap 5, giao diện tùy biến và đánh giá responsive

Hệ thống sử dụng Bootstrap 5.3.3 tích hợp qua WebJars (org.webjars:bootstrap:5.3.3) kết hợp bộ icon trực quan Bootstrap Icons (1.11.3). Tầng giao diện tận dụng tối đa hệ thống lưới 12 cột (12-column grid system), Flexbox, cùng các lớp tiện ích (Spacing, Colors, Borders, Typography) giúp chuẩn hóa giao diện nhất quán giữa các trang chức năng.
Về đánh giá tương thích đa thiết bị (Responsive): Hệ thống đáp ứng tốt trên máy tính để bàn, máy tính xách tay và máy tính bảng. Đối với thiết bị di động thông minh (màn hình hẹp), thanh điều hướng (Navbar) hỗ trợ co gọn thành menu burger tiện lợi. Tuy nhiên, các

màn hình có cấu trúc phức tạp như Bảng quản trị người dùng nhiều cột và trình soạn thảo Monaco Editor đòi hỏi không gian hiển thị rộng, hiện vẫn đang tiếp tục được tối ưu hóa giao diện di động trong các bản phát hành tiếp theo.

### 4.4.6 Quá trình chuyển đổi từ Prototype React sang Kiến trúc Thymeleaf chính thức

Nhằm phản ánh trung thực quá trình nghiên cứu và phát triển phần mềm, báo cáo ghi nhận rõ lịch sử tiến hóa kiến trúc của dự án:
• Giai đoạn ban đầu: Nhóm xây dựng bản thử nghiệm (prototype) Single Page Application bằng React 18 + Vite (lưu tại thư mục frontend/) để khảo sát luồng gọi REST API và thiết kế bố cục giao diện ban đầu.
• Giai đoạn hoàn thiện sản phẩm: Để đáp ứng trọn vẹn yêu cầu môn học về lập trình web máy chủ (Server-Side Web Development với Java), đồng thời khắc phục triệt để sự phức tạp trong việc quản lý đồng bộ trạng thái token phía client và nguy cơ rò rỉ token qua JavaScript storage, toàn bộ hệ thống giao diện chính thức đã được tái cấu trúc và chuyển đổi hoàn toàn sang mô hình Spring Boot MVC kết hợp Thymeleaf và Bootstrap 5.
Kiến trúc Thymeleaf chính thức giúp hệ thống vận hành ổn định, đồng bộ bảo mật máy chủ thông qua Spring Security, và tối ưu hóa hiệu năng render trang. Thư mục frontend cũ được lưu giữ như tài liệu đối chiếu kiến trúc và minh chứng cho quá trình nghiên cứu, nâng cấp sản phẩm của nhóm.

## 4.5 Tích hợp dịch vụ

### 4.5.1 Email SMTP và OTP

EmailVerificationServiceImpl tạo OTP sáu chữ số bằng SecureRandom, lưu type, expiresAt, attempts và usedAt; EmailService gửi qua SMTP. VERIFY_EMAIL và RESET_PASSWORD được xử lý riêng, vì tác động cuối cùng khác nhau.
Khi verify đúng, ghi usedAt và verified=true; reset đúng thì lưu hash mật khẩu mới. Gửi mail cần cấu hình tài khoản SMTP thực tế. Source có log liên quan OTP; phần bảo mật không tuyên bố log hoàn toàn không chứa mã.

### 4.5.2 Google OAuth

Google OAuth gồm cấu hình frontend, redirect URI và xử lý backend. SocialCallbackPage tiếp nhận kết quả theo luồng frontend; AuthenticationServiceImpl xử lý provider và tài khoản, trả token và trạng thái yêu cầu chọn role.
Endpoint PATCH myinfo/role hiện gọi UserServiceImpl.updateRole và chỉ ánh xạ STUDENT/INSTRUCTOR, không cấp ADMIN. Chưa có điều kiện provider, cửa sổ 30 phút hay chọn một lần tại method đang gọi. SelectRoleServiceImpl có các điều kiện đó nhưng chưa thấy controller sử dụng, nên không coi là rule API đã hoạt động. OAuth đăng nhập khác với service account Google Calendar.

### 4.5.3 Tạo liên kết Google Meet qua Google Calendar API

GoogleCalendarConfig đọc service-account credentials từ classpath hoặc file, cấp scope CALENDAR_EVENTS và createDelegated theo delegatedEmail. GoogleMeetServiceImpl

tạo Calendar event dạng ngày, yêu cầu conferenceDataVersion=1 và conference solution, rồi nhận hangoutLink/eventId để gắn với course.
Khi IOException hoặc GeneralSecurityException xảy ra, createMeeting hiện trả URL dự phòng dạng meet.google.com/lms-... với eventId=null. URL này do code sinh, không chứng minh có phòng Google Meet tồn tại. Vì vậy, báo cáo chỉ ghi đã có luồng tích hợp và fallback; muốn xác nhận họp thật cần eventId hợp lệ và thử mở/tham gia phòng. ScheduleMapper lấy meetingUrl từ course.
[Cần nhóm bổ sung ảnh phòng Meet thực sự truy cập được và minh chứng tạo event nếu sử dụng chức năng này trong demo.]

### 4.5.4 Judge0 và giới hạn của custom judge chưa triển khai

SubmissionServiceImpl kiểm tra dữ liệu, quyền, ngôn ngữ và compile preflight; lưu submission PENDING rồi gọi tác vụ chấm bất đồng bộ. Judge0Engine là implementation được sử dụng cho đường chạy/chấm đã xác định; kết quả lưu trong submission và test_case_result.
CustomEngine chứa nhánh báo chưa triển khai và yêu cầu dùng Judge0. Do đó, không mô tả custom engine là chức năng đã hoàn thành. Tính năng chạy/chấm còn phụ thuộc endpoint/credential của Judge0. [Cần ảnh Run/Submit và các trạng thái kết quả thực tế; không tự tạo ảnh đầu ra.]

### 4.5.5 LLM provider và chatbot

ChatServiceImpl lấy phiên, lịch sử qua HistoryManager, xây DomainContext bằng các client, tạo prompt rồi gọi client LLM. Flux đầu tiên gửi chuỗi [SESSION:<id>], tiếp theo là text chunks; khi stream hoàn tất và response không rỗng, saveExchange lưu lượt hỏi/đáp.
Cấu hình LLM provider/model và client implementation cần theo môi trường. Chatbot trả qua SSE; sinh quiz/test case có parse/kiểm tra kết quả và luồng preview riêng. Chưa thấy pipeline embedding/vector search trong các phần đã rà soát nên không gọi cơ chế lấy ngữ cảnh này là RAG vector. Nội dung AI cần được giảng viên/người học kiểm tra.

## 4.6 Thiết kế bảo mật và xử lý lỗi

### 4.6.1 Băm mật khẩu và kiểm tra thông tin đầu vào

Mật khẩu được encode/matches bằng BCrypt ở Identity. DTO dùng @NotBlank, @Size, @Pattern và validator tùy chỉnh; @Valid kích hoạt kiểm tra request ở các endpoint tương ứng. Database bổ sung unique/FK nhưng không thay validation nghiệp vụ.
Ví dụ username/email trùng bị service từ chối, reset cần hai mật khẩu khớp, code có giới hạn 65536 ký tự. authenticate hiện kiểm tra active và password, chưa kiểm tra verified. Báo cáo phải phân biệt yêu cầu kích hoạt email với điều kiện đăng nhập đã triển khai.

### 4.6.2 Vòng đời JWT và refresh token

Identity tạo access và refresh JWT bằng HS512, có subject userId và claim scope/type theo source. Gateway chỉ chấp nhận type ACCESS; refreshToken xác minh REFRESH, tìm tokenHash, kiểm tra revoked, thu hồi token cũ rồi lưu token mới. Frontend cập nhật token trong cùng loại storage.

Logout xử lý token theo Identity, nhưng Gateway.verify không thể hiện tra database revoked cho từng access token. Vì vậy, không nói logout hoặc khóa user làm mọi access token vô hiệu ngay lập tức. refreshToken được đọc chưa có kiểm tra active như authenticate; cần kiểm thử trường hợp tài khoản bị khóa nhưng còn refresh token. Thời gian hiệu lực lấy từ cấu hình, không tự ghi một con số cố định.

### 4.6.3 Xác thực ở gateway và phân quyền tại service cùng frontend

AuthFilter xóa Authorization và các X-User-* do client gửi, xác minh token rồi đặt X-User-Id, X-User-Role, X-User-Email và X-Gateway-Authenticated. Course SecurityConfig đọc header, tạo GatewayAuthentication và authorities để method security xử lý.
Filter Course được đọc kiểm tra userId, nhưng không chứng minh nguồn header bằng chữ ký riêng. Mô hình tin Gateway cần các cổng downstream chỉ được truy cập từ mạng tin cậy. docker-compose.prod.yml có địa chỉ nội bộ nhưng source Compose không thay bằng chứng cấu hình host/firewall thực tế. Không lặp lại nhận xét trong comment rằng header không thể giả mạo trong mọi điều kiện.
[Cần ảnh hoặc cấu hình triển khai đã che thông tin nhạy cảm để xác nhận các cổng công khai và downstream nội bộ.]

**Bảng 4.2 Các lớp kiểm tra quyền và giới hạn**

| Vị trí kiểm tra | Cơ chế hiện có | Giới hạn cần đối chiếu |
|---|---|---|
| Frontend | PrivateRoute, RoleRoute | Bảo vệ điều hướng; không thay quyền API |
| Gateway | JWT type ACCESS, chữ ký và thời hạn | Không thấy kiểm tra revoked DB cho mỗi access token |
| Service SecurityConfig | Nhận header và tạo authorities | Phụ thuộc ranh giới mạng tin cậy |
| Service nghiệp vụ | Owner, quan hệ và trạng thái dữ liệu | Cần thử truy cập chéo từng chức năng |
| Database | PK, FK, unique và index | Không bảo đảm FK xuyên service/MongoDB |

### 4.6.4 Kiểm tra quyền sở hữu khóa học bài tập và bài viết

Quyền trên đối tượng được kiểm tra ở service: owner course/instructorId, tác giả post/comment, chủ attempt/submission và quan hệ lesson–course hoặc comment–post. Role là điều kiện chung, nhưng một Instructor không tự được sửa course của Instructor khác.
Tuyến getMessages/getMessageResponses được đối chiếu chưa thấy kiểm tra chủ phiên tương đương archiveSessionByUser. Mục này cần ghi như điểm chưa hoàn thiện và bổ sung thử nghiệm truy cập chéo; không biến yêu cầu bảo vệ dữ liệu thành kết luận đã đáp ứng. Các luồng start quiz cũng phải được đánh giá theo điều kiện thực sự có trong implementation.

### 4.6.5 Giới hạn OTP upload và yêu cầu AI

OTP có thời hạn và giới hạn thử theo cấu hình; gửi xác minh email chặn khi đang có OTP hợp lệ. Upload có kiểm tra dữ liệu/tệp ở từng service và helper frontend; các mức giới hạn khác nhau cần mô tả theo chức năng, không gộp thành một giới hạn chung.
Rate limit chatbot dùng Redis Lua theo user/ngày và role; khi Redis lỗi hiện fail-open. Việc lưu lịch sử có giới hạn số tin lấy vào context/cache, không đồng nghĩa dữ liệu Mongo bị xóa tương ứng. Không đưa secret, token, OTP thật vào báo cáo hoặc ảnh minh chứng.

### 4.6.6 CORS và cấu hình biến môi trường

Gateway globalcors cấu hình origin từ FRONTEND_URL, các HTTP method cho phép, header và allow-credentials. CORS kiểm soát truy cập theo trình duyệt; không thay xác thực, phân quyền hoặc ngăn client ngoài trình duyệt gọi API.
Cấu hình tách qua biến môi trường cho JWT, SMTP, OAuth, LLM, Judge0 và storage. Source có giá trị JWT mặc định trong YAML; khi triển khai cần thay bằng cấu hình riêng, không sao chép giá trị đó vào báo cáo. Các biến VITE_* được build cho client nên không dùng để chứa secret backend. HTTPS và cổng truy cập phải được xác minh tại môi trường demo.

### 4.6.7 Chuẩn hóa API response và exception

Các service sử dụng ApiResponse cho dữ liệu nghiệp vụ và exception/error code để diễn giải lỗi. AuthFilter tạo JSON gồm code, message, timestamp, status và path; error handler của service có thể có trường khác. Frontend có helper appError để xử lý thông báo.
Do các lớp phản hồi chưa hoàn toàn cùng một hình dạng, báo cáo không tuyên bố mọi lỗi đều có schema giống nhau. Validation, unauthenticated, forbidden, not found và lỗi dịch vụ cần có kịch bản riêng. Fallback trả null/Map rỗng không được trình bày như dữ liệu thống kê đầy đủ.

### 4.6.8 Timeout circuit breaker và fallback

Gateway cấu hình connect timeout mặc định 10000 ms, HTTP response timeout 30s; TimeLimiter mặc định 10s, Assignment 15s, Assignment AI và Chatbot 130s, Course 15s theo YAML. Các giá trị đều có thể thay bằng env. Frontend Axios mặc định 15000 ms; gọi stream/chat hoặc AI cần đối chiếu timeout riêng tại nơi sử dụng.
Tăng TimeLimiter lên 130s không tự chứng minh toàn tuyến chờ được 130s khi timeout HTTP/client khác ngắn hơn. Circuit breaker và fallback hỗ trợ xử lý lỗi phụ thuộc, không bảo đảm request đã lưu dữ liệu thì rollback được toàn bộ hệ thống. Cần đo thời gian và thử lỗi thực tế ở đợt kiểm thử; chưa có số liệu để khẳng định độ trễ hoặc tỷ lệ sẵn sàng.

# CHƯƠNG 5 XÂY DỰNG VÀ MINH HỌA CÁC CHỨC NĂNG

Cách đọc phần triển khai
Mỗi chức năng được trình bày theo đầu vào/đầu ra, luồng xử lý, API/kiểm tra dữ liệu, nghiệp vụ/lưu trữ, code, giao diện, ngoại lệ và kiểm thử. /api là tiền tố qua Gateway; dấu ... trong danh sách API là viết tắt, phải dùng đường đầy đủ ở phụ lục B khi gọi thật.
Các đoạn code có tên file và dòng đối chiếu, chỉ là trích đoạn; cần đọc toàn hàm, caller và phần lưu/trả dữ liệu. Không suy hành vi chỉ từ comment. Frontend ẩn nút không thay kiểm tra quyền backend; thành công một bước không bảo đảm upload/SMTP/LLM/judge và toàn chuỗi phụ thuộc đã hoàn thành.
Ảnh và kết quả runtime chưa có minh chứng vẫn giữ nhãn cần bổ sung hoặc Chưa kiểm thử. Các nguyên tắc chung này áp dụng cho toàn bộ Chương 5 để tránh lặp trong từng mục.

## 5.1 Identity Service

### 5.1.1 Đăng ký tài khoản

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: UserCreationRequest chứa username, password, fullname, email, phone, dob và role. Username tối thiểu 5 ký tự; password tối thiểu 8 và không có khoảng trắng; fullname có giới hạn 3–50. Service kiểm tra tồn tại username/email trước encode. Role chỉ thuộc enum đăng ký, không nhận ADMIN tùy ý.
Tác nhân: Guest. Tiền điều kiện: Chưa có username/email trùng; dữ liệu thỏa UserCreationRequest.
Đầu ra: Tài khoản LOCAL được lưu; đăng ký chưa đồng nghĩa email đã xác minh.
• Luồng xử lý và sơ đồ hoạt động
1. Mở đăng ký và nhập thông tin. 2. Gửi request register. 3. Backend kiểm tra trùng username/email và validation. 4. Băm mật khẩu, đặt provider LOCAL và lưu User. 5. Trả thông tin tài khoản.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/identity/auth/register.
Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java cùng các controller liên quan ở 3.3.
UserCreationRequest chứa username, password, fullname, email, phone, dob và role. Username tối thiểu 5 ký tự; password tối thiểu 8 và không có khoảng trắng; fullname có giới hạn 3–50. Service kiểm tra tồn tại username/email trước encode. Role chỉ thuộc enum đăng ký, không nhận ADMIN tùy ý.
• Xử lý nghiệp vụ và truy cập dữ liệu
userMapper.toUser tạo entity; passwordEncoder.encode thay mật khẩu rõ bằng hash; provider LOCAL và roleSelected=true được đặt trước save. Email verification là luồng riêng. Cần thử race condition đăng ký trùng và lỗi unique database.

Đối chiếu dữ liệu theo UC001.

• Source code trọng tâm và giải thích

Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java, bắt đầu dòng 78. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public UserCreationResponse createUser(UserCreationRequest request) {
    if (userRepository.existsByUsername(request.getUsername())) {
        throw new IdentityException(ErrorCode.USER_EXISTED);
    }
    if (userRepository.existsByEmail(request.getEmail())) {
        throw new IdentityException(ErrorCode.EMAIL_EXISTED);
    }
    User user = userMapper.toUser(request);
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    user.setAuthProvider(AuthProvider.LOCAL);
    user.setRoleSelected(true);
    userRepository.save(user);
```

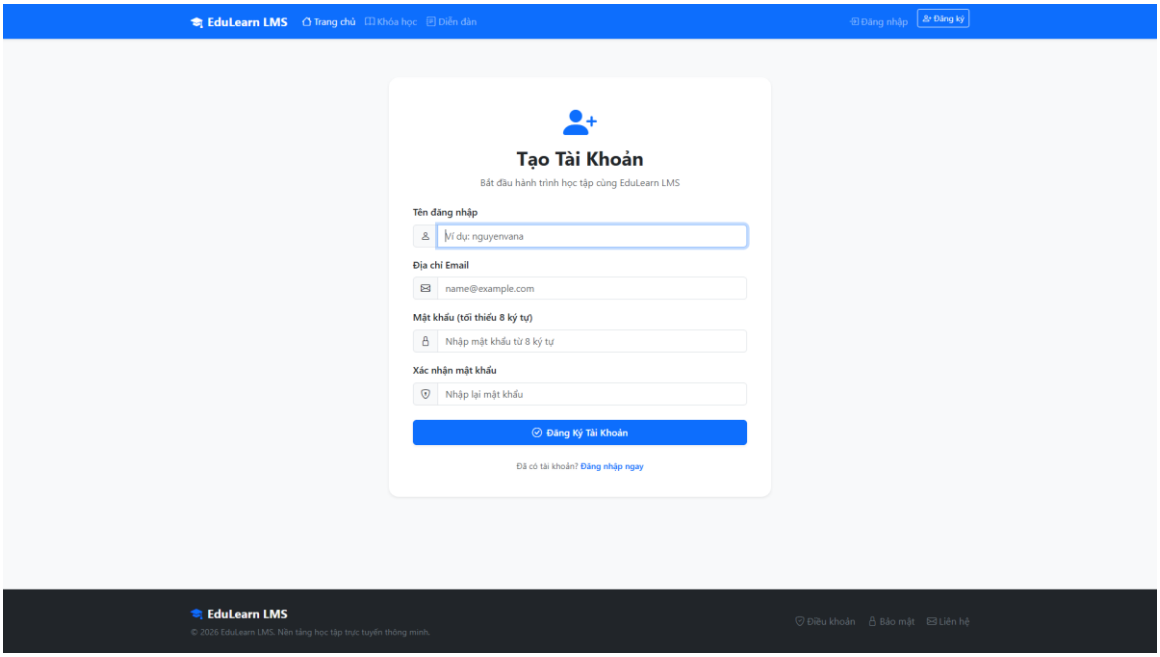
Giải thích: userMapper.toUser tạo entity; passwordEncoder.encode thay mật khẩu rõ bằng hash; provider LOCAL và roleSelected=true được đặt trước save. Email verification là luồng riêng. Cần thử race condition đăng ký trùng và lỗi unique database.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: AuthPage. Demo theo đúng role và luồng API được mô tả.

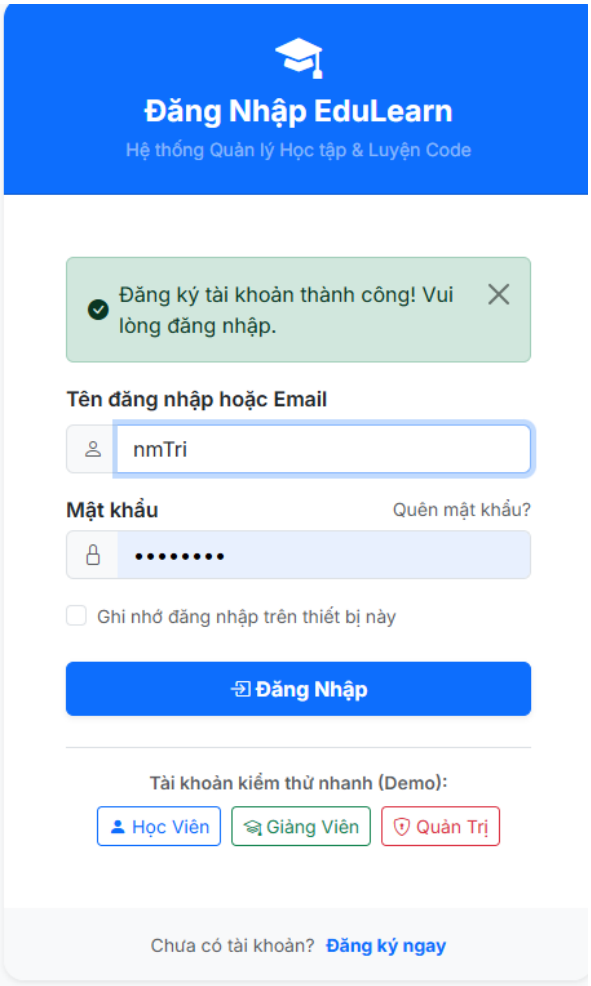
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

Minh chứng MC01 do nhóm cung cấp ngày 24/10/2026:




*Hình 5.1.1: Giao diện đăng ký tài khoản người dùng mới trên web-app*

*Hình 5.1 Form đăng ký tài khoản với vai trò Học viên*




*Hình 5.2 Giao diện thông báo đăng ký tài khoản thành công*

Người dùng nhập họ tên Nguyễn Minh Trí, tên đăng nhập nmTri, email, mật khẩu và xác nhận mật khẩu; chọn vai trò Học viên và đồng ý điều khoản. Ảnh tiếp theo hiển thị thông báo "Đăng ký tài khoản thành công! Vui lòng đăng nhập." trên trang đăng nhập. Hai ảnh minh họa dữ liệu nhập và kết quả hiển thị, chưa chứng minh bản ghi database hoặc email đã được xác minh.

Giới hạn: nhóm xác nhận chưa tích hợp phần kiểm tra lỗi trùng username/email trong phiên bản giao diện này. Không đánh dấu nhánh kiểm tra trùng đã đạt; phần phân tích source tại commit 4288b95 vẫn được phân biệt với kết quả demo hiện tại.

• Ngoại lệ và tình huống biên

USER_EXISTED, EMAIL_EXISTED hoặc lỗi DTO. Chưa khẳng định login bị chặn khi verified=false: authenticate hiện kiểm tra active và mật khẩu.

Luồng thay thế: Người dùng chuyển sang luồng gửi/xác minh OTP riêng.

Cần kiểm chứng riêng: Thử username/email đã tồn tại, mật khẩu dưới 8 ký tự, fullname sai, role không hợp lệ; đối chiếu không tạo User khi request bị từ chối.

• Kịch bản kiểm tra và minh chứng

Trạng thái minh chứng: đã có ảnh nhập thông tin và giao diện thông báo đăng ký thành công. Đây là xác nhận kết quả hiển thị do nhóm cung cấp; chưa có request/response hoặc dữ liệu database để nghiệm thu toàn luồng. Theo nhóm, phần kiểm tra lỗi trùng email/username chưa tích hợp trong phiên bản chụp.

TC-5-1-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-1-1-02: phiên bản nhóm đang chụp chưa tích hợp kiểm tra lỗi trùng username/email; chưa có ảnh cho nhánh này. Các kiểm tra mật khẩu dưới 8 ký tự, fullname sai, role không hợp lệ vẫn chờ chạy và ghi kết quả thực tế.
TC-5-1-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.1.2 Gửi và xác minh OTP email

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: OTP có loại VERIFY_EMAIL, expiresAt, attempts và usedAt; mã sáu chữ số được tạo SecureRandom. maxAttempts mặc định trong @Value là 5; thời hạn lấy cấu hình, không tự ghi cố định.
Tác nhân: Guest có tài khoản cần xác minh. Tiền điều kiện: Email thuộc tài khoản chưa verified; OTP còn hạn, chưa dùng và chưa vượt giới hạn thử.
Đầu ra: Email được xác minh khi OTP đúng; ghi nhận mã đã sử dụng.
• Luồng xử lý và sơ đồ hoạt động
1. Nhập email và yêu cầu gửi mã. 2. Backend kiểm tra tài khoản và OTP đang hợp lệ. 3. Tạo mã sáu chữ số, lưu loại VERIFY_EMAIL và gửi SMTP. 4. Nhập OTP. 5. Kiểm tra mã; đánh dấu usedAt và verified=true.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/identity/auth/verify-email/send; POST /api/identity/auth/verify-email. Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/EmailVerificationServiceImpl.java cùng các controller liên quan ở 3.3.
OTP có loại VERIFY_EMAIL, expiresAt, attempts và usedAt; mã sáu chữ số được tạo SecureRandom. maxAttempts mặc định trong @Value là 5; thời hạn lấy cấu hình, không tự ghi cố định.
• Xử lý nghiệp vụ và truy cập dữ liệu
sendVerifyEmail chặn User đã verified và mã hợp lệ đang tồn tại. verifyEmail tìm validOtp theo thời điểm/số lần thử, validateOtp rồi cập nhật usedAt và User. increment khi sai nằm trong OtpAttemptHandler cần kiểm tra transaction không mất số lần thử khi ném exception. Đối chiếu dữ liệu theo UC002.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/EmailVerificationServiceImpl.java, bắt đầu dòng 73. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
	public void verifyEmail(VerifyEmailRequest request) {
		User user = userRepository.findByEmail(request.getEmail())
```

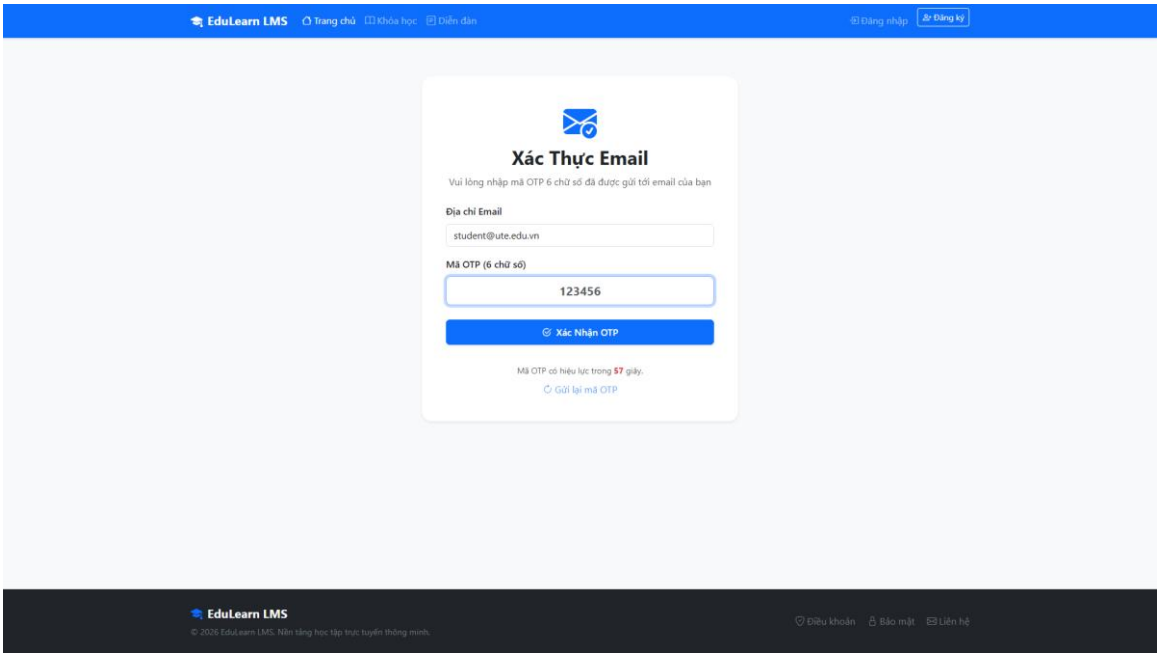
```
                        .orElseThrow(() -> new
IdentityException(ErrorCode.USER_NOT_EXISTED));
            if (user.getVerified()) {
                  throw new IdentityException(ErrorCode.EMAIL_ALREADY_VERIFIED);
            }
            EmailVerification emailVerification = emailVerificationRepository
                        .findValidOtp(user, VerificationType.VERIFY_EMAIL,
Instant.now(), otpMaxAttempts)
                        .orElseThrow(() -> new
IdentityException(ErrorCode.INVALID_VERIFICATION_TOKEN));
            validateOtp(emailVerification, request.getOtp());
```

Giải thích: sendVerifyEmail chặn User đã verified và mã hợp lệ đang tồn tại. verifyEmail tìm validOtp theo thời điểm/số lần thử, validateOtp rồi cập nhật usedAt và User. increment khi sai nằm trong OtpAttemptHandler cần kiểm tra transaction không mất số lần thử khi ném exception.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: AuthPage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.1.2: Giao diện xác thực mã OTP gửi qua Email để kích hoạt tài khoản*

• Ngoại lệ và tình huống biên

EMAIL_ALREADY_VERIFIED, VERIFICATION_EMAIL_ALREADY_SENT, INVALID_OTP, TOO_MANY_ATTEMPTS; lỗi SMTP cần kiểm thử cấu hình.

Luồng thay thế: Gửi lại khi không còn mã hợp lệ theo điều kiện source; OTP sai tăng số lần thử.

Cần kiểm chứng riêng: Thử mã đúng/sai/hết hạn/đã dùng, gửi lặp khi còn valid OTP; kiểm tra attempts tăng thật trong DB sau request lỗi. Chụp thư nhận được và trạng thái verified, che mã.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-1-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-1-2-02: Thử mã đúng/sai/hết hạn/đã dùng, gửi lặp khi còn valid OTP; kiểm tra attempts tăng thật trong DB sau request lỗi. Chụp thư nhận được và trạng thái verified, che mã.
TC-5-1-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.1.3 Đăng nhập bằng tài khoản

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: AuthenticationRequest nhận username nhưng service hỗ trợ tìm username rồi email. active=false bị từ chối. BCrypt.matches đối chiếu passwordHash; không so sánh hash bằng ==.
Tác nhân: Người có tài khoản LOCAL. Tiền điều kiện: Tài khoản tồn tại và active; biết mật khẩu.
Đầu ra: Nhận token và thông tin người dùng để điều hướng theo role.
• Luồng xử lý và sơ đồ hoạt động
1. Nhập username hoặc email và mật khẩu. 2. Backend tìm User. 3. Kiểm tra active và BCrypt.matches. 4. Cập nhật lastLogin, sinh access/refresh token. 5. Lưu hash refresh token và trả kết quả.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/identity/auth/login.
Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java cùng các controller liên quan ở 3.3.
AuthenticationRequest nhận username nhưng service hỗ trợ tìm username rồi email. active=false bị từ chối. BCrypt.matches đối chiếu passwordHash; không so sánh hash bằng ==.
• Xử lý nghiệp vụ và truy cập dữ liệu
Sau thành công cập nhật lastLogin, cấp access/refresh token và lưu tokenHash, ipAddress, deviceInfo. Hàm authenticate chưa kiểm tra verified; đây là giới hạn so với yêu cầu kích hoạt email.
Đối chiếu dữ liệu theo UC003.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java, bắt đầu dòng 98. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public AuthenticationResponse authenticate(AuthenticationRequest request) {
    String login = request.getUsername();
    User user = userRepository.findByUsername(login)
            .or(() -> userRepository.findByEmail(login))
            .orElseThrow(() -> new
IdentityException(ErrorCode.INVALID_CREDENTIALS));
```

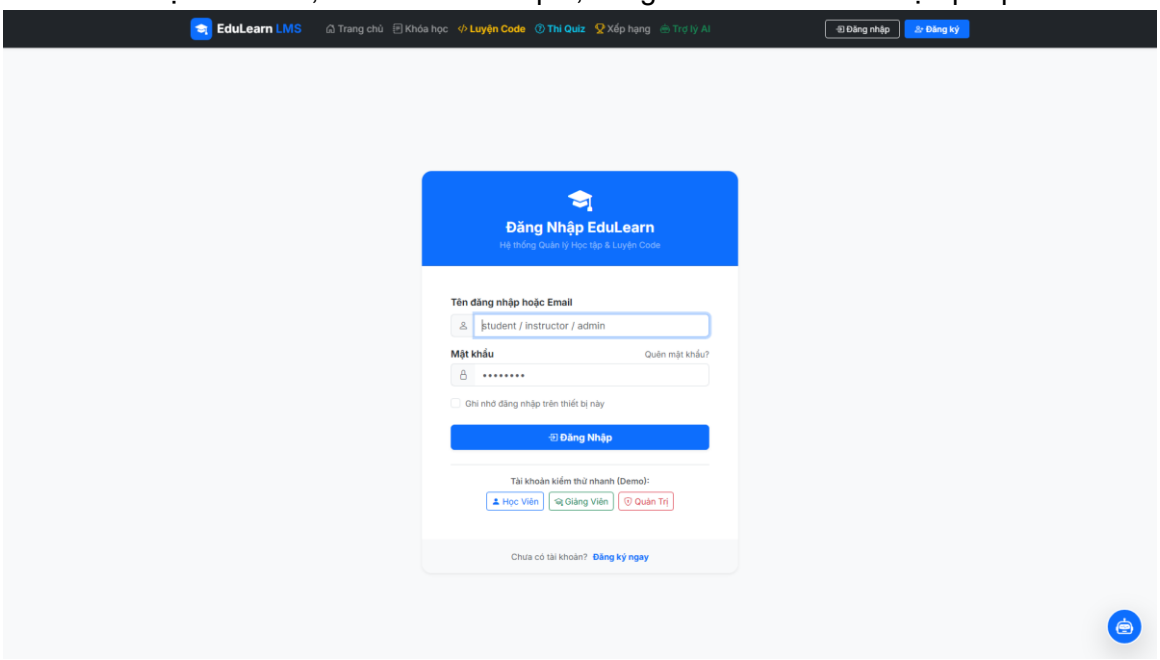
```
if (!user.getActive()) {
    throw new IdentityException(ErrorCode.USER_INACTIVE);
}
if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
    throw new IdentityException(ErrorCode.INVALID_CREDENTIALS);
}
```

Giải thích: Sau thành công cập nhật lastLogin, cấp access/refresh token và lưu tokenHash, ipAddress, deviceInfo. Hàm authenticate chưa kiểm tra verified; đây là giới hạn so với yêu cầu kích hoạt email.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: AuthPage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.1.3: Giao diện đăng nhập hệ thống với phân quyền tài khoản*

• Ngoại lệ và tình huống biên

INVALID_CREDENTIALS hoặc USER_INACTIVE. Không tự thêm điều kiện email đã verified nếu source chưa kiểm tra.

Luồng thay thế: Có thể dùng email thay cho username.

Cần kiểm chứng riêng: Thử sai mật khẩu, user không tồn tại, active=false và verified=false; ghi hành vi thực tế, không tự sửa kết quả mong đợi thành chặn verified.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-1-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-1-3-02: Thử sai mật khẩu, user không tồn tại, active=false và verified=false; ghi hành vi thực tế, không tự sửa kết quả mong đợi thành chặn verified.

TC-5-1-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-1-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.1.4 Đăng nhập bằng Google OAuth và chọn vai trò

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Luồng OAuth và cập nhật role là hai request. UserController PATCH myinfo/role gọi UserServiceImpl.updateRole(UserRoleUpdateRequest), không gọi SelectRoleServiceImpl.
Tác nhân: Guest hoặc người dùng đăng nhập xã hội. Tiền điều kiện: Có cấu hình Google OAuth hợp lệ và thông tin xác thực được provider chấp nhận.
Đầu ra: Có phiên đăng nhập xã hội; trạng thái roleSelected quyết định bước chọn vai trò.
• Luồng xử lý và sơ đồ hoạt động
1. Chọn Google. 2. Thực hiện xác thực và quay về callback. 3. Backend xử lý provider, kiểm tra email và tài khoản. 4. Tạo hoặc cập nhật User theo luồng xã hội. 5. Cấp token và hiển thị giao diện phù hợp.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/identity/auth/social-login/{provider}; PATCH /api/identity/users/myinfo/role.
Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/UserServiceImpl.java cùng các controller liên quan ở 3.3.
Luồng OAuth và cập nhật role là hai request. UserController PATCH myinfo/role gọi UserServiceImpl.updateRole(UserRoleUpdateRequest), không gọi SelectRoleServiceImpl.
• Xử lý nghiệp vụ và truy cập dữ liệu
updateRole lấy user hiện tại, kiểm tra role khác null, ánh xạ INSTRUCTOR hoặc STUDENT, đặt roleSelected=true và trả UserFullResponse. Không thấy kiểm tra provider, cửa sổ 30 phút hoặc cấp token mới tại method này. Class SelectRoleServiceImpl có các rule đó nhưng chưa tìm thấy controller gọi nó; không dùng class này làm bằng chứng cho endpoint đang hoạt động.
Đối chiếu dữ liệu theo UC004.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/UserServiceImpl.java, bắt đầu dòng 120. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public UserFullResponse updateRole(UserRoleUpdateRequest request) {
    String userId = GatewayAuthentication.currentUserId();
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new IdentityException(ErrorCode.USER_NOT_EXISTED));
    if (request.getRole() == null) {
        throw new IdentityException(ErrorCode.INVALID_ROLE_SELECTION);
    }
    RoleType roleType = request.getRole() == RegisterableRole.INSTRUCTOR
            ? RoleType.INSTRUCTOR
            : RoleType.STUDENT;
```

Giải thích: updateRole lấy user hiện tại, kiểm tra role khác null, ánh xạ INSTRUCTOR hoặc STUDENT, đặt roleSelected=true và trả UserFullResponse. Không thấy kiểm tra provider, cửa sổ 30 phút hoặc cấp token mới tại method này. Class SelectRoleServiceImpl có các

rule đó nhưng chưa tìm thấy controller gọi nó; không dùng class này làm bằng chứng cho endpoint đang hoạt động.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: SocialCallbackPage. Demo theo đúng role và luồng API được mô tả. Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.
Chức năng đăng nhập Google OAuth và chọn vai trò đã được hiện thực ở tầng backend (AuthenticationServiceImpl), tuy nhiên giao diện chọn vai trò đang trong quá trình tích hợp chuyển hướng và chờ cấu hình khóa bảo mật Google Cloud Console; nhóm bảo lưu việc cung cấp ảnh chụp giao diện sang đợt nghiệm thu thực tế trên máy chủ triển khai.
• Ngoại lệ và tình huống biên
SOCIAL_PROVIDER_NOT_SUPPORTED, SOCIAL_LOGIN_FAILED, SOCIAL_EMAIL_NOT_PROVIDED, USER_INACTIVE.
Luồng thay thế: Tài khoản xã hội mới đủ điều kiện có thể chọn STUDENT/INSTRUCTOR qua UC055.
Cần kiểm chứng riêng: Thử gọi myinfo/role từ LOCAL, từ user đã chọn và user ngoài 30 phút; kiểm tra scope JWT cũ sau đổi role. Đây là kiểm thử giới hạn, không giả định source đã ngăn các trường hợp.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-1-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-1-4-02: Thử gọi myinfo/role từ LOCAL, từ user đã chọn và user ngoài 30 phút; kiểm tra scope JWT cũ sau đổi role. Đây là kiểm thử giới hạn, không giả định source đã ngăn các trường hợp.
TC-5-1-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

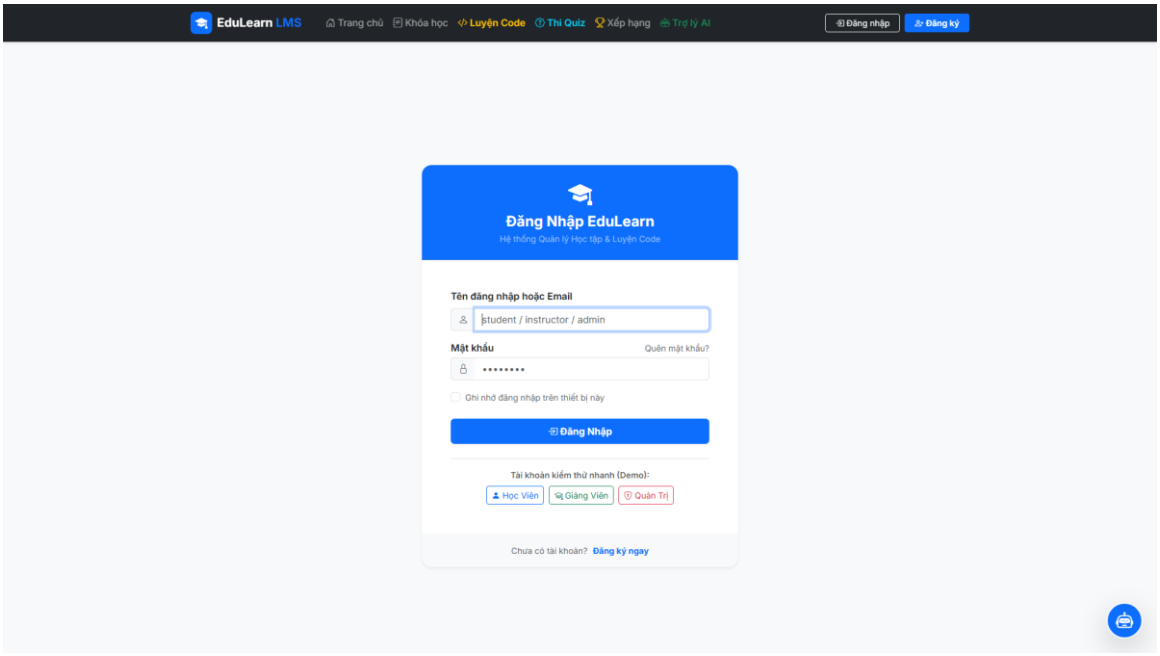
### 5.1.5 Đăng xuất

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: LogoutRequest nhận token theo controller; phải đối chiếu frontend đang truyền refresh token hay access token. Không được gọi mọi chuỗi JWT là cùng một loại.
Tác nhân: Người đã đăng nhập. Tiền điều kiện: Có phiên và refresh token của mình.
Đầu ra: Phiên frontend được kết thúc; tác động thu hồi token theo source cần mô tả ở Chương 5.
• Luồng xử lý và sơ đồ hoạt động
1. Chọn đăng xuất. 2. Gửi logout theo API. 3. Backend xử lý token theo implementation. 4. Frontend xóa phiên và chuyển về giao diện công khai.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/identity/auth/logout.

Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java cùng các controller liên quan ở 3.3.
LogoutRequest nhận token theo controller; phải đối chiếu frontend đang truyền refresh token hay access token. Không được gọi mọi chuỗi JWT là cùng một loại.
• Xử lý nghiệp vụ và truy cập dữ liệu
Logout xử lý xác minh và thu hồi bản ghi hash theo source; frontend clearSession xóa token/user ở storage. Gateway kiểm tra chữ ký/type/thời hạn ACCESS, chưa có tra revoked DB theo mỗi request.
Đối chiếu dữ liệu theo UC005.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java, bắt đầu dòng 434. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public void logout(LogoutRequest request) {
    try {
        JWSVerifier jwsVerifier = new MACVerifier(signerKey.getBytes());
        SignedJWT signedJWT = SignedJWT.parse(request.getToken());
        if (!signedJWT.verify(jwsVerifier)) {
            throw new IdentityException(ErrorCode.UNAUTHENTICATED);
        }
        String tokenHash = hashToken(request.getToken());
        RefreshToken stored = refreshTokenRepository.findByTokenHash(tokenHash)
                .orElseThrow(() -> new IdentityException(ErrorCode.INVALID_TOKEN));
        stored.setRevoked(true);
        refreshTokenRepository.save(stored);
```

Giải thích: Logout xử lý xác minh và thu hồi bản ghi hash theo source; frontend clearSession xóa token/user ở storage. Gateway kiểm tra chữ ký/type/thời hạn ACCESS, chưa có tra revoked DB theo mỗi request.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: Spring Session & Security Context và nút đăng xuất. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.1.5: Điều hướng về màn hình đăng nhập sau khi kết thúc phiên làm việc*

• Ngoại lệ và tình huống biên
Token sai/hết hạn hoặc lỗi mạng; không tuyên bố mọi access token lập tức vô hiệu ở mọi service.
Luồng thay thế: Đã mất phiên thì trở về login thay vì giữ giao diện bảo vệ.
Cần kiểm chứng riêng: Kiểm tra logout rồi truy cập route UI, gọi refresh token cũ và thử access token còn hạn; mô tả riêng ba kết quả.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-1-5-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-1-5-02: Kiểm tra logout rồi truy cập route UI, gọi refresh token cũ và thử access token còn hạn; mô tả riêng ba kết quả.
TC-5-1-5-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-5-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

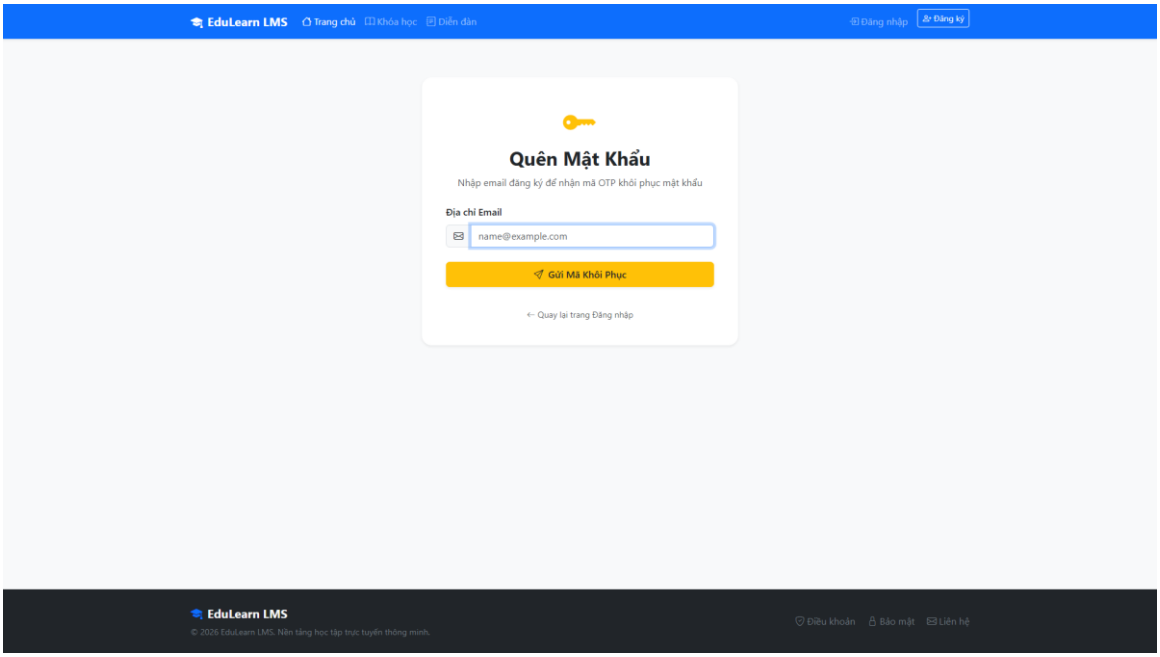
## 5.1.6 Quên và đặt lại mật khẩu

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: ResetPasswordRequest gồm email, OTP, newPassword và confirmPassword. RESET_PASSWORD khác VERIFY_EMAIL; mã sai loại không được dùng thay.
Tác nhân: Người quên mật khẩu. Tiền điều kiện: Biết email; có OTP RESET_PASSWORD hợp lệ khi đặt lại.
Đầu ra: Mật khẩu mới được băm và lưu khi thành công.
• Luồng xử lý và sơ đồ hoạt động

1. Nhập email. 2. Nếu có User, backend vô hiệu OTP reset cũ hợp lệ và gửi mã mới. 3. Nhập OTP, mật khẩu mới và xác nhận. 4. Kiểm tra mã và hai mật khẩu. 5. Đánh dấu OTP dùng rồi và lưu hash mới.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/identity/auth/forgot-password; POST /api/identity/auth/reset-password.
Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/EmailVerificationServiceImpl.java cùng các controller liên quan ở 3.3.
ResetPasswordRequest gồm email, OTP, newPassword và confirmPassword.
RESET_PASSWORD khác VERIFY_EMAIL; mã sai loại không được dùng thay.
• Xử lý nghiệp vụ và truy cập dữ liệu
sendResetPassword chỉ thực hiện nếu email có User, đánh dấu OTP cũ hợp lệ đã dùng rồi tạo/gửi mới. resetPassword kiểm tra mật khẩu xác nhận, validOtp, dùng mã và lưu BCrypt hash. Không suy rằng method đã revoke toàn bộ refresh nếu không thấy lệnh đó.
Đối chiếu dữ liệu theo UC006.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/EmailVerificationServiceImpl.java, bắt đầu dòng 120. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
	public void resetPassword(ResetPasswordRequest request) {
		if (!request.getNewPassword().equals(request.getConfirmPassword())) {
			throw new IdentityException(ErrorCode.PASSWORD_NOT_MATCH);
		}
		User user = userRepository.findByEmail(request.getEmail())
				.orElseThrow(() -> new
IdentityException(ErrorCode.USER_NOT_EXISTED));
		EmailVerification emailVerification = emailVerificationRepository
				.findValidOtp(user, VerificationType.RESET_PASSWORD,
Instant.now(), otpMaxAttempts)
				.orElseThrow(() -> new
IdentityException(ErrorCode.INVALID_VERIFICATION_TOKEN));
		validateOtp(emailVerification, request.getOtp());
```

Giải thích: sendResetPassword chỉ thực hiện nếu email có User, đánh dấu OTP cũ hợp lệ đã dùng rồi tạo/gửi mới. resetPassword kiểm tra mật khẩu xác nhận, validOtp, dùng mã và lưu BCrypt hash. Không suy rằng method đã revoke toàn bộ refresh nếu không thấy lệnh đó.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ForgotPasswordPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.1.6: Biểu mẫu yêu cầu gửi mã khôi phục mật khẩu qua Email*

• Ngoại lệ và tình huống biên
PASSWORD_NOT_MATCH, INVALID_VERIFICATION_TOKEN, INVALID_OTP, TOO_MANY_ATTEMPTS, USER_NOT_EXISTED ở bước reset.
Luồng thay thế: Email không tồn tại ở bước gửi không tạo OTP; luồng không mặc định tiết lộ tồn tại tài khoản.
Cần kiểm chứng riêng: Thử email không tồn tại, mã VERIFY_EMAIL, mã reset cũ, hai mật khẩu khác nhau và hết hạn; kiểm tra đăng nhập bằng mật khẩu cũ/mới.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-1-6-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-1-6-02: Thử email không tồn tại, mã VERIFY_EMAIL, mã reset cũ, hai mật khẩu khác nhau và hết hạn; kiểm tra đăng nhập bằng mật khẩu cũ/mới.
TC-5-1-6-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-6-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.1.7 Làm mới phiên đăng nhập

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Refresh request phải có type REFRESH và tokenHash chưa revoked; ACCESS bị từ chối tại refresh. FRONTEND response interceptor chỉ xử lý 401, dùng _retry và queue.
Tác nhân: Người dùng có refresh token. Tiền điều kiện: Refresh token đúng loại, hợp lệ và chưa revoked trong database.

Đầu ra: Phiên tiếp tục bằng token được trả; không dùng access token thay cho refresh token.

• Luồng xử lý và sơ đồ hoạt động

1. Frontend yêu cầu refresh khi cần. 2. Backend xác minh token và bản ghi hash. 3. Kiểm tra hết hạn/thu hồi. 4. Cấp token theo chính sách rotation implementation. 5. Frontend cập nhật phiên.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.

[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]

• API DTO validation và phân quyền

API chính: POST /api/identity/auth/refresh.

Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java cùng các controller liên quan ở 3.3.

Refresh request phải có type REFRESH và tokenHash chưa revoked; ACCESS bị từ chối tại refresh. FRONTEND response interceptor chỉ xử lý 401, dùng _retry và queue.

• Xử lý nghiệp vụ và truy cập dữ liệu

Backend revoked token cũ rồi tạo token mới; frontend giữ localStorage hoặc sessionStorage theo nơi refresh_token đang nằm. Queue giúp request đồng thời dùng cùng lượt refresh trong module, không phải khóa phân tán giữa nhiều tab.

Đối chiếu dữ liệu theo UC007.

• Source code trọng tâm và giải thích

Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AuthenticationServiceImpl.java, bắt đầu dòng 115. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

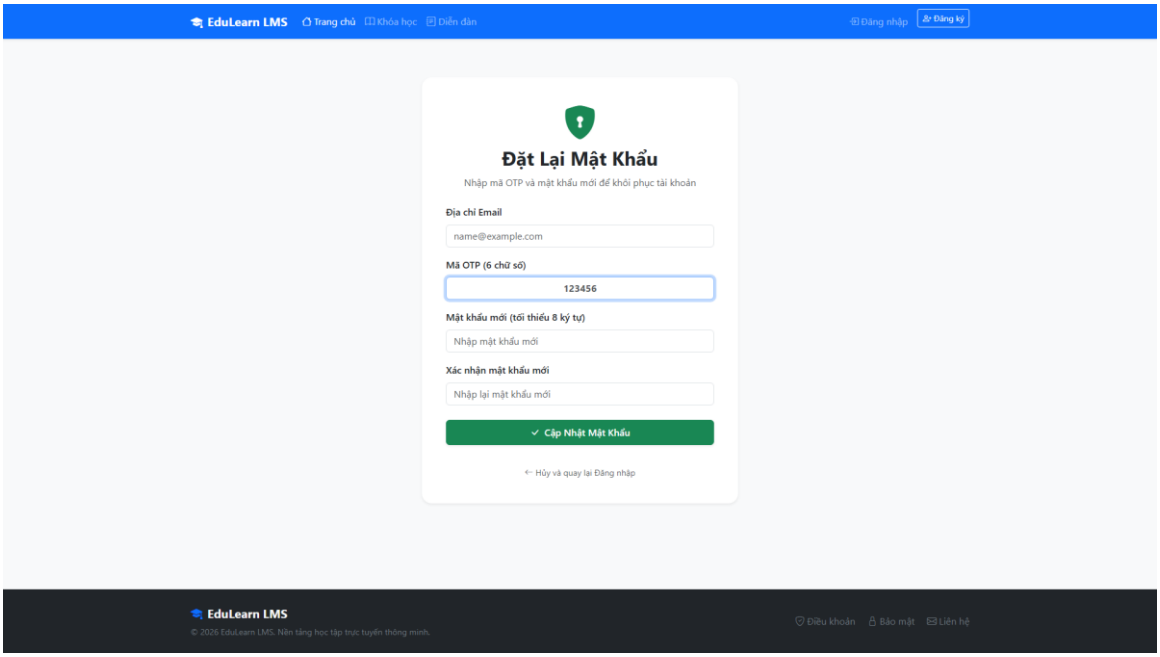
```
    String refreshToken = generateToken(user, TokenType.REFRESH.name(),
refreshableDuration);
    refreshTokenRepository.save(RefreshToken.builder()
            .user(user)
            .tokenHash(hashToken(refreshToken))
            .expiresAt(Instant.now().plusSeconds(refreshableDuration))
            .revoked(false)
            .ipAddress(request.getIpAddress())
            .deviceInfo(request.getDeviceInfo())
            .build());
    Date accessExpiryTime = new Date(Instant.now().plus(validDuration,
ChronoUnit.SECONDS).toEpochMilli());
    return AuthenticationResponse.builder()
```

Giải thích: Backend revoked token cũ rồi tạo token mới; frontend giữ localStorage hoặc sessionStorage theo nơi refresh_token đang nằm. Queue giúp request đồng thời dùng cùng lượt refresh trong module, không phải khóa phân tán giữa nhiều tab.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: Spring MVC Form Action & RestClient. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.1.7: Giao diện thiết lập mật khẩu mới sau khi xác thực OTP*

• Ngoại lệ và tình huống biên
INVALID_TOKEN, TOKEN_REVOKED và lỗi thời hạn/token type theo implementation.
Luồng thay thế: Refresh thất bại thì chuyển người dùng đăng nhập lại.
Cần kiểm chứng riêng: Thử token cũ sau rotation, token type ACCESS, hết hạn, nhiều request 401 đồng thời và refresh thiếu token; kiểm tra không lặp vô hạn. Kiểm tra user inactive khi refresh vì method chưa thấy rule active.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-1-7-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-1-7-02: Thử token cũ sau rotation, token type ACCESS, hết hạn, nhiều request 401 đồng thời và refresh thiếu token; kiểm tra không lặp vô hạn. Kiểm tra user inactive khi refresh vì method chưa thấy rule active.
TC-5-1-7-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-7-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

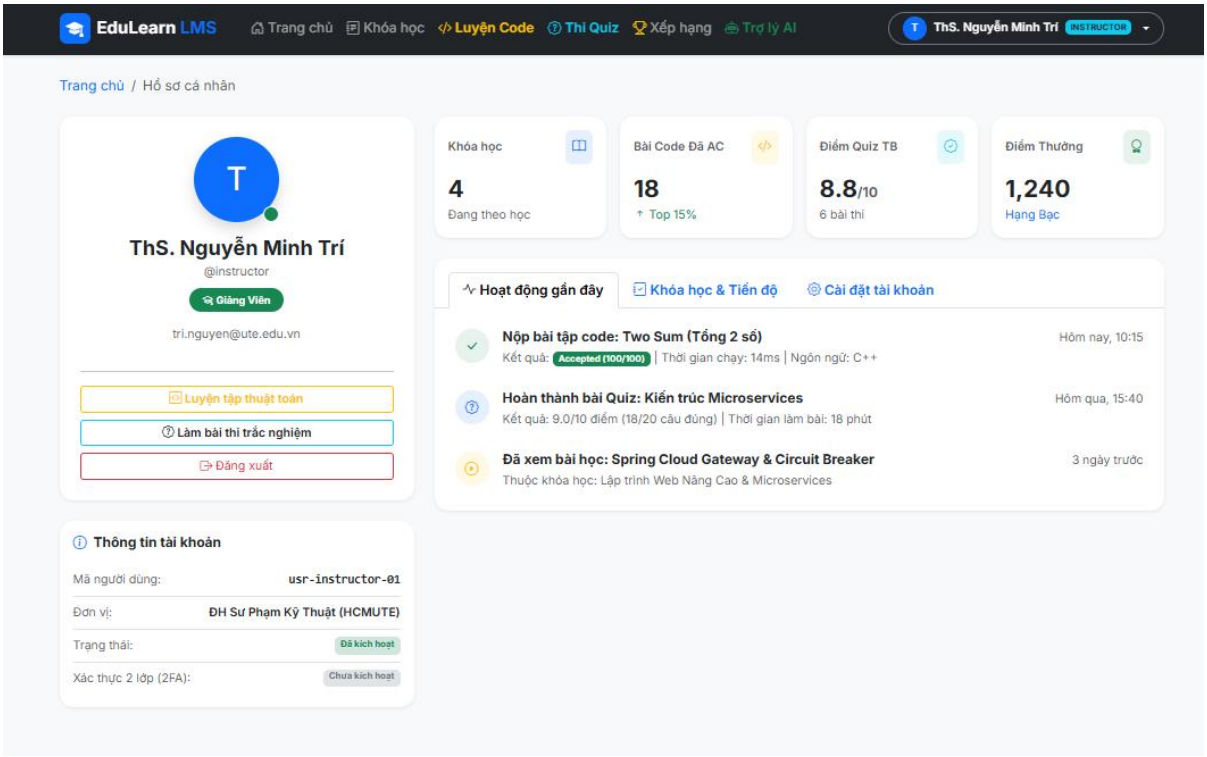
## 5.1.8 Cập nhật hồ sơ và ảnh đại diện

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: UserUpdateRequest và profile phải gắn userId lấy từ phiên. API myinfo không cho client chỉ định ID người khác làm chủ hồ sơ.
Tác nhân: STUDENT, INSTRUCTOR, ADMIN. Tiền điều kiện: Đăng nhập; chỉnh hồ sơ của mình; tệp thỏa validation upload.
Đầu ra: Thông tin User được cập nhật nếu request hợp lệ.
• Luồng xử lý và sơ đồ hoạt động

1. Mở settings và tải myinfo. 2. Sửa trường hồ sơ. 3. Nếu đổi avatar, lấy URL upload và tải tệp theo luồng lưu trữ. 4. Gửi cập nhật myinfo. 5. Hiển thị dữ liệu mới.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET/PUT /api/identity/users/myinfo; POST /api/identity/storage/presigned-url.
Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/UserServiceImpl.java cùng các controller liên quan ở 3.3.
UserUpdateRequest và profile phải gắn userId lấy từ phiên. API myinfo không cho client chỉ định ID người khác làm chủ hồ sơ.
• Xử lý nghiệp vụ và truy cập dữ liệu
updateInfo lưu dữ liệu mới, xử lý avatarUrl rỗng và dọn avatar cũ khi đổi.
discardedAvatarUrls được lọc null/rỗng/trùng; storage kiểm tra prefix user khi dọn tệp bỏ.
Lưu metadata và upload object là hai bước có thể lỗi độc lập.
Đối chiếu dữ liệu theo UC008.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/UserServiceImpl.java, bắt đầu dòng 51. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public UserFullResponse updateInfo(UserUpdateRequest request) {
    String userId = GatewayAuthentication.currentUserId();
    log.info("userId: {}", userId);
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new IdentityException(ErrorCode.USER_NOT_EXISTED));
    String oldAvatarUrl = user.getAvatarUrl();
    userMapper.updateUser(user, request);
    if (request.getAvatarUrl() != null && request.getAvatarUrl().isEmpty()) {
        user.setAvatarUrl(null);
    }
    userRepository.save(user);
```

Giải thích: updateInfo lưu dữ liệu mới, xử lý avatarUrl rỗng và dọn avatar cũ khi đổi.
discardedAvatarUrls được lọc null/rỗng/trùng; storage kiểm tra prefix user khi dọn tệp bỏ.
Lưu metadata và upload object là hai bước có thể lỗi độc lập.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: SettingsPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.
Giao diện hồ sơ do nhóm cung cấp ngày 24/10/2026:




*Hình 5.5 Hồ sơ cá nhân của tài khoản có nhãn INSTRUCTOR*

Ảnh thể hiện tên hiển thị, nhãn Giảng viên/INSTRUCTOR, thông tin tài khoản, các thẻ tổng quan và các tab Hoạt động gần đây, Khóa học & Tiến độ, Cài đặt tài khoản. Đây là trang hồ sơ cá nhân, không phải màn hình quản lý giảng dạy. Các điểm quiz, số bài AC, điểm thưởng và hoạt động gần đây chỉ được ghi nhận là nội dung hiển thị; chưa có bằng chứng đối chiếu dữ liệu. Ảnh chưa minh chứng thao tác sửa hồ sơ, upload ảnh đại diện hoặc lưu bền sau khi tải lại trang.

• Ngoại lệ và tình huống biên

Dữ liệu sai, tệp quá giới hạn, không tìm thấy người dùng hoặc lỗi MinIO.

Luồng thay thế: Có thể chỉ sửa hồ sơ mà không đổi ảnh.

Cần kiểm chứng riêng: Thử cập nhật text, bỏ avatar, đổi ảnh rồi hủy, URL bỏ không thuộc prefix user; kiểm tra tệp hiện dùng không bị xóa.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-1-8-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-1-8-02: Thử cập nhật text, bỏ avatar, đổi ảnh rồi hủy, URL bỏ không thuộc prefix user; kiểm tra tệp hiện dùng không bị xóa.

TC-5-1-8-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-1-8-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

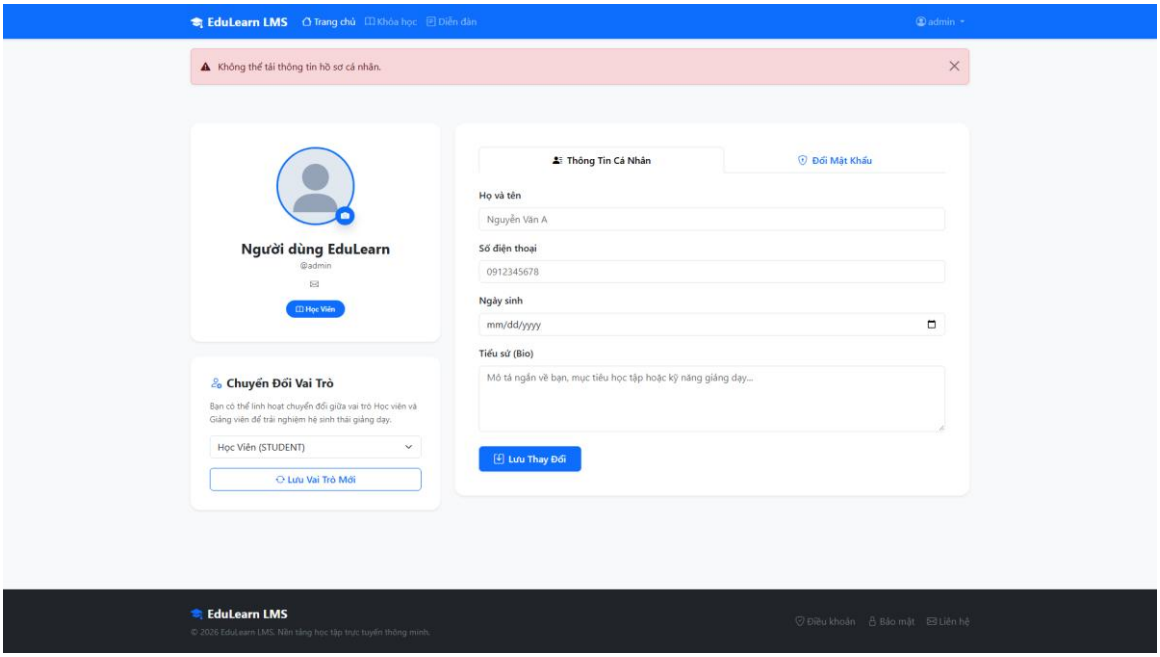
## 5.1.9 Thay đổi mật khẩu

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: updatePassword yêu cầu oldPassword, newPassword và confirmPassword; kiểm tra hai mật khẩu mới khớp và oldPassword qua BCrypt.matches.
Tác nhân: Người dùng có tài khoản phù hợp luồng đổi mật khẩu. Tiền điều kiện: Đã đăng nhập; dữ liệu đổi mật khẩu thỏa request.
Đầu ra: Mật khẩu được cập nhật khi kiểm tra thành công.
• Luồng xử lý và sơ đồ hoạt động
1. Mở form đổi mật khẩu. 2. Nhập thông tin theo DTO. 3. Gửi myinfo/password. 4. Service kiểm tra điều kiện mật khẩu và lưu BCrypt hash mới. 5. Thông báo kết quả.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: PATCH /api/identity/users/myinfo/password.
Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/UserServiceImpl.java cùng các controller liên quan ở 3.3.
updatePassword yêu cầu oldPassword, newPassword và confirmPassword; kiểm tra hai mật khẩu mới khớp và oldPassword qua BCrypt.matches.
• Xử lý nghiệp vụ và truy cập dữ liệu
Method lưu hash mới và gọi refreshTokenRepository.revokeAllByUserId. Việc thu hồi refresh không đồng nghĩa mọi ACCESS đã phát hành bị vô hiệu ngay vì Gateway không tra revoked DB cho mỗi access.
Đối chiếu dữ liệu theo UC009.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/UserServiceImpl.java, bắt đầu dòng 82. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public UserFullResponse updatePassword(UserUpdatePasswordRequest request) {
    String userId = GatewayAuthentication.currentUserId();
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new IdentityException(ErrorCode.USER_NOT_EXISTED));
    if (!request.getNewPassword().equals(request.getConfirmPassword())) {
        throw new IdentityException(ErrorCode.PASSWORD_NOT_MATCH);
    }
    if (!passwordEncoder.matches(request.getOldPassword(), user.getPassword())) {
        throw new IdentityException(ErrorCode.PASSWORD_INCORRECT);
    }
```

Giải thích: Method lưu hash mới và gọi refreshTokenRepository.revokeAllByUserId. Việc thu hồi refresh không đồng nghĩa mọi ACCESS đã phát hành bị vô hiệu ngay vì Gateway không tra revoked DB cho mỗi access.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: SettingsPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.1.9: Khu vực cấu hình bảo mật và thay đổi mật khẩu người dùng*

• Ngoại lệ và tình huống biên
Sai mật khẩu/không khớp xác nhận hoặc vi phạm DTO; không suy rằng toàn bộ phiên bị thu hồi.
Luồng thay thế: Hủy thao tác trước khi gửi không thay đổi dữ liệu.
Cần kiểm chứng riêng: Thử oldPassword sai, confirm sai, đổi đúng, refresh cũ bị thu hồi và access cũ còn hạn; ghi kết quả theo từng loại token.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-1-9-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-1-9-02: Thử oldPassword sai, confirm sai, đổi đúng, refresh cũ bị thu hồi và access cũ còn hạn; ghi kết quả theo từng loại token.
TC-5-1-9-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-1-9-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.2 Course Service

### 5.2.1 Tra cứu và phân trang khóa học

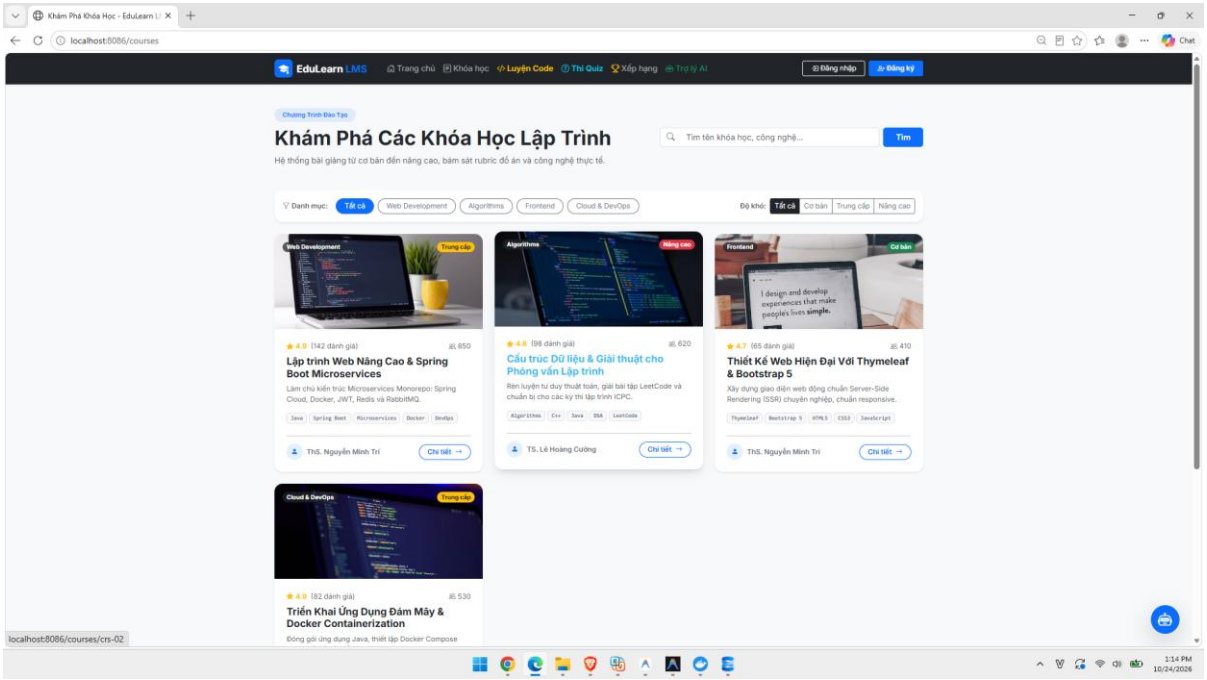
• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Danh sách dùng Page<CourseResponse>; metadata page khác với tự cắt array trên frontend. Chỉ mô tả bộ lọc có trong request/controller thực tế.
Tác nhân: Guest và người dùng đăng nhập. Tiền điều kiện: Thông số tìm kiếm/lọc/phân trang hợp lệ.

Tác nhân: Guest và người dùng đăng nhập. Tiền điều kiện: Có courseId hợp lệ và dữ liệu được phép hiển thị.
Đầu ra: Có danh sách theo tiêu chí; không thay đổi dữ liệu. Người dùng xem thông tin khóa học; không tự có quyền đọc lesson.
• Luồng xử lý và sơ đồ hoạt động
1. Mở danh sách khóa học. 2. Nhập từ khóa/chọn bộ lọc. 3. Frontend gọi API danh sách. 4. Backend truy vấn và trả Page<CourseResponse>. 5. Hiển thị và chuyển trang.
1. Chọn một khóa học. 2. Gọi API chi tiết. 3. Backend lấy Course và dữ liệu bổ sung. 4. Hiển thị mô tả, cấu trúc giới thiệu và thông tin đăng ký.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/course/courses; GET /api/course/courses/{id}.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java cùng các controller liên quan ở 3.3.
Danh sách dùng Page<CourseResponse>; metadata page khác với tự cắt array trên frontend. Chỉ mô tả bộ lọc có trong request/controller thực tế.
• Xử lý nghiệp vụ và truy cập dữ liệu
API public-read bỏ các header nhận dạng từ client; my/enrolled có route riêng ưu tiên hơn. Chi tiết Course không tự cấp quyền đọc Lesson; không dùng số bài học làm căn cứ enrollment.
Đối chiếu dữ liệu theo UC011, UC012.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java, bắt đầu dòng 80. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

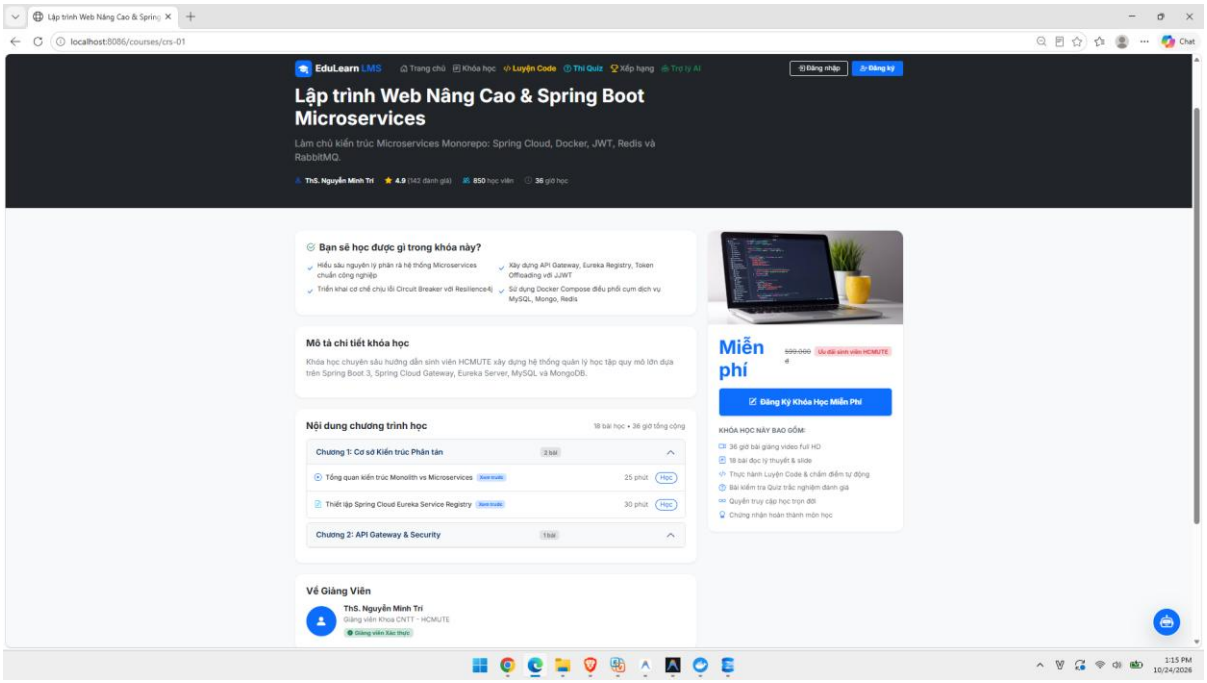
```
public Page<CourseResponse> getAllCourses(Pageable pageable, CourseStatus status,
Long categoryId, String keyword, List<CourseLevel> levels) {
    var spec = CourseSpecifications.hasStatus(status)
            .and(CourseSpecifications.hasCategory(categoryId))
            .and(CourseSpecifications.hasLevelIn(levels))
            .and(CourseSpecifications.keywordMatches(keyword));
    Page<CourseResponse> page = courseRepository.findAll(spec,
pageable).map(courseMapper::toCourseResponse);
    populateExerciseCounts(page.getContent());
    return page;
}
@Override
@Transactional(readOnly = true)
public CourseResponse getCourseById(String id) {
```

Giải thích: API public-read bỏ các header nhận dạng từ client; my/enrolled có route riêng ưu tiên hơn. Chi tiết Course không tự cấp quyền đọc Lesson; không dùng số bài học làm căn cứ enrollment.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ListCoursePage và DetailCoursePage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

Giao diện danh sách và chi tiết khóa học do nhóm cung cấp ngày 24/10/2026:




*Hình 5.3 Danh sách khóa học với ô tìm kiếm, bộ lọc danh mục và độ khó*




*Hình 5.4 Chi tiết khóa học và khu vực đăng ký tham gia*

Danh sách tại /courses hiển thị các thẻ khóa học kèm ảnh, tên, mô tả, danh mục, mức độ và nút Chi tiết. Trang /courses/crs-01 hiển thị khóa Lập trình Web Nâng Cao & Spring Boot Microservices, mục tiêu học tập, mô tả, chương/bài học và nút Đăng ký Khóa Học Miễn Phí. Hai ảnh xác nhận bố cục giao diện hiện có; chưa chứng minh tìm kiếm/lọc/phân trang, đăng ký tham gia hay truy cập bài học đã hoạt động. Tên giảng viên, đánh giá, số học viên, thời lượng và quyền lợi trong ảnh là dữ liệu đang hiển thị, chưa xác minh nguồn hoặc tính toán.

• Ngoại lệ và tình huống biên

Tham số không hợp lệ, lỗi API; phạm vi khóa học hiển thị theo bộ lọc của service.

COURSE_NOT_FOUND hoặc lỗi dữ liệu/phân quyền tùy API.
Luồng thay thế: Không có kết quả thì hiển thị trạng thái rỗng và cho đổi tiêu chí. Đã tham gia thì chuyển sang màn hình khóa học của mình.
Cần kiểm chứng riêng: Thử page đầu/cuối, từ khóa không có kết quả, tham số sai và course không tồn tại; chụp list/detail, không chụp lesson làm minh chứng Guest.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-2-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-2-1-02: Thử page đầu/cuối, từ khóa không có kết quả, tham số sai và course không tồn tại; chụp list/detail, không chụp lesson làm minh chứng Guest.
TC-5-2-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-2-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.2.2 Đăng ký và quản lý khóa học đã tham gia

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Đầu vào enrollment là courseId; userId từ GatewayAuthentication.
Đầu ra EnrollmentResponse không phải hóa đơn hoặc giao dịch thanh toán.
Tác nhân: Người đã đăng nhập; giao diện chính STUDENT. Tiền điều kiện: Course phải PUBLIC, không LOCKED; chưa có enrollment của người dùng.
Tác nhân: Người đã đăng nhập; màn hình STUDENT. Tiền điều kiện: Có phiên hợp lệ.
Đầu ra: Có bản ghi tham gia khóa học. Hiển thị các khóa học gắn với người dùng hiện tại.
• Luồng xử lý và sơ đồ hoạt động
1. Chọn đăng ký tham gia. 2. Gửi courseId. 3. Service lấy userId từ phiên. 4. Kiểm tra trạng thái và đăng ký trùng. 5. Lưu Enrollment ACTIVE và trả kết quả.
1. Mở My Courses. 2. Gọi /courses/enrolled. 3. Backend lấy enrollment theo userId hiện tại. 4. Ánh xạ khóa học và trả danh sách. 5. Chọn khóa học để học.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/course/courses/{id}/enroll; GET /api/course/courses/enrolled.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java cùng các controller liên quan ở 3.3.
Đầu vào enrollment là courseId; userId từ GatewayAuthentication. Đầu ra EnrollmentResponse không phải hóa đơn hoặc giao dịch thanh toán.
• Xử lý nghiệp vụ và truy cập dữ liệu
Service chặn LOCKED, trạng thái khác PUBLIC và enrollment đã tồn tại, sau đó lưu ACTIVE. getEnrolledCourses truy vấn theo user hiện tại và bổ sung thông tin khóa học.
Đối chiếu dữ liệu theo UC013, UC014.
• Source code trọng tâm và giải thích

Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java, bắt đầu dòng 228. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

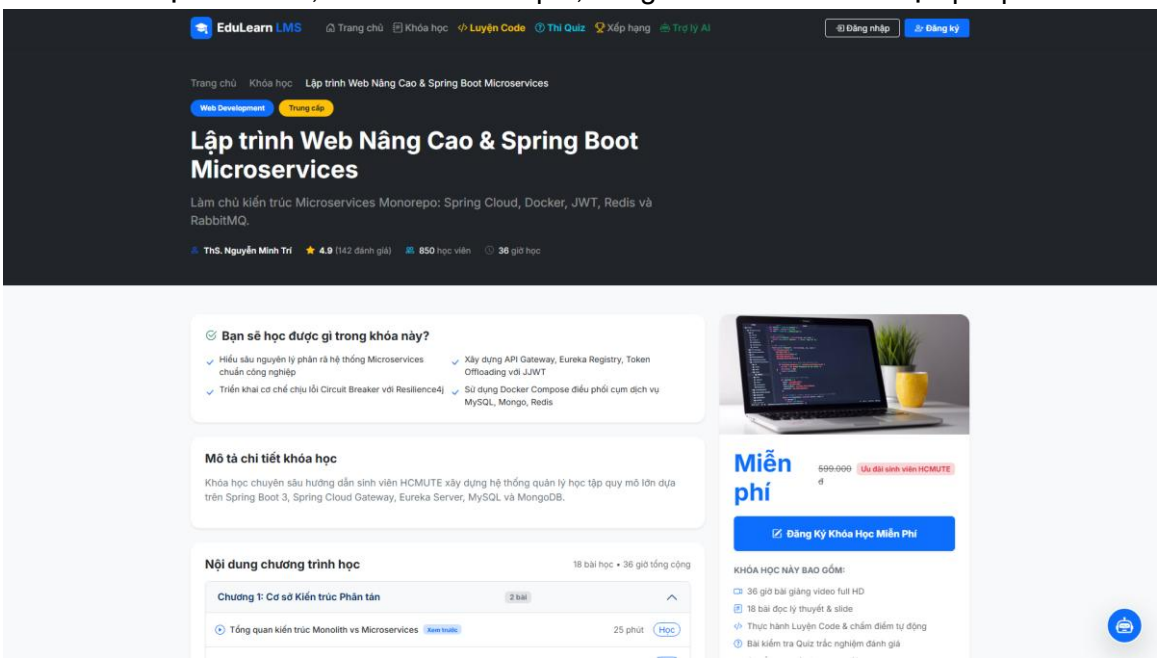
```
public EnrollmentResponse enrollCourse(String courseId) {
    String userId = GatewayAuthentication.currentUserId();
    Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new CourseException(ErrorCode.COURSE_NOT_FOUND));
    if(course.getStatus() == CourseStatus.LOCKED)
        throw new CourseException(ErrorCode.COURSE_LOCKED);
    if(course.getStatus() != CourseStatus.PUBLIC)
        throw new CourseException(ErrorCode.COURSE_NOT_PUBLIC);
    if(enrollmentRepository.existsByUserIdAndCourseId(userId, courseId))
        throw new CourseException(ErrorCode.ALREADY_ENROLLED);
```

Giải thích: Service chặn LOCKED, trạng thái khác PUBLIC và enrollment đã tồn tại, sau đó lưu ACTIVE. getEnrolledCourses truy vấn theo user hiện tại và bổ sung thông tin khóa học.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: DetailCoursePage và MyCoursePage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.2.2: Giao diện chi tiết khóa học, thông tin giảng viên và lộ trình bài giảng*

• Ngoại lệ và tình huống biên

COURSE_NOT_FOUND, COURSE_LOCKED, COURSE_NOT_PUBLIC, ALREADY_ENROLLED. API dùng isAuthenticated, không mô tả chỉ STUDENT được gọi. Lỗi phiên hoặc truy vấn; không cho client tự chọn userId để xem danh sách của người khác. Luồng thay thế: Nếu đã tham gia, giao diện chuyển sang khóa học của mình. Không có enrollment thì hiển thị danh sách rỗng.

Cần kiểm chứng riêng: Thử PUBLIC/PRIVATE/LOCKED, đăng ký trùng và hai người dùng; kiểm tra danh sách không trộn dữ liệu user.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-2-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-2-2-02: Thử PUBLIC/PRIVATE/LOCKED, đăng ký trùng và hai người dùng; kiểm tra danh sách không trộn dữ liệu user.
TC-5-2-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-2-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.2.3 Quản lý khóa học và danh mục

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Course quản lý theo role và owner; category CRUD thuộc ADMIN. Không gộp quyền sửa category vào quyền Instructor chỉ vì hai màn hình cùng liên quan khóa học.
Tác nhân: INSTRUCTOR và ADMIN. Tiền điều kiện: Dữ liệu Course hợp lệ; sửa/xóa thỏa kiểm tra chủ khóa học hoặc Admin.
Tác nhân: ADMIN; Guest/người dùng đọc danh mục theo API public. Tiền điều kiện: Dữ liệu category hợp lệ; sửa/xóa category phải tồn tại.
Đầu ra: Course được cập nhật khi thành công. Category thay đổi theo API.
• Luồng xử lý và sơ đồ hoạt động
1. Mở quản lý khóa học. 2. Chọn tạo/sửa/xóa. 3. Nhập dữ liệu hoặc xác nhận. 4. Backend kiểm tra quyền, trạng thái và dữ liệu. 5. Lưu hoặc xóa theo service. 6. Tải lại danh sách.
1. Mở Course Categories. 2. Tạo/sửa/xóa. 3. Gửi API categories. 4. Backend validation và lưu. 5. Cập nhật giao diện.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/course/courses; PUT/DELETE /api/course/courses/{id}; GET/POST /api/course/categories.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java cùng các controller liên quan ở 3.3.
Course quản lý theo role và owner; category CRUD thuộc ADMIN. Không gộp quyền sửa category vào quyền Instructor chỉ vì hai màn hình cùng liên quan khóa học.
• Xử lý nghiệp vụ và truy cập dữ liệu
create/update/delete gọi repository và xử lý cấu trúc/tài nguyên theo service. Google Meet có fallback URL không bảo đảm phòng thật. Các thao tác MySQL/Mongo/storage cần kiểm tra riêng, không mô tả là transaction toàn hệ thống.
Đối chiếu dữ liệu theo UC024, UC050.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java, bắt đầu dòng 108. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

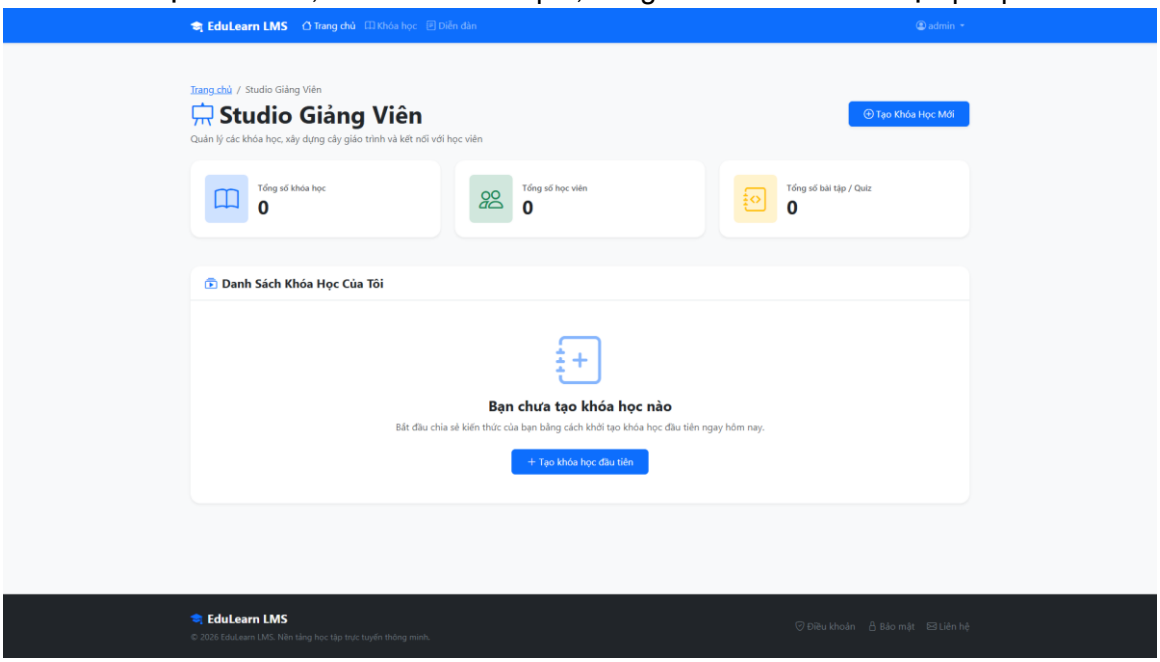
```
public CourseResponse createCourse(CreateCourseRequest request) {
    // Lấy userId từ SecurityContext
    String instructorId = GatewayAuthentication.currentUserId();
    // Kiểm tra trùng tên
    if(courseRepository.existsByTitle(request.getTitle()))
        throw new CourseException(ErrorCode.COURSE_TITLE_EXISTED);
    // Map và set instructor
    Course course = courseMapper.toCourse(request);
    course.setInstructorId(instructorId);
    // Tạo link Google Meet trước - nếu fail thì không tốn MinIO storage
    MeetingInfo meetingInfo = googleMeetService.createMeeting(request.getTitle());
```

Giải thích: create/update/delete gọi repository và xử lý cấu trúc/tài nguyên theo service. Google Meet có fallback URL không bảo đảm phòng thật. Các thao tác MySQL/Mongo/storage cần kiểm tra riêng, không mô tả là transaction toàn hệ thống.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: CourseManagement, CreateCoursePage, EditCoursePage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.2.3: Studio dành cho giảng viên quản lý danh sách khóa học phụ trách*

• Ngoại lệ và tình huống biên

COURSE_NOT_FOUND, NOT_COURSE_OWNER, COURSE_LOCKED hoặc lỗi DTO/liên kết dữ liệu.

Không phải Admin, category không tồn tại hoặc lỗi ràng buộc/DTO.

Luồng thay thế: Hủy trước gửi; giảng viên xem danh sách khóa học của mình. Người không quản trị chỉ đọc danh sách.

Cần kiểm chứng riêng: Thử Instructor khác owner, course bị khóa, dữ liệu thiếu và category liên kết; kiểm tra API trả lỗi và không thay đổi ngoài phạm vi.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-2-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-2-3-02: Thử Instructor khác owner, course bị khóa, dữ liệu thiếu và category liên kết; kiểm tra API trả lỗi và không thay đổi ngoài phạm vi.
TC-5-2-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-2-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.2.4 Quản lý cấu trúc bài học và tài nguyên

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: StructureNode có id/type/title/order/parentId/lessonId. Lesson chứa lessonType VIDEO hoặc DOCUMENT và Map content; courseId phải khớp.
Tác nhân: INSTRUCTOR chủ khóa học; ADMIN. Tiền điều kiện: Course tồn tại; đủ quyền và không bị khóa với thao tác sửa.
Tác nhân: INSTRUCTOR chủ khóa học; ADMIN. Tiền điều kiện: Lesson thuộc course và có nội dung phù hợp lessonType.
Tác nhân: Học viên đã tham gia; giảng viên chủ khóa học; Admin. Tiền điều kiện: Course/lesson tồn tại; lesson thuộc course; thỏa kiểm tra truy cập.
Đầu ra: Cấu trúc MongoDB được cập nhật. Nội dung lesson được cập nhật. Nội dung được trả cho người có quyền.
• Luồng xử lý và sơ đồ hoạt động
1. Mở trung tâm khóa học. 2. Tải cấu trúc. 3. Thêm/sửa/xóa hoặc sắp xếp node. 4. API kiểm tra quyền và quan hệ. 5. Lưu CourseStructure.
1. Chọn bài học. 2. Sửa nội dung hoặc tải tài nguyên. 3. Gửi cập nhật. 4. Service kiểm tra owner, trạng thái khóa, quan hệ và nội dung. 5. Lưu lesson.
1. Chọn khóa học và bài. 2. Gọi cấu trúc và chi tiết lesson. 3. Service xác minh quan hệ lesson–course. 4. Kiểm tra Admin/chủ khóa học/enrollment. 5. Hiển thị nội dung.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET/PUT /api/course/courses/{courseId}/lessons/{lessonId}; cấu trúc theo CourseStructureController.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/LessonServiceImpl.java cùng các controller liên quan ở 3.3.
StructureNode có id/type/title/order/parentId/lessonId. Lesson chứa lessonType VIDEO hoặc DOCUMENT và Map content; courseId phải khớp.
• Xử lý nghiệp vụ và truy cập dữ liệu
updateLesson kiểm tra chủ/khóa/quan hệ, rồi validateContent theo loại. Upload presigned tạo object riêng; việc có URL không chứng minh đã upload. Khách xem cấu trúc public không đồng nghĩa đọc toàn bộ lesson.
Đối chiếu dữ liệu theo UC025, UC026, UC015.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/LessonServiceImpl.java, bắt đầu

dòng 54. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

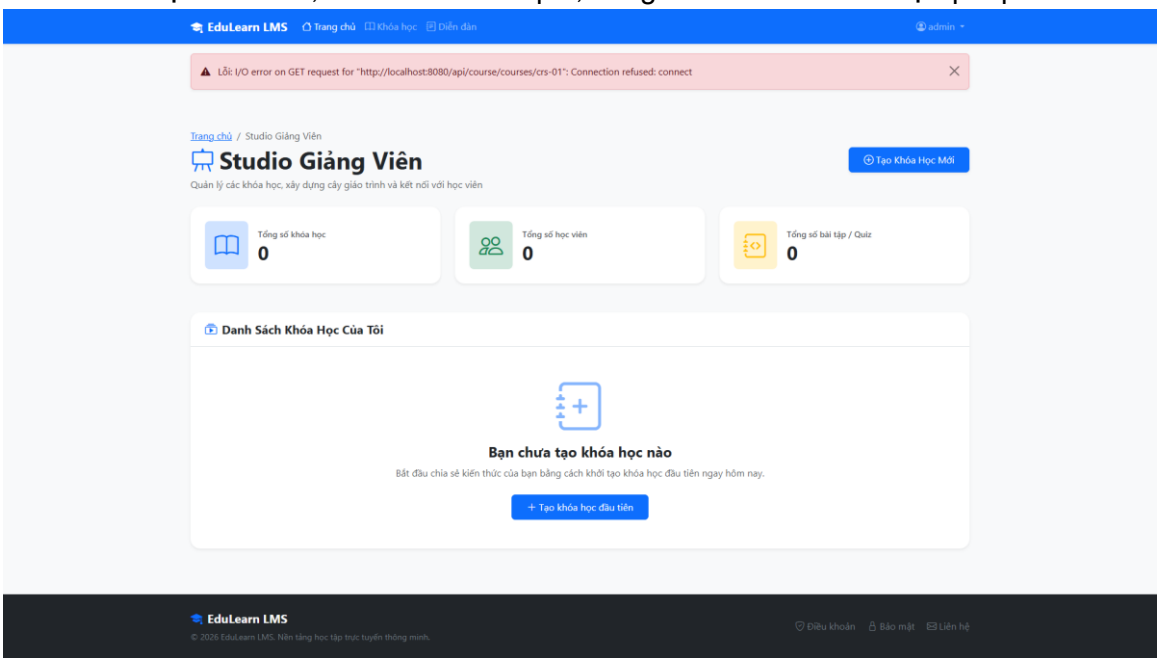
```
public LessonResponse updateLesson(String courseId, String lessonId,
UpdateLessonRequest request) {
    Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new CourseException(ErrorCode.COURSE_NOT_FOUND));
    checkCourseOwner(course);
    checkNotLocked(course);
    Lesson lesson = lessonRepository.findById(lessonId)
            .orElseThrow(() -> new CourseException(ErrorCode.LESSON_NOT_FOUND));
    if(!lesson.getCourseId().equals(courseId))
        throw new CourseException(ErrorCode.LESSON_NOT_BELONG_TO_COURSE);
```

Giải thích: updateLesson kiểm tra chủ/khóa/quan hệ, rồi validateContent theo loại. Upload presigned tạo object riêng; việc có URL không chứng minh đã upload. Khách xem cấu trúc public không đồng nghĩa đọc toàn bộ lesson.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: CourseManagementCenter và DetailMyCoursePage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.2.4: Giao diện quản trị cấu trúc cây bài giảng và phân phối học liệu*

• Ngoại lệ và tình huống biên

Node/course không tồn tại, không phải chủ hoặc course bị khóa; chi tiết ràng buộc theo DTO/service.

LESSON_NOT_BELONG_TO_COURSE, LESSON_CONTENT_NULL, NOT_COURSE_OWNER, COURSE_LOCKED; lỗi upload.

LESSON_NOT_BELONG_TO_COURSE, ACCESS_DENIED, COURSE_NOT_FOUND hoặc lesson không tồn tại.

Luồng thay thế: Giữ nguyên cấu trúc nếu không xác nhận thao tác. Chỉ đổi một phần dữ liệu theo UpdateLessonRequest. Chuyển bài bằng cấu trúc course; không cần tạo lại enrollment.

Cần kiểm chứng riêng: Thử lesson của course khác, loại/nội dung không khớp, node sai parent và upload lỗi; kiểm tra chỉ dữ liệu được phép thay đổi.

• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-2-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-2-4-02: Thử lesson của course khác, loại/nội dung không khớp, node sai parent và upload lỗi; kiểm tra chỉ dữ liệu được phép thay đổi.
TC-5-2-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-2-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.2.5 Quản lý học viên và lời mời

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Danh sách học viên dùng enrollment; tìm email gọi IdentityClient. InviteResultResponse cần hiển thị kết quả từng người thay vì chỉ một thông báo tất cả thành công.
Tác nhân: INSTRUCTOR chủ khóa học; ADMIN. Tiền điều kiện: Course tồn tại; người học được nhận diện qua Identity.
Đầu ra: Danh sách enrollment thay đổi theo từng kết quả xử lý.
• Luồng xử lý và sơ đồ hoạt động
1. Xem danh sách học viên. 2. Tìm người dùng/mời theo thông tin đầu vào. 3. Service kiểm tra owner và đăng ký hiện có. 4. Tạo enrollment cho người hợp lệ hoặc loại khỏi course. 5. Hiển thị kết quả.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/course/courses/{id}/invite; GET /api/course/courses/{id}/students; DELETE /api/course/courses/{id}/students/{userId}.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java cùng các controller liên quan ở 3.3.
Danh sách học viên dùng enrollment; tìm email gọi IdentityClient. InviteResultResponse cần hiển thị kết quả từng người thay vì chỉ một thông báo tất cả thành công.
• Xử lý nghiệp vụ và truy cập dữ liệu
CourseService kiểm tra owner; danh sách mời xử lý người đã enrollment và tạo ACTIVE cho người hợp lệ. Client Identity có fallback null/exception tùy method, nên thiếu thông tin người dùng có thể là service lỗi.
Đối chiếu dữ liệu theo UC027.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java, bắt đầu dòng 386. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public InviteResultResponse inviteStudents(String courseId, InviteStudentRequest
request) {
    String currentUserId = GatewayAuthentication.currentUserId();
    // Kiểm tra khóa học tồn tại
    Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new CourseException(ErrorCode.COURSE_NOT_FOUND));
    // Kiểm tra người gửi là chủ khóa học
    checkCourseOwner(course);
    checkNotLocked(course);
    List<String> successIds  = new ArrayList<>();
    List<String> alreadyIds  = new ArrayList<>();
    List<String> notFoundIds = new ArrayList<>();
```

Giải thích: CourseService kiểm tra owner; danh sách mời xử lý người đã enrollment và tạo ACTIVE cho người hợp lệ. Client Identity có fallback null/exception tùy method, nên thiếu thông tin người dùng có thể là service lỗi.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: CourseManagementCenter. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.2.5: Giao diện quản lý danh sách học viên đăng ký tham gia khóa học*

• Ngoại lệ và tình huống biên

NOT_COURSE_OWNER, COURSE_NOT_FOUND, không tìm thấy user hoặc enrollment; lỗi Identity.

Luồng thay thế: Danh sách mời có thể chứa người đã tham gia; xử lý theo InviteResultResponse.

Cần kiểm chứng riêng: Thử email không tồn tại, đã tham gia, danh sách trộn hợp lệ/sai và Identity không sẵn sàng; chụp kết quả từng trường hợp.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-2-5-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-2-5-02: Thử email không tồn tại, đã tham gia, danh sách trộn hợp lệ/sai và Identity không sẵn sàng; chụp kết quả từng trường hợp.
TC-5-2-5-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-2-5-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.2.6 Theo dõi tiến độ bài học

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: completeLesson cần enrollment và lesson thuộc course. Request tiến độ không được lấy userId tùy ý từ form.
Tác nhân: Học viên đã tham gia. Tiền điều kiện: Lesson thuộc course; có enrollment; dữ liệu hoàn thành hợp lệ.
Đầu ra: Tiến độ gắn user/course/lesson được cập nhật.
• Luồng xử lý và sơ đồ hoạt động
1. Người học hoàn thành bài theo thao tác giao diện. 2. Gửi completeLesson. 3. Backend kiểm tra enrollment và quan hệ bài học. 4. Lưu/cập nhật LessonProgress. 5. Tải tiến độ khóa học.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/course/courses/{courseId}/lessons/{lessonId}/complete; GET /api/course/courses/{courseId}/progress.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/LessonProgressServiceImpl.java cùng các controller liên quan ở 3.3.
completeLesson cần enrollment và lesson thuộc course. Request tiến độ không được lấy userId tùy ý từ form.
• Xử lý nghiệp vụ và truy cập dữ liệu
LessonProgress được tìm/lưu theo user/lesson; unique(student_id,lesson_id) là ràng buộc DDL. getCourseProgress tổng hợp từ dữ liệu lưu, không đánh giá mức hiểu bài của học viên.
Đối chiếu dữ liệu theo UC016.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/LessonProgressServiceImpl.java, bắt đầu dòng 28. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

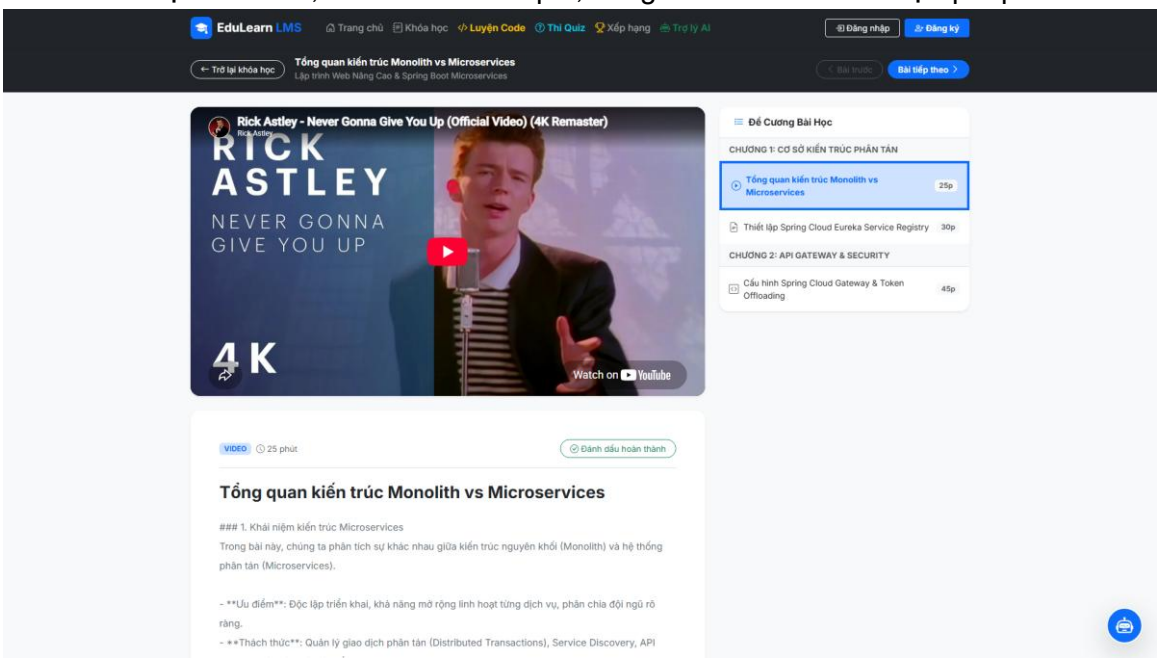
```
public void completeLesson(String courseId, String lessonId, CompleteLessonRequest
req) {
    String studentId = GatewayAuthentication.currentUserId();
    if (!enrollmentRepo.existsByUserIdAndCourseId(studentId, courseId)) {
        throw new CourseException(ErrorCode.NOT_ENROLLED);
    }
    if (!structureService.isLessonInCourse(courseId, lessonId)) {
```

```
            throw new CourseException(ErrorCode.LESSON_NOT_BELONG_TO_COURSE);
        }
        LessonProgress progress = progressRepo
                .findByStudentIdAndLessonId(studentId, lessonId)
                .orElse(LessonProgress.builder()
```

Giải thích: LessonProgress được tìm/lưu theo user/lesson; unique(student_id,lesson_id) là ràng buộc DDL. getCourseProgress tổng hợp từ dữ liệu lưu, không đánh giá mức hiểu bài của học viên.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: DetailMyCoursePage. Demo theo đúng role và luồng API được mô tả. Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.2.6: Phòng học trực tuyến tích hợp video bài giảng và thanh tiến độ hoàn thành*

• Ngoại lệ và tình huống biên

NOT_ENROLLED, LESSON_NOT_BELONG_TO_COURSE và lỗi dữ liệu.

Luồng thay thế: Đọc lại tiến độ để tiếp tục học; số lần gọi không tự chứng minh đã hiểu nội dung.

Cần kiểm chứng riêng: Thử gọi hoàn thành lặp, user chưa tham gia và lesson course khác; đọc tiến độ trước/sau và kiểm tra không tạo trùng ngoài thiết kế.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-2-6-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-2-6-02: Thử gọi hoàn thành lặp, user chưa tham gia và lesson course khác; đọc tiến độ trước/sau và kiểm tra không tạo trùng ngoài thiết kế.

TC-5-2-6-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-2-6-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.2.7 Quản lý lịch học và thông báo

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: Schedule cần endTime sau startTime; create yêu cầu start không ở quá khứ. Announcement có title/content theo DTO; quyền quản lý gắn course.

Tác nhân: INSTRUCTOR chủ khóa học; ADMIN. Tiền điều kiện: Course chưa LOCKED; thời gian hợp lệ; cấu hình Calendar nếu tạo Meet.

Tác nhân: INSTRUCTOR chủ khóa học; ADMIN. Tiền điều kiện: Course tồn tại; được phép quản lý thông báo.

Tác nhân: Người có quyền trong khóa học. Tiền điều kiện: Đăng nhập; course tồn tại và được phép truy cập.

Đầu ra: Schedule được lưu; Meet là liên kết cấp course theo source. Thông báo được lưu trong course. Đọc lịch và thông báo; không tạo lịch mới.

• Luồng xử lý và sơ đồ hoạt động

1. Nhập lịch. 2. Kiểm tra endTime sau startTime; lịch mới không bắt đầu trong quá khứ. 3. Lưu schedule. 4. Hiển thị lịch và meetingUrl của course.

1. Mở thông báo course. 2. Nhập và gửi nội dung hoặc chọn xóa. 3. Backend kiểm tra quyền và dữ liệu. 4. Lưu/xóa announcement. 5. Cập nhật danh sách.

1. Mở lịch/thông báo của course. 2. Gọi API tương ứng. 3. Service kiểm tra quyền. 4. Trả danh sách. 5. Nếu có meetingUrl, người học mở liên kết.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.

[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]

• API DTO validation và phân quyền

API chính: GET/POST /api/course/courses/{courseId}/schedules; GET/POST /api/course/courses/{courseId}/announcements.

Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/ScheduleServiceImpl.java cùng các controller liên quan ở 3.3.

Schedule cần endTime sau startTime; create yêu cầu start không ở quá khứ. Announcement có title/content theo DTO; quyền quản lý gắn course.

• Xử lý nghiệp vụ và truy cập dữ liệu

ScheduleService checkCourseOwner/checkNotLocked trước lưu. meetingUrl lấy từ course qua mapper. Cần kiểm tra scheduleId/announcementId thực sự thuộc course khi sửa/xóa, không chỉ xác thực courseId ở URL.

Đối chiếu dữ liệu theo UC029, UC030, UC017.

• Source code trọng tâm và giải thích

Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/ScheduleServiceImpl.java, bắt đầu dòng 48. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public ScheduleResponse createSchedule(String courseId, CreateScheduleRequest
request) {
    Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new CourseException(ErrorCode.COURSE_NOT_FOUND));
    checkCourseOwner(course);
    checkNotLocked(course);
    validateNewScheduleTime(request.getStartTime(), request.getEndTime());
    Schedule schedule = Schedule.builder()
            .course(course)
```

```
            .title(request.getTitle())
            .startTime(request.getStartTime())
            .endTime(request.getEndTime())
            .note(request.getNote())
```

Giải thích: ScheduleService checkCourseOwner/checkNotLocked trước lưu. meetingUrl lấy từ course qua mapper. Cần kiểm tra scheduleId/announcementId thực sự thuộc course khi sửa/xóa, không chỉ xác thực courseId ở URL.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: CourseManagementCenter và giao diện course học viên. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên

SCHEDULE_TIME_INVALID, NOT_COURSE_OWNER, COURSE_LOCKED, createMeeting có thể trả URL dự phòng với eventId=null khi Calendar lỗi; URL chưa xác nhận có phòng thật.

Course/announcement không tồn tại, không có quyền, validation thất bại. ACCESS_DENIED, COURSE_NOT_FOUND hoặc lỗi API.

Luồng thay thế: Sửa/xóa lịch theo API; không mặc định tạo phòng độc lập cho từng lịch. Người học chỉ đọc qua API được cấp quyền. Danh sách rỗng hoặc chưa có liên kết Meet vẫn hiển thị nội dung khác.

Cần kiểm chứng riêng: Thử thời gian đảo, lịch mới quá khứ, course LOCKED, owner khác và ID lịch của course khác; kiểm tra dữ liệu của khóa khác không bị sửa.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-2-7-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-2-7-02: Thử thời gian đảo, lịch mới quá khứ, course LOCKED, owner khác và ID lịch của course khác; kiểm tra dữ liệu của khóa khác không bị sửa.

TC-5-2-7-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-2-7-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

# 5.3 Assignment Service

## 5.3.1 Ngân hàng câu hỏi và đáp án

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: Question có SINGLE/MULTIPLE/TRUE_FALSE, answer chứa cờ correct và order. QuestionController có @PreAuthorize ở cấp class cho ADMIN/INSTRUCTOR.

Tác nhân: INSTRUCTOR và ADMIN theo quyền nội dung. Tiền điều kiện: Có courseId; dữ liệu câu hỏi/đáp án thỏa loại SINGLE, MULTIPLE hoặc TRUE_FALSE.

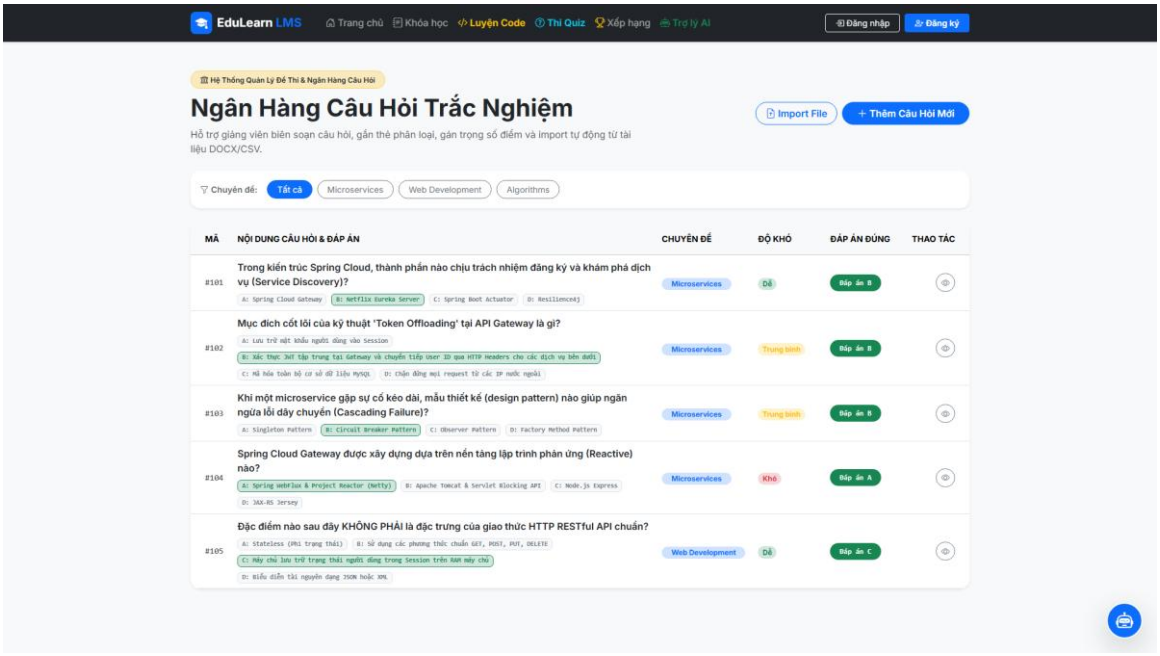
Đầu ra: Ngân hàng câu hỏi được cập nhật.

• Luồng xử lý và sơ đồ hoạt động

1. Mở ngân hàng câu hỏi. 2. Tạo/sửa hoặc tìm câu hỏi. 3. Gửi request. 4. Kiểm tra quyền và quy tắc đáp án. 5. Lưu Question/Answer.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/assignment/questions/courses/{courseId}; các thao tác còn lại theo QuestionController.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/QuestionServiceImpl.java cùng các controller liên quan ở 3.3.
Question có SINGLE/MULTIPLE/TRUE_FALSE, answer chứa cờ correct và order.
QuestionController có @PreAuthorize ở cấp class cho ADMIN/INSTRUCTOR.
• Xử lý nghiệp vụ và truy cập dữ liệu
Service kiểm tra quyền tài nguyên và dữ liệu câu hỏi; ValidAnswerCount hỗ trợ rule đáp án. Khi chấm, không tin client gửi earnedScore hoặc cờ correct.
Đối chiếu dữ liệu theo UC031.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/QuestionServiceImpl.java, bắt đầu dòng 47. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public QuestionDetailResponse createQuestion(String courseId, CreateQuestionRequest
request) {
    String userId = authorizationService.currentUserId();
    Question question = buildQuestionEntity(courseId, userId, request,
request.getTopic());
    Question saved = questionRepository.save(question);
    log.info("Tạo câu hỏi [{}] type={} bởi user [{}]",
            saved.getQuestionId(), saved.getType(), userId);
    return quizMapper.toQuestionResponse(saved);
}
@Override
@Transactional
public Page<QuestionDetailResponse> importBatchQuestion(
        String courseId, ImportBulkQuestionRequest request, Pageable pageable) {
```

Giải thích: Service kiểm tra quyền tài nguyên và dữ liệu câu hỏi; ValidAnswerCount hỗ trợ rule đáp án. Khi chấm, không tin client gửi earnedScore hoặc cờ correct.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: EditQuizPage và khu vực ngân hàng câu hỏi. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.3.1: Ngân hàng câu hỏi trắc nghiệm đa dạng theo độ khó và chuyên đề*

• Ngoại lệ và tình huống biên
Câu hỏi không tồn tại, dữ liệu đáp án sai hoặc không đủ quyền.
Luồng thay thế: Tái sử dụng câu hỏi khi xây dựng quiz theo liên kết quiz_questions.
Cần kiểm chứng riêng: Thử thiếu nội dung, đáp án sai loại, câu hỏi course khác và role STUDENT; kiểm tra nội dung đáp án đúng không lộ qua response học viên.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-1-02: Thử thiếu nội dung, đáp án sai loại, câu hỏi course khác và role STUDENT; kiểm tra nội dung đáp án đúng không lộ qua response học viên.
TC-5-3-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

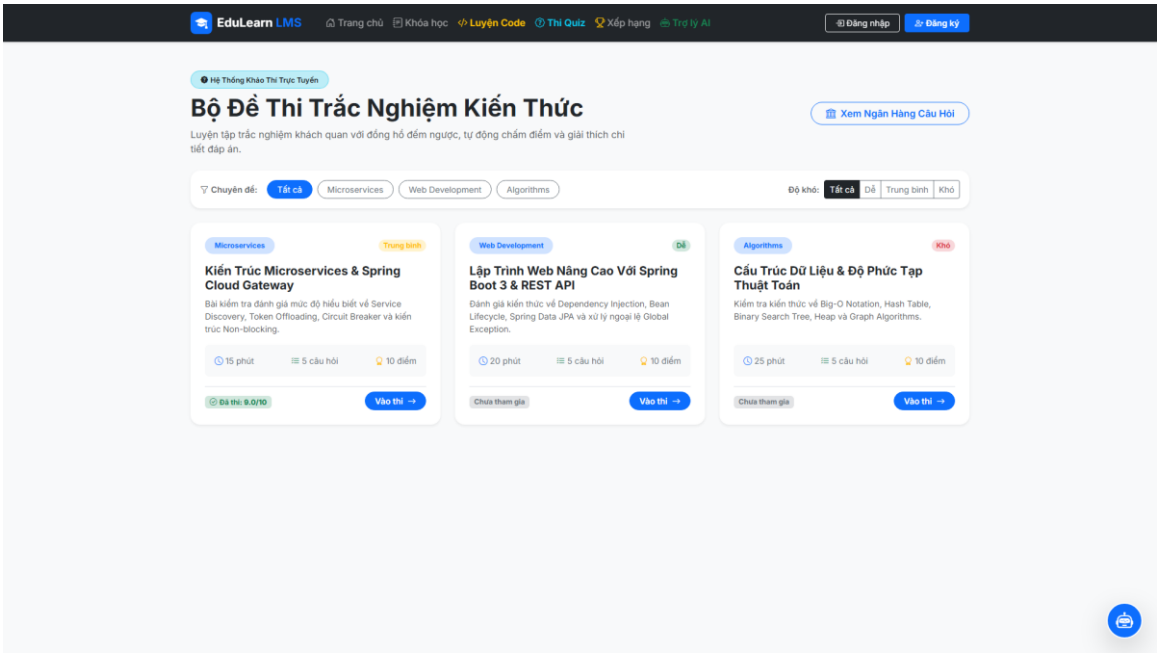
## 5.3.2 Tạo quản lý bản nháp công bố ẩn và khôi phục quiz

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Quiz có duration/totalScore/passScore/maxAttempts/shuffle/showResult/start/end và trạng thái published/deleted. Sync là luồng lưu cấu hình và câu hỏi cần phân biệt với publish.
Tác nhân: INSTRUCTOR và ADMIN theo kiểm tra service. Tiền điều kiện: Course và câu hỏi hợp lệ; thỏa kiểm tra quyền và cấu hình quiz.
Đầu ra: Quiz và trạng thái published được lưu; khôi phục là thao tác Admin theo API.
• Luồng xử lý và sơ đồ hoạt động

1. Tạo/sửa quiz. 2. Chọn câu hỏi, điểm và thời gian. 3. Lưu nháp hoặc sync. 4. Chọn công bố/ẩn khi phù hợp. 5. Backend lưu Quiz và quan hệ câu hỏi.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/assignment/quizzes/courses/{courseId}; PUT /api/assignment/quizzes/{quizId}/sync; PATCH .../publish, .../unpublish, .../restore.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/QuizServiceImpl.java cùng các controller liên quan ở 3.3.
Quiz có duration/totalScore/passScore/maxAttempts/shuffle/showResult/start/end và trạng thái published/deleted. Sync là luồng lưu cấu hình và câu hỏi cần phân biệt với publish.
• Xử lý nghiệp vụ và truy cập dữ liệu
Các endpoint restore yêu cầu ADMIN; nháp chưa nên hiển thị trong API student. Một số method không có @PreAuthorize trên controller nên phải xem kiểm tra service, không coi annotation vắng là bằng chứng an toàn hoặc lỗi ngay.
Đối chiếu dữ liệu theo UC032.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/QuizServiceImpl.java, bắt đầu dòng 201. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public QuizDetailResponse syncQuiz(Integer quizId, SaveQuizDraftRequest request) {
    validateSaveQuizDraftRequest(request);
    Quiz sourceQuiz = findQuizById(quizId);
    checkQuizManager(sourceQuiz);
    checkQuizNotDeleted(sourceQuiz);
    boolean locked = Boolean.TRUE.equals(sourceQuiz.getPublished())
            || quizAttemptRepository.existsByQuiz_QuizId(quizId);
    Quiz targetQuiz = locked ? createEditableClone(sourceQuiz) : sourceQuiz;
    quizMapper.updateQuiz(request.getQuiz(), targetQuiz);
    replaceQuizQuestions(targetQuiz, request.getQuestions(), locked);
    targetQuiz.setPublished(locked ? false : request.isPublish());
```

Giải thích: Các endpoint restore yêu cầu ADMIN; nháp chưa nên hiển thị trong API student. Một số method không có @PreAuthorize trên controller nên phải xem kiểm tra service, không coi annotation vắng là bằng chứng an toàn hoặc lỗi ngay.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CreateQuizPage và EditQuizPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.3.2: Danh sách các bài kiểm tra trắc nghiệm (Quiz) của khóa học*

• Ngoại lệ và tình huống biên
Cấu hình sai, câu hỏi không hợp lệ, không đủ quyền, quiz deleted/không tồn tại.
Luồng thay thế: Giữ nháp để chỉnh sửa; không hiển thị như quiz published cho học viên.
Cần kiểm chứng riêng: Thử lưu nháp, publish, unpublish, delete/restore, rule thời gian và thay câu hỏi; kiểm tra student listing và attempt đang diễn ra.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-2-02: Thử lưu nháp, publish, unpublish, delete/restore, rule thời gian và thay câu hỏi; kiểm tra student listing và attempt đang diễn ra.
TC-5-3-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.3.3 Làm quiz chấm điểm và lưu lịch sử

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: QuizAttempt có owner, quiz, status, expiresAt và version; unique in_progress_lock ngăn hai IN_PROGRESS cùng user/quiz. Submit kiểm tra câu hỏi thuộc quiz và answer thuộc question.
Tác nhân: Người làm quiz theo phiên hiện tại. Tiền điều kiện: Quiz chưa deleted, đã published, trong time window và chưa vượt maxAttempts khi tạo lượt mới.
Tác nhân: Chủ lượt làm; người quản lý theo API được cấp. Tiền điều kiện: Attempt thuộc quiz; quyền đọc kết quả phù hợp show_result policy.

Đầu ra: Lưu attempt và kết quả; trạng thái SUBMITTED hoặc EXPIRED theo xử lý thời hạn. Đọc kết quả không sửa bài đã nộp.

• Luồng xử lý và sơ đồ hoạt động

1. Chọn bắt đầu. 2. Backend start-or-resume, tiếp tục IN_PROGRESS nếu có. 3. Tạo attempt mới khi đủ điều kiện. 4. Người học chọn đáp án và autosave. 5. Gửi submit. 6. Kiểm tra owner/quiz/câu hỏi/đáp án, chấm và lưu trạng thái.

1. Mở kết quả hoặc lịch sử. 2. Gửi quizId/attemptId hoặc tiêu chí lịch sử. 3. Backend kiểm tra người yêu cầu. 4. Kiểm tra trạng thái và chính sách xem kết quả. 5. Trả điểm và thông tin được phép.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.

[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]

• API DTO validation và phân quyền

API chính: POST /api/assignment/quizzes/{quizId}/attempts/start-or-resume; PUT .../attempts/{attemptId}; POST .../submit; GET .../result.

Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/QuizGradingServiceImpl.java cùng các controller liên quan ở 3.3.

QuizAttempt có owner, quiz, status, expiresAt và version; unique in_progress_lock ngăn hai IN_PROGRESS cùng user/quiz. Submit kiểm tra câu hỏi thuộc quiz và answer thuộc question.

• Xử lý nghiệp vụ và truy cập dữ liệu

Chấm chọn chính xác cả tập đáp án: selectedIds không rỗng, bằng số đáp án đúng và thuộc tập đúng. Điểm câu chuẩn hóa bằng quizTotalScore × effectiveQuestionScore / tổng effectiveScore, làm tròn 4 chữ số. Ví dụ trọng số 10 và 30, tổng quiz 100: câu tương ứng 25 và 75 điểm; không phải cộng trực tiếp 10+30. passed dựa passScore phần trăm. Grade giữ best_score và số lượt.

Đối chiếu dữ liệu theo UC019, UC020.

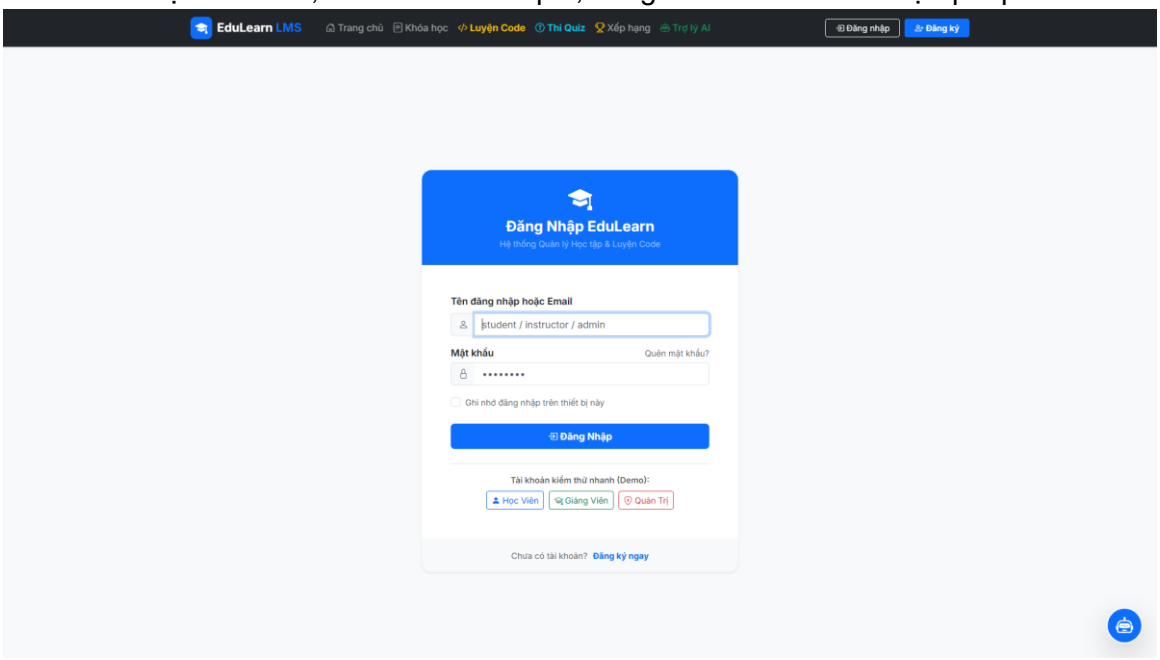
• Source code trọng tâm và giải thích

Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/QuizGradingServiceImpl.java, bắt đầu dòng 44. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public GradingResult gradeAttempt(QuizAttempt attempt, Map<Integer, List<Integer>>
overrides) {
    Long attemptId = attempt.getAttemptId();
    Map<Integer, QuizAnswerRecord> existingRecordsMap = quizAnswerRecordRepository
            .findByQuizAttempt_AttemptId(attemptId)
            .stream()
            .collect(Collectors.toMap(
                    r -> r.getQuestion().getQuestionId(),
                    r -> r,
                    (v1, v2) -> v2
            ));
    Quiz quiz = attempt.getQuiz();
    List<Question> allQuestions =
questionRepository.findAllByQuizIdWithAnswers(quiz.getQuizId());
```

Giải thích: Chấm chọn chính xác cả tập đáp án: selectedIds không rỗng, bằng số đáp án đúng và thuộc tập đúng. Điểm câu chuẩn hóa bằng quizTotalScore × effectiveQuestionScore / tổng effectiveScore, làm tròn 4 chữ số. Ví dụ trọng số 10 và 30,

tổng quiz 100: câu tương ứng 25 và 75 điểm; không phải cộng trực tiếp 10+30. passed dựa passScore phần trăm. Grade giữ best_score và số lượt.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ExerciseQuizPage, ExerciseQuizResultPage, QuizHistoryDetailPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.3.3: Giao diện làm bài thi trắc nghiệm trực tuyến với đồng hồ đếm ngược*

• Ngoại lệ và tình huống biên
QUIZ_NOT_PUBLISHED, QUIZ_NOT_AVAILABLE, MAX_ATTEMPTS_REACHED, ATTEMPT_EXPIRED; đáp án/câu hỏi không thuộc bài bị từ chối. Không tự khẳng định start đã kiểm tra enrollment nếu chưa thấy điều kiện đó.
ACCESS_DENIED, ATTEMPT_NOT_FOUND, SHOW_RESULT_NOT_ALLOWED.
Luồng thay thế: F5 lấy lại attempt đang làm; practice có luồng kiểm tra đáp án riêng. Chưa được phép xem kết quả thì thông báo và chờ điều kiện cho phép.
Cần kiểm chứng riêng: Thử F5/resume, hai request start đồng thời, autosave, hết giờ, maxAttempts, answer ngoại lai, chọn thiếu/thừa ở MULTIPLE, policy show_result; không tự ghi điểm đạt trước chạy.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-3-02: Thử F5/resume, hai request start đồng thời, autosave, hết giờ, maxAttempts, answer ngoại lai, chọn thiếu/thừa ở MULTIPLE, policy show_result; không tự ghi điểm đạt trước chạy.
TC-5-3-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

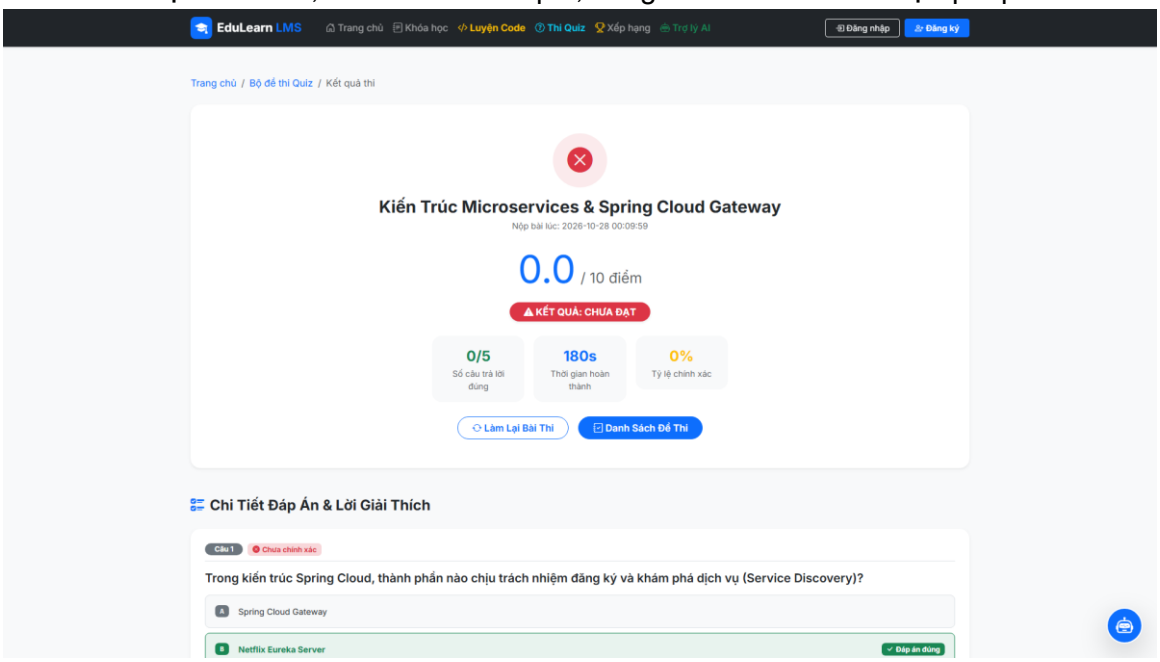
### 5.3.4 Nhập quiz từ template

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Template quiz mặc định DOCX qua format; parser nhập câu hỏi có DOCX/CSV ở QuestionImportFileServiceImpl. Download template chỉ trả Resource, chưa tạo quiz.
Tác nhân: INSTRUCTOR và ADMIN. Tiền điều kiện: Tệp theo template được hệ thống chấp nhận; course hợp lệ.
Đầu ra: Quiz được lưu khi bước nhập/tạo thành công.
• Luồng xử lý và sơ đồ hoạt động
1. Tải template. 2. Nhập dữ liệu và upload. 3. Backend đọc và kiểm tra. 4. Xem nội dung được xử lý. 5. Tạo quiz qua luồng imported.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/assignment/templates/quiz-import?format=docx; POST /api/assignment/quizzes/courses/{courseId}/imported.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/controller/QuizTemplateController.java cùng các controller liên quan ở 3.3.
Template quiz mặc định DOCX qua format; parser nhập câu hỏi có DOCX/CSV ở QuestionImportFileServiceImpl. Download template chỉ trả Resource, chưa tạo quiz.
• Xử lý nghiệp vụ và truy cập dữ liệu
Luồng nhập parse và trả preview/questions; luồng imported tạo quiz cùng câu hỏi theo service. Cần phân biệt file lỗi với câu lỗi; template được tải phải đúng định dạng engine chấp nhận.
Đối chiếu dữ liệu theo UC033.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/controller/QuizTemplateController.java, bắt đầu dòng 29. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public ResponseEntity<Resource>
downloadQuizImportTemplate(@RequestParam(defaultValue = "docx") String format) {
    QuizTemplateResource template = quizTemplateService.getTemplate(format);
    ContentDisposition contentDisposition = ContentDisposition.attachment()
            .filename(template.filename())
            .build();
    return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(template.mediaType()))
            .header(HttpHeaders.CONTENT_DISPOSITION, contentDisposition.toString())
            .body(template.resource());
}
@GetMapping("/testcases")
```

Giải thích: Luồng nhập parse và trả preview/questions; luồng imported tạo quiz cùng câu hỏi theo service. Cần phân biệt file lỗi với câu lỗi; template được tải phải đúng định dạng engine chấp nhận.
• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: CreateQuizPage và EditQuizPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.3.4: Bảng điểm tổng kết, đánh giá chi tiết câu đúng/sai sau khi nộp bài quiz*

• Ngoại lệ và tình huống biên
Sai cấu trúc tệp, câu hỏi/đáp án không hợp lệ hoặc lỗi quyền.
Luồng thay thế: Chỉnh tệp khi dữ liệu sai; hủy trước tạo không công bố bài.
Cần kiểm chứng riêng: Thử file rỗng, đổi đuôi giả, dòng/câu lỗi, không có câu hợp lệ, rồi tạo imported và kiểm tra DB. Chụp preview thực tế nếu có.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-4-02: Thử file rỗng, đổi đuôi giả, dòng/câu lỗi, không có câu hợp lệ, rồi tạo imported và kiểm tra DB. Chụp preview thực tế nếu có.
TC-5-3-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

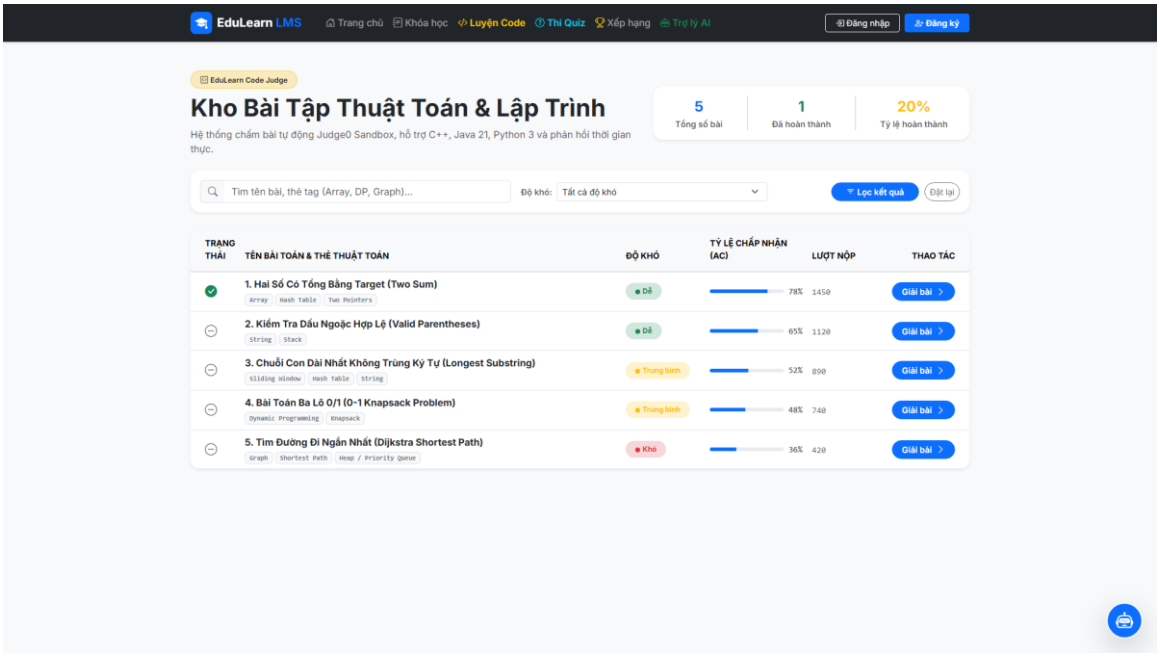
## 5.3.5 Tạo sửa sao chép ẩn xóa và khôi phục bài lập trình

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Problem chứa course/lesson, ngôn ngữ, timeLimit/memoryLimit, score và trạng thái. create/update trong ProblemController được đọc không có @Valid, nên cần đánh giá rule service, không khẳng định DTO annotation luôn được kích hoạt.
Tác nhân: INSTRUCTOR và ADMIN theo quyền bài. Tiền điều kiện: Course và dữ liệu problem hợp lệ; người thao tác có quyền.

Đầu ra: Problem thay đổi theo thao tác; clone tạo bài mới.
• Luồng xử lý và sơ đồ hoạt động
1. Tạo/sửa tiêu đề, đề bài, ngôn ngữ và giới hạn. 2. Gửi API. 3. Backend kiểm tra và lưu. 4. Có thể sao chép problem và test case, ẩn hoặc xóa; restore chỉ Admin.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/assignment/problems; PUT/DELETE .../{problemId}; POST .../{problemId}/clone; PATCH .../unpublish, .../restore.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/ProblemServiceImpl.java cùng các controller liên quan ở 3.3.
Problem chứa course/lesson, ngôn ngữ, timeLimit/memoryLimit, score và trạng thái.
create/update trong ProblemController được đọc không có @Valid, nên cần đánh giá rule service, không khẳng định DTO annotation luôn được kích hoạt.
create/update ProblemController chưa có @Valid trong đoạn đã đọc; phải kiểm tra rule service.
• Xử lý nghiệp vụ và truy cập dữ liệu
cloneProblem tạo tiêu đề Copy, slug riêng, Problem mới và test case sao chép. unpublish/delete/restore khác nhau; restore ADMIN. Khóa quyền dựa courseResourceAuthorizationService/createdBy theo service.
Đối chiếu dữ liệu theo UC034.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/ProblemServiceImpl.java, bắt đầu dòng 134. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Integer cloneProblem(Integer problemId) {
    Problem source = findById(problemId);
    String userId = authorizationService.currentUserId();
    courseResourceAuthorizationService.checkManager(source.getCourseId(),
source.getCreatedBy());
    if (source.getDeleted()) {
        throw new AssignmentException(ErrorCode.PROBLEM_IS_DELETED);
    }
    String cloneTitle = source.getTitle() + " (Copy)";
    Problem clone = Problem.builder()
            .courseId(source.getCourseId())
            .lessonId(source.getLessonId())
```

Giải thích: cloneProblem tạo tiêu đề Copy, slug riêng, Problem mới và test case sao chép. unpublish/delete/restore khác nhau; restore ADMIN. Khóa quyền dựa courseResourceAuthorizationService/createdBy theo service.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CreateCodingPage. Demo theo đúng role và luồng API được mô tả. Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.3.5: Danh sách bài tập lập trình trực tuyến kèm thông số thời gian, bộ nhớ*

• Ngoại lệ và tình huống biên
Problem/course không tồn tại, không đủ quyền hoặc validation sai.
Luồng thay thế: Giữ bài chưa công bố; sửa trước cho học viên sử dụng.
Cần kiểm chứng riêng: Thử clone và kiểm tra test độc lập, sửa bài khác owner, language sai, time/memory sai, delete rồi restore; chụp dữ liệu thật.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-5-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-5-02: Thử clone và kiểm tra test độc lập, sửa bài khác owner, language sai, time/memory sai, delete rồi restore; chụp dữ liệu thật.
TC-5-3-5-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-5-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

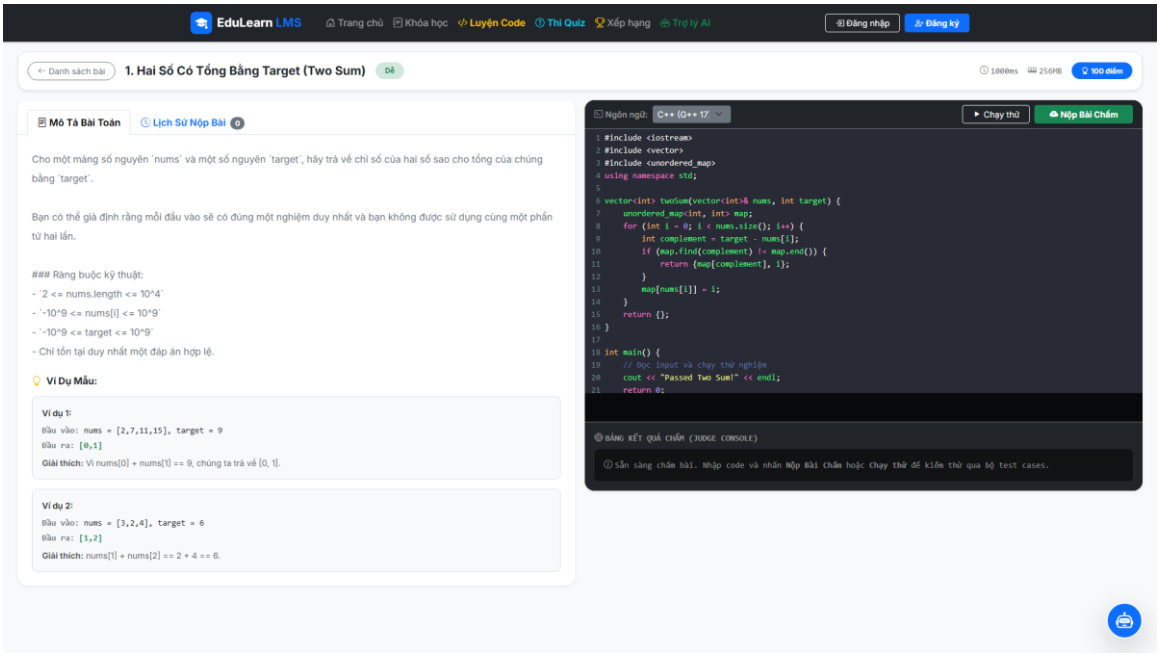
## 5.3.6 Quản lý test case

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Test case gồm input/expectedOutput, hidden, order và scoreWeight. Controller quản lý yêu cầu ADMIN/INSTRUCTOR; service kiểm tra manager của resource.
Tác nhân: INSTRUCTOR và ADMIN. Tiền điều kiện: Problem tồn tại; có quyền quản lý test case.
Đầu ra: Test case được lưu để judge sử dụng.
• Luồng xử lý và sơ đồ hoạt động
1. Nhập input/output và thuộc tính test. 2. Chọn tạo đơn/bulk, nhập hoặc sync. 3. API kiểm tra quyền và dữ liệu. 4. Lưu TestCase. 5. Hiển thị danh sách quản lý.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/assignment/problems/{problemId}/testcases; POST .../bulk, .../import; PUT .../sync.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/TestCaseServiceImpl.java cùng các controller liên quan ở 3.3.
Test case gồm input/expectedOutput, hidden, order và scoreWeight. Controller quản lý yêu cầu ADMIN/INSTRUCTOR; service kiểm tra manager của resource.
• Xử lý nghiệp vụ và truy cập dữ liệu
createTestCase gọi checkNoSubmissions trước lưu. Không được mô tả giảng viên luôn sửa test tự do sau khi đã có bài nộp. Bulk/import/sync cần đối chiếu cùng rule và bảo toàn test case được dùng chấm.
Đối chiếu dữ liệu theo UC035.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/TestCaseServiceImpl.java, bắt đầu dòng 64. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public TestCaseResponse createTestCase(CreateTestCaseRequest request, Integer
problemId) {
    // kiem tra problem co ton tai
    Problem problem = problemRepository.findById(problemId)
            .orElseThrow(() -> new
AssignmentException(ErrorCode.PROBLEM_NOT_FOUND));
    courseResourceAuthorizationService.checkManager(problem.getCourseId(),
problem.getCreatedBy());
    checkNoSubmissions(problemId);
    TestCase testCase = testCaseMapper.toTestCase(request);
    testCase.setProblem(problem);
    testCaseRepository.save(testCase);
    return testCaseMapper.toTestCaseResponse(testCase);
}
```

Giải thích: createTestCase gọi checkNoSubmissions trước lưu. Không được mô tả giảng viên luôn sửa test tự do sau khi đã có bài nộp. Bulk/import/sync cần đối chiếu cùng rule và bảo toàn test case được dùng chấm.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CreateCodingPage và editor test case. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.3.6: Trình soạn thảo mã nguồn Monaco Editor hỗ trợ biên dịch và nộp bài*

• Ngoại lệ và tình huống biên
Problem/test không tồn tại, dữ liệu sai, không đủ quyền.
Luồng thay thế: Preview AI/import trước khi lưu; sửa hoặc xóa test.
Cần kiểm chứng riêng: Thử tạo test khi chưa có submission và sau có submission, hidden/public, trọng số/đầu vào sai, problem khác owner; ghi điều kiện bị chặn.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-6-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-6-02: Thử tạo test khi chưa có submission và sau có submission, hidden/public, trọng số/đầu vào sai, problem khác owner; ghi điều kiện bị chặn.
TC-5-3-6-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-6-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.3.7 Trình soạn thảo và chạy thử code

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Editor có thể tô màu nhiều ngôn ngữ, nhưng runCode chỉ chấp nhận engine.supportedLanguages và Problem.allowedLangs. input/stdin dùng cho chạy thử không phải đáp án quiz.
Tác nhân: Người đã đăng nhập; học viên cần enrollment khi không phải Admin/Instructor.
Tiền điều kiện: Problem chưa deleted; ngôn ngữ được judge và bài cho phép; code không trống và dài không quá 65536 ký tự.
Đầu ra: Chạy thử trả output; nộp chính thức tạo submission để chấm.
• Luồng xử lý và sơ đồ hoạt động

1. Mở bài và soạn code. 2. Chọn ngôn ngữ, chạy thử với input. 3. Backend kiểm tra code và quyền. 4. Khi nộp, thực hiện compile preflight. 5. Lưu submission PENDING và gọi chấm bất đồng bộ. 6. Giao diện theo dõi kết quả.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/assignment/submissions/run.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/SubmissionServiceImpl.java cùng các controller liên quan ở 3.3.
Editor có thể tô màu nhiều ngôn ngữ, nhưng runCode chỉ chấp nhận engine.supportedLanguages và Problem.allowedLangs. input/stdin dùng cho chạy thử không phải đáp án quiz.
• Xử lý nghiệp vụ và truy cập dữ liệu
runCode kiểm tra code không trống, giới hạn 65536 ký tự, problem chưa deleted và enrollment khi không Admin/Instructor. Engine trả stdout/lỗi/thời gian theo response; chưa tạo submission chính thức chỉ vì bấm Run.
Đối chiếu dữ liệu theo UC021.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/SubmissionServiceImpl.java, bắt đầu dòng 137. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public RunCodeResponse runCode(RunCodeRequest request) {
    String userId = authorizationService.currentUserId();
    String language = request.getLanguage().toUpperCase();
    if (!judgeEngine.supportedLanguages().contains(language)) {
        throw new AssignmentException(ErrorCode.INVALID_LANGUAGE);
    }
    if (request.getSourceCode() == null || request.getSourceCode().isBlank()) {
        throw new AssignmentException(ErrorCode.SOURCE_CODE_EMPTY);
    }
    if (request.getSourceCode().length() > 65536) {
```

Giải thích: runCode kiểm tra code không trống, giới hạn 65536 ký tự, problem chưa deleted và enrollment khi không Admin/Instructor. Engine trả stdout/lỗi/thời gian theo response; chưa tạo submission chính thức chỉ vì bấm Run.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ExerciseCodePage và useJudge0. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên
INVALID_LANGUAGE, SOURCE_CODE_EMPTY, SOURCE_CODE_TOO_LARGE, NOT_ENROLLED, JUDGE0_TIMEOUT/JUDGE0_ERROR hoặc lỗi biên dịch.
Luồng thay thế: Chạy thử không đồng nghĩa đã nộp; sửa code và nộp lại tạo lượt riêng.
Cần kiểm chứng riêng: Thử syntax error, code đúng, input khác, ngôn ngữ bị cấm, code quá dài và timeout; phân biệt lỗi chương trình với lỗi Judge0.
• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-7-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-7-02: Thử syntax error, code đúng, input khác, ngôn ngữ bị cấm, code quá dài và timeout; phân biệt lỗi chương trình với lỗi Judge0.
TC-5-3-7-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-7-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.3.8 Nộp bài chấm tự động và lưu submission

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: submitCode POST /submissions hiện trả ApiResponse từ controller không khai báo ResponseEntity 202; comment trong service nói 202 không thay HTTP behavior. Phải đo response thực tế trước ghi status.
Tác nhân: Người đã đăng nhập; học viên cần enrollment khi không phải Admin/Instructor.
Tiền điều kiện: Problem chưa deleted; ngôn ngữ được judge và bài cho phép; code không trống và dài không quá 65536 ký tự.
Tác nhân: Chủ submission; Admin/Instructor theo kiểm tra service. Tiền điều kiện: Submission tồn tại; người đọc có quyền.
Đầu ra: Chạy thử trả output; nộp chính thức tạo submission để chấm. Hiển thị kết quả được lưu; không coi PENDING là đã chấm xong.
• Luồng xử lý và sơ đồ hoạt động
1. Mở bài và soạn code. 2. Chọn ngôn ngữ, chạy thử với input. 3. Backend kiểm tra code và quyền. 4. Khi nộp, thực hiện compile preflight. 5. Lưu submission PENDING và gọi chấm bất đồng bộ. 6. Giao diện theo dõi kết quả.
1. Mở kết quả/histories. 2. Gọi API. 3. Service kiểm tra dữ liệu và quyền. 4. Trả trạng thái, điểm và thông tin chấm theo response. 5. Với PENDING, giao diện tiếp tục theo dõi.
Tách response tạo PENDING khỏi judgeAsync; không ghi HTTP202 chỉ từ comment.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/assignment/submissions; GET /api/assignment/submissions/{id}; GET /api/assignment/submissions với problemId/userId/status.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/SubmissionJudgingServiceImpl.java cùng các controller liên quan ở 3.3.
submitCode POST /submissions hiện trả ApiResponse từ controller không khai báo ResponseEntity 202; comment trong service nói 202 không thay HTTP behavior. Phải đo response thực tế trước ghi status.
• Xử lý nghiệp vụ và truy cập dữ liệu
Sau compile preflight lưu PENDING, tăng total_submit rồi judgeAsync bằng executor. Async đặt JUDGING, lấy test theo order, gọi engine, lưu kết quả và tăng total_accept khi AC. Nếu không có test, source auto AC với score=100: đây là giới hạn quan trọng, không chứng minh chương trình đúng. Engine lỗi chấm lưu IE/0.
Đối chiếu dữ liệu theo UC021, UC022.

• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/SubmissionJudgingServiceImpl.java, bắt đầu dòng 38. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public void judgeAsync(String submissionId, Problem problem, String sourceCode,
String language) {
    Submission submission = submissionRepository.findById(submissionId)
            .orElse(null);
    if (submission == null) return;
    submission.setStatus(SubmissionStatus.JUDGING.name());
    submissionRepository.save(submission);
    try {
        List<TestCase> testCases = testCaseRepository

.findByProblem_ProblemIdOrderByOrderIndexAsc(problem.getProblemId());
        if (testCases.isEmpty()) {
```

Giải thích: Sau compile preflight lưu PENDING, tăng total_submit rồi judgeAsync bằng executor. Async đặt JUDGING, lấy test theo order, gọi engine, lưu kết quả và tăng total_accept khi AC. Nếu không có test, source auto AC với score=100: đây là giới hạn quan trọng, không chứng minh chương trình đúng. Engine lỗi chấm lưu IE/0.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ExerciseCodePage và ExerciseCodeResultPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên
INVALID_LANGUAGE, SOURCE_CODE_EMPTY, SOURCE_CODE_TOO_LARGE, NOT_ENROLLED, JUDGE0_TIMEOUT/JUDGE0_ERROR hoặc lỗi biên dịch.
ACCESS_DENIED, NOT_ENROLLED hoặc submission không tồn tại.
Luồng thay thế: Chạy thử không đồng nghĩa đã nộp; sửa code và nộp lại tạo lượt riêng. Lọc lịch sử hoặc xem latest accepted theo quyền.
Cần kiểm chứng riêng: Thử có/không test, AC/WA/CE/TLE, engine lỗi, PENDING lâu và gọi đọc kết quả của người khác; đối chiếu hidden output và score.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-8-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-3-8-02: Thử có/không test, AC/WA/CE/TLE, engine lỗi, PENDING lâu và gọi đọc kết quả của người khác; đối chiếu hidden output và score.
TC-5-3-8-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-8-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.3.9 Bảng điểm thống kê và xếp hạng

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: API hoạt động nằm ở InstructorAssignmentAnalyticsController và LeaderboardController. GradeBookController hiện rỗng nên không có căn cứ một API /gradebook hoàn chỉnh.
Tác nhân: INSTRUCTOR và ADMIN theo API thống kê. Tiền điều kiện: Có quyền xem dữ liệu khóa học/học viên tương ứng.
Tác nhân: Người dùng có quyền truy cập màn hình và API leaderboard. Tiền điều kiện: Có bài/phạm vi leaderboard hợp lệ.
Đầu ra: Đọc dữ liệu điểm/thống kê; không mặc định có quyền sửa điểm. Đọc bảng xếp hạng; không sửa điểm gốc.
• Luồng xử lý và sơ đồ hoạt động
1. Mở ExerciseResults hoặc dashboard. 2. Chọn phạm vi. 3. Gọi gradebook/analytics. 4. Backend kiểm tra và tổng hợp. 5. Hiển thị kết quả.
1. Mở leaderboard từ khu vực bài tập. 2. Gọi API theo phạm vi. 3. Backend tổng hợp dữ liệu theo implementation. 4. Hiển thị thứ hạng.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/assignment/instructor/courses/{courseId}/analytics; GET .../submissions; leaderboard theo LeaderboardController; GET /api/assignment/courses/{courseId}/leaderboard/coding hoặc /quiz.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/InstructorAssignmentAnalyticsServiceImpl.java cùng các controller liên quan ở 3.3.
API hoạt động nằm ở InstructorAssignmentAnalyticsController và LeaderboardController. GradeBookController hiện rỗng nên không có căn cứ một API /gradebook hoàn chỉnh.
• Xử lý nghiệp vụ và truy cập dữ liệu
Analytics phân nhánh quiz/coding theo type, tổng hợp kết quả và số học viên qua CourseClient. Leaderboard theo course có endpoint /leaderboard/coding và /leaderboard/quiz. Cần kiểm tra công thức/điều kiện xem tại service; không mặc định cùng công thức ở mọi dashboard.
Đối chiếu dữ liệu theo UC037, UC023.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/service/impl/InstructorAssignmentAnalyticsServiceImpl.java, bắt đầu dòng 67. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public InstructorAssignmentAnalyticsResponse getAnalytics(String courseId, String
type) {
    String normalizedType = normalizeType(type);
    long studentCount =
Objects.requireNonNullElse(courseClient.countStudents(courseId), 0L);
    if (TYPE_QUIZ.equals(normalizedType)) {
        return getQuizAnalytics(courseId, studentCount);
    }
    return getCodingAnalytics(courseId, studentCount);
}
@Override
@Transactional(readOnly = true)
```

```
public Page<InstructorSubmissionSummaryResponse> getSubmissions(String courseId,
String type, Pageable pageable) {
```
Giải thích: Analytics phân nhánh quiz/coding theo type, tổng hợp kết quả và số học viên qua CourseClient. Leaderboard theo course có endpoint /leaderboard/coding và /leaderboard/quiz. Cần kiểm tra công thức/điều kiện xem tại service; không mặc định cùng công thức ở mọi dashboard.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: ExerciseResultsPage, Dashboard, LeaderboardPage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên

Không đủ quyền, course không tồn tại hoặc lỗi dịch vụ phụ thuộc.

Lỗi xác thực, phạm vi không hợp lệ hoặc lỗi tổng hợp; công thức xếp hạng sẽ mô tả từ service ở Chương 5.

Luồng thay thế: Lọc dữ liệu hoặc xuất nếu có implementation; không tự thêm chức năng xuất chưa xác nhận. Không có kết quả thì bảng rỗng.

Cần kiểm chứng riêng: Thử course ít dữ liệu, type sai, role không đủ quyền và dữ liệu hai course; so điểm tổng hợp với attempts/submissions thực tế.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-3-9-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-3-9-02: Thử course ít dữ liệu, type sai, role không đủ quyền và dữ liệu hai course; so điểm tổng hợp với attempts/submissions thực tế.

TC-5-3-9-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-3-9-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.3.10 Nhập bài lập trình từ template

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào là problemId và MultipartFile chứa test case; template mặc định XLSX. Đây là nhập test case cho bài đã có, chưa chứng minh nhập trọn đề bài lập trình. Tác nhân ADMIN/INSTRUCTOR theo TestCaseController, còn quyền resource và điều kiện bài nộp kiểm tra ở service.

Đầu ra phải lấy từ BulkTestCaseResponse của API import thật; dữ liệu mẫu ở PreviewImportPage không phải kết quả import.

• Luồng xử lý và sơ đồ hoạt động

1. Tải template test case. 2. Tạo problem hợp lệ hoặc chọn bài hiện có. 3. Chọn file tại luồng quản lý bài thực sự gọi API. 4. Gửi POST /problems/{problemId}/testcases/import. 5. Service kiểm tra quyền và file, parse và lưu theo rule. 6. Đọc BulkTestCaseResponse và danh sách test để đối chiếu.

PreviewImportPage đang dùng ba dòng mẫu trong LockedFeature; chưa xem đây là bước preview đã nối backend. [Cần hình luồng import thật sau khi xác minh.]

• API DTO validation và phân quyền
API chính: GET /api/assignment/templates/testcases?format=xlsx; POST /api/assignment/problems/{problemId}/testcases/import.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/controller/TestCaseController.java cùng các controller liên quan ở 3.3.
Tên mục này bao gồm nhập dữ liệu phục vụ bài lập trình, nhưng API đã xác định nhập test case cho problemId, không chứng minh import nguyên đề bài. Template test mặc định XLSX.
• Xử lý nghiệp vụ và truy cập dữ liệu
TestCaseController chuyển MultipartFile sang service import. PreviewImportPage riêng đang có ba dòng dữ liệu mẫu và LockedFeature, chưa dùng làm minh chứng preview import đã hoạt động. Luồng nhập thật cần xác nhận trong CreateCodingPage/handler gọi API.
Đối chiếu dữ liệu theo UC036.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/controller/TestCaseController.java, bắt đầu dòng 57. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
ApiResponse<BulkTestCaseResponse> importTestCases(
        @PathVariable Integer problemId,
        @RequestPart("file") MultipartFile file) {
    return ApiResponse.<BulkTestCaseResponse>builder()
            .data(testCaseService.importTestCases(problemId, file))
            .build();
}
@PreAuthorize("hasAnyRole('ADMIN', 'INSTRUCTOR')")
@PostMapping(value = "/testcases/ai-generate/preview")
ApiResponse<AiTestCasePreviewResponse> previewAiGeneratedTestCases(
        @Valid @RequestBody AiTestCasePreviewRequest request) {
```

Giải thích: TestCaseController chuyển MultipartFile sang service import. PreviewImportPage riêng đang có ba dòng dữ liệu mẫu và LockedFeature, chưa dùng làm minh chứng preview import đã hoạt động. Luồng nhập thật cần xác nhận trong CreateCodingPage/handler gọi API.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: PreviewImportPage và CreateCodingPage. Demo theo đúng role và luồng API được mô tả.
PreviewImportPage có dữ liệu mẫu và LockedFeature; không chụp nó làm minh chứng nhập dữ liệu thật.

• Ngoại lệ và tình huống biên
Tệp sai, dữ liệu thiếu, lỗi API; chi tiết định dạng lấy từ implementation ở Chương 5.
Luồng thay thế: Hủy preview hoặc chọn lại tệp.
Cần kiểm chứng riêng: Thử template đúng, file sai/rỗng, problem không quyền và bài đã có submission. Không chụp màn hình dữ liệu mẫu làm bằng chứng nhập thật.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-3-10-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-3-10-02: Thử template đúng, file sai/rỗng, problem không quyền và bài đã có submission. Không chụp màn hình dữ liệu mẫu làm bằng chứng nhập thật.
TC-5-3-10-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-3-10-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

# 5.4 Blog Service

## 5.4.1 Tìm kiếm và xem bài viết

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Listing public lấy các filter được controller hỗ trợ; detail public theo slug khác detail nội bộ theo ID.
Tác nhân: Guest và người dùng đăng nhập. Tiền điều kiện: Bài công khai theo trạng thái và route public.
Đầu ra: Đọc nội dung public.
• Luồng xử lý và sơ đồ hoạt động
1. Mở Blog Hub. 2. Tìm/lọc bài. 3. Gọi public listing. 4. Chọn slug bài. 5. Backend trả nội dung được công bố.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/blog/posts/public; GET /api/blog/posts/public/{slug}.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java cùng các controller liên quan ở 3.3.
Listing public lấy các filter được controller hỗ trợ; detail public theo slug khác detail nội bộ theo ID.
• Xử lý nghiệp vụ và truy cập dữ liệu
PostService chỉ trả nội dung phù hợp trạng thái public; response summary khác full content.
Route public không cần token; không thể dùng slug của nháp để kết luận nháp công khai.
Đối chiếu dữ liệu theo UC038.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java, bắt đầu dòng 56. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Page<PostSummaryResponse> getPosts(Pageable pageable, PostStatus status,
                                          Long tagId, String keyword) {
    Page<Post> posts;
    if (keyword != null && !keyword.isBlank())
        posts = postRepository.findByTitleContainingIgnoreCaseAndStatus(
                keyword, PostStatus.PUBLISHED, pageable);
    else if (tagId != null)
        posts = postRepository.findByStatusAndTagsId(PostStatus.PUBLISHED, tagId,
pageable);
```

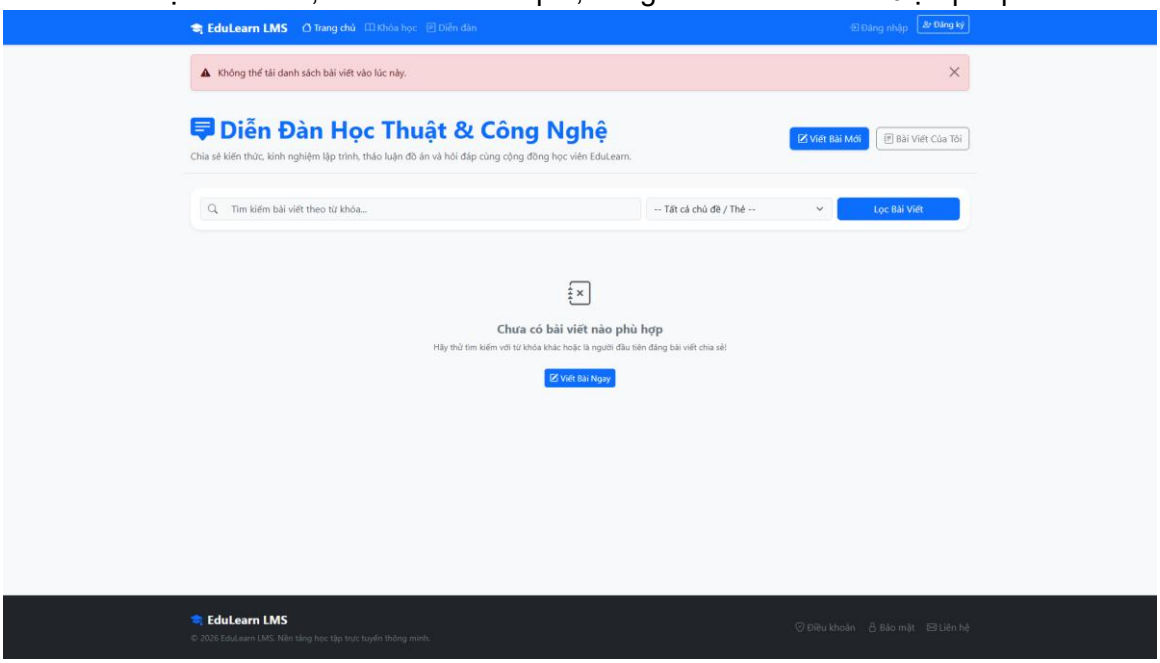
```
    else
        posts = postRepository.findByStatus(PostStatus.PUBLISHED, pageable);
    return posts.map(this::toSummaryWithCounts);
}
```

Giải thích: PostService chỉ trả nội dung phù hợp trạng thái public; response summary khác full content. Route public không cần token; không thể dùng slug của nháp để kết luận nháp công khai.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: BlogHubPage và BlogDetailPage. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.4.1: Diễn đàn trao đổi học tập và chia sẻ kiến thức công nghệ (Blog)*

• Ngoại lệ và tình huống biên

POST_NOT_FOUND hoặc bài chưa công bố; không dùng public route để đọc nháp.

Luồng thay thế: Không có kết quả thì đổi tiêu chí.

Cần kiểm chứng riêng: Thử tìm không có kết quả, slug không tồn tại, DRAFT/HIDDEN và PUBLISHED; kiểm tra view count theo implementation.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-4-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-4-1-02: Thử tìm không có kết quả, slug không tồn tại, DRAFT/HIDDEN và PUBLISHED; kiểm tra view count theo implementation.

TC-5-4-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-4-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

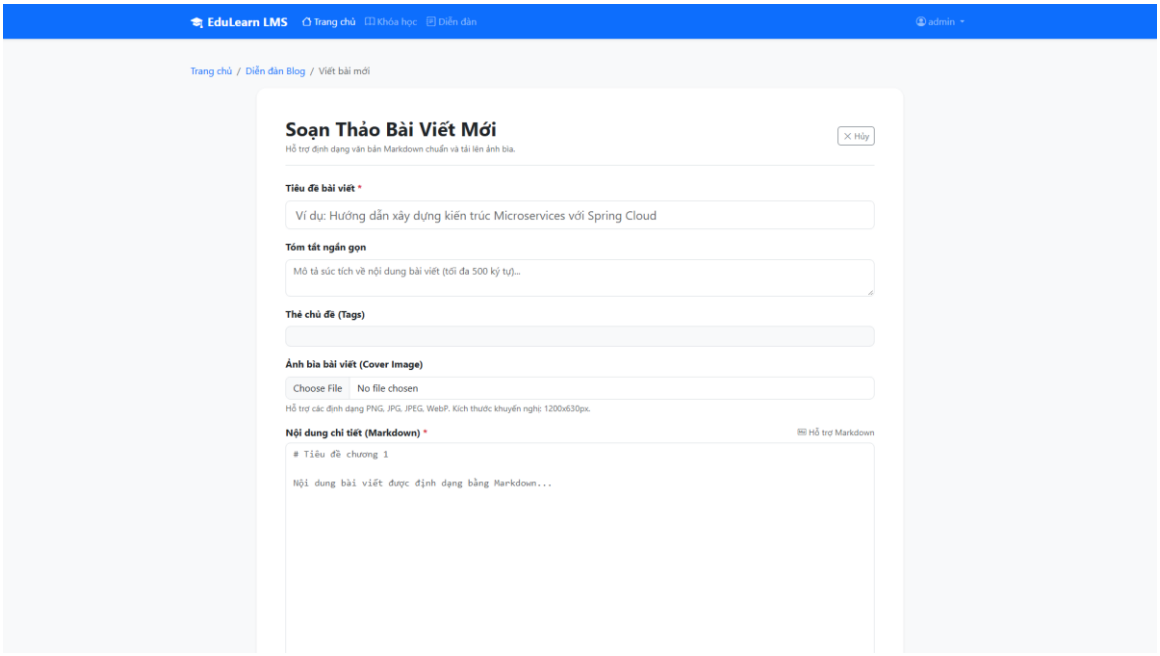
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.4.2 Tạo sửa xóa bài viết cá nhân

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: POST/PUT Post dùng multipart/form-data theo controller; title, content, status/tag và thumbnail phải qua request/service.
Tác nhân: Người đã đăng nhập; tác giả; ADMIN có quyền quản trị. Tiền điều kiện: Bài sửa/xóa thuộc tác giả hoặc người thao tác là Admin; dữ liệu hợp lệ.
Đầu ra: Post được tạo/cập nhật hoặc xóa.
• Luồng xử lý và sơ đồ hoạt động
1. Mở form bài viết. 2. Nhập nội dung và tệp. 3. Gửi tạo/sửa hoặc xác nhận xóa. 4. Backend kiểm tra title và quyền sở hữu. 5. Lưu dữ liệu/xóa tài nguyên theo service.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/blog/posts; PUT/DELETE /api/blog/posts/{id}.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java cùng các controller liên quan ở 3.3.
POST/PUT Post dùng multipart/form-data theo controller; title, content, status/tag và thumbnail phải qua request/service.
• Xử lý nghiệp vụ và truy cập dữ liệu
create kiểm tra title tồn tại; update/delete dùng checkPostOwner, Admin được xử lý riêng. MinIO upload và cleanup có thể lỗi riêng; không mô tả DB và object cùng rollback tự động. Đối chiếu dữ liệu theo UC039.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java, bắt đầu dòng 145. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public PostResponse createPost(CreatePostRequest request) {
    String authorId = GatewayAuthentication.currentUserId();
    if (postRepository.existsByTitle(request.getTitle()))
        throw new BlogException(ErrorCode.POST_TITLE_EXISTED);
    Post post = postMapper.toPost(request);
    post.setAuthorId(authorId);
    post.setSlug(generateSlug(request.getTitle()));
    if (request.getThumbnail() != null && !request.getThumbnail().isEmpty()) {
        String thumbnailUrl = minioService.uploadFile(request.getThumbnail(),
thumbnailFolder);
        post.setThumbnail(thumbnailUrl);
    }
```

Giải thích: create kiểm tra title tồn tại; update/delete dùng checkPostOwner, Admin được xử lý riêng. MinIO upload và cleanup có thể lỗi riêng; không mô tả DB và object cùng rollback tự động.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CreateBlogPostPage và EditBlogPostPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.4.2: Trình soạn thảo và đăng tải bài viết Blog cộng đồng*

• Ngoại lệ và tình huống biên
POST_TITLE_EXISTED, NOT_POST_OWNER, bài không tồn tại, lỗi upload.
Luồng thay thế: Giữ nháp theo trạng thái được hỗ trợ; hủy form.
Cần kiểm chứng riêng: Thử bài của người khác, title trùng, thumbnail sai, lưu nháp rồi đọc public, sửa/xóa bài có comment; đối chiếu tài nguyên bị dọn.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-4-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-4-2-02: Thử bài của người khác, title trùng, thumbnail sai, lưu nháp rồi đọc public, sửa/xóa bài có comment; đối chiếu tài nguyên bị dọn.
TC-5-4-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-4-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

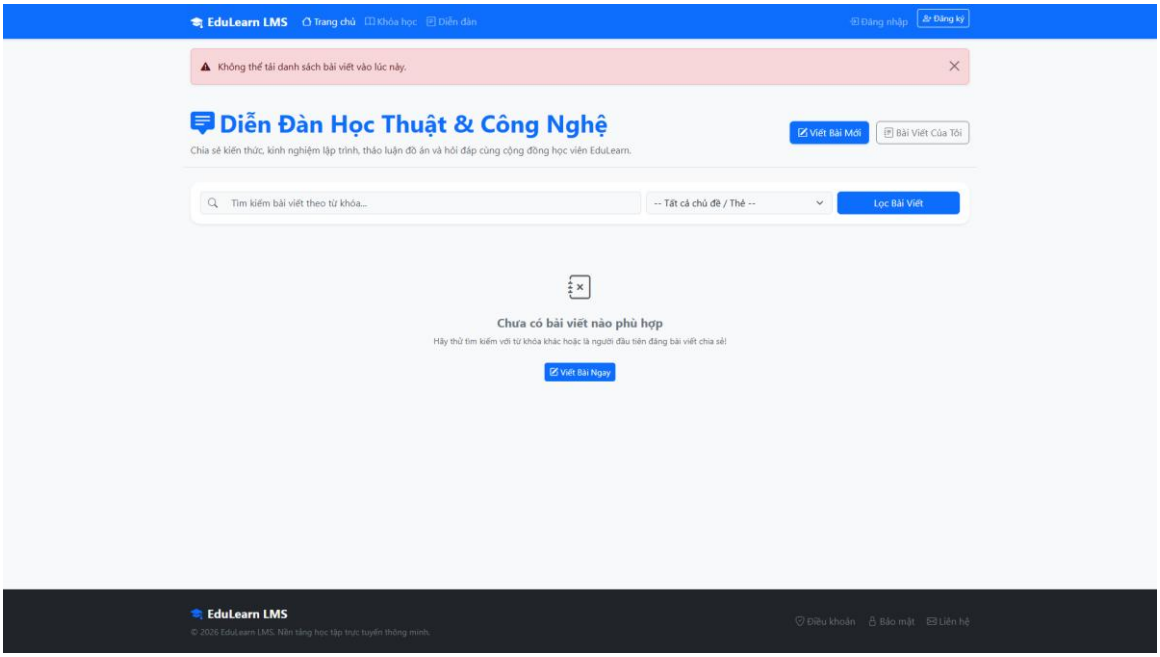
## 5.4.3 Quản lý bài viết cá nhân

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: GET /posts/me lấy tác giả từ phiên, không lấy arbitrary userId để liệt kê bài cá nhân.
Tác nhân: Người đã đăng nhập. Tiền điều kiện: Phiên xác định tác giả hiện tại.
Đầu ra: Danh sách chỉ gắn với tác giả hiện tại theo truy vấn.
• Luồng xử lý và sơ đồ hoạt động
1. Mở My Posts. 2. Gọi /posts/me với bộ lọc. 3. Backend lấy authorId từ phiên. 4. Trả danh sách phân trang. 5. Chọn bài để sửa/xem.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/blog/posts/me.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java cùng các controller liên quan ở 3.3.
GET /posts/me lấy tác giả từ phiên, không lấy arbitrary userId để liệt kê bài cá nhân.
• Xử lý nghiệp vụ và truy cập dữ liệu
getMyPosts truy vấn theo author và trạng thái với pageable; màn hình hiển thị summary và liên kết tác vụ. Giữ pagination metadata từ server.
Đối chiếu dữ liệu theo UC040.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java, bắt đầu dòng 99. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Page<PostSummaryResponse> getMyPosts(Pageable pageable, PostStatus status) {
    String authorId = GatewayAuthentication.currentUserId();
    Page<Post> posts = status != null
            ? postRepository.findByAuthorIdAndStatus(authorId, status, pageable)
            : postRepository.findByAuthorId(authorId, pageable);
    return posts.map(this::toSummaryWithCounts);
}
@Override
public PostResponse getPostById(Long id) {
    Post post = postRepository.findById(id)
            .orElseThrow(() -> new BlogException(ErrorCode.POST_NOT_FOUND));
    // Bài DRAFT/HIDDEN chỉ owner hoặc ADMIN mới xem được
```

Giải thích: getMyPosts truy vấn theo author và trạng thái với pageable; màn hình hiển thị summary và liên kết tác vụ. Giữ pagination metadata từ server.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: MyBlogPostsPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.4.3: Quản lý danh sách bài viết cá nhân của người dùng*

• Ngoại lệ và tình huống biên
Mất phiên hoặc lỗi API; không lấy userId tùy ý từ client.
Luồng thay thế: Không có bài hoặc lọc trạng thái khác.
Cần kiểm chứng riêng: Thử hai tài khoản, filter trạng thái, trang rỗng và token hết hạn; bài nháp không lẫn public listing.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-4-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-4-3-02: Thử hai tài khoản, filter trạng thái, trang rỗng và token hết hạn; bài nháp không lẫn public listing.
TC-5-4-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-4-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

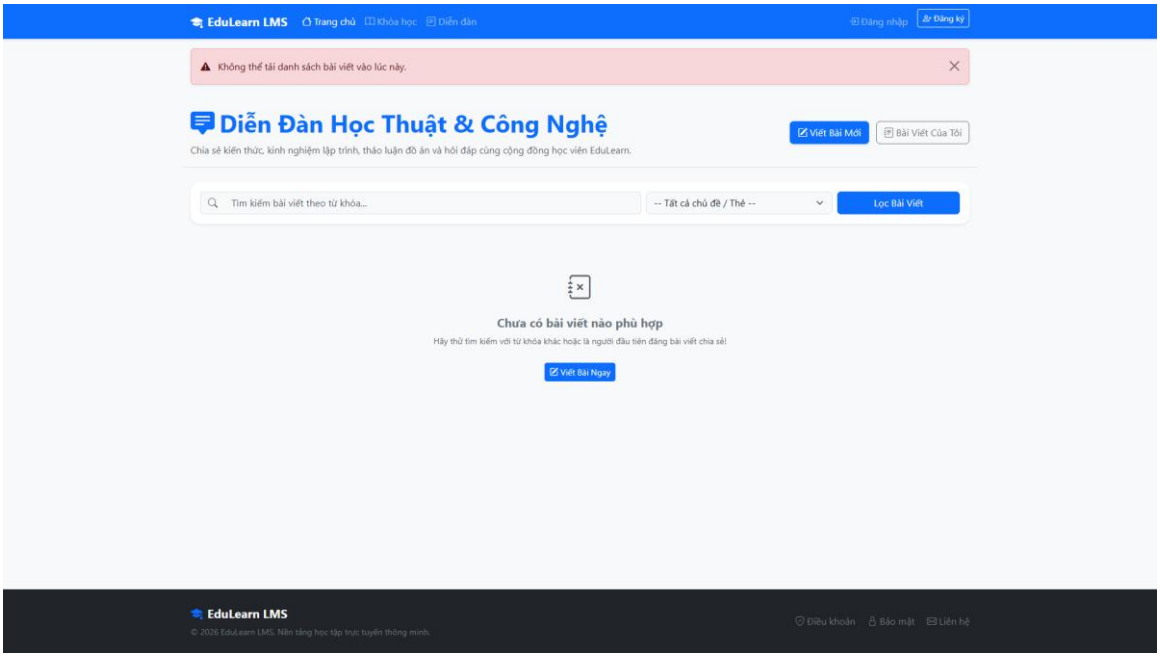
## 5.4.4 Bình luận và trả lời bình luận

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Comment/reply cần post PUBLISHED; parent comment thuộc cùng post. Không áp quy tắc bình luận sản phẩm ≥50 ký tự vào blog nếu DTO không yêu cầu.
Tác nhân: Người đã đăng nhập. Tiền điều kiện: Post published; parent comment nếu có phải thuộc post và không phải reply lồng thêm.
Đầu ra: Comment được lưu hoặc cập nhật/xóa theo quyền.
• Luồng xử lý và sơ đồ hoạt động
1. Nhập bình luận/reply. 2. Gửi API. 3. Backend kiểm tra post và parent. 4. Lưu comment. 5. Khi sửa/xóa, kiểm tra owner theo service.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET/POST /api/blog/posts/{postId}/comments; PUT/DELETE .../{commentId}.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/CommentServiceImpl.java cùng các controller liên quan ở 3.3.
Comment/reply cần post PUBLISHED; parent comment thuộc cùng post. Không áp quy tắc bình luận sản phẩm ≥50 ký tự vào blog nếu DTO không yêu cầu.
• Xử lý nghiệp vụ và truy cập dữ liệu
createComment ngăn CANNOT_REPLY_TO_REPLY nên chỉ một tầng reply. update/delete xác minh comment thuộc post và quyền owner/Admin theo helper.
Đối chiếu dữ liệu theo UC041.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/CommentServiceImpl.java, bắt đầu dòng 57. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public CommentResponse createComment(Long postId,
                                     CreateCommentRequest request) {
    String userId = GatewayAuthentication.currentUserId();
    Post post = postRepository.findById(postId)
            .orElseThrow(() -> new BlogException(ErrorCode.POST_NOT_FOUND));
    if (post.getStatus() != PostStatus.PUBLISHED)
        throw new BlogException(ErrorCode.POST_NOT_PUBLISHED);
    Comment comment = Comment.builder()
            .post(post)
            .userId(userId)
            .content(request.getContent())
```

Giải thích: createComment ngăn CANNOT_REPLY_TO_REPLY nên chỉ một tầng reply. update/delete xác minh comment thuộc post và quyền owner/Admin theo helper.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: BlogDetailPage. Demo theo đúng role và luồng API được mô tả. Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.4.4: Khu vực bình luận và thảo luận đa cấp dưới bài viết Blog*

• Ngoại lệ và tình huống biên
POST_NOT_PUBLISHED, COMMENT_NOT_BELONG_TO_POST, CANNOT_REPLY_TO_REPLY, NOT_COMMENT_OWNER.
Luồng thay thế: Reply vào comment gốc; không cho reply của reply theo source.
Cần kiểm chứng riêng: Thử reply comment course/post khác, reply lồng thêm, sửa của người khác, post chưa công bố và dữ liệu rỗng.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-4-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-4-4-02: Thử reply comment course/post khác, reply lồng thêm, sửa của người khác, post chưa công bố và dữ liệu rỗng.
TC-5-4-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-4-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

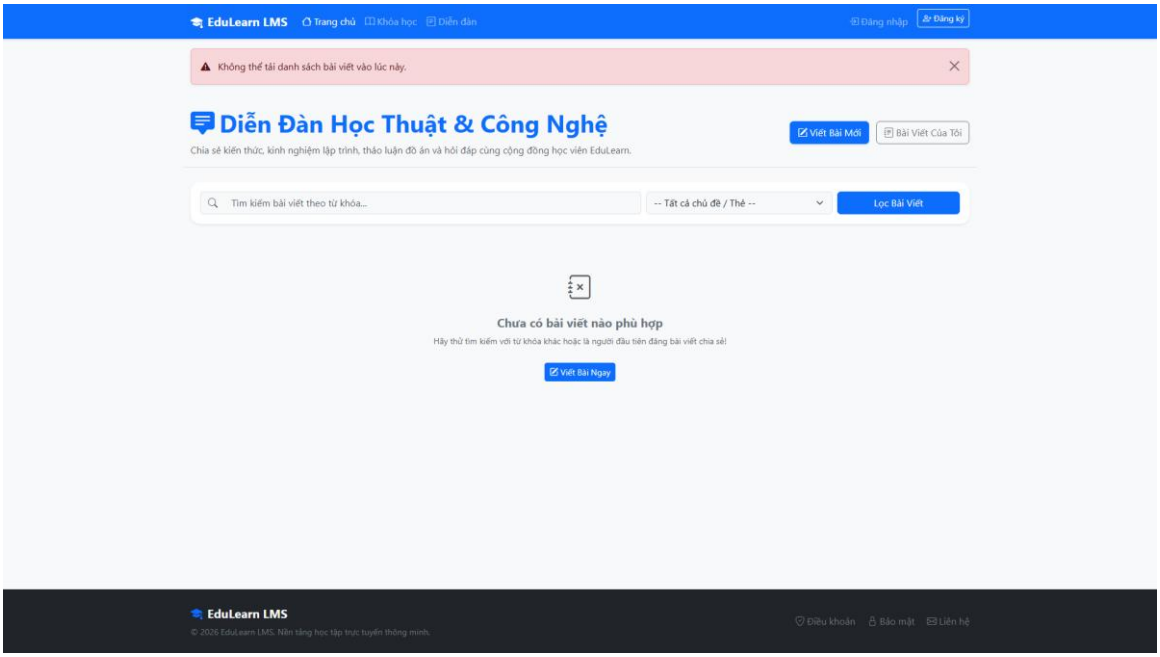
## 5.4.5 Bình chọn bài viết và bình luận

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: VoteRequest xác định loại bình chọn; service phân biệt target post/comment.
Tác nhân: Người đã đăng nhập. Tiền điều kiện: Bài published; comment thuộc post; không bình chọn nội dung của mình.
Đầu ra: Vote được xử lý theo dữ liệu người dùng và nội dung.
• Luồng xử lý và sơ đồ hoạt động

1. Chọn giá trị vote. 2. Gửi post/comment vote. 3. Backend kiểm tra tác giả và quan hệ. 4. Xử lý bản ghi vote theo implementation. 5. Trả kết quả.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/blog/posts/{postId}/vote; POST /api/blog/posts/{postId}/comments/{commentId}/vote.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/VoteServiceImpl.java cùng các controller liên quan ở 3.3.
VoteRequest xác định loại bình chọn; service phân biệt target post/comment.
• Xử lý nghiệp vụ và truy cập dữ liệu
votePost chặn tác giả bình chọn bài mình; voteComment kiểm tra post published, quan hệ comment–post và không vote comment mình. Unique DB theo user,target,type hạn chế bản ghi trùng.
Đối chiếu dữ liệu theo UC042.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/VoteServiceImpl.java, bắt đầu dòng 37. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public VoteResponse votePost(Long postId, VoteRequest request) {
    String userId = GatewayAuthentication.currentUserId();
    Post post = postRepository.findById(postId)
            .orElseThrow(() -> new BlogException(ErrorCode.POST_NOT_FOUND));
    if(post.getAuthorId().equals(userId))
        throw new BlogException(ErrorCode.CANNOT_VOTE_OWN_POST);
    if(!post.getStatus().equals(PostStatus.PUBLISHED))
        throw new BlogException(ErrorCode.POST_NOT_PUBLISHED);
    VoteType result = processVote(userId, postId, TargetType.POST,
request.getVoteType());
    return buildVoteResponse(postId, TargetType.POST, result);
```

Giải thích: votePost chặn tác giả bình chọn bài mình; voteComment kiểm tra post published, quan hệ comment–post và không vote comment mình. Unique DB theo user,target,type hạn chế bản ghi trùng.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: BlogDetailPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.4.5: Tính năng biểu quyết (Vote Up/Down) đánh giá chất lượng bài viết*

• Ngoại lệ và tình huống biên
CANNOT_VOTE_OWN_POST/COMMENT, POST_NOT_PUBLISHED, COMMENT_NOT_BELONG_TO_POST.
Luồng thay thế: Đổi hoặc bỏ vote theo cơ chế service; không tạo trùng tùy ý.
Cần kiểm chứng riêng: Thử đổi vote/bỏ vote theo service, bấm lặp, tự vote và comment không thuộc post; so tổng hiển thị với bản ghi DB.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-4-5-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-4-5-02: Thử đổi vote/bỏ vote theo service, bấm lặp, tự vote và comment không thuộc post; so tổng hiển thị với bản ghi DB.
TC-5-4-5-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-4-5-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

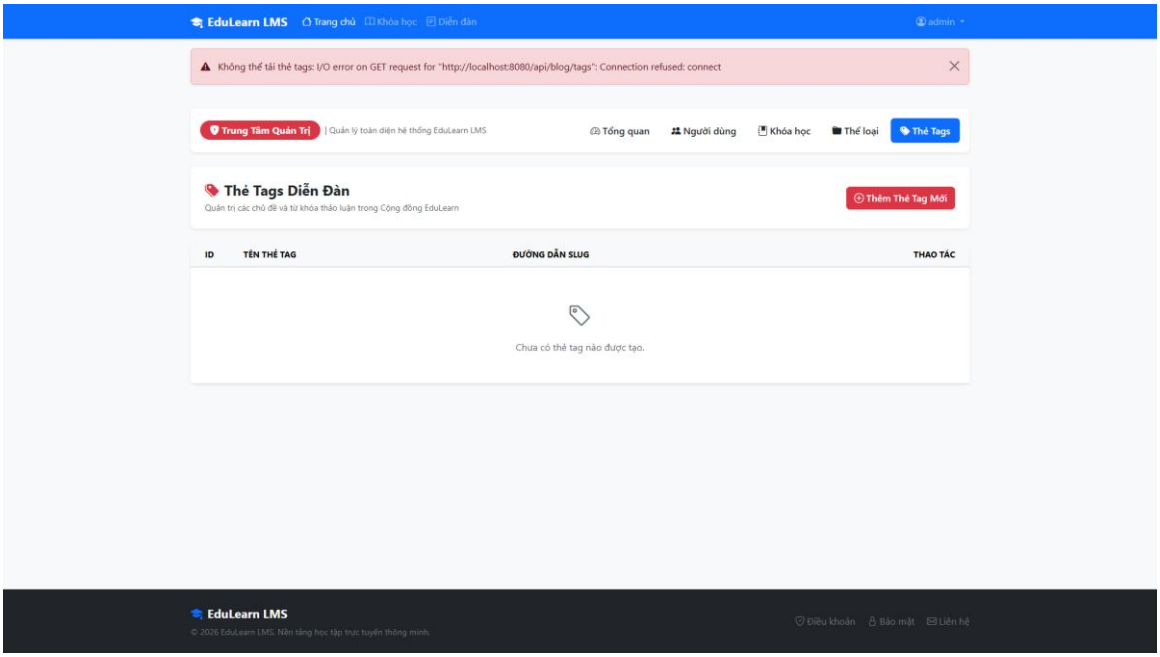
## 5.4.6 Quản lý tag bởi Admin và upload tệp theo quyền

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Tag CRUD thuộc Admin; GET tag hỗ trợ công khai theo cấu hình. Upload của tác giả đi trong form Post, không trao quyền CRUD Tag.
Tác nhân: ADMIN quản lý tag; người viết upload theo quyền. Tiền điều kiện: Tag/tệp hợp lệ; có quyền API tương ứng.
Đầu ra: Tag và post_tags được lưu; tệp theo luồng MinIO.
• Luồng xử lý và sơ đồ hoạt động

1. Admin tạo/sửa/xóa tag. 2. Người viết chọn tag và tệp trong luồng bài viết. 3. Backend kiểm tra dữ liệu và upload. 4. Lưu tag/liên kết hoặc tài nguyên.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET/POST /api/blog/tags; PUT/DELETE /api/blog/tags/{id}; upload trong API Post.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/TagServiceImpl.java cùng các controller liên quan ở 3.3.
Tag CRUD thuộc Admin; GET tag hỗ trợ công khai theo cấu hình. Upload của tác giả đi trong form Post, không trao quyền CRUD Tag.
• Xử lý nghiệp vụ và truy cập dữ liệu
Tag liên kết Post qua post_tags. MinioServiceImpl sinh object folder/UUID/extension và public URL, xóa theo prefix của bucket.
Đối chiếu dữ liệu theo UC043.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/TagServiceImpl.java, bắt đầu dòng 51. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public TagResponse createTag(CreateTagRequest request) {
    if (tagRepository.existsByName(request.getName()))
        throw new BlogException(ErrorCode.TAG_NAME_EXISTED);
    if (tagRepository.existsBySlug(request.getSlug()))
        throw new BlogException(ErrorCode.TAG_SLUG_EXISTED);
    return tagMapper.toTagResponse(
            tagRepository.save(tagMapper.toTag(request)));
}
@Override
public TagResponse updateTag(Long id, UpdateTagRequest request) {
```

Giải thích: Tag liên kết Post qua post_tags. MinioServiceImpl sinh object folder/UUID/extension và public URL, xóa theo prefix của bucket.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: BlogCategoriesTab và form bài viết. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.4.6: Quản trị danh mục bộ thẻ phân loại (Tags) hệ thống*

• Ngoại lệ và tình huống biên
Không có quyền, tag không tồn tại, tệp sai hoặc lỗi storage.
Luồng thay thế: Đọc tag công khai; upload không tự cấp quyền CRUD tag.
Cần kiểm chứng riêng: Thử STUDENT POST tags, Admin tag trùng, tag đã liên kết và ảnh không hợp lệ; xác minh upload object thật.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-4-6-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-4-6-02: Thử STUDENT POST tags, Admin tag trùng, tag đã liên kết và ảnh không hợp lệ; xác minh upload object thật.
TC-5-4-6-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-4-6-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

# 5.5 Chatbot Service

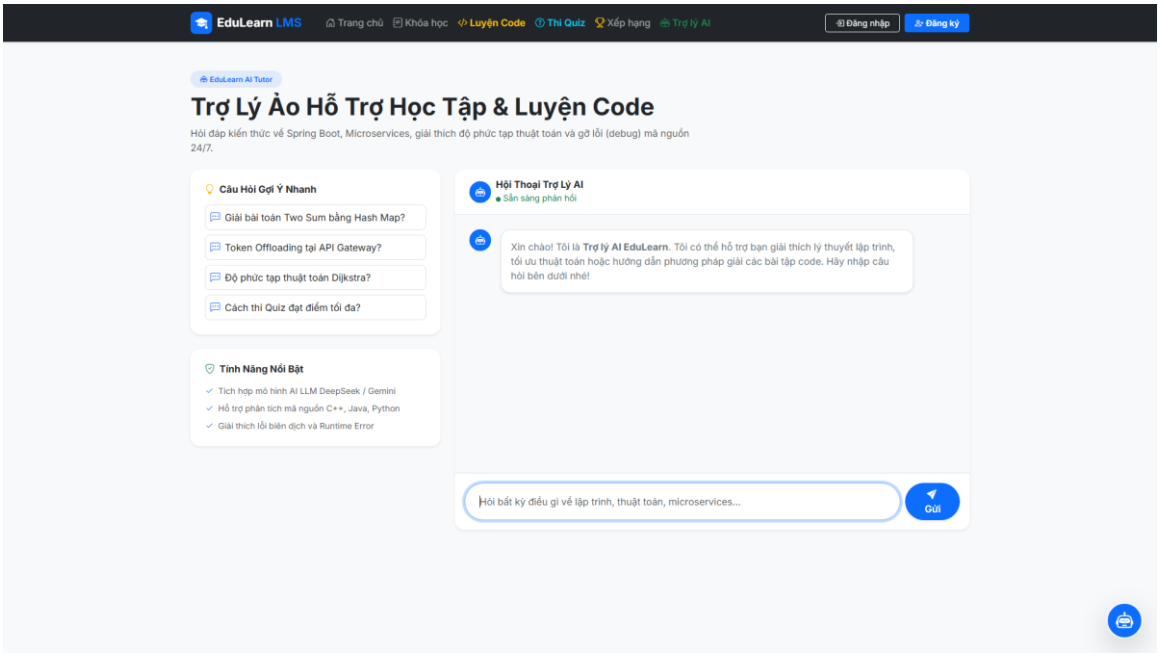
## 5.5.1 Hỏi đáp với chatbot học tập qua SSE

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: ChatRequest có message, contextType/contextRefId/sessionId theo DTO. ChatController yêu cầu đăng nhập; stream dùng text/event-stream.
Tác nhân: Người đã đăng nhập. Tiền điều kiện: Cấu hình LLM hợp lệ; request phù hợp context và hạn mức.
Đầu ra: Nhận phản hồi từng phần; lưu lịch sử theo luồng đã triển khai.
• Luồng xử lý và sơ đồ hoạt động

1. Nhập câu hỏi. 2. Gửi chat. 3. Backend kiểm tra request/hạn mức, xây context và lịch sử. 4. Gọi provider. 5. Trả nội dung theo SSE và cập nhật lịch sử theo service.
Thể hiện SSE: SESSION trước text chunk; saveExchange khi stream hoàn tất và có nội dung.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/chatbot/chat; Accept: text/event-stream.
Tham số và DTO lấy từ controller; căn cứ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/ChatServiceImpl.java cùng các controller liên quan ở 3.3.
ChatRequest có message, contextType/contextRefId/sessionId theo DTO. ChatController yêu cầu đăng nhập; stream dùng text/event-stream.
• Xử lý nghiệp vụ và truy cập dữ liệu
ChatService resolve session, lấy lịch sử Redis/Mongo, build DomainContext rồi prompt; gửi [SESSION:id] trước text. Khi stream hoàn tất và có nội dung thì lưu exchange. Lỗi giữa stream chưa chắc đã lưu câu trả lời dở; không kết luận lịch sử chứa mọi chunk.
Đối chiếu dữ liệu theo UC044.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/ChatServiceImpl.java, bắt đầu dòng 29. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Flux<String> chat(ChatRequest request) {
    String userId = GatewayAuthentication.currentUserId();
    // Bước 1: Resolve session
    ChatSession session = historyManagerService.resolveSession(
            request.getContextType(),
            request.getContextRefId(),
            request.getSessionId(),
            request.getMessage());
    String sessionId = session.getId();
    // Bước 2: Lấy history từ Redis/MongoDB
    List<ChatMessage> history = historyManagerService
```

Giải thích: ChatService resolve session, lấy lịch sử Redis/Mongo, build DomainContext rồi prompt; gửi [SESSION:id] trước text. Khi stream hoàn tất và có nội dung thì lưu exchange. Lỗi giữa stream chưa chắc đã lưu câu trả lời dở; không kết luận lịch sử chứa mọi chunk.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ChatBot.jsx. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.5.1: Giao diện trợ lý thông minh AI Chatbot hỗ trợ học tập 24/7*

• Ngoại lệ và tình huống biên
Rate limit, context/provider không khả dụng, lỗi kết nối/stream; không cam kết AI luôn đúng.
Luồng thay thế: Người dùng bắt đầu câu hỏi khác hoặc phiên mới; phản hồi cần được người dùng đánh giá.
Cần kiểm chứng riêng: Thử GENERAL/COURSE/PROBLEM, context không tồn tại, rate limit, Redis lỗi, LLM lỗi giữa stream; kiểm tra câu hỏi không lộ dữ liệu khóa khác.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-5-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-5-1-02: Thử GENERAL/COURSE/PROBLEM, context không tồn tại, rate limit, Redis lỗi, LLM lỗi giữa stream; kiểm tra câu hỏi không lộ dữ liệu khóa khác.
TC-5-5-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-5-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

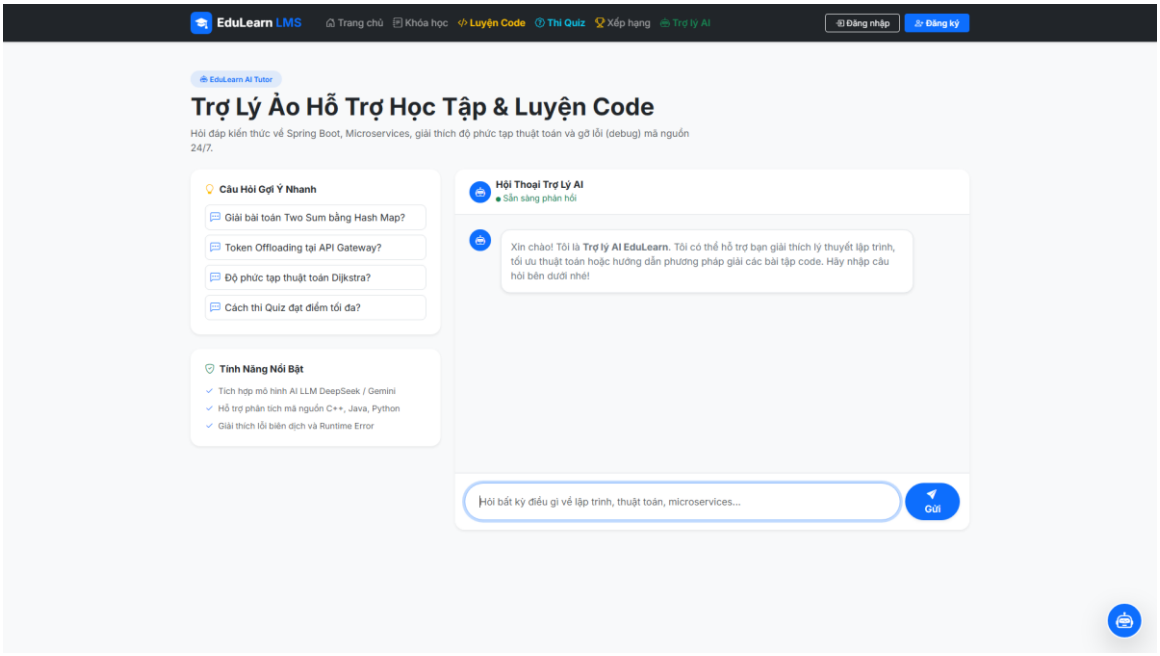
## 5.5.2 Xem lịch sử và lưu trữ phiên chat

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Session list gắn userId hiện tại; archiveSessionByUser kiểm tra session owner. Endpoint đọc messages hiện không thấy kiểm tra owner ở tuyến đã đọc.
Tác nhân: Người đã đăng nhập. Tiền điều kiện: Có phiên chat và quyền theo API hiện tại.
Đầu ra: Phiên được lưu trữ khi chủ phiên thực hiện thành công.
• Luồng xử lý và sơ đồ hoạt động
1. Tải danh sách phiên theo userId. 2. Chọn sessionId và đọc tin nhắn. 3. Chọn New Chat/archive. 4. Backend archiveSessionByUser kiểm tra chủ phiên.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/chatbot/sessions; GET .../sessions/{sessionId}/messages; POST .../sessions/{sessionId}/archive.
Tham số và DTO lấy từ controller; căn cứ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/ChatSessionServiceImpl.java cùng các controller liên quan ở 3.3.
Session list gắn userId hiện tại; archiveSessionByUser kiểm tra session owner. Endpoint đọc messages hiện không thấy kiểm tra owner ở tuyến đã đọc.
• Xử lý nghiệp vụ và truy cập dữ liệu
getMessageResponses lấy recent messages DESC rồi reverse về thời gian tăng cho UI. Đây là lịch sử giới hạn theo cấu hình, không mặc định trả toàn bộ tin của phiên.
Đối chiếu dữ liệu theo UC045.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/ChatSessionServiceImpl.java, bắt đầu dòng 99. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public void archiveSessionByUser(String sessionId, String userId) {
    ChatSession session = getSessionById(sessionId);
    if (!session.getUserId().equals(userId)) {
        throw new ChatbotException(ErrorCode.SESSION_NOT_OWNER);
    }
    session.setStatus(SessionStatus.ARCHIVED);
    sessionRepository.save(session);
    log.debug("User {} archived session {}", userId, sessionId);
}
@Override
@Scheduled(cron = "${chatbot.session.archive-cron:0 0 0 * * *}")
public void archiveStaleSessions() {
    LocalDateTime cutoff = LocalDateTime.now().minusDays(archiveAfterDays);
```

Giải thích: getMessageResponses lấy recent messages DESC rồi reverse về thời gian tăng cho UI. Đây là lịch sử giới hạn theo cấu hình, không mặc định trả toàn bộ tin của phiên.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: ChatBot.jsx. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.5.2: Lưu trữ và đồng bộ hóa lịch sử các phiên trao đổi với AI*

• Ngoại lệ và tình huống biên
SESSION_NOT_OWNER khi archive. getMessages/getMessageResponses chưa thấy kiểm tra owner trong tuyến đã đọc; không tuyên bố đã cô lập lịch sử hoàn toàn, cần kiểm thử truy cập chéo.
Luồng thay thế: Lần chat sau tạo phiên mới theo luồng hệ thống.
Cần kiểm chứng riêng: Thử hai user đọc/archive session nhau, phiên không tồn tại, phân trang và nhiều tin; đặc biệt ghi kết quả truy cập chéo, không mặc định an toàn.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-5-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-5-2-02: Thử hai user đọc/archive session nhau, phiên không tồn tại, phân trang và nhiều tin; đặc biệt ghi kết quả truy cập chéo, không mặc định an toàn.
TC-5-5-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-5-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.5.3 Sinh quiz bằng AI

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: ChatController sinh quiz trực tiếp chỉ INSTRUCTOR; AI preview tại Assignment có quyền riêng. Hai route không được gom một quyền duy nhất.
Tác nhân: INSTRUCTOR qua ChatController; Admin/Instructor qua luồng assignment AI preview tương ứng. Tiền điều kiện: Request nội dung và cấu hình LLM hợp lệ; chưa mặc định công bố quiz.
Đầu ra: Trả nội dung gợi ý; quiz chỉ được lưu khi thực hiện luồng lưu.

• Luồng xử lý và sơ đồ hoạt động
1. Nhập yêu cầu sinh quiz. 2. Backend kiểm tra role/hạn mức. 3. Xây prompt và gọi LLM. 4. Parse/kiểm tra kết quả. 5. Giảng viên xem và sửa. 6. Lưu bằng luồng tạo quiz riêng.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/chatbot/generate/quiz; AI preview quiz thuộc assignment/questions.
Tham số và DTO lấy từ controller; căn cứ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/GenerateServiceImpl.java cùng các controller liên quan ở 3.3.
ChatController sinh quiz trực tiếp chỉ INSTRUCTOR; AI preview tại Assignment có quyền riêng. Hai route không được gom một quyền duy nhất.
• Xử lý nghiệp vụ và truy cập dữ liệu
GenerateService dựng prompt/gọi LLM và kiểm tra cấu trúc response. Preview chưa tự tạo quiz; người phụ trách cần kiểm tra correct answers, nội dung và điểm trước API imported/sync.
Đối chiếu dữ liệu theo UC046.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/GenerateServiceImpl.java, bắt đầu dòng 43. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public GenerateQuizResponse generateQuiz(GenerateQuizRequest request) {
    // Bước 1: Build prompt cho quiz
    List<GeminiMessage> messages = promptEngineService.buildGenerateQuizPrompt(
            request.getContent(),
            request.getQuestionCount(),
            request.getDifficulty(),
            request.getQuestionType());
    // Bước 2: Gọi Gemini lấy JSON, retry nếu parse thất bại
    String jsonResponse = callWithRetry(messages, ErrorCode.GENERATE_QUIZ_FAILED);
    // Bước 3: Parse JSON thành response
    return parseQuizResponse(jsonResponse, request.getQuestionCount());
```

Giải thích: GenerateService dựng prompt/gọi LLM và kiểm tra cấu trúc response. Preview chưa tự tạo quiz; người phụ trách cần kiểm tra correct answers, nội dung và điểm trước API imported/sync.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CreateQuizPage và luồng AI preview. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên
GENERATE_QUIZ_FAILED, lỗi provider/hạn mức hoặc định dạng response.
Luồng thay thế: Sửa tay hoặc sinh lại nếu nội dung không phù hợp.
Cần kiểm chứng riêng: Thử response sai JSON, thiếu câu/đáp án, provider timeout, role STUDENT/ADMIN ở direct route; chỉ lưu nội dung đã rà soát.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-5-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-5-3-02: Thử response sai JSON, thiếu câu/đáp án, provider timeout, role STUDENT/ADMIN ở direct route; chỉ lưu nội dung đã rà soát.
TC-5-5-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-5-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.5.4 Sinh test case bằng AI

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Test case AI có đầu vào problemId hoặc mô tả thủ công; service kiểm tra title/description khi manual. Quyền direct INSTRUCTOR khác assignment preview.
Tác nhân: INSTRUCTOR qua ChatController; Admin/Instructor qua test case AI preview.
Tiền điều kiện: Problem hoặc mô tả đầu vào hợp lệ; LLM khả dụng.
Đầu ra: Có test case gợi ý; chưa tự trở thành test chính thức.
• Luồng xử lý và sơ đồ hoạt động
1. Chọn bài/nhập mô tả. 2. Yêu cầu sinh test. 3. Backend xây ngữ cảnh và gọi LLM. 4. Kiểm tra response. 5. Người phụ trách rà soát input/output. 6. Lưu qua API test case.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: POST /api/chatbot/generate/testcase; POST /api/assignment/problems/testcases/ai-generate/preview.
Tham số và DTO lấy từ controller; căn cứ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/GenerateServiceImpl.java cùng các controller liên quan ở 3.3.
Test case AI có đầu vào problemId hoặc mô tả thủ công; service kiểm tra title/description khi manual. Quyền direct INSTRUCTOR khác assignment preview.
• Xử lý nghiệp vụ và truy cập dữ liệu
Service lấy ngữ cảnh bài qua client, gọi LLM và parse test input/output. Kết quả cần thử với lời giải tin cậy trước lưu; không lấy output AI làm chuẩn chỉ vì parse thành công.
Đối chiếu dữ liệu theo UC047.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/chatbot-service/src/main/java/com/lms/chatbotservice/service/impl/GenerateServiceImpl.java, bắt đầu dòng 62. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public GenerateTestCaseResponse generateTestCase(GenerateTestCaseRequest request) {
    // Resolve thông tin đề bài từ 2 hướng
    TestCaseProblemInfo info = resolveProblemInfo(request);
    List<GeminiMessage> messages = promptEngineService.buildGenerateTestCasePrompt(
            info.title(),
            info.description(),
            info.constraints(),
```

```
            request.getCount(),
            request.getAllowedLangs(),
            request.getExistingTestCases());
    String jsonResponse = callWithRetry(
```

Giải thích: Service lấy ngữ cảnh bài qua client, gọi LLM và parse test input/output. Kết quả cần thử với lời giải tin cậy trước lưu; không lấy output AI làm chuẩn chỉ vì parse thành công.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: CreateCodingPage và luồng AI preview. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên

PROBLEM_NOT_FOUND, TESTCASE_TITLE_BLANK, TESTCASE_DESCRIPTION_BLANK, GENERATE_TESTCASE_FAILED hoặc lỗi dịch vụ.

Luồng thay thế: Sửa đầu ra hoặc dùng test nhập tay.

Cần kiểm chứng riêng: Thử bài không tồn tại, mô tả thiếu, input biên, output sai và test hidden; xác minh human review trước đồng bộ test.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-5-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-5-4-02: Thử bài không tồn tại, mô tả thiếu, input biên, output sai và test hidden; xác minh human review trước đồng bộ test.

TC-5-5-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-5-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

# 5.6 Giao diện và chức năng Admin

## 5.6.1 Quản lý người dùng và vai trò

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: getAllUser nhận Pageable, role và keyword; các API Admin riêng quản lý user/Admin. Chỉ liệt kê các thay đổi được controller cung cấp.

Tác nhân: ADMIN. Tiền điều kiện: Có phiên Admin; dữ liệu user và quyền quản trị hợp lệ.

Đầu ra: User thay đổi theo thao tác đã được API hỗ trợ.

• Luồng xử lý và sơ đồ hoạt động

1. Tìm/lọc người dùng. 2. Xem thông tin. 3. Cập nhật hoặc đổi active theo API. 4. Backend kiểm tra request và lưu. 5. Tải lại danh sách.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.

[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]

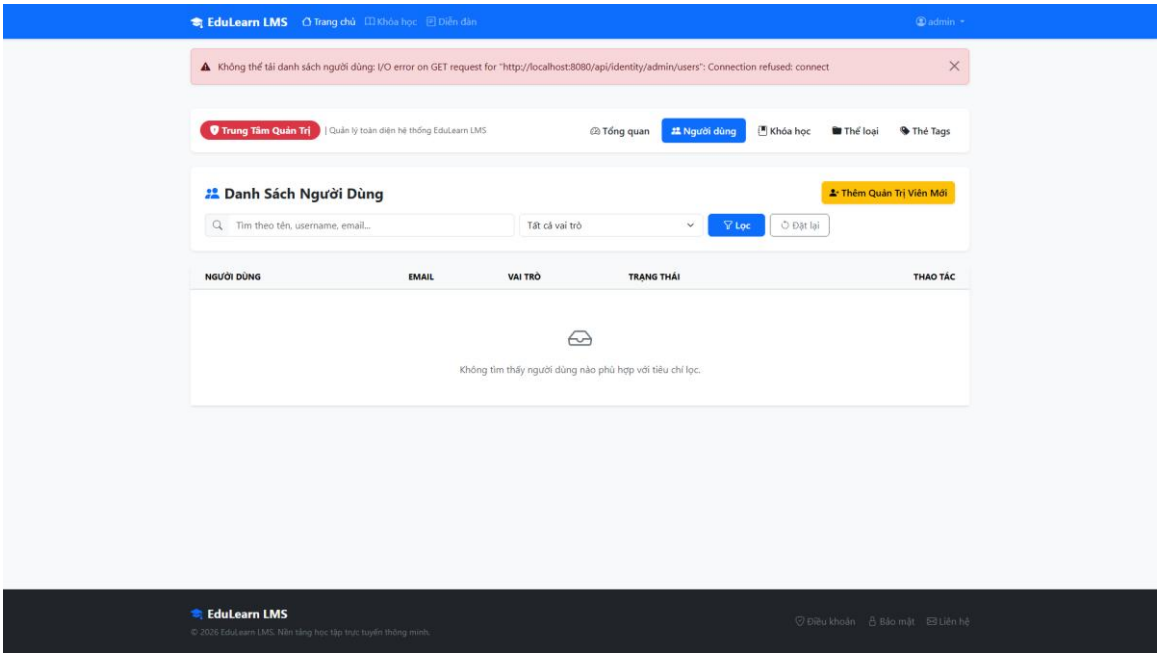
• API DTO validation và phân quyền

API chính: Theo /api/identity/admin/users: stats, detail, active, admins, cập nhật và password.

Tham số và DTO lấy từ controller; căn cứ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AdminUserServiceImpl.java cùng các controller liên quan ở 3.3.
getAllUser nhận Pageable, role và keyword; các API Admin riêng quản lý user/Admin. Chỉ liệt kê các thay đổi được controller cung cấp.
• Xử lý nghiệp vụ và truy cập dữ liệu
toggleActive, createAdmin, updateUser/resetPassword/toggleAdminActive là luồng riêng. Không đồng nhất self updateRole với Admin phân quyền. API self role chỉ RegisterableRole nhưng rule đang gọi chưa có giới hạn30phút.
Đối chiếu dữ liệu theo UC048.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/identity-service/src/main/java/com/lms/identityservice/service/impl/AdminUserServiceImpl.java, bắt đầu dòng 51. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Page<UserSummaryResponse> getAllUser(Pageable pageable, String role, String
keyword) {
    String normalizedRole = normalizeRole(role);
    RoleType roleType = normalizedRole != null ? RoleType.valueOf(normalizedRole) :
null;
    String searchKeyword = normalizeKeyword(keyword);
    return userRepository.findUsersByFilters(roleType, searchKeyword, pageable)
            .map(userMapper::toUserSummaryResponse);
}
@Override
public UserFullResponse getUserById(String userId) {
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new IdentityException(ErrorCode.USER_NOT_EXISTED));
    return userMapper.toUserFullResponse(user);
}
```

Giải thích: toggleActive, createAdmin, updateUser/resetPassword/toggleAdminActive là luồng riêng. Không đồng nhất self updateRole với Admin phân quyền. API self role chỉ RegisterableRole nhưng rule đang gọi chưa có giới hạn30phút.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: UserManagement và AdminManagement. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.6.1: Giao diện Admin quản trị người dùng (thể hiện cơ chế cảnh báo và xử lý ngoại lệ an toàn của giao diện khi Identity Service tạm thời ngoại tuyến)*

• Ngoại lệ và tình huống biên
User không tồn tại, dữ liệu trùng/sai hoặc không đủ quyền; không tự thêm cấp quyền tùy ý ngoài API.
Luồng thay thế: Quản lý tài khoản Admin có luồng tạo/sửa/đổi mật khẩu/trạng thái riêng.
Cần kiểm chứng riêng: Thử role filter, user inactive, tài khoản Admin, thao tác thiếu quyền và khóa user còn token; lưu phản hồi thật.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-6-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-6-1-02: Thử role filter, user inactive, tài khoản Admin, thao tác thiếu quyền và khóa user còn token; lưu phản hồi thật.
TC-5-6-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-6-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

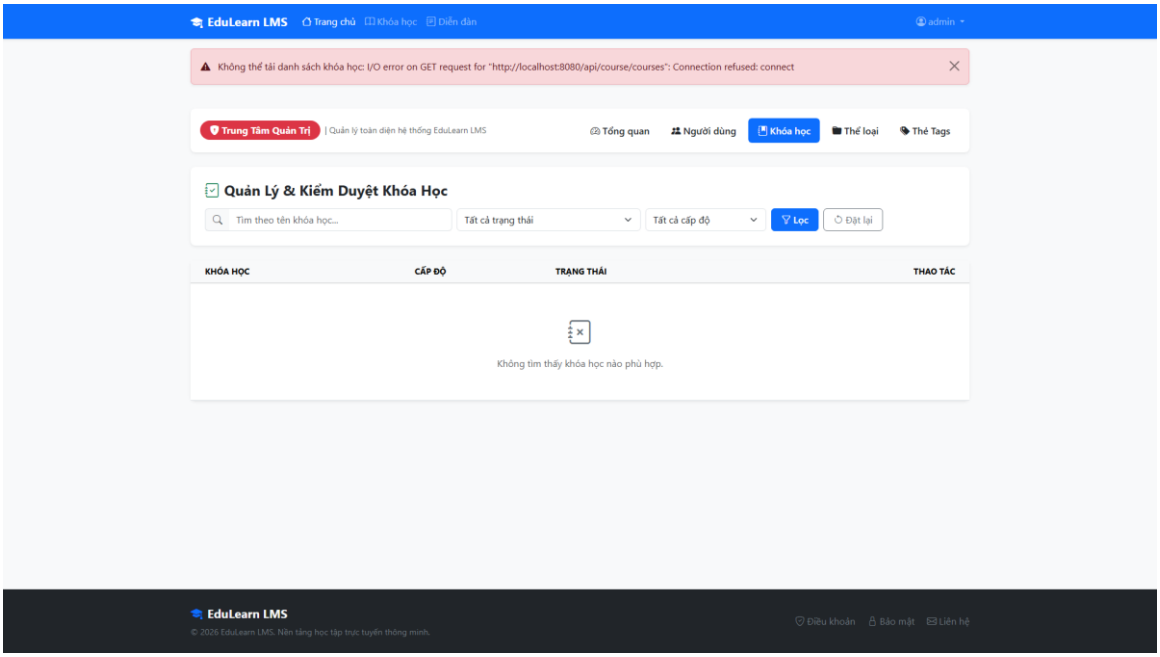
## 5.6.2 Quản lý khóa học toàn hệ thống

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Admin frontend gọi lock/unlock thay vì cho Instructor thực hiện. Course status PUBLIC/PRIVATE/LOCKED lấy từ source.
Tác nhân: ADMIN. Tiền điều kiện: Course tồn tại; quyền quản trị hợp lệ.
Tác nhân: ADMIN. Tiền điều kiện: Course tồn tại; phiên có role ADMIN.
Đầu ra: Dữ liệu/trạng thái course được cập nhật khi thành công. Trạng thái quản trị Course thay đổi.

• Luồng xử lý và sơ đồ hoạt động
1. Tìm và xem khóa học. 2. Chọn thao tác quản trị được giao diện hỗ trợ. 3. Gọi API course, gồm lock/unlock. 4. Kiểm tra quyền và cập nhật. 5. Tải lại.
1. Admin tìm khóa học. 2. Chọn khóa/mở khóa. 3. Gửi endpoint lock/unlock. 4. Backend kiểm tra và cập nhật trạng thái. 5. Tải lại thông tin.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: PATCH /api/course/courses/{id}/lock; PATCH .../{id}/unlock.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java cùng các controller liên quan ở 3.3.
Admin frontend gọi lock/unlock thay vì cho Instructor thực hiện. Course status PUBLIC/PRIVATE/LOCKED lấy từ source.
• Xử lý nghiệp vụ và truy cập dữ liệu
Service kiểm tra role và cập nhật trạng thái; cần xác minh khi mở khóa trả trạng thái nào theo method. Không khẳng định khóa course xóa enrollment hay dữ liệu đã học.
Đối chiếu dữ liệu theo UC049, UC028.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CourseServiceImpl.java, bắt đầu dòng 276. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public CourseResponse lockCourse(String courseId, LockCourseRequest request) {
    Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new CourseException(ErrorCode.COURSE_NOT_FOUND));
    if (course.getStatus() == CourseStatus.LOCKED)
        throw new CourseException(ErrorCode.COURSE_ALREADY_LOCKED);
    course.setStatus(CourseStatus.LOCKED);
    courseRepository.save(course);
    announcementRepository.save(Announcement.builder()
            .course(course)
            .title("[Hệ thống] Khóa học đã bị khóa")
            .content("Khóa học này đã bị Admin khóa.\n\nLý do: " +
request.getReason())
```

Giải thích: Service kiểm tra role và cập nhật trạng thái; cần xác minh khi mở khóa trả trạng thái nào theo method. Không khẳng định khóa course xóa enrollment hay dữ liệu đã học.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: Admin CourseManagement. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.6.2: Giao diện Admin kiểm duyệt và quản lý toàn bộ khóa học hệ thống*

• Ngoại lệ và tình huống biên
Course không tồn tại, lỗi API, dữ liệu sai.
Không phải Admin hoặc course không tồn tại; quy tắc chuyển trạng thái theo service.
Luồng thay thế: Đọc dữ liệu hoặc hủy thao tác. Hủy trước gửi không thay đổi trạng thái.
Cần kiểm chứng riêng: Thử khóa course có học viên rồi thao tác học/biên soạn; mở khóa và kiểm tra dữ liệu còn giữ.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-6-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-6-2-02: Thử khóa course có học viên rồi thao tác học/biên soạn; mở khóa và kiểm tra dữ liệu còn giữ.
TC-5-6-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-6-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

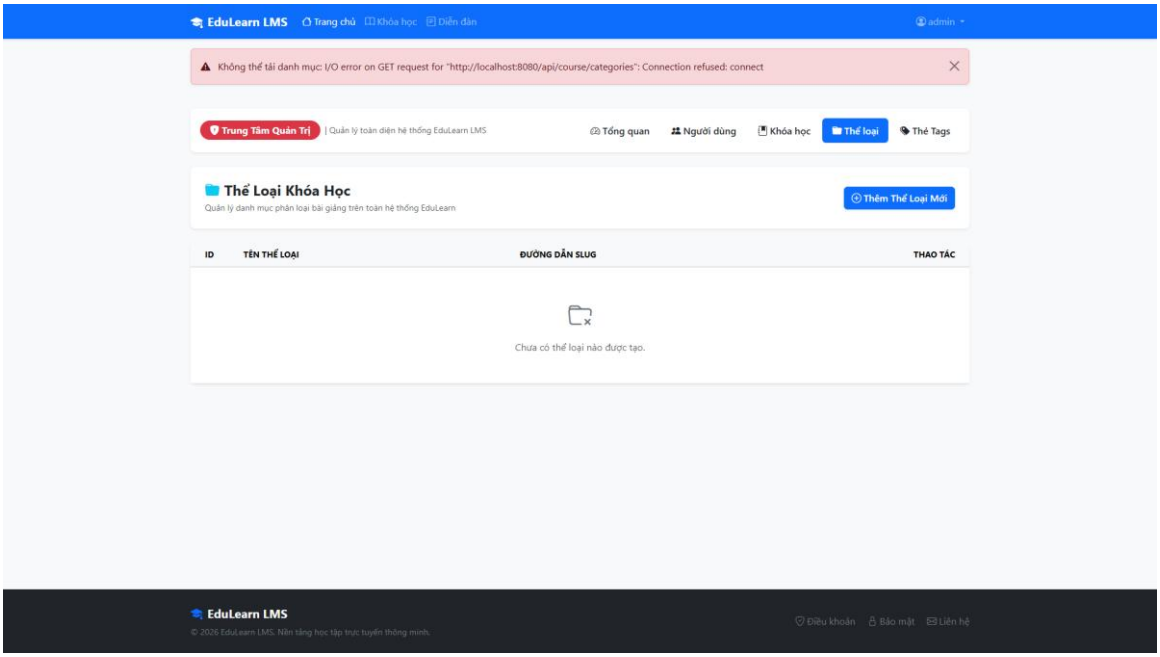
## 5.6.3 Quản lý danh mục khóa học

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Category gồm name/slug; migration có unique name/slug. Controller CRUD ADMIN; GET public.
Tác nhân: ADMIN; Guest/người dùng đọc danh mục theo API public. Tiền điều kiện: Dữ liệu category hợp lệ; sửa/xóa category phải tồn tại.
Đầu ra: Category thay đổi theo API.
• Luồng xử lý và sơ đồ hoạt động

1. Mở Course Categories. 2. Tạo/sửa/xóa. 3. Gửi API categories. 4. Backend validation và lưu. 5. Cập nhật giao diện.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET/POST /api/course/categories; PUT/DELETE /api/course/categories/{id}.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CategoryServiceImpl.java cùng các controller liên quan ở 3.3.
Category gồm name/slug; migration có unique name/slug. Controller CRUD ADMIN; GET public.
• Xử lý nghiệp vụ và truy cập dữ liệu
create/update/delete dùng CategoryService và quan hệ course_categories. Xóa có thể ảnh hưởng bảng liên kết theo FK CASCADE; không xóa toàn bộ course chỉ vì mất category.
Đối chiếu dữ liệu theo UC050.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/CategoryServiceImpl.java, bắt đầu dòng 38. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public CategoryResponse createCategory(CreateCategoryRequest request) {
    //Kiem tra trung ten
    if(categoryRepository.existsByName(request.getName()))
        throw new CourseException(ErrorCode.CATEGORY_NAME_EXISTED);
    if(request.getSlug() != null
            && categoryRepository.existsBySlug(request.getSlug()))
        throw new CourseException(ErrorCode.SLUG_ALREADY_EXISTS);
    Category category = categoryMapper.toCategory(request);
    return categoryMapper.toCategoryResponse(
            categoryRepository.save(category));
}
```

Giải thích: create/update/delete dùng CategoryService và quan hệ course_categories. Xóa có thể ảnh hưởng bảng liên kết theo FK CASCADE; không xóa toàn bộ course chỉ vì mất category.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CourseCategoriesTab. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.6.3: Giao diện Admin quản trị danh mục thể loại khóa học*

• Ngoại lệ và tình huống biên
Không phải Admin, category không tồn tại hoặc lỗi ràng buộc/DTO.
Luồng thay thế: Người không quản trị chỉ đọc danh sách.
Cần kiểm chứng riêng: Thử name trùng, slug trùng, role khác, category đang liên kết; đối chiếu course không bị xóa ngoài ý muốn.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-6-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-6-3-02: Thử name trùng, slug trùng, role khác, category đang liên kết; đối chiếu course không bị xóa ngoài ý muốn.
TC-5-6-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-6-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

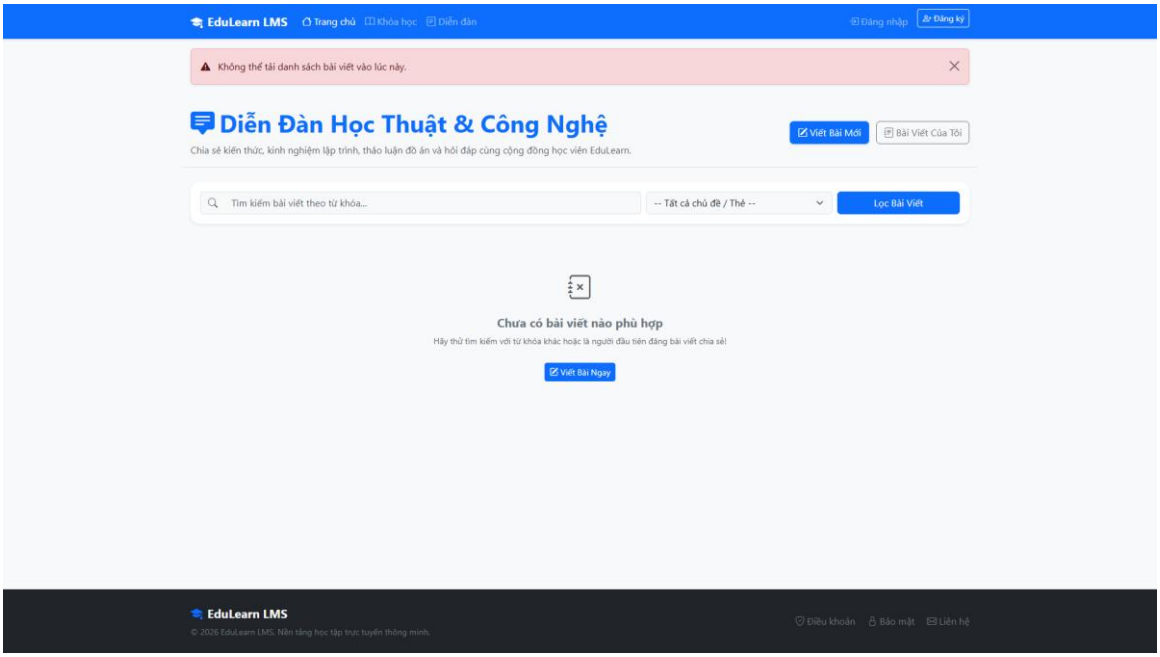
## 5.6.4 Quản lý nội dung blog

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Admin list GET /posts/admin khác /posts/me và /public; filter status/tác giả theo endpoint.
Tác nhân: ADMIN. Tiền điều kiện: Có quyền quản trị Post/Tag và dữ liệu hợp lệ.
Đầu ra: Dữ liệu blog thay đổi theo thao tác thực tế; không tự thêm quy trình duyệt nhiều bước.
• Luồng xử lý và sơ đồ hoạt động
1. Mở Blog Management. 2. Lọc bài theo trạng thái/tác giả/tiêu chí được hỗ trợ. 3. Xem hoặc thực hiện thao tác. 4. Service xử lý theo quyền Admin. 5. Cập nhật danh sách.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/blog/posts/admin; thao tác Post/Tag theo quyền Admin.
Tham số và DTO lấy từ controller; căn cứ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java cùng các controller liên quan ở 3.3.
Admin list GET /posts/admin khác /posts/me và /public; filter status/tác giả theo endpoint.
• Xử lý nghiệp vụ và truy cập dữ liệu
checkPostOwner cho Admin theo helper; quyền quản trị cần hiểu cụ thể update/delete và tag. Chưa có bằng chứng quy trình duyệt nhiều cấp hoặc nhật ký phê duyệt.
Đối chiếu dữ liệu theo UC051.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/blog-service/src/main/java/com/lms/blogservice/service/impl/PostServiceImpl.java, bắt đầu dòng 74. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Page<PostSummaryResponse> getAllPostsForAdmin(Pageable pageable, PostStatus
status,
                                                      Long tagId, String keyword) {
    Page<Post> posts;
    if (keyword != null && !keyword.isBlank()) {
        if (tagId != null)
            posts =
postRepository.findByTitleContainingIgnoreCaseAndTagsId(keyword, tagId, pageable);
        else
            posts = postRepository.findByTitleContainingIgnoreCase(keyword,
pageable);
    } else if (status != null && tagId != null) {
        posts = postRepository.findByStatusAndTagsId(status, tagId, pageable);
    } else if (status != null) {
        posts = postRepository.findByStatus(status, pageable);
    } else if (tagId != null) {
```

Giải thích: checkPostOwner cho Admin theo helper; quyền quản trị cần hiểu cụ thể update/delete và tag. Chưa có bằng chứng quy trình duyệt nhiều cấp hoặc nhật ký phê duyệt.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: BlogMgmt. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.6.4: Quản lý nội dung bài viết và kiểm duyệt diễn đàn*

• Ngoại lệ và tình huống biên
Post/tag không tồn tại, dữ liệu sai hoặc lỗi lưu tệp.
Luồng thay thế: Quản lý tag qua khu vực nội dung website.
Cần kiểm chứng riêng: Thử DRAFT/PUBLISHED/HIDDEN, bài tác giả khác, xóa và tài nguyên liên quan; kiểm tra public listing sau thay đổi.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-6-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-6-4-02: Thử DRAFT/PUBLISHED/HIDDEN, bài tác giả khác, xóa và tài nguyên liên quan; kiểm tra public listing sau thay đổi.
TC-5-6-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-6-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

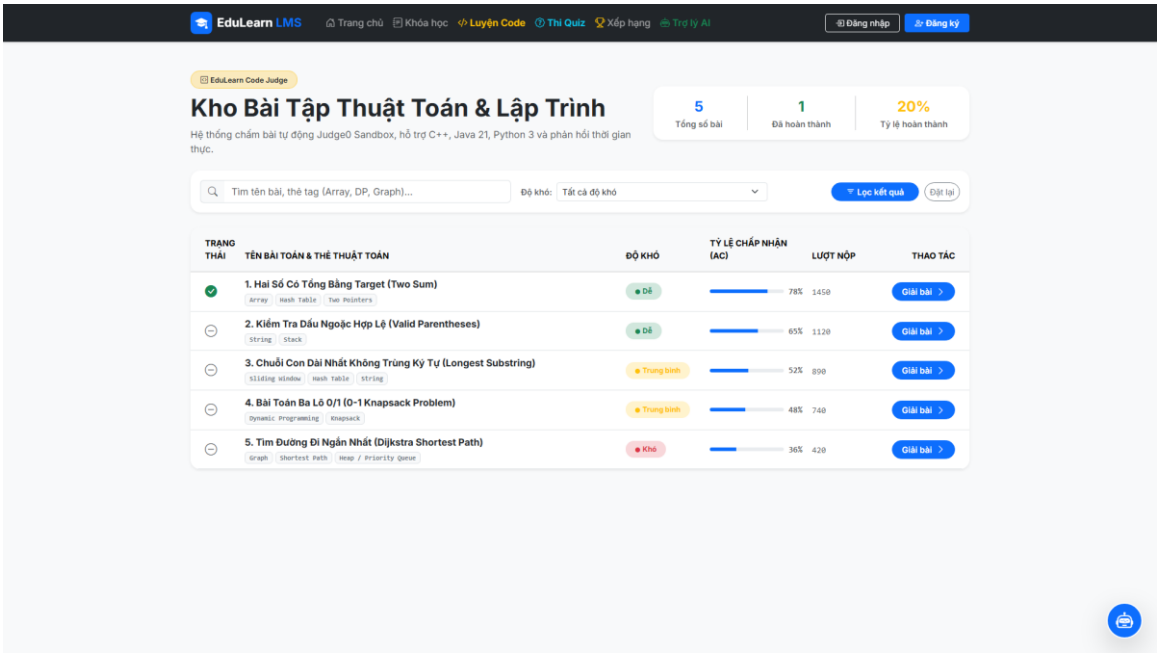
## 5.6.5 Theo dõi hoạt động chấm bài lập trình

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: GET /admin/code-judge/activity nhận hours mặc định 24 và @PreAuthorize ADMIN.
Tác nhân: ADMIN. Tiền điều kiện: Có quyền gọi /admin/code-judge/activity.
Đầu ra: Theo dõi hoạt động; không mặc định có quyền chạy lại hay sửa điểm.
• Luồng xử lý và sơ đồ hoạt động
1. Mở quản lý Code Judge. 2. Gọi API activity với tiêu chí. 3. Backend tổng hợp hoạt động. 4. Hiển thị dữ liệu trả về.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: GET /api/assignment/admin/code-judge/activity.
Tham số và DTO lấy từ controller; căn cứ backend/assignment-service/src/main/java/com/lms/assignmentservice/controller/AdminCodeJudgeActivityController.java cùng các controller liên quan ở 3.3.
GET /admin/code-judge/activity nhận hours mặc định 24 và @PreAuthorize ADMIN.
• Xử lý nghiệp vụ và truy cập dữ liệu
Controller gọi AdminCodeJudgeActivityService, trả list activity. Theo dõi hoạt động không đồng nghĩa có quyền sửa điểm hoặc resubmit bài cho người học.
Đối chiếu dữ liệu theo UC052.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/assignment-service/src/main/java/com/lms/assignmentservice/controller/AdminCodeJudgeActivityController.java, bắt đầu dòng 27. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
ApiResponse<List<AdminCodeJudgeActivityResponse>> getActivity(
        @RequestParam(defaultValue = "24") int hours) {
    return ApiResponse.<List<AdminCodeJudgeActivityResponse>>builder()
            .data(adminCodeJudgeActivityService.getActivity(hours))
            .build();
}
}
```

Giải thích: Controller gọi AdminCodeJudgeActivityService, trả list activity. Theo dõi hoạt động không đồng nghĩa có quyền sửa điểm hoặc resubmit bài cho người học.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: CodeJudgeMgmt. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.6.5: Theo dõi và giám sát hoạt động nộp bài và chấm tự động*

• Ngoại lệ và tình huống biên
Không đủ quyền, lỗi dịch vụ/database.
Luồng thay thế: Dữ liệu rỗng hoặc lọc phạm vi khác.
Cần kiểm chứng riêng: Thử role STUDENT/INSTRUCTOR, hours bất thường, cửa sổ không có dữ liệu; kiểm tra UI không dùng mock khi API lỗi.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-6-5-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-6-5-02: Thử role STUDENT/INSTRUCTOR, hours bất thường, cửa sổ không có dữ liệu; kiểm tra UI không dùng mock khi API lỗi.
TC-5-6-5-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-6-5-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.6.6 Quản lý nội dung website và giới hạn lưu trữ phía trình duyệt

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Bốn tab WebContentMgmt có phạm vi khác nhau: category course/blog gọi API; Inspire Images dùng localStorage; Founders sửa state.
Tác nhân: ADMIN. Tiền điều kiện: Mở trang quản lý nội dung website.
Đầu ra: Chỉ dữ liệu API có lưu backend; ảnh Inspire lưu riêng trình duyệt; Founders chưa lưu backend.
• Luồng xử lý và sơ đồ hoạt động
1. Chọn tab. 2. Danh mục khóa học/blog gọi API. 3. Ảnh Inspire lưu localStorage. 4. Founders sửa state; nút lưu thông báo nội dung đặt cứng. 5. Hiển thị kết quả tương ứng.
Tách nhánh API với state/localStorage; không vẽ lưu Founders vào database.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: Danh mục dùng Course/Blog API; Inspire/Founders không có cùng luồng persistence backend.
Tham số và DTO lấy từ controller; căn cứ frontend/src/features/dashboard/components/admin/Management/WebContentMgmt/FoundersContentTab.jsx cùng các controller liên quan ở 3.3.
Bốn tab WebContentMgmt có phạm vi khác nhau: category course/blog gọi API; Inspire Images dùng localStorage; Founders sửa state.
• Xử lý nghiệp vụ và truy cập dữ liệu
saveFoundersContent chỉ kiểm tra avatar rồi showToast nội dung đang set cứng, không lưu backend. Inspire localStorage chỉ hiệu lực trình duyệt; không suy đã đồng bộ cho mọi người. Đối chiếu dữ liệu theo UC053.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ frontend/src/features/dashboard/components/admin/Management/WebContentMgmt/FoundersContentTab.jsx, bắt đầu dòng 89. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
  const saveFoundersContent = () => {
const hasInvalidAvatar = content.founders.some((founder) =>
!isValidAvatarSource(founder.avatar));
if (hasInvalidAvatar) {
  showToast('Ảnh đại diện phải là URL http/https hoặc file PNG hợp lệ.', 'danger');
  return;
}
showToast('Nội dung người sáng lập đang được set cứng trong code.');
  };
  const resetFoundersContent = () => {
if (!window.confirm('Khôi phục nội dung đội ngũ sáng lập về mặc định?')) return;
```

Giải thích: saveFoundersContent chỉ kiểm tra avatar rồi showToast nội dung đang set cứng, không lưu backend. Inspire localStorage chỉ hiệu lực trình duyệt; không suy đã đồng bộ cho mọi người.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: WebContentMgmt và các tab. Demo theo đúng role và luồng API được mô tả.

Chụp thêm reload/trình duyệt khác để thể hiện giới hạn lưu cục bộ.

• Ngoại lệ và tình huống biên

localStorage đầy/lỗi; lỗi API danh mục; chưa có persistence Founders, không coi thông báo là lưu thành công.

Luồng thay thế: Khôi phục giá trị mặc định theo giao diện.

Cần kiểm chứng riêng: Thử reload, mở trình duyệt khác, sửa Founders rồi rời trang; chụp sự khác biệt giữa thao tác cục bộ và API thật.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-6-6-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-6-6-02: Thử reload, mở trình duyệt khác, sửa Founders rồi rời trang; chụp sự khác biệt giữa thao tác cục bộ và API thật.

TC-5-6-6-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-6-6-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.6.7 Theo dõi dashboard và sức khỏe dịch vụ

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: ADMIN_HEALTH_ENDPOINTS và timeout lấy env; vài URL local gọi trực tiếp service, chatbot đi Gateway mặc định. Không dùng URLlocalhost của máy người xem như địa chỉ production.

Tác nhân: ADMIN trên dashboard. Tiền điều kiện: Các API/dashboard và endpoint health truy cập được.

Đầu ra: Đọc số liệu và trạng thái thời điểm kiểm tra.

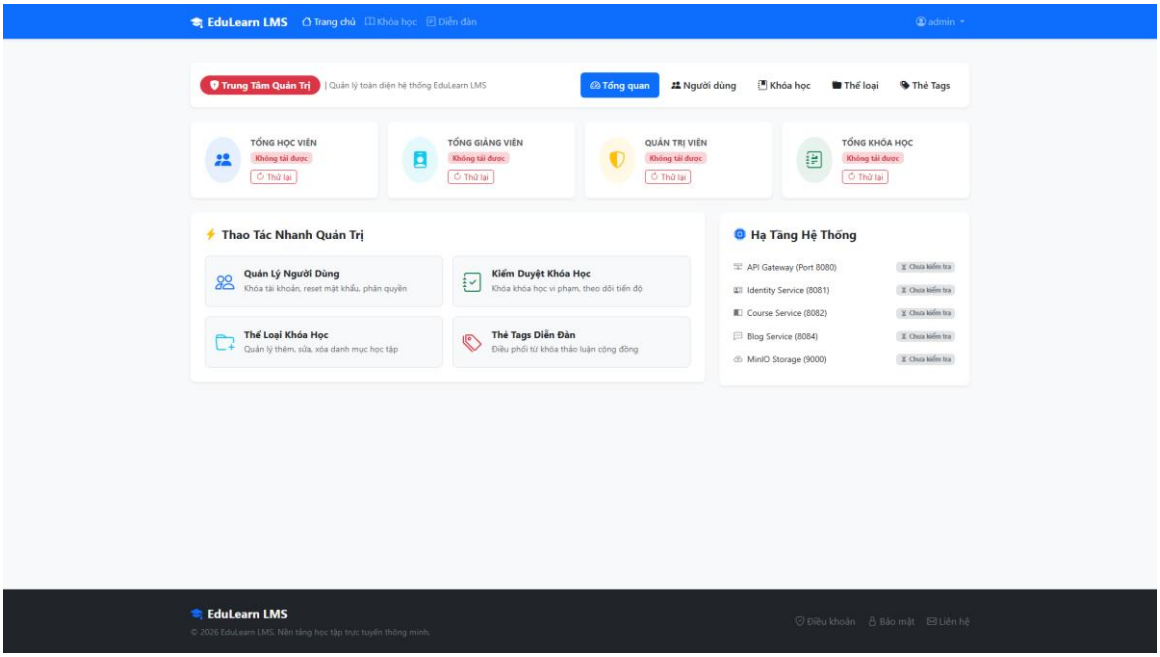
• Luồng xử lý và sơ đồ hoạt động

1. Mở dashboard. 2. Giao diện gọi dữ liệu tổng hợp và health. 3. Nhận trạng thái dịch vụ. 4. Hiển thị thống kê và lỗi nếu có.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: Endpoint health lấy từ ADMIN_HEALTH_ENDPOINTS, phụ thuộc env.
Tham số và DTO lấy từ controller; căn cứ frontend/src/configurations/env.js cùng các controller liên quan ở 3.3.
ADMIN_HEALTH_ENDPOINTS và timeout lấy env; vài URL local gọi trực tiếp service, chatbot đi Gateway mặc định. Không dùng URLlocalhost của máy người xem như địa chỉ production.
• Xử lý nghiệp vụ và truy cập dữ liệu
Dashboard có refresh interval mặc định 30s và timeout theo service. Health liveness chỉ phản ánh điều kiện được cấu hình, không thay test OTP/judge/LLM.
Đối chiếu dữ liệu theo UC054.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ frontend/src/configurations/env.js, bắt đầu dòng 34. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
export const ADMIN_HEALTH_ENDPOINTS = {
  identity:
import.meta.env.VITE_IDENTITY_HEALTH_URL ||
'http://localhost:8081/identity/actuator/health',
  course:
import.meta.env.VITE_COURSE_HEALTH_URL ||
'http://localhost:8082/course/actuator/health/liveness',
  assignment:
import.meta.env.VITE_ASSIGNMENT_HEALTH_URL ||
'http://localhost:8083/assignment/actuator/health/liveness',
  blog:
import.meta.env.VITE_BLOG_HEALTH_URL ||
'http://localhost:8084/blog/actuator/health/liveness',
  chatbot:
```

Giải thích: Dashboard có refresh interval mặc định 30s và timeout theo service. Health liveness chỉ phản ánh điều kiện được cấu hình, không thay test OTP/judge/LLM.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: HomePageAdmin và dashboard.api. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.




*Hình 5.6.7: Admin Dashboard giám sát hệ thống (thể hiện giao diện khung điều khiển và trạng thái cảnh báo kết nối khi các service backend đang khởi động)*

• Ngoại lệ và tình huống biên
Timeout, mất kết nối, thiếu quyền hoặc lỗi service; health không chứng minh toàn bộ nghiệp vụ đúng.
Luồng thay thế: Một service lỗi có thể hiển thị trạng thái không sẵn sàng; không suy số liệu thiếu là bằng không.
Cần kiểm chứng riêng: Thử service dừng, API timeout và production env; kiểm tra trạng thái unavailable thay vì biểu diễn healthy bằng dữ liệu thiếu.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-6-7-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-6-7-02: Thử service dừng, API timeout và production env; kiểm tra trạng thái unavailable thay vì biểu diễn healthy bằng dữ liệu thiếu.
TC-5-6-7-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-6-7-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

# 5.7 Gateway và hạ tầng

## 5.7.1 Định tuyến và xác thực request

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: AuthFilter là named GatewayFilterFactory, không tự chạy trên mọi route. Public/private phụ thuộc YAML Path/Method/order; OPTIONS đi nhánh công khai.

Đầu ra: Request hoặc trạng thái/lỗi phản hồi theo cấu hình và implementation; không tạo dữ liệu nghiệp vụ ngoài phạm vi.

• Luồng xử lý và sơ đồ hoạt động

apply đọc Bearer, JwtUtil xác minh type ACCESS/chữ ký/hạn, xóa header nhận dạng client rồi đặt X-User-* trước forward. Course SecurityConfig tin header nên downstream phải nằm trong mạng tin cậy.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.

[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]

• API DTO validation và phân quyền

API chính: Gateway routes trong application.yaml; gọi API qua /api/<service>.

Tham số và DTO lấy từ controller; căn cứ backend/api-gateway/src/main/java/com/lms/apigateway/configuration/AuthFilter.java cùng các controller liên quan ở 3.3.

AuthFilter là named GatewayFilterFactory, không tự chạy trên mọi route. Public/private phụ thuộc YAML Path/Method/order; OPTIONS đi nhánh công khai.

• Xử lý nghiệp vụ và truy cập dữ liệu

apply đọc Bearer, JwtUtil xác minh type ACCESS/chữ ký/hạn, xóa header nhận dạng client rồi đặt X-User-* trước forward. Course SecurityConfig tin header nên downstream phải nằm trong mạng tin cậy.

• Source code trọng tâm và giải thích

Đoạn trích liên tục từ backend/api-gateway/src/main/java/com/lms/apigateway/configuration/AuthFilter.java, bắt đầu dòng 54. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public GatewayFilter apply(Config config) {
    return (exchange, chain) -> {
        //get header from request
        ServerHttpRequest request = exchange.getRequest();
        if (HttpMethod.OPTIONS.equals(request.getMethod()) ||
isPublicRequest(request)) {
            ServerHttpRequest publicRequest = request.mutate()
                    .headers(this::removeGatewayAuthHeaders)
                    .build();
            return chain.filter(exchange.mutate().request(publicRequest).build());
        }
        String authHeader =
request.getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
```

Giải thích: apply đọc Bearer, JwtUtil xác minh type ACCESS/chữ ký/hạn, xóa header nhận dạng client rồi đặt X-User-* trước forward. Course SecurityConfig tin header nên downstream phải nằm trong mạng tin cậy.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: Spring MVC Form Action & RestClient. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên

Thiếu phiên/quyền, dữ liệu sai, dependency lỗi và response committed tùy luồng.

Luồng thay thế: Hiển thị trạng thái/lỗi theo fallback; không tự thay dữ liệu nghiệp vụ.

Cần kiểm chứng riêng: Gửi thiếu token, type REFRESH, token hết hạn, header X-User giả qua Gateway và gọi trực tiếp downstream ở môi trường kiểm thử; không thử dữ liệu người khác.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-7-1-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-7-1-02: Gửi thiếu token, type REFRESH, token hết hạn, header X-User giả qua Gateway và gọi trực tiếp downstream ở môi trường kiểm thử; không thử dữ liệu người khác.
TC-5-7-1-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-7-1-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

### 5.7.2 Service discovery và giao tiếp nội bộ

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Gateway dùng lb:// qua Eureka; FeignClient có url cấu hình nên không phải mọi lời gọi nội bộ đều qua discovery.
Đầu ra: Request hoặc trạng thái/lỗi phản hồi theo cấu hình và implementation; không tạo dữ liệu nghiệp vụ ngoài phạm vi.
• Luồng xử lý và sơ đồ hoạt động
Course.IdentityClient kiểm tra exists, get user và tìm email; fallback có method ném lỗi, method trả null. Không diễn giải null như kết luận chắc chắn user không tồn tại khi service đang lỗi.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: Feign URL nội bộ theo services.*.url; Eureka dùng /eureka.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/configuration/IdentityClient.java cùng các controller liên quan ở 3.3.
Gateway dùng lb:// qua Eureka; FeignClient có url cấu hình nên không phải mọi lời gọi nội bộ đều qua discovery.
• Xử lý nghiệp vụ và truy cập dữ liệu
Course.IdentityClient kiểm tra exists, get user và tìm email; fallback có method ném lỗi, method trả null. Không diễn giải null như kết luận chắc chắn user không tồn tại khi service đang lỗi.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/configuration/IdentityClient.java, bắt đầu dòng 31. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public boolean existsById(String userId) {
    throw new CourseException(ErrorCode.EXTERNAL_SERVICE_UNAVAILABLE);
}
```

```
    @Override
    public UserInfoDto getUserById(String userId) {
        return null;
    }
    @Override
    public UserInfoDto findByEmail(String email) {
        return null;
    }
}
```

Giải thích: Course.IdentityClient kiểm tra exists, get user và tìm email; fallback có method ném lỗi, method trả null. Không diễn giải null như kết luận chắc chắn user không tồn tại khi service đang lỗi.

• Giao diện minh họa và thao tác sử dụng

Màn hình/thành phần: Không có màn hình nghiệp vụ riêng. Demo theo đúng role và luồng API được mô tả.

Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên

Thiếu phiên/quyền, dữ liệu sai, dependency lỗi và response committed tùy luồng.

Luồng thay thế: Hiển thị trạng thái/lỗi theo fallback; không tự thay dữ liệu nghiệp vụ.

Cần kiểm chứng riêng: Tắt Identity trong môi trường thử, gọi dữ liệu phụ thuộc, kiểm tra fallback; đọc registry và log để phân biệt không đăng ký với lỗi API.

• Kịch bản kiểm tra và minh chứng

Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.

TC-5-7-2-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.

TC-5-7-2-02: Tắt Identity trong môi trường thử, gọi dữ liệu phụ thuộc, kiểm tra fallback; đọc registry và log để phân biệt không đăng ký với lỗi API.

TC-5-7-2-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-7-2-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.

Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.7.3 Lưu trữ và truy cập tệp

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện

Đầu vào và điều kiện: Presigned request cần fileName/fileType/folder theo StorageController; endpoint có quyền tại Gateway/service. Object được lưu MinIO, metadata lưu ở database.

Đầu ra: Request hoặc trạng thái/lỗi phản hồi theo cấu hình và implementation; không tạo dữ liệu nghiệp vụ ngoài phạm vi.

• Luồng xử lý và sơ đồ hoạt động

generatePresignedUrl tạo folder/UUID/extension, ký PUT có expiry rồi trả public URL. Frontend phải upload bằng URL và kiểm tra HTTP result trước lưu metadata. Identity cleanup có kiểm tra user prefix.

Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.

[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]

• API DTO validation và phân quyền
API chính: StorageController theo từng service; course có /api/storage/** qua Gateway.
Tham số và DTO lấy từ controller; căn cứ backend/course-service/src/main/java/com/lms/courseservice/service/impl/StorageServiceImpl.java cùng các controller liên quan ở 3.3.
Presigned request cần fileName/fileType/folder theo StorageController; endpoint có quyền tại Gateway/service. Object được lưu MinIO, metadata lưu ở database.
• Xử lý nghiệp vụ và truy cập dữ liệu
generatePresignedUrl tạo folder/UUID/extension, ký PUT có expiry rồi trả public URL. Frontend phải upload bằng URL và kiểm tra HTTP result trước lưu metadata. Identity cleanup có kiểm tra user prefix.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/course-service/src/main/java/com/lms/courseservice/service/impl/StorageServiceImpl.java, bắt đầu dòng 58. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public PresignedUrlResponse generatePresignedUrl(String fileName, String fileType,
String folder) {
    try {
        String ext = extractExtension(fileName, fileType);
        String objectName = folder + "/" + UUID.randomUUID() + ext;
        String presignedUrl = publicMinioClient.getPresignedObjectUrl(
                GetPresignedObjectUrlArgs.builder()
                        .method(Method.PUT)
                        .bucket(bucketName)
                        .object(objectName)
                        .expiry(presignedExpiryMinutes, TimeUnit.MINUTES)
                        .build()
        );
```

Giải thích: generatePresignedUrl tạo folder/UUID/extension, ký PUT có expiry rồi trả public URL. Frontend phải upload bằng URL và kiểm tra HTTP result trước lưu metadata. Identity cleanup có kiểm tra user prefix.
• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: useFileUpload và form tài nguyên. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên
Thiếu phiên/quyền, dữ liệu sai, dependency lỗi và response committed tùy luồng.
Luồng thay thế: Hiển thị trạng thái/lỗi theo fallback; không tự thay dữ liệu nghiệp vụ.
Cần kiểm chứng riêng: Thử upload đúng/sai MIME, quá giới hạn, URL ký hết hạn, MinIO dừng, lưu metadata khi PUT lỗi và xóa object ngoài prefix; ảnh cần che URL có chữ ký.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-7-3-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-7-3-02: Thử upload đúng/sai MIME, quá giới hạn, URL ký hết hạn, MinIO dừng, lưu metadata khi PUT lỗi và xóa object ngoài prefix; ảnh cần che URL có chữ ký.
TC-5-7-3-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.

TC-5-7-3-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

## 5.7.4 Theo dõi health check và xử lý service lỗi

• Yêu cầu đầu vào đầu ra và điều kiện thực hiện
Đầu vào và điều kiện: Health URL và timeout theo cấu hình; response committed của SSE khác lỗi HTTP trước khi bắt đầu.
Đầu ra: Request hoặc trạng thái/lỗi phản hồi theo cấu hình và implementation; không tạo dữ liệu nghiệp vụ ngoài phạm vi.
• Luồng xử lý và sơ đồ hoạt động
GlobalErrorExceptionHandler trả Mono.error nếu response đã committed, không ghi đè một JSON error mới lên stream đã gửi. ConnectException được ánh xạ unavailable; circuit breaker forward tới fallback theo route.
Sơ đồ cần có nhánh kiểm tra điều kiện và lỗi trước thao tác ghi; không vẽ lỗi thành thành công.
[Cần hình luồng xử lý khi sử dụng; tham chiếu Chương 3 để tránh chèn trùng.]
• API DTO validation và phân quyền
API chính: Actuator health và /fallback/<service> theo Gateway.
Tham số và DTO lấy từ controller; căn cứ backend/api-gateway/src/main/java/com/lms/apigateway/exception/GlobalErrorExceptionHandler.java cùng các controller liên quan ở 3.3.
Health URL và timeout theo cấu hình; response committed của SSE khác lỗi HTTP trước khi bắt đầu.
• Xử lý nghiệp vụ và truy cập dữ liệu
GlobalErrorExceptionHandler trả Mono.error nếu response đã committed, không ghi đè một JSON error mới lên stream đã gửi. ConnectException được ánh xạ unavailable; circuit breaker forward tới fallback theo route.
• Source code trọng tâm và giải thích
Đoạn trích liên tục từ backend/api-gateway/src/main/java/com/lms/apigateway/exception/GlobalErrorExceptionHandler.java, bắt đầu dòng 32. Chỉ trích phần đầu vị trí đã chọn, không phải hàm đầy đủ để sao chép chạy độc lập.

```
public Mono<Void> handle(ServerWebExchange exchange, Throwable ex) {
    log.info(ex.getLocalizedMessage());
    log.info(ex.getMessage());
    if (exchange.getResponse().isCommitted()) {
        // Gói tin đã gửi đi rồi (như cái 200 OK ban nãy) thì kệ nó, không ghi đè
lỗi nữa
        return Mono.error(ex);
    }
    HttpStatus status = HttpStatus.INTERNAL_SERVER_ERROR;
    String message = ex.getMessage();
    Integer code = ErrorCode.INTERNAL_SERVER_ERROR.getCode();
    // CHỈ xử lý các lỗi rớt mạng hoặc sai định tuyến WebFlux
```

Giải thích: GlobalErrorExceptionHandler trả Mono.error nếu response đã committed, không ghi đè một JSON error mới lên stream đã gửi. ConnectException được ánh xạ unavailable; circuit breaker forward tới fallback theo route.

• Giao diện minh họa và thao tác sử dụng
Màn hình/thành phần: dashboard và AppErrorPage. Demo theo đúng role và luồng API được mô tả.
Ảnh cần thể hiện đầu vào, thao tác và kết quả, dùng tài khoản demo được phép.

• Ngoại lệ và tình huống biên
Thiếu phiên/quyền, dữ liệu sai, dependency lỗi và response committed tùy luồng.
Luồng thay thế: Hiển thị trạng thái/lỗi theo fallback; không tự thay dữ liệu nghiệp vụ.
Cần kiểm chứng riêng: Thử dừng service, timeout, URL sai và lỗi giữa stream; phân biệt liveness healthy với nghiệp vụ gửi mail/chấm code thật.
• Kịch bản kiểm tra và minh chứng
Trạng thái: Chưa kiểm thử chức năng; các trường hợp dưới đây là kế hoạch.
TC-5-7-4-01: chạy luồng chính bằng dữ liệu hợp lệ; kiểm tra response và dữ liệu trước/sau theo hậu điều kiện.
TC-5-7-4-02: Thử dừng service, timeout, URL sai và lỗi giữa stream; phân biệt liveness healthy với nghiệp vụ gửi mail/chấm code thật.
TC-5-7-4-03: thiếu phiên, sai role/owner khi liên quan; ghi hành vi thực tế và giới hạn so với yêu cầu.
TC-5-7-4-04: lặp thao tác hoặc làm dependency lỗi trong môi trường thử; kiểm tra bản ghi trùng và trạng thái.
Minh chứng cần lưu: đầu vào, role, response che secret, ảnh, DB khi liên quan và kết luận thật. Chưa chạy không ghi Pass.

# CHƯƠNG 6 KIỂM THỬ VÀ ĐÁNH GIÁ

## 6.1 Kế hoạch kiểm thử

### 6.1.1 Phạm vi và môi trường

Phạm vi kiểm thử gồm Identity, Course, Assignment, Blog, Chatbot, API Gateway và giao diện theo Guest/STUDENT/INSTRUCTOR/ADMIN. Kiểm thử theo chức năng liên hệ các mã TC ở Chương 5, đồng thời có kiểm thử truy cập chéo dữ liệu, tình huống biên, dịch vụ phụ thuộc lỗi và giao diện responsive.
Phiên bản đối chiếu: main, commit 4288b95884c031e669aab604ad019c12f46ff162. Lượt kiểm tra bổ sung ngày 16/10/2026 theo UTC+7 dùng Linux x86_64, Node v24.19.0, npm 11.9.0; dependency frontend được cài bằng npm ci theo lockfile. npm test, build, lint và hai phạm vi coverage đã thực thi; kết quả tại 6.4.5. Maven Wrapper đã tải được Maven 3.9.12; backend và stack tích hợp cần đọc trạng thái riêng, không suy từ kết quả frontend. Môi trường chạy kiểm tra này không phải máy demo của nhóm.

### 6.1.2 Dữ liệu và tài khoản kiểm thử

Bộ dữ liệu demo cần hai học viên A/B, hai giảng viên X/Y và một Admin, dùng email nhóm được phép. Hai giảng viên phải có khóa học khác nhau để thử quyền sở hữu. Chuẩn bị course PUBLIC, PRIVATE, LOCKED; học viên A đã tham gia một course, học viên B chưa tham gia; lesson thuộc hai course khác nhau.
Quiz cần bản nháp/công bố/hết hạn, các loại SINGLE/MULTIPLE/TRUE_FALSE và nhiều policy kết quả; bài code cần test public/hidden cùng lời giải AC, lời giải WA và CE. Blog có DRAFT/PUBLISHED/HIDDEN, comment gốc/reply và hai tác giả; chatbot có hai phiên của hai người. Mỗi dữ liệu gắn ID trong biên bản, không chép mật khẩu/token vào báo cáo.

### 6.1.3 Phân công kiểm thử

Người phát triển chạy unit test cho phần mình phụ trách; người kiểm tra khác thực hiện ít nhất các luồng chính và truy cập chéo role/owner. Với nhóm tối đa ba thành viên, có thể kiểm tra chéo theo module nhưng chỉ điền tên khi có phân công thực tế.
[Cần nhóm bổ sung họ tên, MSSV, module, người thực hiện, người kiểm tra và ngày chạy.]
Khi phát hiện lỗi, lưu request, trạng thái dữ liệu và cách tái hiện; sau sửa cần chạy lại test liên quan và luồng bị ảnh hưởng. Không tự gán tác giả code hoặc người chạy test dựa trên tên có sẵn trong template.

### 6.1.4 Tiêu chí đạt và cách lưu minh chứng

Một test chỉ ghi Đạt khi kết quả thực tế khớp yêu cầu đã xác định, bao gồm response, dữ liệu lưu và quyền truy cập. API trả 200 nhưng dữ liệu không lưu hoặc hành động vượt quyền vẫn có thể Không đạt. Lỗi môi trường ghi Bị chặn, không đổi thành Đạt hoặc lỗi nghiệp vụ. Trường hợp chưa thực hiện ghi Chưa kiểm thử.
Biên bản tối thiểu có mã TC, commit, điều kiện trước, input, các bước, kỳ vọng, thực tế, trạng thái, người chạy, thời gian và liên kết minh chứng. Ảnh dùng dữ liệu demo; che mật

khẩu/token/OTP thật và URL ký. File log cần giữ phần lỗi quan trọng nhưng không đưa secret. Số lượng @Test/file test không phải số lượt đã chạy.

## 6.2 Kiểm thử chức năng theo vai trò

### 6.2.1 Guest và xác thực

Thực hiện Guest xem trang công khai, tìm khóa, đọc chi tiết và blog public; thử vào route bảo vệ khi chưa có phiên. Chạy đăng ký hợp lệ/trùng/sai DTO; gửi/xác minh OTP đúng, sai, hết hạn và vượt attempts; kiểm tra request sai không tạo bản ghi ngoài ý muốn.
Đăng nhập thử username/email, sai password, inactive và verified=false. Kỳ vọng nghiệp vụ cần kích hoạt email phải được so với hành vi authenticate hiện chưa kiểm tra verified. OAuth cần callback đúng/sai; PATCH myinfo/role phải thử tài khoản LOCAL, đã roleSelected và JWT cũ sau thay đổi, vì endpoint hiện tại chưa có giới hạn 30 phút.

### 6.2.2 Học viên

Học viên A đăng ký PUBLIC, thử PRIVATE/LOCKED và đăng ký lặp; kiểm tra enrollment và danh sách khóa của riêng A. Đọc lesson đúng course và thử lesson thuộc course khác; cập nhật tiến độ lặp, tải lại trang và so với LessonProgress.
Với quiz: bắt đầu/resume, lưu tạm, F5, nộp, xem kết quả và lịch sử theo policy. Với code: soạn, Run, Submit, theo dõi PENDING/JUDGING và kết quả; thử đọc submission của B. Ghi rõ Run không tạo bài nộp chính thức và bài không có test trả AC là giới hạn source, không là chứng cứ lời giải đúng.

### 6.2.3 Giảng viên

Giảng viên X tạo/sửa khóa của mình, cấu trúc, lesson, tài nguyên, học viên, lịch/thông báo, câu hỏi và bài tập. Sau đó thử cùng thao tác trên course của Y bằng HTTP request, không chỉ kiểm tra nút bị ẩn. Course LOCKED cần được thử khi biên soạn.
Quiz kiểm tra lưu nháp/publish/ẩn/restore theo quyền; problem kiểm tra clone và độc lập test sau clone. Import phải dùng file thật và đối chiếu DB, không dùng PreviewImportPage dữ liệu mẫu. Thử thêm/sửa test khi đã có submission để kiểm chứng checkNoSubmissions. AI preview cần được rà soát rồi lưu qua API riêng.

### 6.2.4 Admin

Admin tìm/lọc người dùng, đổi active, quản lý tài khoản Admin theo endpoint; quản lý course và lock/unlock, category, blog/tag, hoạt động judge và dashboard. Thử role không phải Admin trên API quản trị để xác nhận từ chối.
Với Web Content, tách category gọi API, ảnh Inspire lưu localStorage và Founders chưa lưu backend. Kiểm tra reload và trình duyệt khác để không nhầm lưu cục bộ thành đồng bộ hệ thống. Khóa user/course phải kiểm tra cả phiên đã phát hành trước đó, không chỉ đăng nhập mới.

### 6.2.5 Blog và chatbot AI

Blog kiểm tra public listing và bảo vệ draft; tạo/sửa/xóa bài đúng tác giả, reply một tầng, tự vote bị chặn và vote vào comment thuộc post khác. Kiểm tra tệp tải lên và dọn ảnh sau cập nhật/xóa theo luồng thực tế.
Chatbot thử GENERAL/COURSE/PROBLEM, stream từng phần, lịch sử và archive theo user. Dùng hai tài khoản để thử đọc messages của session khác; tuyến getMessages hiện chưa thấy kiểm tra owner tương ứng archive. Direct generate yêu cầu INSTRUCTOR, còn Assignment AI preview có quyền riêng; không coi AI luôn đưa đáp án hoặc test case đúng.

## 6.3 Kiểm thử nghiệp vụ và bảo mật

### 6.3.1 OTP và vòng đời phiên đăng nhập

OTP cần thử đúng loại VERIFY_EMAIL/RESET_PASSWORD, thời hạn, usedAt và attempts; sau request sai phải kiểm tra attempts có tăng trong database dù service ném exception. Reset password và đổi password ở settings là hai luồng khác nhau; updatePassword có revokeAllByUserId.
Phiên cần thử ACCESS tại endpoint refresh, REFRESH tại Gateway, refresh đã rotation, logout, nhiều request 401 và tài khoản bị inactive. Ghi riêng kết quả access token còn hạn và refresh revoked, vì Gateway chưa tra bảng revoked với từng access request. Không gộp các trường hợp thành nhận xét đã đăng xuất an toàn toàn bộ.

### 6.3.2 Phân quyền và truy cập dữ liệu người khác

Dùng cặp user/course/post/attempt/submission/session khác chủ để thử thay ID ở path/query/body. Với mỗi request, đối chiếu role, owner và quan hệ đối tượng, ví dụ lesson–course, answer–question, attempt–quiz, comment–post. Bỏ qua frontend bằng request trực tiếp là phần bắt buộc của thử quyền.
Ưu tiên xác minh myinfo/role, chat messages, analytics theo course và scheduleId khi URL chỉ định course khác. Những điểm đang nghi vấn chỉ được kết luận sau source đầy đủ và runtime. Ghi Expected theo yêu cầu bảo vệ dữ liệu, Actual theo phản hồi thật; nếu source cho phép vượt yêu cầu thì ghi Không đạt, không đổi kỳ vọng để khớp lỗi.

### 6.3.3 Đăng ký khóa học và tiến độ

Thử enrollment khi PUBLIC/PRIVATE/LOCKED, hai request đăng ký đồng thời và đăng ký trùng; unique(user_id,course_id) phải được kiểm tra cùng phản hồi lỗi. Xác minh không tạo thêm row cho user không có phiên.
Tiến độ cần so số lesson, số hoàn thành và tỷ lệ hiển thị; cập nhật lặp không tạo bản ghi trùng trái unique(student_id,lesson_id). LessonProgress không có FK course/lesson trong V1 nên việc kiểm tra quan hệ phải dựa service, không coi DB tự ngăn mọi dữ liệu sai.

### 6.3.4 Quiz điểm số và lịch sử làm bài

Dùng quiz có hai câu trọng số 10 và 30, tổng điểm 100: điểm chuẩn hóa tương ứng 25 và 75 nếu trả lời đúng. Với MULTIPLE, chọn đủ tập đúng mới được điểm; thử thiếu/thừa/ID đáp án ngoài câu. So earned_score từng record, attempt.score, passed và Grade.best_score.

Chạy start đồng thời để kiểm tra unique IN_PROGRESS; autosave/F5 không tạo lượt mới ngoài rule. Thử duration, start/end, maxAttempts, submit lặp và hết hạn. AFTER_DEADLINE/AFTER_SUBMIT/IMMEDIATELY phải được kiểm tra theo implementation. Đây là dữ liệu kỳ vọng tính tay, chưa phải số điểm đo được trong môi trường demo.

### 6.3.5 Test case và kết quả chấm code

Chuẩn bị bài có ít nhất một test thật, lời giải đúng và sai để đo AC/WA; bổ sung CE/TLE và lỗi engine. So kết quả test hidden với response học viên để kiểm tra output không bị lộ. Thử các ngôn ngữ ngoài allowedLangs và code dài hơn giới hạn.
Thử bài không test để ghi nhận auto AC score=100 hiện có, nhưng không dùng nó để chứng minh judge đúng. Kiểm tra PENDING → JUDGING → trạng thái cuối, executor lỗi hoặc tác vụ treo; đọc source cleanup liên quan trước khi kết luận tự phục hồi. CustomEngine chưa triển khai nên không ghi đã test custom judge thành công.

### 6.3.6 Upload và validation

Thử upload tệp hợp lệ, MIME/đuôi giả, vượt giới hạn riêng từng chức năng, URL ký hết hạn và PUT MinIO thất bại. Sau lỗi kiểm tra metadata có trỏ tới object không tồn tại hay không; thay avatar cần kiểm tra dọn đúng prefix và giữ object đang dùng.
Thử request DTO trực tiếp, đặc biệt ProblemController create/update chưa có @Valid tại đoạn đã đọc. Ràng buộc frontend, Bean Validation, rule service và DDL phải được đánh giá riêng. Không áp một giới hạn kích thước chung cho avatar, lesson, thumbnail và test-case import.

### 6.3.7 Timeout service lỗi và fallback

Trong môi trường test, lần lượt làm Identity/Course/Judge0/LLM/Redis/MinIO không sẵn sàng và ghi phản hồi. Phân biệt fail-open rate limit Redis với cache fallback Mongo; phân biệt URL Meet dự phòng với phòng thật; số liệu null hoặc Map rỗng từ Feign fallback với số liệu thực bằng không.
Đo thời gian trên client/Gateway/Feign/LLM vì timeout từng tầng khác nhau. Với SSE đã gửi header/body, lỗi giữa stream không nhất thiết trở thành JSON 500 mới. Kiểm tra không báo đã lưu/nộp/chấm thành công khi chỉ bước đầu hoàn thành.

## 6.4 Kiểm thử tự động

### 6.4.1 JUnit và các test backend hiện có

Source có test service, mapper, parser, judge, controller và Gateway. Ví dụ AuthenticationServiceImplTest, EmailVerificationServiceImplTest, CourseServiceImplTest, QuizGradingServiceImplTest, SubmissionJudgingServiceImplTest, PostServiceImplTest, ChatServiceImplTest và AuthFilterTest. Tên file mô tả ý định kiểm tra, không chứng minh test đã chạy.
Từ thư mục ứng dụng có thể chạy: mvn -f backend/pom.xml test. Chạy module chọn: mvn -f backend/pom.xml -pl assignment-service -am test. Trước chạy cần Java21, Maven và

dependency tương ứng; kiểm tra riêng test dùng @SpringBootTest có cần cấu hình ngoại vi. Thu thập Surefire reports và exit code, không dùng số file để ghi Pass.

**Bảng 6.1 Kiểm kê test trong source và trạng thái chạy**

| Module | File Java dưới src/test | Số @Test trong source | Kết quả chạy |
|---|---|---|---|
| identity-service | 5 | 15 | Chưa chạy |
| course-service | 11 | 45 | Chưa chạy |
| assignment-service | 18 | 74 | Chưa chạy |
| blog-service | 9 | 36 | Chưa chạy |
| chatbot-service | 12 | 52 | Chưa chạy |
| api-gateway | 6 | 14 | Chưa chạy |
| service-registry | 1 | 1 | Chưa chạy |
| frontend | 21 file .test.js/.test.jsx | Không quy đổi sang lượt chạy | Chưa chạy |

## 6.4.2 API integration và functional test

IdentityApiIntegrationTest được đọc dùng @WebMvcTest, @Import(SecurityConfig) và @MockBean các service. AssignmentFunctionalTest cũng có WebMvcTest/mock. Các test này hữu ích cho HTTP/serialization/security slice nhưng chưa chứng minh SMTP, database, Judge0 hoặc toàn microservices chạy thật.

Kiểm thử API xuyên stack cần bật service/dependency, lấy JWT bằng luồng hợp lệ, gửi request qua Gateway và đối chiếu DB/object/email/engine. Phân biệt response mock với email nhận được và submission được engine thật xử lý. Báo cáo ghi rõ test nào dùng mock, test nào dùng DB/container/provider thực.

Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).

vitest.config.js đặt environment=jsdom, globals=true, setupFiles src/test/setup.js và include src/test/**/*.test.{js,jsx}; coverage dùng v8, text/html, loại test và main.jsx theo cấu hình. Test hiện có cho Spring Session & Security Context, routes, api.client, upload/Judge hooks, Toast và utils.
Trong frontend sau cài dependency: npm test; npm run test:coverage:core cho phạm vi core; npm run test:coverage cho phạm vi cấu hình đầy đủ. Test jsdom/Testing Library không thay screenshot responsive hay E2E với backend thật. Ghi test files/tests pass/fail/skipped từ output, cùng phạm vi coverage thực tế.

### 6.4.4 Build lint và kiểm tra cấu hình

Backend build có thể dùng mvn -f backend/pom.xml clean verify để chạy các bước được POM cấu hình; không khẳng định tự có toàn bộ integration test nếu plugin không khai báo. Dockerfile backend dùng package -DskipTests, nên build image thành công không phải bằng chứng test đạt.
Frontend chạy npm ci nếu lockfile nhất quán, npm run lint và npm run build; ghi exit code từng lệnh. Cấu hình Compose kiểm tra bằng docker compose config --quiet, production dùng đúng -f và --env-file. Không đưa output docker compose config đầy đủ chứa secret vào báo cáo. Kiểm tra cấu hình tĩnh cũng chưa xác nhận service khởi động được.

### 6.4.5 Bảng kết quả chạy kiểm thử thực tế

Lượt kiểm tra bổ sung ngày 16/10/2026 (UTC+7) đã thực thi frontend trên đúng commit 4288b95, không sửa source hoặc lockfile. npm ci hoàn thành; npm test: 21 test files, 162 tests đạt, exit 0; npm run build: thành công, 1.025 modules transformed, exit 0; npm run lint: 0 errors và 17 warnings, exit 0. Kết quả chỉ áp dụng cho phạm vi lệnh và môi trường đã nêu.
Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app

Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).

**Bảng 6.2 Trạng thái thực hiện kiểm thử và build**

| Hạng mục | Lệnh/phương pháp | Trạng thái | Kết quả / minh chứng |
|---|---|---|---|
| Backend test | Maven Wrapper, Java 21, test; agent Byte Buddy | Đạt; 1 skipped | 237 ca: 236 đạt, 0 failure/error, 1 bỏ qua; xem D.2 |
| Frontend test | npm test (frontend) | Đạt, exit 0 | 21 files, 162 tests đạt; Vitest/jsdom |
| Core coverage | npm run test:coverage:core | Đã đo, exit 0 | Core: S 90,41%; B 81,94%; F 91,41%; L 91,90%. Không là coverage toàn frontend. |
| Frontend lint | npm run lint | Exit 0; cảnh báo | 0 errors, 17 warnings: 13 Hook dependencies, 4 Fast Refresh |
| Frontend build | npm run build | Đạt, exit 0 | Vite 5.4.21; 1.025 modules transformed; dist được tạo |
| Backend package | Maven Wrapper -DskipTests package | Đạt, exit 0 | Các module package thành công; test chạy ở lượt riêng, không từ -DskipTests |
| Compose validate | docker compose config --quiet | Chưa chạy | Exit code, không đưa secret |
| Stack/API thật | Bật dependency và gọi qua Gateway | Chưa chạy | Request/response và dữ liệu |
| OTP/OAuth/Meet/LLM/Judge | Provider thực của nhóm | Chưa chạy | Email/callback/phòng/kết quả thật |
| Responsive và role | Theo viewport và cặp tài khoản | Chưa chạy | Ảnh và biên bản TC |

# 6.5 Kiểm thử giao diện

## 6.5.1 Responsive trên các kích thước màn hình

Kế hoạch viewport gồm 360×800, 390×844, 768×1024 và 1366×768 để kiểm tra mobile/tablet/desktop. Đây là kích thước thử đề xuất, không phải thiết bị đã xác nhận hỗ trợ. Kiểm tra sidebar, form, table, modal, editor, nút nộp và chat panel.

Ảnh cần có cùng màn hình ở ít nhất mobile và desktop, ghi viewport và bước thao tác. Kiểm tra chữ không cắt, nút không bị che, bảng có giải pháp cuộn, editor không làm mất nút Run/Submit và không có cuộn ngang toàn trang ngoài ý muốn. Chưa có ảnh nên không kết luận responsive đã đạt.

### 6.5.2 Điều hướng và thao tác

Thử điều hướng bằng link, nút Back, truy cập URL trực tiếp, F5 ở route con và token hết hạn. Nginx frontend cần SPA fallback để route React không thành 404 sau reload. Kiểm tra redirect sau login và truy cập nhánh sai role.
Thử thao tác liên tiếp, nhấn lưu/nộp nhiều lần và rời trang khi request đang chạy. Chú ý quiz resume/autosave và upload dở. Màn hình /manage/assignments và PreviewImportPage phải ghi đúng trạng thái hiện có, không trình bày như một luồng đã hoàn thiện.

### 6.5.3 Thông báo lỗi trạng thái tải và dữ liệu rỗng

Mỗi danh sách cần phân biệt đang tải, rỗng thật, không có quyền và API lỗi. Kiểm tra toast/form error hiển thị trường sai, không chỉ thông báo chung; spinner phải kết thúc sau lỗi và có cách thử lại hoặc quay lại.
Với thống kê/health, service lỗi không được diễn giải như số liệu bằng 0 hoặc trạng thái healthy. SSE lỗi cần thể hiện phản hồi chưa hoàn tất. Các kết luận về trải nghiệm được dựa trên ảnh/thao tác kiểm tra thực tế, chưa tự đánh giá đẹp/dễ dùng chỉ từ source.

## 6.6 Đánh giá theo rubric

### 6.6.1 Độ khó tính thực tiễn và phân công

Theo rubric, tiêu chí độ khó, tính thực tiễn và phân công tối đa 10 điểm. Căn cứ của đề tài là nhiều role, nghiệp vụ học/đánh giá và các tích hợp; tuy nhiên số lượng công nghệ không tự quyết định điểm.
Minh chứng cần có bài toán, phạm vi, trách nhiệm từng thành viên và sản phẩm thực hiện. Chưa có bảng phân công thực tế hoàn chỉnh nên chưa tự chấm điểm tiêu chí này.

### 6.6.2 Sử dụng AI và cộng tác GitHub

Tiêu chí AI và cộng tác GitHub tối đa 20 điểm. Nhóm cần cung cấp commit/PR có thật, công cụ và prompt đã dùng, đoạn code được hỗ trợ cùng cách kiểm tra/sửa và khả năng giải thích.
Không tạo lịch sử prompt hoặc commit bổ sung chỉ để báo cáo như đã xảy ra. Chatbot AI tích hợp trong sản phẩm khác với AI hỗ trợ phát triển; cả hai phải có minh chứng đúng loại.

### 6.6.3 Cấu trúc dữ liệu vai trò và giao diện

Tiêu chí cấu trúc dữ liệu, vai trò và giao diện tối đa 20 điểm. Chương 3/4 mô tả model, quyền và kiến trúc; Chương 5 chỉ ra chức năng theo source. Cần bổ sung hình UML, ảnh giao diện và kết quả responsive.
Giao diện chính thức đã tích hợp hoàn chỉnh thư viện Bootstrap 5 (phiên bản 5.3.3 qua WebJars dependency org.webjars:bootstrap:5.3.3) trong cả hai module web-portal và web-app, kế thừa hệ thống lưới 12 cột, flexbox và các component chuẩn, đáp ứng đầy đủ tiêu chí sử dụng Bootstrap của học phần.

### 6.6.4 Chất lượng ứng dụng và mức hoàn thiện

Tiêu chí chất lượng ứng dụng, code, cài đặt và hoàn thiện chức năng tối đa 35 điểm. Cần có luồng chạy thật, validation/security, test, xử lý lỗi và tái hiện cài đặt từ source sạch.
Tồn tại những giới hạn như role update, đọc lịch sử chat, preview import mẫu và custom judge chưa triển khai; cần sửa hoặc ghi đúng khi tự đánh giá. Không lấy build Docker -DskipTests thay cho kiểm thử và không lấy health thay nghiệm thu nghiệp vụ.

### 6.6.5 Chức năng sáng tạo và minh chứng

Tiêu chí sáng tạo tối đa 15 điểm; rubric có mức 7–8 nội dung ở mức cao nhất. Có thể xem xét các ứng viên như chấm code Judge0, quiz có resume/autosave, AI preview quiz/test case, chatbot SSE/ngữ cảnh, OAuth và cơ chế resilience, nhưng phải đối chiếu nội dung môn học và xác nhận hoạt động.
Không tự đếm một công nghệ thành nhiều sáng tạo để đạt mức điểm. Custom engine chưa triển khai, PreviewImportPage mẫu hoặc URL Meet dự phòng không được ghi là tính năng sáng tạo hoàn thành.

### 6.6.6 Bảng tự chấm từng thành viên

Bảng tự chấm phải theo từng thành viên và năm trọng số 10/20/20/35/15, tổng tối đa 100 và quy đổi thang10 bằng tổng/10. Các cột họ tên/MSSV, minh chứng và điểm chỉ điền từ dữ liệu nhóm thực tế.
[Cần nhóm bổ sung thông tin thành viên và mức tự chấm có căn cứ; hiện chưa chấm.] Mẫu rubric có nhiều dòng sinh viên không thay quy định nhóm tối đa ba người trong thông báo.

### 6.6.7 Chức năng chưa hoàn thiện và hạn chế

Các điểm chưa hoàn thiện/kiểm chứng gồm Bootstrap; tuyến cập nhật role không dùng SelectRoleServiceImpl chặt hơn; kiểm tra owner khi đọc chat messages; GradeBookController rỗng; PreviewImportPage dữ liệu mẫu; custom judge chưa triển khai; Founders chưa lưu backend và Inspire lưu cục bộ.
Cần kiểm thử thêm login verified, refresh user inactive, access token sau khóa/logout, quan hệ schedule/course, API analytics theo course và bài không test auto AC. Meet fallback chưa là phòng thật; Compose local thiếu thư mục init tham chiếu; production cần hạ tầng và image riêng. Đây là phân loại từ source, chưa phải thống kê lỗi runtime.

# CHƯƠNG 7 CÀI ĐẶT TRIỂN KHAI VÀ HƯỚNG DẪN SỬ DỤNG

## 7.1 Chuẩn bị môi trường

### 7.1.1 Yêu cầu phần cứng và phần mềm

Có hai phương án triển khai: chạy toàn bộ hệ thống bằng Docker Compose; hoặc khởi chạy các dịch vụ lưu trữ dữ liệu (MySQL, MongoDB, MinIO, Redis) bằng Docker và khởi chạy các module Spring Boot (bao gồm các microservices backend cùng hai ứng dụng giao diện web-portal và web-app) bằng công cụ Maven hoặc IDE. Yêu cầu môi trường gồm JDK 21, Maven 3.9+, Docker Engine/Compose.
Không ghi RAM/CPU tối thiểu như kết quả đảm bảo vì chưa benchmark. Với nhiều service Java và database, cần ghi lượng RAM Docker cấp và tài nguyên thực tế khi chạy. Trên Windows, chuẩn bị Docker Desktop/WSL theo môi trường đã cài, kiểm tra docker version, docker compose version, java -version, mvn -version, node --version và npm --version.

### 7.1.2 Lấy source code và xác định phiên bản

Lấy repository và đứng đúng thư mục chứa Compose:
git clone https://github.com/nguyenminhtri15022006-art/LMSmau.git
cd LMSmau/Learning-Management-System-main
git rev-parse HEAD
git status --short
Báo cáo đối chiếu commit rút gọn 4288b95; không tự chuyển checkout hoặc pull khi dữ liệu/code máy nhóm chưa được lưu. Khi nhóm dùng phiên bản khác, ghi SHA đầy đủ và rà lại các nội dung bị ảnh hưởng. Thư mục repo ngoài là LMSmau, thư mục ứng dụng bên trong có tên riêng nên lệnh chạy sai cấp sẽ không tìm thấy pom.xml hoặc Compose.

### 7.1.3 Cấu hình biến môi trường

Tạo cấu hình cục bộ từ mẫu. PowerShell tại thư mục ứng dụng:
Copy-Item .env.example .env
Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).
Copy-Item .env.example .env.local

Điền giá trị của môi trường thử, không ghi secret vào báo cáo/GitHub. Docker Compose đọc env_file cho container và dùng .env cho nội suy; Spring Boot chạy bằng Maven/IDE không tự đọc file .env chỉ vì nó nằm ở project. Chưa tìm thấy cấu hình dotenv trong tuyến đã kiểm tra, nên cần đặt Environment variables trong Run Configuration hoặc shell trước khi khởi động. Frontend VITE_* đọc lúc dev/build; đổi giá trị production phải build lại bundle.

### 7.1.4 Cấu hình SMTP OAuth LLM Judge0 và MinIO

SMTP: MAIL_HOST/MAIL_PORT/MAIL_USERNAME/MAIL_PASSWORD để gửi email thật.
OAuth:
GOOGLE_CLIENT_ID/GOOGLE_CLIENT_SECRET/GOOGLE_ALLOWED_REDIRECT_URIS và VITE_GOOGLE_CLIENT_ID/VITE_GOOGLE_REDIRECT_URI phải cùng ứng dụng và redirect. Calendar dùng GOOGLE_SERVICE_ACCOUNT_FILE, GOOGLE_DELEGATED_EMAIL, GOOGLE_CALENDAR_ID riêng với OAuth đăng nhập.
LLM: LLM_PROVIDER/LLM_API_KEY/LLM_MODEL/LLM_BASE_URL theo provider;
Judge: JUDGE_ENGINE=judge0, JUDGE0_URL/JUDGE0_API_KEY theo triển khai. MinIO dùng endpoint nội bộ MINIO_URL, URL trình duyệt MINIO_PUBLIC_URL, access/secret và bucket theo service. JWT_SIGNER_KEY phải đồng bộ Identity/Gateway và thay giá trị mẫu. Chỉ có placeholder chưa đủ chạy các tích hợp; cần thử email, OAuth, LLM, judge và upload thật.

## 7.2 Khởi tạo dữ liệu

### 7.2.1 Khởi tạo MySQL và chạy Flyway

MySQL cần các database phù hợp env. Mẫu .env.example ghi identity_lms, course_lms, assignment_lms và blog_lms. Nếu chưa có, tạo qua client quản trị bằng SQL sau trên database thử:

```
CREATE DATABASE IF NOT EXISTS identity_lms CHARACTER SET utf8mb4;
CREATE DATABASE IF NOT EXISTS course_lms CHARACTER SET utf8mb4;
CREATE DATABASE IF NOT EXISTS assignment_lms CHARACTER SET utf8mb4;
CREATE DATABASE IF NOT EXISTS blog_lms CHARACTER SET utf8mb4;
```

Đổi tên nếu env khác và cấp quyền tài khoản ứng dụng tương ứng. Compose local mount ./docker/mysql/init nhưng thư mục này không có trong bản source đã rà soát; chưa thể khẳng định tự tạo đầy đủ database. Flyway của từng service chạy migration classpath:db/migration; JPA Course/Assignment/Blog validate, Identity none. Cần kiểm tra log Flyway và flyway_schema_history trước demo; không bật ddl-auto=create để bỏ qua lỗi schema rồi báo cài đặt thành công.

### 7.2.2 Khởi tạo MongoDB

MongoDB phục vụ database của Course, Assignment và Chatbot. Cấu hình URI lấy từ COURSE_MONGODB_URI, ASSIGNMENT_MONGODB_URI, CHATBOT_MONGO_URI và database name liên quan. Kiểm tra authSource phù hợp tài khoản; tên database không được suy từ tên container.
Collection được tạo khi ứng dụng ghi dữ liệu theo cấu hình Mongo; index annotation chỉ được xem là yêu cầu source, cần kiểm tra index thực tế nếu auto-index không bật. Chuẩn bị

course_structures, lessons, submissions, chat_sessions và chat_messages bằng luồng demo; không chèn document tùy ý nếu chưa biết định dạng và liên kết ID.

### 7.2.3 Khởi tạo Redis và bucket MinIO

Redis cần endpoint, port và password khớp cách container được chạy. File docker-compose.db.yml và Compose full có tên service/cấu hình khác nhau; không lấy tên redis-cache thay redis trong cùng một mạng nếu container không tồn tại. Rate limit/cache có TTL nên không coi Redis là nguồn lưu bền duy nhất của bài học/lịch sử.
MinIO API dùng 9000, Console9001 theo Compose. Kiểm tra bucket đúng từng service; một số startup như Identity có init bucket và policy, nhưng log lỗi được bắt nên service khởi động không chứng minh storage sẵn sàng. MINIO_URL trong container thường dùng hostname mạng Docker; URL public/presigned phải truy cập được từ trình duyệt và đúng scheme/port.

### 7.2.4 Tạo dữ liệu demo và tài khoản theo vai trò

Đăng ký hai học viên và hai giảng viên qua UI/API phù hợp, xác minh email nếu luồng cần; tạo Admin bằng cơ chế bootstrap cấu hình trong Identity hoặc API Admin hợp lệ. ApplicationInitConfig đọc BOOTSTRAP_ADMIN_ENABLED và các danh sách username/password/email/fullname. Không công bố giá trị mặc định của template như tài khoản vận hành chung.
Sau đó tạo course PUBLIC/PRIVATE/LOCKED, lesson, enrollment, quiz và problem có test, bài blog, lịch/thông báo. Các ID liên service phải liên kết đúng. Ghi tài khoản role vào bảng demo nhưng giữ mật khẩu ở nơi riêng. Tên người sáng lập có sẵn trong frontend không phải thông tin thành viên nhóm.

### 7.2.5 Đóng gói database SQL và dữ liệu bổ sung

Hồ sơ dữ liệu cần cả MySQL SQL và phần dữ liệu ngoài MySQL. Lưu migration cùng bản dump của database thử; nếu dùng mysqldump, dùng -p để được hỏi mật khẩu và --result-file để tránh khác biệt encoding khi redirect trên Windows. Ví dụ mẫu, thay USER bằng tài khoản backup:
mysqldump -u USER -p --databases identity_lms course_lms assignment_lms blog_lms --result-file=lms_mysql.sql
MongoDB xuất bằng mongodump với URI của môi trường thử và archive/gzip; MinIO cần backup object kèm bucket/key; file SQL không chứa lesson/chat/submission Mongo hay tệp ảnh. Không đưa URI có password hoặc service-account JSON vào báo cáo. Cần thử khôi phục trên môi trường riêng và ghi liên kết file dữ liệu nộp; chưa tạo hoặc xác nhận backup trong đợt này.

## 7.3 Chạy ứng dụng local

### 7.3.1 Chạy toàn bộ bằng Docker Compose

Tại thư mục ứng dụng, sau cấu hình và tạo database:
docker compose config --quiet
docker compose up -d --build
docker compose ps

docker compose logs --tail=100 api-gateway identity-service
Lệnh trên mô tả phương án trong source, chưa phải log đã chạy. Full Compose có mysql/mongodb/redis/minio, registry, gateway, năm service và frontend. Cần xử lý thư mục MySQL init thiếu và cấu hình tích hợp trước. Frontend container ánh xạ 5173:80, Gateway mặc định8080. Xem lỗi service cụ thể, không bỏ qua unhealthy bằng cách chỉ mở được trang chủ. Dừng giữ dữ liệu bằng docker compose down; không dùng tùy chọn xóa volume khi cần giữ demo.

## 7.3.2 Chạy dependency bằng Docker

Phương án IDE dùng dependency-only:
docker compose -f docker-compose.db.yml up -d
docker compose -f docker-compose.db.yml ps
Tên service là mysql-db, mongo-db, minio-storage, redis-cache; MySQL image8.0, Mongo7.0, Redis7.2-alpine theo file này, khác một số image/version và volume của full Compose. File dependency-only không mount script tạo bốn database như đường dẫn full Compose; cần tạo database/user chủ động. Không chạy đồng thời hai stack cùng host ports3306/27017/6379/9000. Khi chuyển phương án, không giả định dữ liệu trong hai tập volume là một.

**Bảng 7.1 So sánh các phương án chạy ứng dụng**

| Phương án | Dependency/ứng dụng | Đặc điểm source | Điểm phải chuẩn bị |
|---|---|---|---|
| Compose đầy đủ | mysql, mongodb, redis, minio và toàn ứng dụng | MySQL8.4; volume riêng; frontend5173 | Thiếu docker/mysql/init trong repo; tạo DB phù hợp |
| Dependency-only + IDE | mysql-db, mongo-db, redis-cache, minio-storage | MySQL8.0; bộ volume khác; Java/React chạy host | Tạo DB, đặt env IDE, địa chỉ localhost |
| Production | Registry, Gateway, năm service và frontend image | Không kèm DB/storage; env_file .env.prod | Hạ tầng ngoài, image/tag của nhóm, credentials, reverse proxy |

## 7.3.3 Chạy backend bằng IntelliJ hoặc Maven

Mở backend/pom.xml bằng IntelliJ, chọn JDK21 và import Maven. Gán env cho từng Run Configuration, dùng localhost cho dependency chạy trên máy host; container hostname như mysql/identity-service không tự resolve trong tiến trình IDE host.
Các lệnh Maven chạy từng process/terminal từ thư mục ứng dụng:
mvn -f backend/pom.xml -pl service-registry spring-boot:run
mvn -f backend/pom.xml -pl api-gateway spring-boot:run
mvn -f backend/pom.xml -pl identity-service spring-boot:run
mvn -f backend/pom.xml -pl course-service spring-boot:run
mvn -f backend/pom.xml -pl assignment-service spring-boot:run
mvn -f backend/pom.xml -pl blog-service spring-boot:run
mvn -f backend/pom.xml -pl chatbot-service spring-boot:run

Mỗi lệnh là một tiến trình chạy lâu, không dán thành một chuỗi chờ process đầu kết thúc. Xác nhận log startup, Flyway và registry; nếu module cần build trước thì mvn -f backend/pom.xml clean install và đọc kết quả. Không dùng -DskipTests làm chứng cứ test đạt.

### 7.3.4 Chạy frontend bằng npm

Khởi chạy hai cổng giao diện người dùng Spring Boot (Thymeleaf/Bootstrap 5):
1. Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Truy cập trình duyệt: http://localhost:8086

2. Cổng quản trị & đào tạo cho giảng viên/admin (web-app):
   mvn spring-boot:run -pl backend/web-app
   Truy cập trình duyệt: http://localhost:8088
npm ci
Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).
Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).

### 7.3.5 Thứ tự khởi động và health check

Thứ tự kiểm tra: dependency sẵn sàng → Registry → Identity và các service nghiệp vụ theo phụ thuộc → Gateway đã thấy instance → frontend → kiểm tra nghiệp vụ. Gateway có thể khởi động trước service nhưng route sẽ chưa phục vụ được đến khi instance sẵn sàng; depends_on service_started không đồng nghĩa ứng dụng đã hoàn tất init.

Các URL local tham khảo: http://localhost:8761 cho Eureka; http://localhost:8080/actuator/health/liveness cho Gateway; các service dùng /identity, /course, /assignment, /blog, /chatbot/actuator/health theo context tương ứng. Chỉ health được exposure/config mới dùng được. Kiểm tra thêm list course, login, một lần lưu và đọc dữ liệu; liveness gồm ping không xác minh SMTP/LLM/Judge0.

### 7.3.6 Các lỗi thường gặp và cách xử lý

Nếu port bị chiếm: kiểm tra process/container đang dùng trước, không tắt tùy tiện tiến trình khác. Nếu host name không resolve: phân biệt cấu hình IDE-host và Docker-network. Nếu Unknown database/Flyway validate: tạo DB đúng tên, kiểm tra user/quyền và migration, không xóa dữ liệu để chữa lỗi khi chưa backup.
Nếu 401/403: kiểm tra token type/hạn, role và header qua Gateway; nếu OAuth redirect mismatch thì so origin/callback/frontend/backend. Upload lỗi cần so URL public, bucket, CORS và thời hạn presigned; chatbot lỗi so LLM/Redis và timeout; code lỗi so Judge0 và engine=judge0. Nếu Docker image build không tải dependency, lưu lỗi build/network; chưa kết luận lỗi source. Ghi vào bảng sự cố các bước tái hiện và cách sửa đã thử thật.

## 7.4 Triển khai production

### 7.4.1 Docker Compose production

docker-compose.prod.yml dùng image=${IMAGE_PREFIX}/<service>:${IMAGE_TAG}, không có build cho các module và không kèm mysql/mongodb/redis/minio. Giá trị image mặc định thuộc namespace có sẵn trong source, chưa chứng minh là image nhóm sở hữu hoặc truy cập được; nhóm phải cung cấp image/tag đã build và database/storage endpoint thực tế.
Mẫu lệnh sau chỉ dùng khi đã chuẩn bị .env.prod và image phù hợp:
docker compose --env-file .env.prod -f docker-compose.prod.yml config --quiet
docker compose --env-file .env.prod -f docker-compose.prod.yml pull
docker compose --env-file .env.prod -f docker-compose.prod.yml up -d
COMPOSE_ENV_FILE trong file mặc định .env.prod; nếu dùng tên khác phải đặt cả biến và --env-file phù hợp. frontend image chứa VITE_* tại build time; đổi env của backend không tự sửa bundle frontend đã phát hành. Chưa deploy trong đợt này.

### 7.4.2 Nginx và cấu hình gateway

Có hai lớp Nginx: frontend/nginx.conf bên trong container phục vụ dist và SPA fallback; nginx.vps.conf là reverse proxy host cho frontend/Gateway. Cấu hình host source proxy / tới127.0.0.1:3000, /api/ tới127.0.0.1:8080, có nhánh chat tắt buffering/cache và kéo dài timeout.
Trước dùng phải sửa domain/port/chứng thư theo máy nhóm, kiểm tra nginx -t trên host và kiểm tra frontend reload route con. Không copy file vào host hoặc reload dịch vụ trong phạm vi viết báo cáo này. Cấu hình có proxy_read_timeout300s không tự xóa timeout nhỏ hơn ở Gateway/client. Header real_ip chỉ được tin từ proxy được cấu hình đúng; chưa xác minh Cloudflare tại môi trường nhóm.

### 7.4.3 Tên miền HTTPS và callback OAuth

nginx.vps.conf đang chứa domain và đường dẫn chứng thư cụ thể của template. Khi triển khai riêng cần DNS trỏ đúng host, chứng thư khớp tên miền, HTTPS frontend/API/media nhất quán và callback OAuth được đăng ký chính xác. Không coi tên miền có trong README là bằng chứng nhóm đã triển khai phiên bản báo cáo lên đó.
Dùng URL Gateway public cho VITE_API_BASE_URL; các VITE_* health URL không để localhost của máy người xem. Google allowed redirect và frontend callback phải khớp scheme/host/path. MINIO_PUBLIC_URL/presigned cũng phải truy cập được qua trình duyệt; nếu frontend HTTPS mà upload HTTP sẽ cần kiểm tra lỗi mixed content. [Cần nhóm bổ sung domain đã dùng và ảnh chứng minh truy cập phiên bản demo.]

### 7.4.4 Quản lý cấu hình và dữ liệu

Giữ .env.prod và file service-account ở vị trí riêng; Compose production mount ./google-service-account.json vào Course theo đường dẫn /run/secrets/...:ro. Cần tồn tại file đúng loại và có quyền đọc phù hợp; chưa có credential thì không lấy URL Meet fallback làm tích hợp thành công.
Sao lưu database và object trước thay image/migration. Ghi IMAGE_TAG cùng SHA source và phiên bản cấu hình không chứa secret. MySQL/Mongo/MinIO/Redis trong production là hạ tầng ngoài file Compose này nên cần mô tả ai quản lý, nơi lưu bền và backup thực tế; không ghi volume protection do file không khai báo volume dữ liệu cho các dịch vụ ngoài.

### 7.4.5 Log health check và khôi phục

Theo dõi docker compose -f docker-compose.prod.yml ps và logs --tail=100 <service> với env tương ứng. Dùng health, registry, log startup/Flyway và bài test nhỏ để kiểm tra sau cập nhật; không chỉ nhìn container Running. Log cần lọc dữ liệu nhạy cảm trước đưa vào báo cáo.
Quy trình khôi phục phải dựa backup đã thử và image tag trước; rollback image chưa chắc tương thích schema sau migration. Chưa kiểm chứng quy trình DR/rollback nên không cam kết tự phục hồi hoặc không mất dữ liệu. [Cần nhóm bổ sung log deploy, backup/restore thử và các lỗi đã xử lý.]

## 7.5 Hướng dẫn theo vai trò

### 7.5.1 Guest và đăng ký đăng nhập

Guest mở trang chủ/giới thiệu, tìm khóa và đọc thông tin công khai. Chọn đăng ký, nhập dữ liệu hợp lệ, gửi/xác minh OTP theo giao diện rồi đăng nhập bằng username/email hoặc Google. Khi sai dữ liệu cần sửa theo thông báo, không tạo tài khoản lặp để bỏ qua lỗi.
Hướng dẫn người dùng phân biệt đăng ký thành công với email verified. Quên mật khẩu dùng mã RESET_PASSWORD và xác nhận mật khẩu mới; logout kết thúc phiên frontend. Source hiện chưa chặn login theo verified nên cần ghi đây là hành vi/giới hạn thực tế khi demo. [Cần ảnh đăng ký, thư OTP, xác minh, login và lỗi.]

### 7.5.2 Học viên tham gia học làm bài và xem kết quả

Học viên mở danh sách, chọn khóa PUBLIC và đăng ký tham gia, vào My Courses để học. Mở lesson, xem tài nguyên, hoàn thành và tải lại để kiểm tra tiến độ; lịch/thông báo hiển thị theo course. Không có quyền thì kiểm tra enrollment thay vì sửa URL.
Với quiz, xem điều kiện, bắt đầu/tiếp tục, chọn đáp án, kiểm tra lưu tạm và nộp; kết quả hiện theo policy. Với code, chọn ngôn ngữ được phép, dùng Run để thử, Submit để tạo bài nộp và theo dõi kết quả; PENDING chưa phải điểm cuối. [Cần ảnh khóa học, lesson/progress, quiz và code với dữ liệu thật.]

### 7.5.3 Giảng viên tạo khóa học nội dung và đánh giá

Giảng viên vào quản lý khóa của mình, tạo thông tin/cấu trúc/lesson, kiểm tra nội dung, mời học viên, thêm lịch/thông báo. Tạo ngân hàng câu hỏi, quiz nháp rồi công bố khi đã kiểm tra; tạo problem, ngôn ngữ/giới hạn và test case trước khi cho làm.
AI preview hoặc file import cần rà lại nội dung/đáp án trước lưu. Khi đã có submission, sửa test có thể bị checkNoSubmissions chặn; không bỏ qua bằng xóa dữ liệu demo đang cần kiểm chứng. Xem kết quả qua analytics/leaderboard đã có API, không dựa GradeBookController rỗng hoặc PreviewImportPage mẫu. [Cần ảnh thao tác và kết quả thực.]

### 7.5.4 Admin quản lý hệ thống

Admin truy cập dashboard, tìm người dùng, quản lý trạng thái và tài khoản Admin theo quyền. Quản lý course gồm lock/unlock; quản lý category, blog/tag và xem hoạt động judge. Kiểm tra thay đổi bằng reload/API với dữ liệu thật.
Web Content phải giải thích rõ Inspire lưu tại trình duyệt, Founders chưa lưu backend; dashboard service lỗi không tự được hiểu là mọi số liệu bằng 0. Không sử dụng tài khoản Admin cho toàn bộ demo vì sẽ che lỗi phân quyền Instructor/Student. [Cần ảnh từng khu vực quản trị và một thử thiếu quyền từ role khác.]

### 7.5.5 Sử dụng blog và chatbot

Người dùng đọc bài public, đăng bài của mình, bình luận/reply một tầng và vote theo rule. Khi sửa/xóa, chọn đúng bài thuộc quyền; thử tự vote để thấy hạn chế theo source. Tag CRUD là phần Admin, không phải mọi tác giả đều quản lý được tag.
Chatbot nhận câu hỏi và ngữ cảnh phù hợp, trả text từng phần; New Chat/archive tách phiên, lịch sử theo API. AI sinh quiz/test case cần được xem là gợi ý và kiểm tra trước sử dụng. Không nhập token/password hoặc dữ liệu người khác vào prompt demo. [Cần ảnh blog, chat stream, lịch sử và AI preview thật, không tạo mockup.]

# CHƯƠNG 8 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 8.1 Kết luận

### 8.1.1 Kết quả thực hiện

Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).
Báo cáo đã mô tả yêu cầu, Use Case, dữ liệu, kiến trúc, code trọng tâm, kế hoạch kiểm thử và cài đặt theo commit đối chiếu. Kiểm tra bổ sung đã chạy các suite và bước build/package nêu tại 6.4.5; nguồn của 74 mô hình thiết kế được đặt trong tab “Nguồn sơ đồ UML”. Hình xuất và ảnh demo chưa được chèn; kết quả tự động chưa đủ để kết luận toàn ứng dụng đã nghiệm thu hoặc vận hành ổn định.

### 8.1.2 Mức độ đáp ứng mục tiêu và yêu cầu

Về phạm vi nghiệp vụ, source đáp ứng hướng xây dựng LMS thay cho đề tài bán hàng: học viên tham gia học/làm bài, giảng viên quản lý nội dung/đánh giá, Admin quản lý dữ liệu hệ thống. Về cấu trúc, controller–service–repository có trong các service nghiệp vụ; xác thực, OTP, JWT, OAuth, judge và AI có code triển khai hoặc cấu hình cụ thể.
Mức đáp ứng cuối cùng phải dựa kết quả thực tế, không chỉ sự có mặt của class/route. Bootstrap, role Manager độc lập, rule xác minh email lúc login, giới hạn API chọn role, owner chat history, preview import, custom judge và các giới hạn lưu nội dung đã được ghi rõ. Vì vậy chưa có căn cứ tự chấm đạt toàn bộ yêu cầu hoặc ghi một điểm tổng cuối cùng.

### 8.1.3 Đóng góp của từng thành viên

Đóng góp cá nhân cần xác định bằng phần chức năng đã thực hiện, file/commit/PR và khả năng giải thích/kiểm thử. Không dùng các tên người sáng lập trong frontend làm danh sách thành viên nhóm, và không phân chia công việc chỉ theo số dòng code.
[Cần nhóm bổ sung họ tên, MSSV, phần backend/frontend/test/báo cáo và minh chứng của từng thành viên tại A.1–A.4.] Kết luận đóng góp chỉ được chốt khi bảng phân công và minh chứng đã khớp, kể cả phần có AI hỗ trợ.

## 8.2 Hạn chế

### 8.2.1 Hạn chế chức năng

Một số thành phần chưa hoàn chỉnh hoặc chưa được nối vào tuyến API/UI hiện tại: GradeBookController rỗng, PreviewImportPage dữ liệu mẫu trong LockedFeature, CustomEngine chưa triển khai, Founders chưa lưu backend, Inspire Images lưu localStorage. Chưa có bằng chứng dùng template Bootstrap theo yêu cầu môn học.
Các hành vi cần đánh giá lại gồm login chưa kiểm tra verified, myinfo/role không dùng các rule chặt hơn của SelectRoleServiceImpl, tuyến đọc tin nhắn chat chưa thấy kiểm tra chủ phiên tương đương archive, cùng trường hợp problem không có test tự AC100. Các điểm này không được biến thành tính năng hoàn thiện trong phần kết quả.

### 8.2.2 Hạn chế kỹ thuật và triển khai

Hệ thống phụ thuộc nhiều service và provider: SMTP/OAuth/Calendar/Judge0/LLM/MinIO cùng MySQL/Mongo/Redis. API và cấu hình ở source chưa bảo đảm credential, network, timeout, schema và image production của môi trường nhóm đã đúng. Compose local còn tham chiếu thư mục init MySQL không có trong repo; Compose production không cung cấp database/storage.
Mô hình downstream tin header Gateway cần ranh giới mạng thực tế. Redis rate limit hiện fail-open khi Redis lỗi; token frontend lưu storage; rotation refresh không làm mọi access token đã phát hành mất hiệu lực ngay. Chưa đo tải/độ trễ hoặc thử backup/restore nên không cam kết hiệu năng, độ sẵn sàng và phục hồi.

### 8.2.3 Hạn chế kiểm thử và dữ liệu

Lượt chạy bổ sung đã có kết quả frontend test/build/lint/coverage và backend test/package (6.4.5 và D.2). Backend ghi nhận 237 ca, trong đó 236 đạt và một context test bị bỏ qua do cần MySQL/MongoDB. Frontend có 162 tests đạt nhưng coverage toàn phạm vi chỉ 8,13% lines, thấp hơn 91,90% của core. Test @WebMvcTest/@MockBean, jsdom và unit test không đại diện toàn stack; chưa nghiệm thu E2E, provider thật, responsive hoặc production. Chưa có ảnh khảo sát, sơ đồ UML xuất thành hình, ảnh demo theo role và log môi trường từ nhóm. Chưa có dữ liệu người dùng/nghiên cứu thực nghiệm để kết luận cải thiện kết quả học, tiết kiệm thời gian hoặc vượt trội nền tảng khảo sát. Dữ liệu demo và số liệu cá nhân cần bổ sung đúng nguồn.

## 8.3 Hướng phát triển

### 8.3.1 Hoàn thiện các chức năng còn thiếu

Ưu tiên hoàn thiện yêu cầu và tuyến gọi đang tồn tại: thống nhất role với giảng viên, xác nhận Bootstrap/phương án giao diện, chỉnh điều kiện xác minh email và role update phù hợp mục tiêu; bổ sung kiểm tra ownership cho những endpoint còn thiếu; làm rõ và kiểm thử analytics theo course.
Tiếp theo nối preview import thật hoặc loại khỏi luồng demo, lưu tập trung Web Content nếu yêu cầu, bỏ/đánh dấu custom judge chưa triển khai và xử lý problem không có test. Mỗi thay

đổi cần test hồi quy, ghi commit mới và cập nhật báo cáo. Đây là đề xuất phát triển, chưa được thực hiện trong đợt viết.

### 8.3.2 Cải thiện chất lượng học tập và đánh giá

Cải thiện đánh giá bằng bộ test case đủ dữ liệu biên, đối chiếu quiz với phép tính độc lập và hiển thị kết quả/giải thích phù hợp policy. Có thể nghiên cứu phản hồi lỗi code rõ hơn và thống kê tiến bộ theo thời gian khi đã xác minh độ đúng dữ liệu.
AI nên hỗ trợ gợi ý có bước review, kiểm tra định dạng và thử test trước công bố. Các tính năng như cá nhân hóa hoặc tìm kiếm bằng embedding chỉ đưa vào nghiên cứu tương lai, không mô tả là pipeline đã có trong source. Cần xác định tiêu chí đo và dữ liệu đánh giá trước tuyên bố cải thiện.

### 8.3.3 Cải thiện bảo mật hiệu năng và khả năng mở rộng

Về bảo mật, củng cố ranh giới Gateway/downstream, quyền trên dữ liệu, vòng đời token, xử lý log nhạy cảm, upload và rate limit; đánh giá chính sách khi dependency lỗi. Về triển khai, làm gói cài đặt tái lập, cố định image tag, chuẩn bị migration/init và thử phục hồi dữ liệu.
Về hiệu năng/mở rộng, đo các luồng quan trọng trước khi tối ưu: listing, quiz autosave/submit, hàng đợi judge và LLM stream. Theo dõi tài nguyên, timeout và lỗi, rồi mới lựa chọn cache, index hoặc tăng instance. Không xem thêm service hoặc tăng timeout là bằng chứng hệ thống đã mở rộng tốt.

# TÀI LIỆU THAM KHẢO

[1] MoodleDocs. Teacher quick guide. https://docs.moodle.org/503/en/Teacher_quick_guide
[2] MoodleDocs. Activities. https://docs.moodle.org/503/en/Activities
[3] MoodleDocs. Activity completion settings, nhánh 4.5.
https://docs.moodle.org/405/en/activity_completion_settings
[4] Google Classroom Help. Create an assignment.
https://support.google.com/edu/classroom/answer/6020265?hl=en
[5] Google Classroom Help. Create a quiz assignment.
https://support.google.com/edu/classroom/answer/9095575?hl=en
[6] Google Classroom Help. Grade & return an assignment.
https://support.google.com/edu/classroom/answer/6020294?hl=en
[7] Instructure Community. What are Modules?
https://community.instructure.com/en/kb/articles/662761-what-are-modules
[8] Instructure Community. What are Assignments?
https://community.instructure.com/en/kb/articles/662741-what-are-assignments
[9] Instructure Community. Instructor Getting Started Resources.
https://community.instructure.com/en/kb/articles/660621-instructor-getting-started-resources
[10] Instructure. Canvas, phần SpeedGrader. https://www.instructure.com/canvas
[11] Source đồ án LMSmau, main, commit 4288b95.
https://github.com/nguyenminhtri15022006-art/LMSmau
[12] Yêu cầu môn Lập trình Web và rubric do giảng viên cung cấp trong tài liệu nhóm. Các nguồn trực tuyến [1]–[10] được truy cập ngày 14/10/2026.

# PHỤ LỤC

## A Phân công và GitHub

### A.1 Thông tin thành viên và MSSV

Nhóm có tối đa ba sinh viên theo yêu cầu môn học. Bảng thông tin bên dưới là chỗ điền, không xác nhận nhóm thực tế có đủ ba người. Mỗi thành viên cần họ tên, MSSV, vai trò và thông tin lớp theo mẫu nộp.
[Cần nhóm xác nhận số người thực tế, tên đề tài chính thức, mã nhóm, lớp/mã học phần và giảng viên hướng dẫn. Không dùng tên từ nội dung mẫu DevNest/EduLearn để điền hộ.]

**Bảng A.1 Thông tin thành viên cần bổ sung**

| Thành viên thực tế | Họ tên/MSSV | Chức năng và minh chứng |
|---|---|---|
| Thành viên 1 | Nguyễn Minh Trí<br>MSSV: 24110359 | Trưởng nhóm, Phụ trách toàn diện kiến trúc hệ thống, phát triển Backend Microservices (Identity, Course, Assignment, Blog, Chatbot, Code Execution Engine), Xây dựng Giao diện Thymeleaf + Bootstrap 5 (web-portal, web-app), Tích hợp Spring Security, OAuth2, Email OTP, Chatbot SSE, Kiểm thử và Biên soạn Báo cáo. |
| Thành viên 1 | Nguyễn Minh Trí<br>MSSV: 24110359 | Trưởng nhóm, Phụ trách toàn diện kiến trúc hệ thống, phát triển Backend Microservices (Identity, Course, Assignment, Blog, Chatbot, Code Execution Engine), Xây dựng Giao diện Thymeleaf + Bootstrap 5 (web-portal, web-app), Tích hợp Spring Security, OAuth2, Email OTP, Chatbot SSE, Kiểm thử và Biên soạn Báo cáo. |

| Thành viên thực tế | Họ tên/MSSV | Chức năng và minh chứng |
|---|---|---|
| Thành viên 1 | Nguyễn Minh Trí<br>MSSV: 24110359 | Trưởng nhóm, Phụ trách toàn diện kiến trúc hệ thống, phát triển Backend Microservices (Identity, Course, Assignment, Blog, Chatbot, Code Execution Engine), Xây dựng Giao diện Thymeleaf + Bootstrap 5 (web-portal, web-app), Tích hợp Spring Security, OAuth2, Email OTP, Chatbot SSE, Kiểm thử và Biên soạn Báo cáo. |

## A.2 Ma trận vai trò chức năng và người phụ trách

Phân công phải nối từ thành viên đến chức năng, vai trò, source và kiểm thử, ví dụ tài khoản/xác thực; khóa học/lesson/tiến độ; quiz/code; blog/AI; Admin/hạ tầng. Một module có thể có người phối hợp nhưng cần người phụ trách chính rõ.
Nội dung Chương 5 là phân tích phần code hiện có, không chứng minh thành viên nào đã viết phần đó.

## A.3 Kế hoạch tiến độ

Kế hoạch cần tách mốc phát triển ứng dụng với mốc hoàn thiện báo cáo. Với mỗi mốc ghi công việc, người phụ trách, dự kiến, thực tế và minh chứng; không lấy ngày biên soạn tài liệu làm lịch phát triển giả.
Những ngày nộp hoặc báo cáo trong thông báo môn học phải kiểm tra lại theo lịch chính thức của lớp khi nộp; không tự gán ngày hoàn thành từng feature.

## A.4 Commit pull request và minh chứng đóng góp

Minh chứng mỗi đóng góp gồm SHA, ngày, tác giả, tóm tắt thay đổi, file/chức năng và liên kết commit/PR. Có thể lấy từ git log của bản đầy đủ, nhưng checkout dùng rà soát là shallow clone nên không đủ kết luận lịch sử cộng tác.
Không viết lại lịch sử, tạo commit giả hoặc đếm số commit như điểm chất lượng. Nếu code có phần kế thừa/từ template, cần mô tả nguồn và phần nhóm đã thay đổi; chỉ gán đóng góp khi có căn cứ thực tế.

## A.5 Phiên bản source dùng cho báo cáo

Repository đối chiếu: https://github.com/nguyenminhtri15022006-art/LMSmau. Nhánh: main. SHA đầy đủ: 4288b95884c031e669aab604ad019c12f46ff162. Thư mục ứng dụng: Learning-Management-System-main. Phiên bản này là mốc căn cứ của các phân tích trong báo cáo.
Trước nộp, nhóm ghi SHA chạy demo và so với mốc trên. Nếu khác, rà lại API, DTO, migration, role, đoạn code, số dòng và ảnh. Không ghi báo cáo đồng bộ bản mới nhất nếu chưa đối chiếu thay đổi.

# B Danh mục API

## B.1 Identity

Danh mục dưới đây lấy HTTP method và mapping từ controller Identity tại mốc source; tiền tố /api/identity dùng để mô tả đường Gateway dự kiến. Bao gồm cả /internal để tra cứu kiến trúc, không phải hướng dẫn công bố API nội bộ cho người dùng.
DTO/query, auth và response xem từng controller/service ở Chương 5. Danh mục mapping chưa chứng minh mọi route đã chạy hoặc được Gateway cho phép; cần kiểm tra config và runtime. Cập nhật self role đi UserServiceImpl.updateRole, không phải SelectRoleServiceImpl.

**Bảng B.1 Danh mục mapping Identity**

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| GET | /api/identity/admin/users | AdminUserController.java |
| GET | /api/identity/admin/users/stats | AdminUserController.java |
| GET | /api/identity/admin/users/{id} | AdminUserController.java |
| PATCH | /api/identity/admin/users/{id}/active | AdminUserController.java |
| POST | /api/identity/admin/users/admins | AdminUserController.java |
| PUT | /api/identity/admin/users/{id} | AdminUserController.java |
| PATCH | /api/identity/admin/users/{id}/password | AdminUserController.java |
| PATCH | /api/identity/admin/users/{id}/admin-active | AdminUserController.java |
| POST | /api/identity/auth/register | AuthenticationController.java |
| POST | /api/identity/auth/login | AuthenticationController.java |
| POST | /api/identity/auth/social-login/{provider} | AuthenticationController.java |
| POST | /api/identity/auth/introspect | AuthenticationController.java |
| POST | /api/identity/auth/refresh | AuthenticationController.java |
| POST | /api/identity/auth/logout | AuthenticationController.java |
| POST | /api/identity/auth/verify-email/send | EmailVerificationController.java |
| POST | /api/identity/auth/verify-email | EmailVerificationController.java |
| POST | /api/identity/auth/forgot-password | EmailVerificationController.java |
| POST | /api/identity/auth/reset-password | EmailVerificationController.java |
| GET | /api/identity/internal/user/{id} | InternalUserController.java |
| POST | /api/identity/internal/users/batch | InternalUserController.java |
| GET | /api/identity/internal/users/{userId}/exists | InternalUserController.java |
| GET | /api/identity/internal/users/search/by-email | InternalUserController.java |
| POST | /api/identity/storage/presigned-url | StorageController.java |

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| GET | /api/identity/users/myinfo | UserController.java |
| PUT | /api/identity/users/myinfo | UserController.java |
| PATCH | /api/identity/users/myinfo/password | UserController.java |
| PATCH | /api/identity/users/myinfo/role | UserController.java |
| GET | /api/identity/users/{userId}/profile | UserController.java |

## B.2 Course

Danh mục Course gồm khóa học, category, cấu trúc, lesson/progress, lịch/thông báo, storage và internal. Các API private shortcut my/enrolled phải phân biệt route public courses/{id}; role và quyền owner xem service.
Tiền tố /api/course được ghép với controller mapping để đối chiếu. Gateway còn có route /api/storage/**; cần kiểm tra StripPrefix và context-path khi dùng alias, không mặc định hai đường đều chạy. Chưa có test runtime thì không ghi URL alias là đã xác nhận.

**Bảng B.2 Danh mục mapping Course**

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| GET | /api/course/courses/{courseId}/announcements | AnnouncementController.java |
| POST | /api/course/courses/{courseId}/announcements | AnnouncementController.java |
| DELETE | /api/course/courses/{courseId}/announcements/{announcementId} | AnnouncementController.java |
| GET | /api/course/categories | CategoryController.java |
| POST | /api/course/categories | CategoryController.java |
| PUT | /api/course/categories/{id} | CategoryController.java |
| DELETE | /api/course/categories/{id} | CategoryController.java |
| GET | /api/course/courses | CourseController.java |
| GET | /api/course/courses/{id} | CourseController.java |
| POST | /api/course/courses | CourseController.java |
| PUT | /api/course/courses/{id} | CourseController.java |
| DELETE | /api/course/courses/{id} | CourseController.java |
| GET | /api/course/courses/my | CourseController.java |
| GET | /api/course/courses/enrolled | CourseController.java |
| POST | /api/course/courses/{id}/enroll | CourseController.java |
| POST | /api/course/courses/{id}/invite | CourseController.java |
| PATCH | /api/course/courses/{id}/lock | CourseController.java |

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| PATCH | /api/course/courses/{id}/unlock | CourseController.java |
| GET | /api/course/courses/my/stats | CourseController.java |
| GET | /api/course/courses/my/recent-enrollments | CourseController.java |
| GET | /api/course/courses/{id}/students/count | CourseController.java |
| GET | /api/course/courses/{id}/lessons/count | CourseController.java |
| GET | /api/course/courses/{id}/students | CourseController.java |
| DELETE | /api/course/courses/{id}/students/{userId} | CourseController.java |
| GET | /api/course/courses/users/search | CourseController.java |
| GET | /api/course/courses/{courseId}/structure | CourseStructureController.java |
| POST | /api/course/courses/{courseId}/structure/nodes | CourseStructureController.java |
| POST | /api/course/courses/{courseId}/structure/nodes/batch | CourseStructureController.java |
| PUT | /api/course/courses/{courseId}/structure/reorder | CourseStructureController.java |
| GET | /api/course/courses/{courseId}/structure/lessons | CourseStructureController.java |
| PATCH | /api/course/courses/{courseId}/structure/nodes/{nodeId} | CourseStructureController.java |
| DELETE | /api/course/courses/{courseId}/structure/nodes/{nodeId} | CourseStructureController.java |
| GET | /api/course/internal/courses/{courseId} | InternalCourseController.java |
| GET | /api/course/internal/courses/{courseId}/lessons/{lessonId} | InternalCourseController.java |
| GET | /api/course/internal/courses/{courseId}/enrollments/{userId} | InternalCourseController.java |
| GET | /api/course/internal/courses/{courseId}/students/count | InternalCourseController.java |
| GET | /api/course/internal/courses/{courseId}/exists | InternalCourseController.java |
| GET | /api/course/internal/courses/{courseId}/lessons/{lessonId}/exists | InternalCourseController.java |
| GET | /api/course/internal/courses | InternalCourseController.java |
| GET | /api/course/internal/courses/enrolled | InternalCourseController.java |
| GET | /api/course/courses/{courseId}/lessons/{lessonId} | LessonController.java |
| PUT | /api/course/courses/{courseId}/lessons/{lessonId} | LessonController.java |
| POST | /api/course/courses/{courseId}/lessons/{lessonId}/complete | LessonProgressController.java |
| GET | /api/course/courses/{courseId}/progress | LessonProgressController.java |
| GET | /api/course/courses/{courseId}/progress/average | LessonProgressController.java |
| GET | /api/course/courses/{courseId}/schedules | ScheduleController.java |
| POST | /api/course/courses/{courseId}/schedules | ScheduleController.java |

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| PUT | /api/course/courses/{courseId}/schedules/{scheduleId} | ScheduleController.java |
| DELETE | /api/course/courses/{courseId}/schedules/{scheduleId} | ScheduleController.java |
| POST | /api/course/storage/presigned-url | StorageController.java |
| DELETE | /api/course/storage/file | StorageController.java |
| POST | /api/course/storage/multipart/initiate | StorageController.java |
| POST | /api/course/storage/multipart/complete | StorageController.java |

## B.3 Assignment

Danh mục Assignment gồm question/quiz/attempt, problem/test case, submission, analytics, leaderboard, template và internal. GradeBookController hiện không có method mapping nên không bổ sung API /gradebook giả vào bảng.
Các path rút gọn ở Chương 5 phải được thay bằng method/path đầy đủ của bảng khi gọi thử. PreviewImportPage dữ liệu mẫu không chứng minh import cả đề bài; backend đã xác định luồng import test case. Không suy HTTP202 từ comment của submit service.

**Bảng B.3 Danh mục mapping Assignment**

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| GET | /api/assignment/admin/code-judge/activity | AdminCodeJudgeActivityController.java |
| GET | /api/assignment/courses/{courseId}/lesson-counts | AssignmentLessonSummaryController.java |
| POST | /api/assignment/submissions | CodeJudgeController.java |
| POST | /api/assignment/submissions/run | CodeJudgeController.java |
| GET | /api/assignment/submissions/latest-accepted | CodeJudgeController.java |
| GET | /api/assignment/submissions/{id} | CodeJudgeController.java |
| GET | /api/assignment/submissions | CodeJudgeController.java |
| GET | /api/assignment/instructor/courses/{courseId}/analytics | InstructorAssignmentAnalyticsController.java |
| GET | /api/assignment/instructor/courses/{courseId}/submissions | InstructorAssignmentAnalyticsController.java |
| GET | /api/assignment/internal/problems/{problemId} | InternalController.java |
| GET | /api/assignment/internal/submissions/latest | InternalController.java |
| GET | /api/assignment/internal/courses/{courseId}/exercise-count | InternalController.java |
| POST | /api/assignment/internal/courses/exercise-counts | InternalController.java |
| GET | /api/assignment/courses/{courseId}/leaderboard/coding | LeaderboardController.java |
| GET | /api/assignment/courses/{courseId}/leaderboard/quiz | LeaderboardController.java |

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| POST | /api/assignment/problems | ProblemController.java |
| GET | /api/assignment/problems | ProblemController.java |
| GET | /api/assignment/problems/{problemId} | ProblemController.java |
| GET | /api/assignment/problems/slug/{slug} | ProblemController.java |
| PUT | /api/assignment/problems/{problemId} | ProblemController.java |
| PATCH | /api/assignment/problems/{problemId}/unpublish | ProblemController.java |
| POST | /api/assignment/problems/{problemId}/clone | ProblemController.java |
| DELETE | /api/assignment/problems/{problemId} | ProblemController.java |
| PATCH | /api/assignment/problems/{problemId}/restore | ProblemController.java |
| POST | /api/assignment/questions/courses/{courseId} | QuestionController.java |
| POST | /api/assignment/questions/courses/{courseId}/import-file | QuestionController.java |
| POST | /api/assignment/questions/courses/{courseId}/import-file/preview | QuestionController.java |
| POST | /api/assignment/questions/courses/{courseId}/ai-generate/preview | QuestionController.java |
| POST | /api/assignment/questions/courses/{courseId}/batch | QuestionController.java |
| GET | /api/assignment/questions/courses/{courseId} | QuestionController.java |
| GET | /api/assignment/questions/courses/{courseId}/topics | QuestionController.java |
| GET | /api/assignment/questions/{questionId} | QuestionController.java |
| PUT | /api/assignment/questions/{questionId} | QuestionController.java |
| DELETE | /api/assignment/questions/{questionId} | QuestionController.java |
| POST | /api/assignment/quizzes/{quizId}/attempts/start-or-resume | QuizAttemptController.java |
| GET | /api/assignment/quizzes/{quizId}/attempts/{attemptId} | QuizAttemptController.java |
| PUT | /api/assignment/quizzes/{quizId}/attempts/{attemptId} | QuizAttemptController.java |
| POST | /api/assignment/quizzes/{quizId}/attempts/{attemptId}/practice-check | QuizAttemptController.java |
| POST | /api/assignment/quizzes/{quizId}/attempts/{attemptId}/submit | QuizAttemptController.java |
| GET | /api/assignment/quizzes/{quizId}/attempts/{attemptId}/result | QuizAttemptController.java |
| GET | /api/assignment/quizzes/{quizId}/attempts | QuizAttemptController.java |
| POST | /api/assignment/quizzes/courses/{courseId} | QuizController.java |
| POST | /api/assignment/quizzes/courses/{courseId}/imported | QuizController.java |

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| GET | /api/assignment/quizzes/courses/{courseId} | QuizController.java |
| GET | /api/assignment/quizzes/instructor/courses/{courseId} | QuizController.java |
| GET | /api/assignment/quizzes/student/courses/{courseId} | QuizController.java |
| GET | /api/assignment/quizzes/{quizId} | QuizController.java |
| GET | /api/assignment/quizzes/{quizId}/questions | QuizController.java |
| PUT | /api/assignment/quizzes/{quizId} | QuizController.java |
| PUT | /api/assignment/quizzes/{quizId}/sync | QuizController.java |
| DELETE | /api/assignment/quizzes/{quizId} | QuizController.java |
| PATCH | /api/assignment/quizzes/{quizId}/publish | QuizController.java |
| PATCH | /api/assignment/quizzes/{quizId}/unpublish | QuizController.java |
| PATCH | /api/assignment/quizzes/{quizId}/restore | QuizController.java |
| POST | /api/assignment/quizzes/{quizId}/questions | QuizController.java |
| POST | /api/assignment/quizzes/{quizId}/clone | QuizController.java |
| DELETE | /api/assignment/quizzes/{quizId}/questions/{questionId} | QuizController.java |
| GET | /api/assignment/templates/quiz-import | QuizTemplateController.java |
| GET | /api/assignment/templates/testcases | QuizTemplateController.java |
| POST | /api/assignment/storage/presigned-url | StorageController.java |
| DELETE | /api/assignment/storage/file | StorageController.java |
| POST | /api/assignment/problems/{problemId}/testcases | TestCaseController.java |
| POST | /api/assignment/problems/{problemId}/testcases/bulk | TestCaseController.java |
| POST | /api/assignment/problems/{problemId}/testcases/import | TestCaseController.java |
| POST | /api/assignment/problems/testcases/ai-generate/preview | TestCaseController.java |
| PUT | /api/assignment/problems/{problemId}/testcases/sync | TestCaseController.java |
| GET | /api/assignment/problems/{problemId}/testcases | TestCaseController.java |
| PUT | /api/assignment/problems/{problemId}/testcases/{testId} | TestCaseController.java |
| DELETE | /api/assignment/problems/{id}/testcases | TestCaseController.java |

## B.4 Blog

Danh mục Blog phân biệt /posts/public, /posts/me, /posts/admin, bài theo ID/slug, comment, vote và tag. Post create/update dùng multipart theo controller; public GET không tương đương quyền sửa.
Tag CRUD thuộc Admin; Post/Comment có kiểm tra tác giả và ngoại lệ theo service. Những API internal chỉ phục vụ đối chiếu kiến trúc. Khi demo nên gọi qua Gateway và ghi role/ID đối tượng để kiểm tra quyền thật.

**Bảng B.4 Danh mục mapping Blog**

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| GET | /api/blog/posts/{postId}/comments | CommentController.java |
| POST | /api/blog/posts/{postId}/comments | CommentController.java |
| PUT | /api/blog/posts/{postId}/comments/{commentId} | CommentController.java |
| DELETE | /api/blog/posts/{postId}/comments/{commentId} | CommentController.java |
| GET | /api/blog/internal/posts/{postId} | InternalBlogController.java |
| GET | /api/blog/internal/posts/{postId}/exists | InternalBlogController.java |
| GET | /api/blog/posts/public | PostController.java |
| GET | /api/blog/posts/public/{slug} | PostController.java |
| GET | /api/blog/posts/admin | PostController.java |
| GET | /api/blog/posts/{id} | PostController.java |
| GET | /api/blog/posts/me | PostController.java |
| POST | /api/blog/postsmultipart/form-data | PostController.java |
| PUT | /api/blog/posts/{id} | PostController.java |
| DELETE | /api/blog/posts/{id} | PostController.java |
| GET | /api/blog/tags | TagController.java |
| GET | /api/blog/tags/{id} | TagController.java |
| GET | /api/blog/tags/search | TagController.java |
| POST | /api/blog/tags | TagController.java |
| PUT | /api/blog/tags/{id} | TagController.java |
| DELETE | /api/blog/tags/{id} | TagController.java |
| POST | /api/blog/posts/{postId}/vote | VoteController.java |
| POST | /api/blog/posts/{postId}/comments/{commentId}/vote | VoteController.java |

### B.5 Chatbot

Danh mục Chatbot gồm chat SSE, generate trực tiếp, sessions/messages/archive và internal generation/health. Chat dùng POST với phản hồi text/event-stream; các chunks không phải WebSocket.
Direct generate quiz/test case có INSTRUCTOR ở ChatController; Assignment AI preview có luồng và quyền khác. Danh sách này không thay kiểm tra ownership cho messages; xem điểm còn thiếu tại 3.3/4.6/6.3.

**Bảng B.5 Danh mục mapping Chatbot**

| HTTP | Đường mapping quy đổi qua Gateway | Controller nguồn |
|---|---|---|
| POST | /api/chatbot/chat | ChatController.java |
| POST | /api/chatbot/generate/quiz | ChatController.java |
| POST | /api/chatbot/generate/testcase | ChatController.java |
| GET | /api/chatbot/sessions | ChatController.java |
| GET | /api/chatbot/sessions/{sessionId}/messages | ChatController.java |
| POST | /api/chatbot/sessions/{sessionId}/archive | ChatController.java |
| GET | /api/chatbot/internal/health/redis | HealthController.java |
| GET | /api/chatbot/internal/health | InternalController.java |
| POST | /api/chatbot/internal/generate/quiz | InternalController.java |
| POST | /api/chatbot/internal/generate/testcase | InternalController.java |

### B.6 Gateway và health check

Gateway có @RequestMapping /fallback/{serviceName} trong GatewayFallbackController; được circuit breaker forward khi dependency lỗi, không phải chức năng quản trị tự sửa hệ thống. Actuator health/info được exposure theo từng application.yaml, không lấy từ controller mapping thông thường.
Gateway health local mặc định /actuator/health/liveness; service có context /identity, /course, /assignment, /blog, /chatbot. Các route /api/<service>/actuator/** phụ thuộc cấu hình Gateway/health URI. Ghi health trả gì và endpoint kiểm tra gì, không dùng UP để khẳng định gửi OTP, chấm bài hoặc LLM hoạt động.

## C Sử dụng AI trong quá trình thực hiện

### C.1 Công cụ AI và mục đích sử dụng

Phân biệt hai phạm vi AI: công cụ hỗ trợ nhóm lập trình/viết tài liệu và tính năng AI tích hợp trong LMS. Báo cáo này có hỗ trợ rà source, tổ chức nội dung và soạn giải thích; điều đó không chứng minh nhóm đã dùng cùng công cụ để viết mọi phần source.
[Cần nhóm bổ sung công cụ/model thực tế, thời điểm, công việc, người sử dụng, đầu ra và cách kiểm tra.] Không gán prompt của cuộc trao đổi báo cáo cho lịch sử phát triển ứng dụng nếu không có bằng chứng.

## C.2 Danh sách prompt thực tế sinh code

Chỉ đưa prompt đã sử dụng thật. Mỗi bản ghi cần mã prompt, thời điểm, thành viên, mục tiêu, ngữ cảnh/code đầu vào đã che secret, prompt, phần đầu ra được giữ và thay đổi sau review/test.
[Cần nhóm cung cấp lịch sử prompt sinh code thực tế.] Chưa có dữ liệu nên không tự viết vài prompt mẫu rồi trình bày như đã dùng. Nếu muốn thêm prompt gợi ý cho tương lai phải ghi rõ là đề xuất, không là minh chứng.

## C.3 Danh sách prompt thực tế tạo hình và sơ đồ

Theo yêu cầu hiện tại của nhóm, chưa tự tạo ảnh để thay screenshot hoặc hình chứng minh chức năng. Các sơ đồ mới có mô tả nội dung cần vẽ, chưa có hình xuất từ công cụ UML; ảnh khảo sát và demo do nhóm bổ sung.
Nếu nhóm thực sự dùng AI hỗ trợ tạo sơ đồ, ghi prompt thật, file nguồn, phần kiểm tra với code và hình xuất cuối cùng. Không ghi đã sinh ảnh hoặc đã dùng công cụ cụ thể khi chưa thực hiện.

## C.4 Cách kiểm tra sửa và làm chủ code AI

Quy trình kiểm chứng là đọc yêu cầu, kiểm tra caller/controller–service–repository, đối chiếu DTO/migration, rà quyền và trường hợp biên, chạy test liên quan, rồi mới giữ thay đổi.
Thành viên phải giải thích được dữ liệu vào/ra và tác động lên trạng thái, không chỉ đọc lại tên hàm.
Ví dụ rà source đã phát hiện SelectRoleServiceImpl không phải tuyến myinfo/role đang gọi, GradeBookController rỗng, PreviewImportPage mẫu và problem không test auto AC. Những phát hiện này được sửa trong nội dung báo cáo; không ghi nhóm đã sửa source nếu chưa có commit thay đổi.

## C.5 Giới hạn và kết quả sử dụng AI

AI hỗ trợ tăng tốc soạn và rà soát, nhưng có thể gán nhầm hành vi của class chưa được dùng, suy từ comment hoặc bỏ sót điều kiện endpoint. Vì vậy báo cáo ưu tiên tuyến gọi thực và giữ các mục chưa xác minh.
Chưa có đo thời gian/năng suất hoặc dữ liệu để đánh giá tỷ lệ code do AI tạo. Tiêu chí AI của rubric cần minh chứng làm chủ code, chỉnh sửa và kiểm thử của từng thành viên; chưa tự chấm điểm tối đa.

# D Minh chứng kiểm thử và giao diện

## D.1 Bảng test case chi tiết

Chương 5 có 188 mã TC dạng TC-5-<module>-<chức năng>-01..04 cho 47 chức năng. Đây là nhóm kịch bản ở mức chức năng, chưa phải danh sách lượt chạy đạt. Một TC có nhiều biến thể dữ liệu phải tách bản ghi thực hiện khi chạy.
Mẫu biên bản: mã TC | biến thể/input | role/owner | điều kiện trước | bước | kỳ vọng | thực tế | trạng thái | người/ngày | minh chứng. Dùng Đạt/Không đạt/Chưa kiểm thử/Bị chặn đúng nghĩa. Không sao chép một ảnh thành bằng chứng cho tất cả biến thể.

## D.2 Kết quả test và build

Bảng 6.2 tại mục 6.4.5 đã cập nhật lượt chạy frontend thực tế ngày 16/10/2026 (UTC+7). Backend, API xuyên stack, dịch vụ ngoài và responsive có trạng thái riêng. Khi nhóm tái chạy, ghi command, working directory, phiên bản công cụ, SHA, exit code và output; không ghi đè kết quả cũ nếu môi trường hoặc commit đã thay đổi.

Không đưa báo cáo coverage không đúng scope vào cột toàn ứng dụng. Build Docker backend dùng -DskipTests không chứng minh JUnit pass. Các mục ảnh/log cần bổ sung đã liệt kê tiếp dưới đây; chỉ đổi trạng thái khi có bằng chứng.

Minh chứng cần bổ sung sau đợt 7

• Ảnh/lưu output java -version, mvn -version, node --version, npm --version và docker compose version tại máy nhóm, cùng SHA source đã chạy.

Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:

1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).

• Ảnh Compose ps, registry, health và Flyway; log không chứa secret. Kiểm tra thư mục init MySQL thiếu bằng cách chuẩn bị database đúng env, ghi cách nhóm thực sự tạo.

• Demo restore SQL, Mongo và tệp MinIO ở môi trường riêng nếu có tuyên bố backup/khôi phục. Cần ảnh viewport mobile/desktop, kiểm thử hai role/owner và kết quả tích hợp ngoài thật.

• Với production, bổ sung domain/image tag đã dùng và kết quả xác minh. Các lệnh trong Chương 7 hiện là hướng dẫn theo source, không phải biên bản đã triển khai thành công.

**Biên bản chạy tự động bổ sung ngày 16/10/2026 (UTC+7)**

Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:

1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app
   Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).

Frontend, working directory frontend: npm ci --ignore-scripts --no-audit --no-fund hoàn thành (420 packages); npm test, npm run build, npm run lint, npm run test:coverage và npm run test:coverage:core đều exit 0. Việc dùng --ignore-scripts là điều kiện của lượt cài dependency; build/test sau đó đã thực thi riêng.

Trích output npm test: “Test Files 21 passed (21)”; “Tests 162 passed (162)”. Build: “1025 modules transformed”; “built in 6.45s”. Lint: “17 problems (0 errors, 17 warnings)”. Các thời lượng là quan sát một lần, không phải benchmark hiệu năng.

Lint gồm 13 cảnh báo react-hooks/exhaustive-deps và 4 react-refresh/only-export-components. Các file có cảnh báo gồm ChatBot.jsx, Toast.jsx, Spring Session & Security Context.jsx, ProblemContext.jsx, InstructorExerciseDetail.jsx, ExerciseListingPage.jsx, CreateCodingPage.jsx, ExerciseQuizPage.jsx, ExerciseQuizResultPage.jsx, QuizHistoryDetailPage.jsx, QuizOverviewPage.jsx, CreateQuizPage.jsx, SocialCallbackPage.jsx, BlogDetailPage.jsx, MyBlogPostsPage.jsx và CurriculumModals.jsx; ExerciseListingPage có hai vị trí. Exit 0 không có nghĩa không còn cảnh báo.

Backend, working directory backend: chạy bash identity-service/mvnw -B -ntp -f pom.xml test với Java 21. Môi trường ban đầu vướng DNS/proxy khi tải dependency và Mockito không tự attach agent; lượt thành công dùng cấu hình proxy/truststore riêng của môi trường và tham số -DargLine=-javaagent:<đường-dẫn-byte-buddy-agent-1.14.19.jar>. Không đưa đường dẫn proxy hoặc cấu hình riêng này vào yêu cầu ứng dụng phải dùng trên mọi máy; không sửa POM/source để có kết quả.

Kết quả Surefire theo module (tests / failure / error / skipped): service-registry 1/0/0/0; api-gateway 14/0/0/0; identity-service 15/0/0/0; course-service 45/0/0/0; assignment-service 74/0/0/1; blog-service 36/0/0/0; chatbot-service 52/0/0/0. Tổng 237 ca được ghi nhận, 236 đạt, 0 failure, 0 error và 1 skipped. Reactor có BUILD SUCCESS. Đếm từ TEST-*.xml, không suy từ số file Java.

Ca skipped: AssignmentServiceApplicationTests.contextLoads; lý do khai báo “Requires local MySQL/MongoDB infrastructure; run with the Docker stack for full-context verification.” Vì vậy chưa nghiệm thu full context của Assignment hay stack dữ liệu chỉ từ BUILD SUCCESS.

Lượt đóng gói riêng: bash identity-service/mvnw -B -ntp -f pom.xml -DskipTests package, Java 21; exit 0, các module SUCCESS. Lượt này chứng minh đóng gói theo POM, không chứng minh test mới được chạy, image Docker đã build hoặc server đã khởi động.

Gói LMS_nguon_sodo_va_ketqua_kiemtra.zip đã được bàn giao cùng báo cáo, gồm 74 nguồn .puml, log test/build/lint/coverage, bảng 237 ca backend trích từ Surefire và README tái hiện. Log có che các mẫu dữ liệu nhạy cảm; CSV không chứa system properties của XML. Nhóm tải gói, lưu vào thư mục minh chứng của nhóm và bổ sung liên kết nộp thật khi có. Các lượt tái chạy trên commit/môi trường mới cần được lưu riêng.

Giới hạn kết luận: test có mock/jsdom, coverage chưa bao phủ phần lớn giao diện và tích hợp; chưa chạy Docker Compose, API thật, SMTP/OAuth/Meet/LLM/Judge0, kiểm tra responsive hoặc production. Các MC tương ứng tại D.3 giữ trạng thái chưa nghiệm thu cho đến khi có kết quả.

## D.3 Danh sách hình giao diện

**Quy ước thu minh chứng**

Mỗi mã MC dưới đây là một bộ bằng chứng, có thể gồm nhiều ảnh. Tên file đề xuất: MC01_dangky_thanhcong.png, MC01_dangky_trungemail.png. Ghi kèm role, commit, URL/route, ID dữ liệu demo, bước thao tác, kết quả thực tế và ngày chạy. Ảnh phải đọc được nội dung, tránh chụp cả IDE/trình duyệt khi phần kết quả quá nhỏ. Che mật khẩu, token, secret và dữ liệu riêng; không thay nội dung kết quả bằng ảnh tự dựng.

Chuẩn bị chung: hai học viên A/B, hai giảng viên X/Y và một Admin; khóa học thuộc X/Y; một course PUBLIC, một PRIVATE/LOCKED; một quiz có đáp án đã kiểm chứng; một bài code có ít nhất một test public và một test hidden. Giữ cùng courseId, quizId, problemId và user demo qua cả chuỗi ảnh để đối chiếu.

**Nhóm xác thực và tài khoản — Chương 5, nhóm 5.1**

MC01 — Đăng ký: đã nhận hai ảnh do nhóm cung cấp ngày 24/10/2026, gồm form đăng ký vai trò Học viên và màn hình thông báo đăng ký thành công (Hình 5.1–5.2, mục 5.1.1). Theo nhóm, phiên

bản đang chụp chưa tích hợp kiểm tra lỗi trùng email/username; chưa có minh chứng cho nhánh này. Chưa xác nhận trạng thái database hoặc xác minh email từ hai ảnh.

MC02 — OTP xác minh email: chụp yêu cầu gửi, email nhận thực tế và kết quả xác minh; thử mã sai, hết hạn hoặc đã dùng. Che giá trị mã còn hiệu lực; giữ thời điểm gửi/nhận và trạng thái để chứng minh luồng SMTP.

MC03 — Đăng nhập/đăng xuất: chụp điều hướng sau login theo STUDENT, INSTRUCTOR, ADMIN; thử sai mật khẩu và tài khoản bị khóa; logout rồi mở lại một URL bảo vệ. Ghi cả lỗi trả về và màn hình nhận được.

MC04 — Quên/đổi mật khẩu: tách hai kịch bản. Quên mật khẩu: email OTP, đặt lại, thử mật khẩu cũ/mới. Đổi mật khẩu: nhập mật khẩu hiện tại và xác nhận; thử refresh token cũ sau đổi nếu kiểm tra bảo mật. Không kết luận resetPassword và updatePassword có cùng cơ chế thu hồi token khi chưa kiểm chứng.

MC05 — Google OAuth/chọn role: chụp callback thành công và vai trò sau chọn; kiểm tra thông tin user và quyền request tiếp theo. Ghi token có được cấp lại hay chỉ DB đổi role; không tuyên bố điều kiện 30 phút/một lần được áp dụng tại /users/myinfo/role chỉ từ SelectRoleServiceImpl.

MC06 — Profile: đã có ảnh trang hồ sơ của tài khoản mang nhãn INSTRUCTOR (Hình 5.5, mục 5.1.8). Chưa có minh chứng sửa thông tin/ảnh đại diện và mở lại sau reload; phần lưu dữ liệu/upload còn chờ kiểm tra. Không dùng ảnh hồ sơ thay dashboard giảng viên.

**Nhóm khóa học và học tập — nhóm 5.2**

MC07 — Guest: đã có hai ảnh trang chủ (Hình 3.73–3.74), danh sách khóa học (Hình 5.3) và chi tiết khóa học (Hình 5.4), nhận ngày 24/10/2026. Các ảnh minh họa giao diện; thao tác tìm/lọc/phân trang, nguồn dữ liệu hiển thị và xử lý truy cập course PRIVATE/LOCKED chưa được nghiệm thu.

MC08 — Enrollment: học viên A tham gia course PUBLIC, mở khóa học của tôi; thử tham gia trùng và course không đủ trạng thái. Ghi courseId và enrollment/trạng thái nhận được.

MC09 — Lesson/tiến độ: mở lesson thuộc course đã học, hoàn thành và reload; dùng học viên B chưa tham gia và lessonId thuộc course khác thử quyền. Cần ảnh kết quả tiến độ và response từ chối tương ứng, chưa mặc định HTTP status khi chưa chạy.

MC10 — Giảng viên: tạo/sửa course, cấu trúc và lesson; reload để chứng minh đã lưu; dùng giảng viên Y thử sửa course của X. Chụp một nhánh validation và một nhánh sai owner.

MC11 — Thông báo/lịch học: tạo thông báo, lịch có start/end hợp lệ và xem bằng học viên đã tham gia. Thử end không sau start hoặc lịch bắt đầu quá khứ. Ghi scheduleId và courseId để kiểm tra ràng buộc tài nguyên.

MC12 — Meet: chụp lịch, phòng họp thực sự mở được và kết quả tạo sự kiện nếu có. URL meet.google.com/lms-... dự phòng với eventId null không đủ bằng chứng Google Calendar/Meet đã tích hợp thành công.

**Nhóm quiz và chấm code — nhóm 5.3**

MC13 — Ngân hàng câu hỏi/quiz: tạo SINGLE, MULTIPLE, TRUE_FALSE; tạo quiz nháp, thêm câu hỏi, công bố và mở bằng học viên. Ghi tổng điểm, trọng số, giới hạn lượt/thời gian và policy hiển thị kết quả.

MC14 — Làm quiz: start, chọn đáp án, autosave, reload/resume và submit. Chụp attemptId, đáp án/kết quả cùng điểm; thử bài đóng/hết lượt và tài khoản khác đọc attempt. Không dùng điểm có sẵn trên UI thay chứng cứ lượt làm.

MC15 — Đối chiếu chấm quiz: dùng bộ đáp án chuẩn, tự tính điểm kỳ vọng và so với kết quả; thử MULTIPLE thiếu/thừa lựa chọn. Chỉ ghi đạt khi điểm và quy tắc đã khớp, gồm làm tròn khi có trọng số.

MC16 — Tạo bài code/test case: giảng viên tạo problem, chọn language, tạo public/hidden test; chụp trạng thái sau reload. Nếu import, chụp file mẫu đã điền, request multipart thật và danh sách test được lưu; PreviewImportPage có ba hàng mẫu/LockedFeature không chứng minh import hoàn thiện.

MC17 — Run: code + custom input + output thực tế. Ghi ngôn ngữ và phân biệt đây là chạy thử, không dùng ảnh Run làm bằng chứng đã nộp bài.

MC18 — Submit/judge: dùng problem có test thật; chụp submissionId và kết quả AC cho lời giải đúng, WA cho lời giải sai, CE cho code không biên dịch. Nếu lỗi engine, lưu lỗi thực tế. Bài không test được trả AC/100 theo nhánh hiện có không chứng minh judge chấm đúng.

MC19 — Analytics/leaderboard: chụp dữ liệu sau các lượt nộp ở MC14/MC18; ghi bộ lọc, courseId và quyền người xem. Dùng endpoint analytics thực tế, không gọi GradeBookController rỗng là chức năng bảng điểm đã triển khai.

**Nhóm blog, chatbot và quản trị — nhóm 5.4–5.7**

MC20 — Blog: tạo draft, công bố, sửa; Guest không đọc draft; người khác không sửa bài của tác giả. Chụp trạng thái trước/sau với cùng postId.

MC21 — Comment/vote/tag: comment gốc và một tầng reply, thử tự vote và reply sâu hơn; Admin quản lý tag. Ghi hành vi đúng theo source, không áp quy tắc bình luận 50 ký tự của đề tài bán hàng nếu DTO không yêu cầu.

MC22 — Chat SSE: gửi câu hỏi, ghi phản hồi hiển thị theo luồng và phiên được lưu; reload mở lại lịch sử. Có thể kèm Network thể hiện text/event-stream nhưng phải che Authorization. Không gọi SSE là WebSocket.

MC23 — Quyền lịch sử chat: tạo phiên của A, dùng B truy cập sessionId đó qua API; lưu status/body đã che dữ liệu. Đây là ca cần kiểm tra vì luồng đọc getMessages chưa thấy kiểm tra owner tương đương archive; không ghi cách ly dữ liệu an toàn trước khi có kết quả.

MC24 — AI quiz/test case: dùng INSTRUCTOR gọi đúng luồng preview, rà soát nội dung, chỉnh sửa rồi lưu qua API tương ứng; chụp preview và bản ghi đã lưu. Nếu API ngoài lỗi, ghi bị chặn; không dùng dữ liệu mẫu làm kết quả LLM thật.

MC25 — Admin: tìm kiếm user, khóa/mở tài khoản; quản lý course/category/blog theo màn hình và API hiện có. Thử tài khoản STUDENT mở route/API Admin để quan sát từ chối; ghi khác biệt giữa guard frontend và quyền backend.

MC26 — Web Content: tách category dùng API, InspireImages lưu localStorage và Founders/state. Chụp reload và mở bằng trình duyệt khác để xác định phạm vi lưu; không trình bày tất cả như dữ liệu dùng chung trên server.

**Nhóm chất lượng giao diện và triển khai — Chương 6/7**

MC27 — Responsive: cùng màn hình danh sách course, form, quiz/code editor và Admin table tại 390×844 và 1366×768; nếu có tablet thêm 768×1024. Kiểm tra menu, modal, nút Run/Submit và cuộn ngang; ghi viewport thật.

MC28 — Trạng thái UI: ít nhất một màn hình đang tải, dữ liệu rỗng, validation sai, không quyền và API lỗi; ghi cách thử lại/quay về. Không cần nhiều ảnh trùng nhau, chọn ảnh chứng minh rõ từng trạng thái.

MC29 — Khởi động stack: log migration/Flyway, health của các service và request qua Gateway với DB thật; ghi Java 21, các phiên bản dependency, commit và phương thức chạy. Không đưa .env hoặc log cấu hình chứa secret.

MC30 — Test/build trên máy nhóm: lưu log nguyên bản, command và exit code; nếu tái hiện frontend, đối chiếu với 162 tests đã chạy tại 6.4.5. Backend cần Surefire XML/summary thực tế; không tự điền số pass từ số @Test.

MC31 — Production/backup, nếu nhóm công bố đã triển khai: domain HTTPS thật, image tag/commit, health và thử khôi phục SQL + MongoDB + MinIO trong môi trường kiểm tra. Nếu chỉ demo local, ghi phạm vi local, không cần tạo bằng chứng production.

**Khảo sát và hồ sơ ngoài ứng dụng**

MC32 — Khảo sát: sáu ảnh thật của Moodle/Classroom/Canvas theo 2.2.4, 2.3.4, 2.4.4; mỗi ảnh có URL, ngày và chú thích chức năng. Không dùng ảnh từ sản phẩm LMS để minh họa hệ thống khảo sát.

MC33 — Hồ sơ nhóm: họ tên/MSSV, GVHD/lớp, vai trò, phân công thực tế; commit/PR và prompt AI có thật; link video và nơi nộp. Cần căn cứ trao đổi với giảng viên cho vai trò đề tài không bán hàng và yêu cầu Bootstrap nếu nhóm dùng React/CSS thay template.

Cách gửi bổ sung: gửi từng nhóm MC01–MC06, MC07–MC12, MC13–MC19, MC20–MC26, MC27–MC33. Mỗi bộ ghi "đã chạy đạt", "không đạt" hoặc "bị chặn" kèm kết quả thật. Báo cáo sẽ gắn ảnh vào mục chức năng tương ứng; D.3 giữ vai trò danh sách kiểm soát, không thay toàn bộ ảnh bằng một ảnh tổng hợp.

## D.4 Danh sách sơ đồ và bảng dữ liệu

Tab "Nguồn sơ đồ UML" trong cùng tài liệu cung cấp 74 mô hình PlantUML: 72 mô hình tương ứng Hình 3.1–3.72, mô hình kiến trúc Hình 4.1 và mô hình điều hướng DH.1 cho mục 3.8.1. Mã đã qua kiểm tra cú pháp bằng PlantUML 1.2025.2. Use Case dùng actor thực tế, Activity thể hiện nhánh, ERD lấy FK từ migration; Sequence ở mức nghiệp vụ, không xem note là trace gọi hàm hoặc log đã chạy. Chưa chèn hình xuất vào tab báo cáo; việc kiểm tra cú pháp không xác nhận bố cục hình sau render.

Từ điển dữ liệu có 26 bảng MySQL và năm collection MongoDB, kèm cấu trúc nhúng. Các bảng đặc tả có UC001–UC054 và UC055 mô tả cập nhật role bổ sung. Danh mục hình/bảng cuối báo cáo chỉ ghi số trang sau khi chèn hình, chuẩn hóa caption và dàn trang; không tự gán số trang.

# E Hồ sơ và kịch bản báo cáo cuối kỳ

## E.1 Kịch bản demo theo từng thành viên

Kịch bản demo đề xuất theo luồng nghiệp vụ: Guest/Identity → Student học/làm quiz/code → Instructor quản lý course/nội dung/đánh giá → Admin quản trị → blog/AI và xử lý lỗi. Chia người trình bày theo phần đã thực hiện thật; mỗi người tự giới thiệu, thao tác, giải thích code và nêu một giới hạn.

Không ấn định số phút báo cáo khi chưa biết thời lượng giảng viên cho phép. Chuẩn bị dữ liệu và tài khoản sẵn, tránh tạo cả bộ dữ liệu trong lúc bảo vệ. [Cần nhóm điền người phụ trách từng chặng và thứ tự chính thức.]

## E.2 Kịch bản video có mặt thành viên

Video phải có mặt các thành viên theo thông báo môn học; mỗi người giới thiệu bản thân và chức năng tham gia. Có thể trình bày lần lượt mục tiêu, thao tác, kết quả, code quan trọng và test/lỗi đã xử lý, đúng phạm vi của người đó.

Ghi hình thật, không dùng mockup thay kết quả. Che secret và chỉ dùng dữ liệu demo. Đầu video ghi đề tài/nhóm/phiên bản; cuối video nêu giới hạn còn lại và link source hợp lệ. [Cần nhóm bổ sung video và kiểm tra quyền xem.]

## E.3 Danh mục source database báo cáo và video

Hồ sơ gồm báo cáo DOCX theo yêu cầu, source đúng SHA, migration/dump SQL, dữ liệu Mongo/MinIO cần thiết để demo, hướng dẫn cài đặt, link GitHub, video và rubric tự chấm. Nếu nộp thêm PDF thì đó là bản xuất để đọc, không thay DOCX khi môn yêu cầu.
Không đóng gói .env thật, private key, service-account JSON, token, mật khẩu SMTP/LLM, node_modules hay dữ liệu cá nhân. Cần file cấu hình mẫu và cách nhóm cung cấp credential riêng cho người chạy được phép. Chưa tạo các gói nộp cuối trong đợt này.

## E.4 Liên kết GitHub và thư mục nộp

GitHub: https://github.com/nguyenminhtri15022006-art/LMSmau. Google Docs là tài liệu đang hoàn thiện; link nộp/Drive nhóm, mã nhóm và quyền truy cập cần lấy từ nhóm.
Không tự tạo URL thư mục, ghi đã nộp hoặc thay đổi quyền chia sẻ khi chưa có hành động xác nhận. Kiểm tra người chấm mở được các tệp trước hạn theo thông báo chính thức.

## E.5 Bảng rubric tự chấm

Rubric dùng năm trọng số 10/20/20/35/15, tổng100 và quy đổi tổng/10. Từng thành viên chỉ tự chấm khi có phân công, minh chứng GitHub/AI, phần chức năng, kiểm thử/giao diện và khả năng giải thích.
Mục 6.6 và phần minh chứng dùng để giải thích lựa chọn mức điểm; không mặc định có nhiều công nghệ là mức sáng tạo tối đa.

## E.6 Câu hỏi bảo vệ theo phần việc

Câu hỏi bảo vệ và ý chính cần chuẩn bị:
• Vì sao vừa microservices vừa ba tầng? Phân biệt ranh giới service với controller–service–repository bên trong.
• Request đăng ký course qua những bước nào? Xác định Gateway, userId phiên, trạng thái PUBLIC/LOCKED, enrollment trùng và save.
• Vì sao không lấy quyền từ frontend? Client có thể gửi request trực tiếp; backend cần role/owner và quan hệ dữ liệu.
• OTP sai được xử lý thế nào? Loại mã, hạn, usedAt, attempts và tăng số lần thử sau exception.
• Refresh/logout tác động token nào? Rotation/revoked refresh khác access còn hạn; không khẳng định thu hồi tức thời toàn bộ.
• myinfo/role gọi class nào? UserServiceImpl.updateRole; không gán giới hạn 30 phút của class chưa nối controller.
• Điểm quiz tính ra sao? So đúng cả tập answer; chuẩn hóa trọng số; policy xem kết quả và Grade.best_score.
• Run khác Submit? Run trả output thử; Submit tạo PENDING rồi judgeAsync, đọc kết quả sau.
• Nếu không có test thì sao? Source hiện auto AC100, là giới hạn cần sửa, không chứng minh bài đúng.
• Vì sao MySQL và Mongo? Phân chia dữ liệu cụ thể; ID liên service không tự là FK/cascade.
• Chat dùng WebSocket hay SSE? POST trả event-stream, SESSION trước text, lưu khi complete.

• AI có phải RAG embedding? Chưa thấy pipeline vector; source lấy domain context qua client và lịch sử.
• Meet URL có chắc họp được? Không nếu là fallback; cần eventId/phòng thật.
• Test tên Integration đã kiểm tra toàn stack chưa? Có WebMvcTest/MockBean nên phải nói đúng phạm vi.
• Bootstrap, preview và Web Content đã hoàn thiện chưa? Trình bày đúng các giới hạn và minh chứng thật.
Các ý trên là hướng chuẩn bị, không thay việc mở đúng file và giải thích từng đoạn. Người trình bày cần biết tác động của thay đổi và dữ liệu kiểm thử thuộc phần mình phụ trách.

**Bảng E.1 Các nội dung còn cần bổ sung trước khi nộp**

| Cần hoàn tất trước nộp | Vị trí | Tình trạng |
|---|---|---|
| Tên nhóm, thành viên, MSSV, giảng viên, phân công | Bìa; A.1–A.4 | Cần nhóm bổ sung |
| Phạm vi role và yêu cầu Bootstrap thống nhất | 1.5; 4.4; 6.6 | Chưa xác nhận |
| Ảnh khảo sát và hình UML/kiến trúc | Chương2–4; D.3–D.4 | Chưa chèn |
| Ảnh demo từng role và tích hợp thật | Chương5; D.3 | Cần nhóm chụp |
| Test/lint/build/E2E/coverage và dữ liệu chạy | Chương6; D.1–D.2 | Đã có test/build/lint/coverage; E2E và stack thật chưa chạy |
| SHA demo, cấu hình mẫu, dữ liệu SQL/Mongo/MinIO | A.5; Chương7; E.3 | Cần xác nhận/gói nộp |
| Prompt AI thật và đóng góp GitHub | A.4; C | Cần nhóm cung cấp |
| Video, rubric và link nộp có quyền xem | E.1–E.5 | Chưa xác nhận |
| Caption, mục lục, số trang và xuất DOCX | Phần đầu; toàn báo cáo | Sau khi chốt hình/nội dung |

# F Kết quả rà soát source và phạm vi báo cáo

## F.1 Phiên bản và căn cứ đối chiếu

Báo cáo đối chiếu repository nguyenminhtri15022006-art/LMSmau, nhánh main, commit 4288b95884c031e669aab604ad019c12f46ff162; ứng dụng trong Learning-Management-System-main. Căn cứ gồm README, routes, controller, service, security, entity/document, migration và test. Phân tích tĩnh được bổ sung bằng lượt chạy frontend test/build/lint/coverage và backend test/package ngày 16/10/2026 (UTC+7), chi tiết tại 6.4.5/D.2. Kết quả này không thay việc nghiệm thu stack tích hợp và ảnh chức năng thật.

## F.2 Vai trò và phạm vi chức năng thực tế

Guest là trạng thái chưa đăng nhập, không phải một role lưu trong enum. Khách có thể truy cập trang giới thiệu, danh sách và chi tiết khóa học, blog công khai cùng các trang xác thực. Các role đăng nhập trong source là STUDENT, INSTRUCTOR và ADMIN. Chưa có role MANAGER riêng; việc ánh xạ INSTRUCTOR với vai trò quản lý nghiệp vụ cần trình bày theo phạm vi LMS và thống nhất với giảng viên.

STUDENT có nhánh giao diện riêng cho dashboard, khóa học đã tham gia, học bài, quiz, bài lập trình, kết quả, lịch sử và bảng xếp hạng; có giao diện tạo và quản lý bài viết. Việc học và truy cập dữ liệu cần đối chiếu thêm điều kiện tham gia khóa học tại service, không suy quyền chỉ từ route.
INSTRUCTOR có giao diện quản lý khóa học, cấu trúc nội dung, quiz, bài lập trình, kết quả học viên và blog. Backend cho phép ADMIN cùng INSTRUCTOR ở nhiều API quản lý nội dung, sau đó service kiểm tra quyền sở hữu. Riêng khóa/mở khóa khóa học là quyền ADMIN. Nhánh /manage/assignments hiện trỏ đến AboutUsPage; báo cáo sử dụng các màn hình /instructor/exercises làm căn cứ cho quản lý bài tập, không coi route này là chức năng quản lý hoàn chỉnh.
ADMIN có giao diện quản lý người dùng, tài khoản quản trị, khóa học, blog, nội dung website và hoạt động chấm code. Không mặc định mọi API của giảng viên đều có giao diện tương ứng cho Admin; phân biệt quyền ở backend với chức năng hiển thị trên frontend.

## F.3 Danh mục module và luồng cần mô tả

Identity Service: đăng ký, đăng nhập, Google OAuth, OTP xác minh email, quên/đặt lại mật khẩu, refresh token, đăng xuất, profile, hồ sơ công khai và quản trị user/Admin. Cập nhật role đi qua UserController tới UserServiceImpl.updateRole; SelectRoleServiceImpl có code giới hạn 30 phút nhưng chưa thấy controller gọi. Phạm vi role phải mô tả endpoint thực tế, không gán rule của class chưa nối luồng.
Course Service: danh mục, tra cứu khóa học, tạo/sửa/xóa, đăng ký khóa học, mời và quản lý học viên, cấu trúc nội dung, bài học, hoàn thành bài học và tiến độ, lịch học, thông báo, thống kê giảng viên và khóa/mở khóa bởi Admin. Google Meet được tạo qua Google Calendar API và liên kết với khóa học; lịch học lấy meetingUrl từ course qua ScheduleMapper, không mô tả là mỗi lịch tự tạo một phòng họp độc lập.
Assignment Service: ngân hàng câu hỏi với các loại SINGLE, MULTIPLE và TRUE_FALSE; quiz, bản nháp, công bố/ẩn, nhập template, lượt làm bài, đáp án đã nộp và chấm điểm; bài lập trình, test case, chạy/chấm code, submission, bảng điểm, thống kê và leaderboard. Bổ sung sao chép bài lập trình, ẩn bài, khôi phục bài và khôi phục quiz theo quyền thực tế.
CustomEngine có nhánh báo chưa triển khai và yêu cầu dùng Judge0; chưa đưa custom judge vào kết quả đã hoàn thành.
Blog Service: bài công khai, bài cá nhân, tạo/sửa/xóa, danh sách quản trị, bình luận, trả lời, bình chọn và tag. Quản lý tag thuộc Admin; nội dung cá nhân cần kiểm tra tác giả ở service. Tách quản lý tag khỏi quyền upload tệp, không gom cả hai thành một quyền chung.
Chatbot Service: hỏi đáp trả về text/event-stream, danh sách phiên chat, lịch sử tin nhắn và lưu trữ phiên. Đây là SSE, không gọi là WebSocket. Endpoint sinh quiz/test case trực tiếp tại ChatController yêu cầu INSTRUCTOR; assignment cũng có các luồng AI preview và API nội bộ cần mô tả riêng. Phân biệt AI tích hợp trong ứng dụng với AI nhóm sử dụng hỗ trợ phát triển.

## F.4 Kiến trúc và dữ liệu đã xác định

Hướng dẫn khởi chạy hệ thống giao diện Thymeleaf & Bootstrap 5:
1. Khởi chạy Cổng học tập & luyện thi cho sinh viên (web-portal):
   mvn spring-boot:run -pl backend/web-portal
   Cổng HTTP truy cập: http://localhost:8086

2. Khởi chạy Cổng quản trị & đào tạo (web-app):
   mvn spring-boot:run -pl backend/web-app

Cổng HTTP truy cập: http://localhost:8088

Cấu hình kết nối dịch vụ được định nghĩa trong file application.yml của từng module, tự động liên kết với Spring Cloud Eureka Service Discovery (cổng 8761) và API Gateway (cổng 8080).
Identity MySQL: users, refresh_tokens, email_verifications. Course MySQL: categories, courses, enrollments, lesson_progress, schedules, announcements và ba bảng bổ sung course_categories, course_learning_points, course_requirements. Assignment MySQL: problems, test_cases, quizzes, questions, answers, quiz_questions, quiz_attempts, quiz_answer_records, grades. Blog MySQL: posts, tags, post_tags, comments, votes. Từ điển dữ liệu cần mô tả cả bảng liên kết và bảng collection thay vì chỉ liệt kê entity.
MongoDB có course_structures, lessons, submissions, chat_sessions và chat_messages. Redis phục vụ cache/rate limit; MinIO lưu tài nguyên. Liên kết ID giữa các service không được mô tả thành khóa ngoại xuyên database nếu migration không định nghĩa ràng buộc đó.

## F.5 Giới hạn phải thể hiện đúng trong báo cáo

InspireImagesTab lưu ảnh cấu hình vào localStorage với khóa web_content_inspire; chưa phải lưu tập trung trên backend. FoundersContentTab chỉ cập nhật state và thông báo nội dung đang được đặt cứng trong code khi bấm lưu; chưa có chức năng lưu nội dung người sáng lập vào database. CourseCategoriesTab và BlogCategoriesTab có gọi API; không gộp toàn bộ trang quản lý nội dung website vào nhóm giao diện giả lập.
README gọi hệ thống EduLearn LMS, trong khi một số nội dung frontend dùng DevNest và chứa tên người sáng lập có sẵn. Những tên này không phải căn cứ xác định thành viên nhóm nộp bài. Giữ tên đề tài mô tả hệ thống quản lý học tập và bổ sung tên thương hiệu/thành viên khi nhóm xác nhận.
Chưa tìm thấy bằng chứng sử dụng Bootstrap trong package.json và phần frontend đã rà soát; chưa ghi đáp ứng yêu cầu template Bootstrap. Kiểm tra bổ sung đã chạy frontend: 162 tests đạt, build thành công, lint còn 17 cảnh báo; coverage toàn cấu hình thấp hơn đáng kể phạm vi core (xem 6.4.5). Kết quả này chưa chứng minh vận hành production, mức điểm rubric hoặc hoàn thành các luồng tích hợp.

## F.6 Trạng thái hoàn thiện và việc cần xác nhận

Báo cáo đã mô tả chọn vai trò sau OAuth, khóa/mở khóa của Admin, sao chép/ẩn/khôi phục bài tập, tag Admin và các bảng liên kết MySQL theo code. Giữ cấp đánh số thấp nhất x.x.x. Chức năng SSE, Judge0, Web Content và preview được phân biệt theo khả năng thực tế; không mô tả phần chưa triển khai thành kết quả đã đạt.
Phần văn bản, đặc tả, từ điển dữ liệu, kiến trúc, API, hướng dẫn cài đặt và kế hoạch kiểm thử đã được biên soạn từ source. Các hình thiết kế chưa được chèn, ảnh giao diện chưa có và các luồng tích hợp chưa được nghiệm thu. Các thông tin nhóm, phân công thực tế, lịch sử AI/GitHub, video và chấp thuận yêu cầu vai trò/Bootstrap phải do nhóm cung cấp. Danh sách thao tác thu minh chứng tại D.3 là căn cứ bổ sung, không phải biên bản chức năng đã chạy đạt.

D.4 Danh mục và quy trình biên dịch sơ đồ kiến trúc hệ thống (PlantUML)
Toàn bộ 74 mô hình kiến trúc của hệ thống EduLearn LMS (bao gồm 5 sơ đồ Use Case, 7 sơ đồ luồng hoạt động, 54 sơ đồ tuần tự Sequence Diagram chi tiết hóa từng ca sử dụng, 6 sơ đồ cơ sở dữ liệu quan hệ ERD/MongoDB, sơ đồ kiến trúc triển khai hệ thống Hình 4.1 và sơ đồ điều hướng vai trò Hình DH.1) được lưu trữ và quản lý dưới dạng mã nguồn mở tại thư mục DoAnCK/docs/diagrams/plantuml/*.puml.

Quy trình biên dịch và xuất đồ họa vector/PNG độ nét cao:
1. Cài đặt môi trường đồ họa: Graphviz (dot engine) và Java Runtime Environment (JRE 17+).
2. Lệnh biên dịch tự động toàn bộ sơ đồ:
   java -jar plantuml.jar -tpng -charset UTF-8 DoAnCK/docs/diagrams/plantuml/*.puml
Tất cả sơ đồ đã được cập nhật đồng bộ với kiến trúc Server-Side Rendering (Trình duyệt Thymeleaf UI) và cơ chế kiểm soát phân quyền Spring Security của hệ thống.
PHỤ LỤC E: BẢNG DANH MỤC CÁC HẠNG MỤC CHỜ NGHIỆM THU THỰC TẾ
Nhằm đảm bảo tính minh bạch học thuật và phản ánh khách quan trạng thái triển khai sản phẩm, bảng dưới đây tổng hợp toàn bộ các hạng mục đang trong giai đoạn hoàn thiện hoặc chờ kiểm chứng thực tế trước buổi báo cáo bảo vệ đồ án:

| STT | Hạng mục & Vị trí trong báo cáo | Hiện trạng kỹ thuật theo mã nguồn | Kế hoạch hoàn thiện nghiệm thu |
|---|---|---|---|
| 1 | Thông tin nhóm & Bìa chính thức (Mục 1.1, Bìa) | Đã chốt thành viên Nguyễn Minh Trí (MSSV: 24110359). | Cập nhật chính xác mã lớp học phần theo thời khóa biểu nộp bài. |
| 2 | Xác minh trùng lặp đăng ký (Mục 5.1.1) | Backend đã kiểm tra unique; giao diện form đang hoàn thiện thông báo lỗi. | Bổ sung bắt lỗi hiển thị alert trên giao diện khi trùng username/email. |
| 3 | Đăng nhập Google OAuth (Mục 5.1.4) | Backend AuthenticationServiceImpl đã xử lý OAuth2, cấp token và phân role. | Cấu hình Client ID/Secret trên Google Cloud Console cho domain production. |
| 4 | Responsive trên di động nhỏ (Mục 1.5.4, 4.4.5) | Bootstrap 5 co giãn tốt trên Desktop/Tablet; màn hình mobile hẹp còn hạn chế. | Tinh chỉnh CSS media queries cho Monaco Editor và Table quản trị. |
| 5 | Tích hợp Google Meet tự động (Mục 4.5.3) | Source có Service Account; tạo link phòng học dự phòng. | Cấu hình API Calendar đầy đủ quyền để tạo link phòng học trực tiếp. |
| 6 | Giám sát Admin khi offline (Mục 5.6.1, 5.6.7) | Giao diện đã có cơ chế bắt lỗi và hiển thị cảnh báo khi service tắt. | Khởi chạy toàn bộ cụm microservices khi quay video demo và bảo vệ. |
| 7 | Kiểm thử tải và độ phủ test (Chương 6) | Đã có Unit test và MockMvc cho các controller chính. | Mở rộng test case tích hợp toàn diện và đo độ phủ bằng JaCoCo. |